



OSR eHSM Host API 驱动库参考手册

版本	1.0
作者	固件开发组
部门	OSR IP
日期	2026 - 06 - 26
分类	用户手册

Copyright Notice and Proprietary Information Notice

Copyright © 2014 - 2026 Open Security Research, Inc. All rights reserved. This documentation contains confidential and proprietary information that is the property of Open Security Research, Inc. The documentation is transported under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, translated, distributed, disclosed, announced or copied in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Open Security Research, Inc., or as expressly provided by the license agreement.

版权声明和专有信息声明

版权所有 © 2014 - 2026 深圳市纽创信安科技开发有限公司 保留所有权利。本文档包含机密和专有信息，属于深圳市纽创信安科技开发有限公司的财产。该文档根据对应的许可协议传播，且只能根据许可协议的条款使用或复制。未经深圳市纽创信安科技开发有限公司事先书面许可或经许可协议明确规定，该文档和对应软件的全部或任何部分均不得以任何形式或方式（电子的，机械的，手动的，光学的或其他的）被复制、传播、翻译、分发、披露、宣布或抄袭。

修订历史

版本	修改日期	描述
1.0	2025-09-30	初版

目录

1 eHSM Host API	19
1.1 基础库	19
2 基础驱动库 API	20
2.1 基本接口	20
2.1.1 驱动初始化与上下文管理	20
2.2 API 功能分类概览	20
2.2.1 随机数生成	20
2.2.2 HASH/摘要算法	20
2.2.3 对称加密算法	21
2.2.4 非对称加密	23
2.2.5 数字签名	23
2.2.6 密钥管理	25
2.2.7 系统管理	25
2.3 BL 专有接口	26
2.3.1 自检功能	26
2.3.2 镜像验证与固件管理	27
2.3.3 固件认证	27
2.3.4 密钥生成与加密	27
2.3.5 系统信息与配置	27
2.3.6 OTP 与寄存器操作	27
3 驱动移植指南	28
3.1 概述	28
3.2 文件结构	28
3.3 宏定义	28
3.3.1 示例	28
3.4 接口实现	29
3.4.1 ehsm_port_init	29
3.4.2 ehsm_port_addr_to_raddr	29
3.4.3 ehsm_port_raddr_to_addr	30
3.4.4 ehsm_port_flush_and_invalidate_cache	30
3.4.5 ehsm_port_enable_mailbox_int	30
3.4.6 ehsm_port_reset_ehsm	31
3.4.7 ehsm_port_read_reg	31
3.4.8 ehsm_port_write_reg	32
3.4.9 ehsm_port_read_otp	32
3.4.10 ehsm_port_write_otp	32
3.4.11 ehsm_port_create_timer 和 ehsm_port_is_timeout	33
3.5 构建脚本适配	33
4 Host demo 指南	35
4.1 概述	35
4.2 Demo 示例功能	35

4.2.1	Bootloader demo	35
4.2.2	Firmware demo	35
4.2.3	编译说明	36
4.2.4	运行说明	37
5	密钥管理方案	38
5.1	密钥权限	38
5.2	密钥导入/导出数据结构说明	38
5.2.1	密钥头	38
5.2.2	密钥数据格式	38
5.3	明文密钥数据格式	40
6	共享内存机制	42
7	eHSM Speed Test 程序使用说明	43
7.1	概述	43
7.2	程序结构	43
7.3	编译方法	43
7.3.1	使用 CMake 编译	43
7.3.2	编译产物	44
7.4	运行方式	44
7.4.1	运行所有测试	44
7.5	测试类型详解	44
7.5.1	1. SKE (对称密钥加密) 测试	44
7.5.2	2. HASH (哈希算法) 测试	45
7.5.3	3. PKE (公钥加密) 测试	45
7.5.4	4. MAC (消息认证码) 测试	45
7.5.5	5. AEAD (认证加密) 测试	45
7.5.6	6. CHACHA 测试	45
7.6	测试参数配置	45
7.6.1	默认配置	45
7.6.2	性能指标	45
7.7	硬件要求	46
7.7.1	eHSM 硬件	46
7.7.2	OTP 配置	46
7.8	输出示例	46
7.9	注意事项	46
7.10	故障排除	47
7.10.1	常见错误	47
7.10.2	调试建议	47
8	API Reference	48
8.1	Groups	48
8.1.1	Group FW (Firmware) 相关的 API 接口	48
8.1.2	Group 密钥导入/导入数据格式结构	57
8.1.3	Group 密钥权限	61
8.1.4	Group BL (Bootloader) 相关的 API 接口	61
8.1.5	Group 自检算法	62

8.2	Files	63
8.2.1	File api.h	63
8.2.2	File bl_api.h	69
8.2.3	File key_assist.h	70
8.2.4	File mailbox.h	73
8.2.5	File ehsm_host_port.h	74
8.2.6	File types.h	75
8.2.7	File version.h	75
8.3	Structs	76
8.3.1	ehsm_ctx_st	76
8.3.2	ehsm_dh_params_1024_st	76
8.3.3	ehsm_dh_params_2048_st	76
8.3.4	ehsm_dh_params_3072_st	76
8.3.5	ehsm_dh_params_4096_st	77
8.3.6	ehsm_ecc_key_st	77
8.3.7	ehsm_emu_status_st	77
8.3.8	ehsm_key_format_dh_1024_cipher_keypair_st	78
8.3.9	ehsm_key_format_dh_1024_cipher_privkey_st	78
8.3.10	ehsm_key_format_dh_1024_plain_keypair_st	78
8.3.11	ehsm_key_format_dh_1024_plain_privkey_st	79
8.3.12	ehsm_key_format_dh_1024_plain_pubkey_st	79
8.3.13	ehsm_key_format_dh_2048_cipher_keypair_st	79
8.3.14	ehsm_key_format_dh_2048_cipher_privkey_st	80
8.3.15	ehsm_key_format_dh_2048_plain_keypair_st	80
8.3.16	ehsm_key_format_dh_2048_plain_privkey_st	80
8.3.17	ehsm_key_format_dh_2048_plain_pubkey_st	81
8.3.18	ehsm_key_format_dh_3072_cipher_keypair_st	81
8.3.19	ehsm_key_format_dh_3072_cipher_privkey_st	81
8.3.20	ehsm_key_format_dh_3072_plain_keypair_st	82
8.3.21	ehsm_key_format_dh_3072_plain_privkey_st	82
8.3.22	ehsm_key_format_dh_3072_plain_pubkey_st	82
8.3.23	ehsm_key_format_dh_4096_cipher_keypair_st	83
8.3.24	ehsm_key_format_dh_4096_cipher_privkey_st	83
8.3.25	ehsm_key_format_dh_4096_plain_keypair_st	83
8.3.26	ehsm_key_format_dh_4096_plain_privkey_st	84
8.3.27	ehsm_key_format_dh_4096_plain_pubkey_st	84
8.3.28	ehsm_key_format_ecc_160_cipher_keypair_st	84
8.3.29	ehsm_key_format_ecc_160_cipher_privkey_st	85
8.3.30	ehsm_key_format_ecc_160_plain_keypair_st	85
8.3.31	ehsm_key_format_ecc_160_plain_privkey_st	86
8.3.32	ehsm_key_format_ecc_160_plain_pubkey_st	86
8.3.33	ehsm_key_format_ecc_192_cipher_keypair_st	86
8.3.34	ehsm_key_format_ecc_192_cipher_privkey_st	87
8.3.35	ehsm_key_format_ecc_192_plain_keypair_st	87

8.3.36	ehsm_key_format_ecc_192_plain_privkey_st	87
8.3.37	ehsm_key_format_ecc_192_plain_pubkey_st	88
8.3.38	ehsm_key_format_ecc_224_cipher_keypair_st	88
8.3.39	ehsm_key_format_ecc_224_cipher_privkey_st	88
8.3.40	ehsm_key_format_ecc_224_plain_keypair_st	89
8.3.41	ehsm_key_format_ecc_224_plain_privkey_st	89
8.3.42	ehsm_key_format_ecc_224_plain_pubkey_st	89
8.3.43	ehsm_key_format_ecc_25519_cipher_keypair_st	90
8.3.44	ehsm_key_format_ecc_25519_cipher_privkey_st	90
8.3.45	ehsm_key_format_ecc_25519_plain_keypair_st	91
8.3.46	ehsm_key_format_ecc_25519_plain_privkey_st	91
8.3.47	ehsm_key_format_ecc_25519_plain_pubkey_st	91
8.3.48	ehsm_key_format_ecc_256_cipher_keypair_st	92
8.3.49	ehsm_key_format_ecc_256_cipher_privkey_st	92
8.3.50	ehsm_key_format_ecc_256_plain_keypair_st	92
8.3.51	ehsm_key_format_ecc_256_plain_privkey_st	93
8.3.52	ehsm_key_format_ecc_256_plain_pubkey_st	93
8.3.53	ehsm_key_format_ecc_320_cipher_keypair_st	93
8.3.54	ehsm_key_format_ecc_320_cipher_privkey_st	94
8.3.55	ehsm_key_format_ecc_320_plain_keypair_st	94
8.3.56	ehsm_key_format_ecc_320_plain_privkey_st	94
8.3.57	ehsm_key_format_ecc_320_plain_pubkey_st	95
8.3.58	ehsm_key_format_ecc_384_cipher_keypair_st	95
8.3.59	ehsm_key_format_ecc_384_cipher_privkey_st	96
8.3.60	ehsm_key_format_ecc_384_plain_keypair_st	96
8.3.61	ehsm_key_format_ecc_384_plain_privkey_st	96
8.3.62	ehsm_key_format_ecc_384_plain_pubkey_st	97
8.3.63	ehsm_key_format_ecc_512_cipher_keypair_st	97
8.3.64	ehsm_key_format_ecc_512_cipher_privkey_st	97
8.3.65	ehsm_key_format_ecc_512_plain_keypair_st	98
8.3.66	ehsm_key_format_ecc_512_plain_privkey_st	98
8.3.67	ehsm_key_format_ecc_512_plain_pubkey_st	98
8.3.68	ehsm_key_format_ecc_521_cipher_keypair_st	99
8.3.69	ehsm_key_format_ecc_521_cipher_privkey_st	99
8.3.70	ehsm_key_format_ecc_521_plain_keypair_st	99
8.3.71	ehsm_key_format_ecc_521_plain_privkey_st	100
8.3.72	ehsm_key_format_ecc_521_plain_pubkey_st	100
8.3.73	ehsm_key_format_rsa1024_cipher_keypair_st	101
8.3.74	ehsm_key_format_rsa1024_cipher_privkey_st	101
8.3.75	ehsm_key_format_rsa1024_cert_cipher_keypair_st	101
8.3.76	ehsm_key_format_rsa1024_cert_cipher_privkey_st	102
8.3.77	ehsm_key_format_rsa1024_cert_plain_keypair_st	102
8.3.78	ehsm_key_format_rsa1024_cert_plain_privkey_st	103
8.3.79	ehsm_key_format_rsa1024_plain_keypair_st	103

8.3.80	ehsm_key_format_rsa1024_plain_privkey_st	104
8.3.81	ehsm_key_format_rsa1024_plain_pubkey_st	104
8.3.82	ehsm_key_format_rsa2048_cipher_keypair_st	104
8.3.83	ehsm_key_format_rsa2048_cipher_privkey_st	105
8.3.84	ehsm_key_format_rsa2048_crt_cipher_keypair_st	105
8.3.85	ehsm_key_format_rsa2048_crt_cipher_privkey_st	106
8.3.86	ehsm_key_format_rsa2048_crt_plain_keypair_st	106
8.3.87	ehsm_key_format_rsa2048_crt_plain_privkey_st	107
8.3.88	ehsm_key_format_rsa2048_plain_keypair_st	107
8.3.89	ehsm_key_format_rsa2048_plain_privkey_st	108
8.3.90	ehsm_key_format_rsa2048_plain_pubkey_st	108
8.3.91	ehsm_key_format_rsa3072_cipher_keypair_st	108
8.3.92	ehsm_key_format_rsa3072_cipher_privkey_st	109
8.3.93	ehsm_key_format_rsa3072_crt_cipher_keypair_st	109
8.3.94	ehsm_key_format_rsa3072_crt_cipher_privkey_st	110
8.3.95	ehsm_key_format_rsa3072_crt_plain_keypair_st	110
8.3.96	ehsm_key_format_rsa3072_crt_plain_privkey_st	111
8.3.97	ehsm_key_format_rsa3072_plain_keypair_st	111
8.3.98	ehsm_key_format_rsa3072_plain_privkey_st	111
8.3.99	ehsm_key_format_rsa3072_plain_pubkey_st	112
8.3.100	ehsm_key_format_rsa4096_cipher_keypair_st	112
8.3.101	ehsm_key_format_rsa4096_cipher_privkey_st	112
8.3.102	ehsm_key_format_rsa4096_crt_cipher_keypair_st	113
8.3.103	ehsm_key_format_rsa4096_crt_cipher_privkey_st	113
8.3.104	ehsm_key_format_rsa4096_crt_plain_keypair_st	114
8.3.105	ehsm_key_format_rsa4096_crt_plain_privkey_st	114
8.3.106	ehsm_key_format_rsa4096_plain_keypair_st	115
8.3.107	ehsm_key_format_rsa4096_plain_privkey_st	115
8.3.108	ehsm_key_format_rsa4096_plain_pubkey_st	116
8.3.109	ehsm_key_format_sm2_cipher_keypair_st	116
8.3.110	ehsm_key_format_sm2_cipher_privkey_st	116
8.3.111	ehsm_key_format_sm2_plain_keypair_st	117
8.3.112	ehsm_key_format_sm2_plain_privkey_st	117
8.3.113	ehsm_key_format_sm2_plain_pubkey_st	117
8.3.114	ehsm_key_format_st	118
8.3.115	ehsm_key_format_symm_cipher_128_st	118
8.3.116	ehsm_key_format_symm_cipher_192_st	119
8.3.117	ehsm_key_format_symm_cipher_256_st	119
8.3.118	ehsm_key_format_symm_cipher_384_st	119
8.3.119	ehsm_key_format_symm_cipher_512_st	120
8.3.120	ehsm_key_format_symm_cipher_64_st	120
8.3.121	ehsm_key_format_symm_plain_128_st	120
8.3.122	ehsm_key_format_symm_plain_192_st	121
8.3.123	ehsm_key_format_symm_plain_256_st	121

8.3.124	ehsm_key_format_symm_plain_384_st	121
8.3.125	ehsm_key_format_symm_plain_512_st	122
8.3.126	ehsm_key_format_symm_plain_64_st	122
8.3.127	ehsm_mb_cmd_st	122
8.3.128	ehsm_mb_packet_st	123
8.3.129	ehsm_mb_rsp_st	123
8.3.130	ehsm_rsa_key_st	123
8.3.131	ehsm_self_test_result_st	123
8.3.132	ehsm_session_st	124
8.3.133	ehsm_sm2_key_st	124
8.3.134	ehsm_sm2_params_st	124
8.3.135	ehsm_sm9_params_st	124
8.3.136	ehsm_soc_dbg_bitmap_st	125
8.3.137	ehsm_version_st	125
8.4	Functions	126
8.4.1	ehsm_mb_init	126
8.4.2	ehsm_mb_send	126
8.4.3	ehsm_mb_poll	126
8.4.4	ehsm_port_init	126
8.4.5	ehsm_port_addr_to_raddr	127
8.4.6	ehsm_port_raddr_to_addr	127
8.4.7	ehsm_port_flush_and_invalidate_cache	127
8.4.8	ehsm_port_enable_mailbox_int	128
8.4.9	ehsm_port_reset_ehsm	128
8.4.10	ehsm_port_read_reg	128
8.4.11	ehsm_port_write_reg	129
8.4.12	ehsm_port_read_otp	129
8.4.13	ehsm_port_write_otp	129
8.4.14	ehsm_port_create_timer	130
8.4.15	ehsm_port_is_timeout	130
8.4.16	ehsm_port_get_time_ms	130
8.4.17	ehsm_port_memory_barrier_write	131
8.4.18	ehsm_port_memory_barrier_read	131
8.4.19	ehsm_port_enter_critical	131
8.4.20	ehsm_port_exit_critical	131
8.4.21	ehsm_driver_init_library	131
8.4.22	ehsm_driver_get_version	132
8.4.23	ehsm_ctx_init	132
8.4.24	ehsm_ctx_poll	133
8.4.25	ehsm_gen_random	133
8.4.26	ehsm_hash_init	134
8.4.27	ehsm_hash_update	134
8.4.28	ehsm_hash_finish	135
8.4.29	ehsm_hash_onepass	135

8.4.30	ehsm_hmac_init	136
8.4.31	ehsm_hmac_update	137
8.4.32	ehsm_hmac_finish_gen	137
8.4.33	ehsm_hmac_finish_verify	138
8.4.34	ehsm_hmac_onepass_gen	138
8.4.35	ehsm_hmac_onepass_verify	139
8.4.36	ehsm_hmac_init_with_plain_key	139
8.4.37	ehsm_hmac_onepass_gen_with_plain_key	140
8.4.38	ehsm_hmac_onepass_verify_with_plain_key	141
8.4.39	ehsm_mac_init	142
8.4.40	ehsm_mac_update	142
8.4.41	ehsm_mac_finish_gen	143
8.4.42	ehsm_mac_finish_verify	144
8.4.43	ehsm_mac_onepass_gen	144
8.4.44	ehsm_mac_onepass_verify	145
8.4.45	ehsm_mac_init_with_plain_key	146
8.4.46	ehsm_mac_onepass_gen_with_plain_key	147
8.4.47	ehsm_mac_onepass_verify_with_plain_key	147
8.4.48	ehsm_symm_cipher_init	148
8.4.49	ehsm_symm_cipher_update	149
8.4.50	ehsm_symm_cipher_finish	150
8.4.51	ehsm_symm_cipher_onepass	151
8.4.52	ehsm_symm_cipher_init_with_plain_key	152
8.4.53	ehsm_symm_cipher_onepass_with_plain_key	153
8.4.54	ehsm_sm2_cipher	154
8.4.55	ehsm_sm2_cipher_with_plain_key	155
8.4.56	ehsm_sm2_sign_init	156
8.4.57	ehsm_sm2_sign_init_with_plain_key	156
8.4.58	ehsm_sm2_sign_update	157
8.4.59	ehsm_sm2_sign_finish_gen	157
8.4.60	ehsm_sm2_sign_finish_verify	158
8.4.61	ehsm_sm2_sign_onepass_ex	159
8.4.62	ehsm_sm2_sign_onepass_gen	160
8.4.63	ehsm_sm2_sign_onepass_verify	160
8.4.64	ehsm_sm2_sign_onepass_gen_with_digest	161
8.4.65	ehsm_sm2_sign_onepass_verify_with_digest	161
8.4.66	ehsm_sm2_sign_onepass_gen_with_plain_key	162
8.4.67	ehsm_sm2_sign_onepass_verify_with_plain_key	163
8.4.68	ehsm_sm2_sign_onepass_gen_with_plain_key_and_digest	163
8.4.69	ehsm_sm2_sign_onepass_verify_with_plain_key_and_digest	164
8.4.70	ehsm_rsa_cipher	165
8.4.71	ehsm_rsa_cipher_with_plain_key	166
8.4.72	ehsm_rsa_sign_init	166
8.4.73	ehsm_rsa_sign_init_with_plain_key	167

8.4.74	ehsm_rsa_sign_update	168
8.4.75	ehsm_rsa_sign_finish_gen	168
8.4.76	ehsm_rsa_sign_finish_verify	169
8.4.77	ehsm_rsa_sign_onepass_ex	169
8.4.78	ehsm_rsa_sign_onepass_gen	171
8.4.79	ehsm_rsa_sign_onepass_verify	171
8.4.80	ehsm_rsa_sign_onepass_gen_with_digest	172
8.4.81	ehsm_rsa_sign_onepass_verify_with_digest	173
8.4.82	ehsm_rsa_sign_onepass_gen_with_plain_key	174
8.4.83	ehsm_rsa_sign_onepass_verify_with_plain_key	175
8.4.84	ehsm_rsa_sign_onepass_gen_with_plain_key_and_digest	175
8.4.85	ehsm_rsa_sign_onepass_verify_with_plain_key_and_digest	176
8.4.86	ehsm_ecdsa_init	177
8.4.87	ehsm_ecdsa_init_with_plain_key	178
8.4.88	ehsm_ecdsa_update	178
8.4.89	ehsm_ecdsa_finish_gen	179
8.4.90	ehsm_ecdsa_finish_verify	179
8.4.91	ehsm_ecdsa_onepass_ex	180
8.4.92	ehsm_ecdsa_onepass_gen	181
8.4.93	ehsm_ecdsa_onepass_verify	182
8.4.94	ehsm_ecdsa_onepass_gen_with_digest	182
8.4.95	ehsm_ecdsa_onepass_verify_with_digest	183
8.4.96	ehsm_ecdsa_onepass_gen_with_plain_key	184
8.4.97	ehsm_ecdsa_onepass_verify_with_plain_key	184
8.4.98	ehsm_ecdsa_onepass_gen_with_plain_key_and_digest	185
8.4.99	ehsm_ecdsa_onepass_verify_with_plain_key_and_digest	186
8.4.100	ehsm_sm9_cipher	186
8.4.101	ehsm_sm9_cipher_with_plain_key	188
8.4.102	ehsm_sm9_sign_onepass_gen	189
8.4.103	ehsm_sm9_sign_onepass_verify	189
8.4.104	ehsm_sm9_sign_onepass_gen_with_plain_key	190
8.4.105	ehsm_chacha_init	191
8.4.106	ehsm_chacha_update	192
8.4.107	ehsm_chacha_finish_enc	192
8.4.108	ehsm_chacha_finish_dec	193
8.4.109	ehsm_chacha_onepass_enc	194
8.4.110	ehsm_chacha_onepass_dec	194
8.4.111	ehsm_chacha_init_with_plain_key	195
8.4.112	ehsm_chacha_onepass_enc_with_plain_key	196
8.4.113	ehsm_chacha_onepass_dec_with_plain_key	197
8.4.114	ehsm_aead_init	198
8.4.115	ehsm_aead_init_with_plain_key	199
8.4.116	ehsm_aead_update	200
8.4.117	ehsm_aead_finish_enc	201

8.4.118 ehsm_aead_finish_dec	201
8.4.119 ehsm_aead_onepass_enc	202
8.4.120 ehsm_aead_onepass_dec	203
8.4.121 ehsm_aead_onepass_enc_with_plain_key	204
8.4.122 ehsm_aead_onepass_dec_with_plain_key	205
8.4.123 ehsm_km_gen_key	206
8.4.124 ehsm_km_import_key	207
8.4.125 ehsm_km_export_key	208
8.4.126 ehsm_km_derive_key	209
8.4.127 ehsm_km_derive_key_to_soc	210
8.4.128 ehsm_km_exchange_key	210
8.4.129 ehsm_km_sm9_exchange_key	211
8.4.130 ehsm_km_get_pub_from_priv	212
8.4.131 ehsm_km_remove_key	213
8.4.132 ehsm_get_version	213
8.4.133 ehsm_verify_image	214
8.4.134 ehsm_upgrade_fw_image_ex	215
8.4.135 ehsm_upgrade_fw_image	215
8.4.136 ehsm_upgrade_fw_image_init	216
8.4.137 ehsm_upgrade_fw_image_update	216
8.4.138 ehsm_upgrade_fw_image_finish	217
8.4.139 ehsm_set_utc_time	218
8.4.140 ehsm_get_utc_time	218
8.4.141 ehsm_create_counter	219
8.4.142 ehsm_read_counter	219
8.4.143 ehsm_increase_counter	220
8.4.144 ehsm_delete_counter	220
8.4.145 ehsm_enter_wfi	221
8.4.146 ehsm_set_uart_baudrate	221
8.4.147 ehsm_install_random_key	221
8.4.148 ehsm_install_encrypted_key	222
8.4.149 ehsm_read_otp	223
8.4.150 ehsm_write_otp	223
8.4.151 ehsm_read_reg	224
8.4.152 ehsm_write_reg	224
8.4.153 ehsm_get_challenge	225
8.4.154 ehsm_debug_auth	225
8.4.155 ehsm_close_debug	226
8.4.156 ehsm_get_emu_status	227
8.4.157 ehsm_change_lifecycle	227
8.4.158 ehsm_change_control_field	228
8.4.159 ehsm_bl_self_test	228
8.4.160 ehsm_bl_get_self_test_result	229
8.4.161 ehsm_bl_fw_auth	229

8.4.162	ehsm_bl_get_random_key	230
8.4.163	ehsm_bl_encrypt_key	230
8.5	Enums	231
8.5.1	ehsm_reg_e	231
8.5.2	ehsm_hash_algo_e	232
8.5.3	ehsm_rng_algo_e	232
8.5.4	ehsm_symm_algo_e	233
8.5.5	ehsm_mac_mode_e	233
8.5.6	ehsm_padding_mode_e	233
8.5.7	ehsm_rsa_padding_mode_e	233
8.5.8	ehsm_cipher_mode_e	233
8.5.9	ehsm_aead_mode_e	234
8.5.10	ehsm_key_type_e	234
8.5.11	ehsm_key_part_e	236
8.5.12	ehsm_derive_algo_e	236
8.5.13	ehsm_derive_type_e	236
8.5.14	ehsm_sm2_role_e	236
8.5.15	ehsm_sm9_enc_type_e	236
8.5.16	ehsm_sm9_padding_mode_e	236
8.5.17	ehsm_sm9_role_e	237
8.5.18	ehsm_proc_mode_e	237
8.5.19	ehsm_drv_mode_e	237
8.5.20	ehsm_key_level_e	237
8.5.21	ehsm_install_key_type_e	237
8.5.22	ehsm_challenge_type_e	238
8.5.23	ehsm_auth_algo_e	238
8.5.24	ehsm_lifecycle_e	238
8.5.25	ehsm_ctrl_field_e	239
8.5.26	ehsm_bl_gen_key_type_e	239
8.6	Defines	239
8.6.1	ehsm_port_printf	239
8.6.2	NULL	239
8.6.3	EHSM_OK	239
8.6.4	EHSM_ERR_PARAM_ERROR	239
8.6.5	EHSM_ERR_BUSY	239
8.6.6	EHSM_ERR_CTX_NOT_INIT	239
8.6.7	EHSM_ERR_CTX_STATUS	239
8.6.8	EHSM_ERR_TIMEOUT	240
8.6.9	EHSM_ERR_CTX_INVALID	240
8.6.10	EHSM_ERR_ILLEGAL_USE	240
8.6.11	EHSM_ERR_DEV_ERROR	240
8.6.12	EHSM_ERR_FW_BASE	240
8.6.13	EHSM_ERR_NEED_POLL	240
8.6.14	EHSM_SHM	240

8.6.15	EHSM_DRV_VERSION_MAJOR	240
8.6.16	EHSM_DRV_VERSION_MINOR	240
8.6.17	EHSM_DRV_VERSION_PATCH	240
8.6.18	EHSM_DRV_VERSION_PRE_RELEASE	240
8.6.19	EHSM_KEY_HANDLE_INVALID	240
8.6.20	EHSM_KEY_PRIV_SIGN	241
8.6.21	EHSM_KEY_PRIV_VERIFY	241
8.6.22	EHSM_KEY_PRIV_ENCRYPT	241
8.6.23	EHSM_KEY_PRIV_DECRYPT	241
8.6.24	EHSM_KEY_PRIV_TIMESTAMP	241
8.6.25	EHSM_KEY_PRIV_SECURE_BOOT	241
8.6.26	EHSM_KEY_PRIV_SECURE_STORAGE	241
8.6.27	EHSM_KEY_PRIV_KEY_CREATION	241
8.6.28	EHSM_KEY_PRIV_TRANSPORT_KEY_CREATION	241
8.6.29	EHSM_KEY_PRIV_UTCSYNC	241
8.6.30	EHSM_KEY_PRIV_TRANSPORT	242
8.6.31	EHSM_KEY_PRIV_REMOVE	242
8.6.32	EHSM_KEY_PRIV_IMPORT_PLAIN	242
8.6.33	EHSM_KEY_PRIV_IMPORT_CIPHER	242
8.6.34	EHSM_KEY_PRIV_EXPORT_PLAIN	242
8.6.35	EHSM_KEY_PRIV_EXPORT_CIPHER	242
8.6.36	EHSM_KEY_PRIV_BOOT_FAIL_USAGE	242
8.6.37	EHSM_KEY_PRIV_DEBUG_USAGE	242
8.6.38	EHSM_KEY_PRIV_WILDCARD_PROTECT	242
8.6.39	EHSM_KEY_PRIV_WRITE_PROTECT	243
8.6.40	ehsm_bl_get_version	243
8.6.41	ehsm_bl_verify_image	243
8.6.42	ehsm_bl_upgrade_fw_image	243
8.6.43	ehsm_bl_upgrade_fw_image_init	243
8.6.44	ehsm_bl_upgrade_fw_image_update	243
8.6.45	ehsm_bl_upgrade_fw_image_finish	243
8.6.46	ehsm_bl_set_uart_baudrate	243
8.6.47	ehsm_bl_get_challenge	243
8.6.48	ehsm_bl_debug_auth	243
8.6.49	ehsm_bl_close_debug	244
8.6.50	ehsm_bl_read_reg	244
8.6.51	ehsm_bl_write_reg	244
8.6.52	ehsm_bl_read_otp	244
8.6.53	ehsm_bl_write_otp	244
8.6.54	EHSM_SELF_TEST_SM4_ECB	244
8.6.55	EHSM_SELF_TEST_SM4_CBC	244
8.6.56	EHSM_SELF_TEST_SM4_CFB	244
8.6.57	EHSM_SELF_TEST_SM4_OFB	244
8.6.58	EHSM_SELF_TEST_SM4_CTR	244

8.6.59	EHSM_SELF_TEST_SM2_ENC	244
8.6.60	EHSM_SELF_TEST_SM2_VERIFY	245
8.6.61	EHSM_SELF_TEST_SM2_EXCH	245
8.6.62	EHSM_SELF_TEST_SM3	245
8.6.63	EHSM_SELF_TEST_DES	245
8.6.64	EHSM_SELF_TEST_TDES	245
8.6.65	EHSM_SELF_TEST_AES	245
8.6.66	EHSM_SELF_TEST_RSA	245
8.6.67	EHSM_SELF_TEST_ECC	245
8.6.68	EHSM_SELF_TEST_MD5	245
8.6.69	EHSM_SELF_TEST_SHA1	245
8.6.70	EHSM_SELF_TEST_SHA2	245
8.6.71	EHSM_SELF_TEST_SHA3	245
8.6.72	EHSM_SELF_TEST_SHA256	245
8.6.73	EHSM_SELF_TEST_TRNG	245
8.7	Typedefs	246
8.7.1	ehsm_mb_interrupt_func_t	246
8.7.2	ehsm_mailbox_isr_f	246
8.7.3	ehsm_port_timer_t	246
8.7.4	bool_t	246
8.7.5	raddr_t	246
8.7.6	ehsm_rsp_cb_func_t	246
8.7.7	ehsm_key_format_des_plain_st	246
8.7.8	ehsm_key_format_des_cipher_st	246
8.7.9	ehsm_key_format_aes128_plain_st	246
8.7.10	ehsm_key_format_aes128_cipher_st	246
8.7.11	ehsm_key_format_tdes128_plain_st	247
8.7.12	ehsm_key_format_tdes128_cipher_st	247
8.7.13	ehsm_key_format_sm4_plain_st	247
8.7.14	ehsm_key_format_sm4_cipher_st	247
8.7.15	ehsm_key_format_tdes192_plain_st	247
8.7.16	ehsm_key_format_tdes192_cipher_st	247
8.7.17	ehsm_key_format_aes192_plain_st	247
8.7.18	ehsm_key_format_aes192_cipher_st	247
8.7.19	ehsm_key_format_aes256_plain_st	247
8.7.20	ehsm_key_format_aes256_cipher_st	247
8.7.21	ehsm_key_format_aes128_xts_plain_st	248
8.7.22	ehsm_key_format_aes128_xts_cipher_st	248
8.7.23	ehsm_key_format_sm4_xts_plain_st	248
8.7.24	ehsm_key_format_sm4_xts_cipher_st	248
8.7.25	ehsm_key_format_aes192_xts_plain_st	248
8.7.26	ehsm_key_format_aes192_xts_cipher_st	248
8.7.27	ehsm_key_format_aes256_xts_plain_st	248
8.7.28	ehsm_key_format_aes256_xts_cipher_st	248

8.7.29	ehsm_key_format_hmac64_plain_st	248
8.7.30	ehsm_key_format_hmac64_cipher_st	249
8.7.31	ehsm_key_format_hmac128_plain_st	249
8.7.32	ehsm_key_format_hmac128_cipher_st	249
8.7.33	ehsm_key_format_hmac192_plain_st	249
8.7.34	ehsm_key_format_hmac192_cipher_st	249
8.7.35	ehsm_key_format_hmac256_plain_st	249
8.7.36	ehsm_key_format_hmac256_cipher_st	249
8.7.37	ehsm_key_format_hmac384_plain_st	249
8.7.38	ehsm_key_format_hmac384_cipher_st	249
8.7.39	ehsm_key_format_hmac512_plain_st	249
8.7.40	ehsm_key_format_hmac512_cipher_st	250
8.7.41	ehsm_key_format_ecc_bp_p160r1_plain_pubkey_st	250
8.7.42	ehsm_key_format_ecc_bp_p160r1_plain_privkey_st	250
8.7.43	ehsm_key_format_ecc_bp_p160r1_cipher_privkey_st	250
8.7.44	ehsm_key_format_ecc_bp_p160r1_plain_keypair_st	250
8.7.45	ehsm_key_format_ecc_bp_p160r1_cipher_keypair_st	250
8.7.46	ehsm_key_format_ecc_bp_p192r1_plain_pubkey_st	250
8.7.47	ehsm_key_format_ecc_bp_p192r1_plain_privkey_st	250
8.7.48	ehsm_key_format_ecc_bp_p192r1_cipher_privkey_st	250
8.7.49	ehsm_key_format_ecc_bp_p192r1_plain_keypair_st	250
8.7.50	ehsm_key_format_ecc_bp_p192r1_cipher_keypair_st	251
8.7.51	ehsm_key_format_ecc_bp_p224r1_plain_pubkey_st	251
8.7.52	ehsm_key_format_ecc_bp_p224r1_plain_privkey_st	251
8.7.53	ehsm_key_format_ecc_bp_p224r1_cipher_privkey_st	251
8.7.54	ehsm_key_format_ecc_bp_p224r1_plain_keypair_st	251
8.7.55	ehsm_key_format_ecc_bp_p224r1_cipher_keypair_st	251
8.7.56	ehsm_key_format_ecc_bp_p256r1_plain_pubkey_st	251
8.7.57	ehsm_key_format_ecc_bp_p256r1_plain_privkey_st	251
8.7.58	ehsm_key_format_ecc_bp_p256r1_cipher_privkey_st	251
8.7.59	ehsm_key_format_ecc_bp_p256r1_plain_keypair_st	251
8.7.60	ehsm_key_format_ecc_bp_p256r1_cipher_keypair_st	251
8.7.61	ehsm_key_format_ecc_bp_p320r1_plain_pubkey_st	252
8.7.62	ehsm_key_format_ecc_bp_p320r1_plain_privkey_st	252
8.7.63	ehsm_key_format_ecc_bp_p320r1_cipher_privkey_st	252
8.7.64	ehsm_key_format_ecc_bp_p320r1_plain_keypair_st	252
8.7.65	ehsm_key_format_ecc_bp_p320r1_cipher_keypair_st	252
8.7.66	ehsm_key_format_ecc_bp_p384r1_plain_pubkey_st	252
8.7.67	ehsm_key_format_ecc_bp_p384r1_plain_privkey_st	252
8.7.68	ehsm_key_format_ecc_bp_p384r1_cipher_privkey_st	252
8.7.69	ehsm_key_format_ecc_bp_p384r1_plain_keypair_st	252
8.7.70	ehsm_key_format_ecc_bp_p384r1_cipher_keypair_st	252
8.7.71	ehsm_key_format_ecc_bp_p512r1_plain_pubkey_st	252
8.7.72	ehsm_key_format_ecc_bp_p512r1_plain_privkey_st	253

8.7.73	ehsm_key_format_ecc_bp_p512r1_cipher_privkey_st	253
8.7.74	ehsm_key_format_ecc_bp_p512r1_plain_keypair_st	253
8.7.75	ehsm_key_format_ecc_bp_p512r1_cipher_keypair_st	253
8.7.76	ehsm_key_format_ecc_sec_p192r1_plain_pubkey_st	253
8.7.77	ehsm_key_format_ecc_sec_p192r1_plain_privkey_st	253
8.7.78	ehsm_key_format_ecc_sec_p192r1_cipher_privkey_st	253
8.7.79	ehsm_key_format_ecc_sec_p192r1_plain_keypair_st	253
8.7.80	ehsm_key_format_ecc_sec_p192r1_cipher_keypair_st	253
8.7.81	ehsm_key_format_ecc_sec_p224r1_plain_pubkey_st	253
8.7.82	ehsm_key_format_ecc_sec_p224r1_plain_privkey_st	253
8.7.83	ehsm_key_format_ecc_sec_p224r1_cipher_privkey_st	254
8.7.84	ehsm_key_format_ecc_sec_p224r1_plain_keypair_st	254
8.7.85	ehsm_key_format_ecc_sec_p224r1_cipher_keypair_st	254
8.7.86	ehsm_key_format_ecc_sec_p256r1_plain_pubkey_st	254
8.7.87	ehsm_key_format_ecc_sec_p256r1_plain_privkey_st	254
8.7.88	ehsm_key_format_ecc_sec_p256r1_cipher_privkey_st	254
8.7.89	ehsm_key_format_ecc_sec_p256r1_plain_keypair_st	254
8.7.90	ehsm_key_format_ecc_sec_p256r1_cipher_keypair_st	254
8.7.91	ehsm_key_format_ecc_sec_p384r1_plain_pubkey_st	254
8.7.92	ehsm_key_format_ecc_sec_p384r1_plain_privkey_st	254
8.7.93	ehsm_key_format_ecc_sec_p384r1_cipher_privkey_st	254
8.7.94	ehsm_key_format_ecc_sec_p384r1_plain_keypair_st	255
8.7.95	ehsm_key_format_ecc_sec_p384r1_cipher_keypair_st	255
8.7.96	ehsm_key_format_ecc_sec_p521r1_plain_pubkey_st	255
8.7.97	ehsm_key_format_ecc_sec_p521r1_plain_privkey_st	255
8.7.98	ehsm_key_format_ecc_sec_p521r1_cipher_privkey_st	255
8.7.99	ehsm_key_format_ecc_sec_p521r1_plain_keypair_st	255
8.7.100	ehsm_key_format_ecc_sec_p521r1_cipher_keypair_st	255
8.7.101	ehsm_key_format_ecc_sec_p160k1_plain_pubkey_st	255
8.7.102	ehsm_key_format_ecc_sec_p160k1_plain_privkey_st	255
8.7.103	ehsm_key_format_ecc_sec_p160k1_cipher_privkey_st	255
8.7.104	ehsm_key_format_ecc_sec_p160k1_plain_keypair_st	255
8.7.105	ehsm_key_format_ecc_sec_p160k1_cipher_keypair_st	256
8.7.106	ehsm_key_format_ecc_sec_p192k1_plain_pubkey_st	256
8.7.107	ehsm_key_format_ecc_sec_p192k1_plain_privkey_st	256
8.7.108	ehsm_key_format_ecc_sec_p192k1_cipher_privkey_st	256
8.7.109	ehsm_key_format_ecc_sec_p192k1_plain_keypair_st	256
8.7.110	ehsm_key_format_ecc_sec_p192k1_cipher_keypair_st	256
8.7.111	ehsm_key_format_ecc_sec_p224k1_plain_pubkey_st	256
8.7.112	ehsm_key_format_ecc_sec_p224k1_plain_privkey_st	256
8.7.113	ehsm_key_format_ecc_sec_p224k1_cipher_privkey_st	256
8.7.114	ehsm_key_format_ecc_sec_p224k1_plain_keypair_st	256
8.7.115	ehsm_key_format_ecc_sec_p224k1_cipher_keypair_st	256
8.7.116	ehsm_key_format_ecc_sec_p256k1_plain_pubkey_st	257

8.7.117 ehsm_key_format_ecc_sec_p256k1_plain_privkey_st	257
8.7.118 ehsm_key_format_ecc_sec_p256k1_cipher_privkey_st	257
8.7.119 ehsm_key_format_ecc_sec_p256k1_plain_keypair_st	257
8.7.120 ehsm_key_format_ecc_sec_p256k1_cipher_keypair_st	257

1 eHSM Host API

eHSM Host API 库，提供在 Host 端操作 eHSM 的相关 API 接口。

1.1 基础库

基础库的头文件位于 inc/ehsmdrv/basic 目录下，请阅读以下章节内容以了解基础库的使用方式：

- [基础驱动库 API](#)
- [密钥管理方案](#)
- [共享内存机制](#)
- [驱动移植指南](#)
- [Host demo 指南](#)

2 基础驱动库 API

2.1 基本接口

2.1.1 驱动初始化与上下文管理

- [ehsm_driver_init_library\(\)](#)- 初始化驱动库
- [ehsm_driver_get_version\(\)](#)- 获取驱动库版本
- [ehsm_ctx_init\(\)](#)- 初始化操作上下文
- [ehsm_ctx_poll\(\)](#)- 轮询异步操作完成状态

所有对 eHSM 下位机的功能调用，都需要通过一个 `ctx` 进行，因此，需要从[共享内存](#)中分配一个 `ctx` 空间，然后调用 [ehsm_ctx_init\(\)](#) 初始化这个 `ctx`。

初始化时可指定此次调用是同步模式还是异步模式，其特性如下：

- 如果指定了同步模式，函数会在发送了命令数据后等待 eHSM 发送响应值，然后返回
- 如果指定了异步模式，函数会在发送了命令数据后立刻返回 [EHSM_ERR_NEED_POLL](#)，然后
 - 如果提供了回调函数，当收到响应时，`ctx` 对应的回调函数会被调用（回调函数是在中断函数中被调用的，请只处理简单工作，以免阻塞其它任务）
 - 如果未提供回调函数，可以反复调用 [ehsm_ctx_poll\(\)](#) 来判断任务是否完成

在使用异步模式时，传入 API 函数的指针所指向的内存空间，应当保证其在操作完成前都是有效的，不要使用指向局部变量的空间。

可参考 demo 代码。

- FW 完整 API 见 [FW \(Firmware\) 相关的 API 接口](#)。
- BL 完整 API 见 [BL \(Bootloader\) 相关的 API 接口](#)。

2.2 API 功能分类概览

2.2.1 随机数生成

- [ehsm_gen_random\(\)](#)- 生成随机数

2.2.2 HASH/摘要算法

2.2.2.1 HASH

三段式接口：

- [ehsm_hash_init\(\)](#)- 初始化 HASH 计算
- [ehsm_hash_update\(\)](#)- 更新 HASH 数据
- [ehsm_hash_finish\(\)](#)- 完成 HASH 计算

一次性接口:

- [ehsm_hash_onepass\(\)](#)- 一次性 HASH 计算

2.2.2.2 HMAC

三段式接口:

- [ehsm_hmac_init\(\)](#)- 初始化 HMAC 计算
- [ehsm_hmac_update\(\)](#)- 更新 HMAC 数据
- [ehsm_hmac_finish_gen\(\)](#)- 完成 HMAC 生成
- [ehsm_hmac_finish_verify\(\)](#)- 完成 HMAC 验证

一次性接口:

- [ehsm_hmac_onepass_gen\(\)](#)- 一次性生成 HMAC
- [ehsm_hmac_onepass_verify\(\)](#)- 一次性验证 HMAC

明文密钥接口:

- [ehsm_hmac_init_with_plain_key\(\)](#)- 使用明文密钥初始化
- [ehsm_hmac_onepass_gen_with_plain_key\(\)](#)- 使用明文密钥生成
- [ehsm_hmac_onepass_verify_with_plain_key\(\)](#)- 使用明文密钥验证

2.2.3 对称加密算法

2.2.3.1 对称加密 (CBC/ECB/CFB/OFB/CTR 等)

三段式接口:

- [ehsm_symm_cipher_init\(\)](#)- 初始化对称加解密
- [ehsm_symm_cipher_update\(\)](#)- 更新加解密数据
- [ehsm_symm_cipher_finish\(\)](#)- 完成加解密

一次性接口:

- [ehsm_symm_cipher_onepass\(\)](#)- 一次性对称加解密

明文密钥接口:

- [ehsm_symm_cipher_init_with_plain_key\(\)](#)- 使用明文密钥初始化
- [ehsm_symm_cipher_onepass_with_plain_key\(\)](#)- 使用明文密钥一次性加解密

2.2.3.2 MAC

三段式接口:

- [ehsm_mac_init\(\)](#)- 初始化 MAC 计算
- [ehsm_mac_update\(\)](#)- 更新 MAC 数据
- [ehsm_mac_finish_gen\(\)](#)- 完成 MAC 生成

- [ehsm_mac_finish_verify\(\)](#)- 完成 MAC 验证

一次性接口:

- [ehsm_mac_onepass_gen\(\)](#)- 一次性生成 MAC
- [ehsm_mac_onepass_verify\(\)](#)- 一次性验证 MAC

明文密钥接口:

- [ehsm_mac_init_with_plain_key\(\)](#)- 使用明文密钥初始化
- [ehsm_mac_onepass_gen_with_plain_key\(\)](#)- 使用明文密钥生成
- [ehsm_mac_onepass_verify_with_plain_key\(\)](#)- 使用明文密钥验证

2.2.3.3 AEAD

三段式接口:

- [ehsm_aead_init\(\)](#)- 初始化 AEAD 加解密
- [ehsm_aead_update\(\)](#)- 更新 AEAD 数据
- [ehsm_aead_finish_enc\(\)](#)- 完成 AEAD 加密
- [ehsm_aead_finish_dec\(\)](#)- 完成 AEAD 解密

一次性接口:

- [ehsm_aead_onepass_enc\(\)](#)- 一次性 AEAD 加密
- [ehsm_aead_onepass_dec\(\)](#)- 一次性 AEAD 解密

明文密钥接口:

- [ehsm_aead_init_with_plain_key\(\)](#)- 使用明文密钥初始化
- [ehsm_aead_onepass_enc_with_plain_key\(\)](#)- 使用明文密钥加密
- [ehsm_aead_onepass_dec_with_plain_key\(\)](#)- 使用明文密钥解密

2.2.3.4 ChaCha

三段式接口:

- [ehsm_chacha_init\(\)](#)- 初始化 ChaCha 加解密
- [ehsm_chacha_update\(\)](#)- 更新 ChaCha 数据
- [ehsm_chacha_finish_enc\(\)](#)- 完成 ChaCha 加密
- [ehsm_chacha_finish_dec\(\)](#)- 完成 ChaCha 解密

一次性接口:

- [ehsm_chacha_onepass_enc\(\)](#)- 一次性 ChaCha 加密
- [ehsm_chacha_onepass_dec\(\)](#)- 一次性 ChaCha 解密

明文密钥接口:

- [ehsm_chacha_init_with_plain_key\(\)](#)- 使用明文密钥初始化

- [ehsm_chacha_onepass_enc_with_plain_key\(\)](#)- 使用明文密钥加密
- [ehsm_chacha_onepass_dec_with_plain_key\(\)](#)- 使用明文密钥解密

2.2.4 非对称加密

2.2.4.1 SM2 加解密

- [ehsm_sm2_cipher\(\)](#)- SM2 加解密
- [ehsm_sm2_cipher_with_plain_key\(\)](#)- 使用明文密钥的 SM2 加解密

2.2.4.2 RSA 加解密

- [ehsm_rsa_cipher\(\)](#)- RSA 加解密
- [ehsm_rsa_cipher_with_plain_key\(\)](#)- 使用明文密钥的 RSA 加解密

2.2.4.3 SM9 加解密

- [ehsm_sm9_cipher\(\)](#)- SM9 加解密
- [ehsm_sm9_cipher_with_plain_key\(\)](#)- 使用明文密钥的 SM9 加解密

2.2.5 数字签名

2.2.5.1 SM2 签名

三段式接口:

- [ehsm_sm2_sign_init\(\)](#)- 初始化 SM2 签名
- [ehsm_sm2_sign_update\(\)](#)- 更新签名数据
- [ehsm_sm2_sign_finish_gen\(\)](#)- 完成签名生成
- [ehsm_sm2_sign_finish_verify\(\)](#)- 完成签名验证

一次性接口:

- [ehsm_sm2_sign_onepass_gen\(\)](#)- 一次性生成签名
- [ehsm_sm2_sign_onepass_verify\(\)](#)- 一次性验证签名
- [ehsm_sm2_sign_onepass_gen_with_digest\(\)](#)- 使用摘要生成签名
- [ehsm_sm2_sign_onepass_verify_with_digest\(\)](#)- 使用摘要验证签名

明文密钥接口:

- [ehsm_sm2_sign_init_with_plain_key\(\)](#)- 使用明文密钥初始化
- [ehsm_sm2_sign_onepass_gen_with_plain_key\(\)](#)- 使用明文密钥生成签名
- [ehsm_sm2_sign_onepass_verify_with_plain_key\(\)](#)- 使用明文密钥验证签名
- [ehsm_sm2_sign_onepass_gen_with_plain_key_and_digest\(\)](#)- 使用明文密钥和摘要生成
- [ehsm_sm2_sign_onepass_verify_with_plain_key_and_digest\(\)](#)- 使用明文密钥和摘要验证

扩展接口:

- [ehsm_sm2_sign_onepass_ex\(\)](#)- 扩展的一次性签名接口

2.2.5.2 RSA 签名

三段式接口:

- [ehsm_rsa_sign_init\(\)](#)- 初始化 RSA 签名
- [ehsm_rsa_sign_update\(\)](#)- 更新签名数据
- [ehsm_rsa_sign_finish_gen\(\)](#)- 完成签名生成
- [ehsm_rsa_sign_finish_verify\(\)](#)- 完成签名验证

一次性接口:

- [ehsm_rsa_sign_onepass_gen\(\)](#)- 一次性生成签名
- [ehsm_rsa_sign_onepass_verify\(\)](#)- 一次性验证签名
- [ehsm_rsa_sign_onepass_gen_with_digest\(\)](#)- 使用摘要生成签名
- [ehsm_rsa_sign_onepass_verify_with_digest\(\)](#)- 使用摘要验证签名

明文密钥接口:

- [ehsm_rsa_sign_init_with_plain_key\(\)](#)- 使用明文密钥初始化
- [ehsm_rsa_sign_onepass_gen_with_plain_key\(\)](#)- 使用明文密钥生成签名
- [ehsm_rsa_sign_onepass_verify_with_plain_key\(\)](#)- 使用明文密钥验证签名
- [ehsm_rsa_sign_onepass_gen_with_plain_key_and_digest\(\)](#)- 使用明文密钥和摘要生成
- [ehsm_rsa_sign_onepass_verify_with_plain_key_and_digest\(\)](#)- 使用明文密钥和摘要验证

扩展接口:

- [ehsm_rsa_sign_onepass_ex\(\)](#)- 扩展的一次性签名接口

2.2.5.3 ECDSA 签名

三段式接口:

- [ehsm_ecdsa_init\(\)](#)- 初始化 ECDSA 签名
- [ehsm_ecdsa_update\(\)](#)- 更新签名数据
- [ehsm_ecdsa_finish_gen\(\)](#)- 完成签名生成
- [ehsm_ecdsa_finish_verify\(\)](#)- 完成签名验证

一次性接口:

- [ehsm_ecdsa_onepass_gen\(\)](#)- 一次性生成签名
- [ehsm_ecdsa_onepass_verify\(\)](#)- 一次性验证签名
- [ehsm_ecdsa_onepass_gen_with_digest\(\)](#)- 使用摘要生成签名
- [ehsm_ecdsa_onepass_verify_with_digest\(\)](#)- 使用摘要验证签名

明文密钥接口:

- [ehsm_ecdsa_init_with_plain_key\(\)](#)- 使用明文密钥初始化
- [ehsm_ecdsa_onepass_gen_with_plain_key\(\)](#)- 使用明文密钥生成签名
- [ehsm_ecdsa_onepass_verify_with_plain_key\(\)](#)- 使用明文密钥验证签名
- [ehsm_ecdsa_onepass_gen_with_plain_key_and_digest\(\)](#)- 使用明文密钥和摘要生成
- [ehsm_ecdsa_onepass_verify_with_plain_key_and_digest\(\)](#)- 使用明文密钥和摘要验证

扩展接口:

- [ehsm_ecdsa_onepass_ex\(\)](#)- 扩展的一次性签名接口

2.2.5.4 SM9 签名

一次性接口:

- [ehsm_sm9_sign_onepass_gen\(\)](#)- 一次性生成 SM9 签名
- [ehsm_sm9_sign_onepass_verify\(\)](#)- 一次性验证 SM9 签名

明文密钥接口:

- [ehsm_sm9_sign_onepass_gen_with_plain_key\(\)](#)- 使用明文密钥生成签名

2.2.6 密钥管理

- [ehsm_km_gen_key\(\)](#)- 生成密钥
- [ehsm_km_import_key\(\)](#)- 导入密钥
- [ehsm_km_export_key\(\)](#)- 导出密钥
- [ehsm_km_remove_key\(\)](#)- 删除密钥
- [ehsm_km_derive_key\(\)](#)- 派生密钥
- [ehsm_km_exchange_key\(\)](#)- 密钥交换
- [ehsm_km_sm9_exchange_key\(\)](#)- SM9 密钥交换
- [ehsm_km_get_pub_from_priv\(\)](#)- 从私钥获取公钥

2.2.7 系统管理

2.2.7.1 版本与信息

- [ehsm_get_version\(\)](#)- 获取 eHSM 固件版本

2.2.7.2 镜像管理

- [ehsm_verify_image\(\)](#)- 验证镜像
- [ehsm_upgrade_fw_image\(\)](#)- 升级固件镜像一次性接口
- [ehsm_upgrade_fw_image_init\(\)](#)- 升级固件镜像三段式接口
- [ehsm_upgrade_fw_image_update\(\)](#)- 升级固件镜像三段式接口
- [ehsm_upgrade_fw_image_finish\(\)](#)- 升级固件镜像三段式接口

2.2.7.3 时间管理

- [ehsm_set_utc_time\(\)](#)- 设置 UTC 时间
- [ehsm_get_utc_time\(\)](#)- 获取 UTC 时间

2.2.7.4 计数器管理

- [ehsm_create_counter\(\)](#)- 创建计数器
- [ehsm_read_counter\(\)](#)- 读取计数器
- [ehsm_increase_counter\(\)](#)- 增加计数器
- [ehsm_delete_counter\(\)](#)- 删除计数器

2.2.7.5 调试与鉴权

- [ehsm_get_challenge\(\)](#)- 获取挑战值
- [ehsm_debug_auth\(\)](#)- 调试鉴权
- [ehsm_close_debug\(\)](#)- 关闭调试

2.2.7.6 生命周期与配置

- [ehsm_change_lifecycle\(\)](#)- 改变生命周期状态
- [ehsm_change_control_field\(\)](#)- 修改控制字段

2.2.7.7 OTP 与寄存器操作

- [ehsm_read_otp\(\)](#)- 读取 OTP
- [ehsm_write_otp\(\)](#)- 写入 OTP
- [ehsm_read_reg\(\)](#)- 读取寄存器
- [ehsm_write_reg\(\)](#)- 写入寄存器

2.2.7.8 其他

- [ehsm_set_uart_baudrate\(\)](#)- 设置串口波特率
- [ehsm_enter_wfi\(\)](#)- 进入 WFI 模式
- [ehsm_install_random_key\(\)](#)- 安装随机密钥
- [ehsm_install_encrypted_key\(\)](#)- 安装加密密钥

2.3 BL 专有接口

BL (Bootloader) 模式下的专有接口，大部分是对 FW API 的宏封装：

2.3.1 自检功能

- [ehsm_bl_self_test\(\)](#)- 启动算法自检
- [ehsm_bl_get_self_test_result\(\)](#)- 获取自检结果

2.3.2 镜像验证与固件管理

- [ehsm_bl_verify_image\(\)](#)- 验证镜像
- [ehsm_bl_upgrade_fw_image\(\)](#)- 升级固件镜像一次性接口
- [ehsm_bl_upgrade_fw_image_init\(\)](#)- 升级固件镜像三段式接口
- [ehsm_bl_upgrade_fw_image_update\(\)](#)- 升级固件镜像三段式接口
- [ehsm_bl_upgrade_fw_image_finish\(\)](#)- 升级固件镜像三段式接口

2.3.3 固件认证

- [ehsm_bl_get_challenge\(\)](#)- 获取挑战值
- [ehsm_bl_fw_auth\(\)](#)- 国密固件认证
- [ehsm_bl_debug_auth\(\)](#)- 调试鉴权
- [ehsm_bl_close_debug\(\)](#)- 关闭调试

2.3.4 密钥生成与加密

- [ehsm_bl_get_random_key\(\)](#)- 生成随机密钥（用于 OTP）
- [ehsm_bl_encrypt_key\(\)](#)- 加密外部密钥

2.3.5 系统信息与配置

- [ehsm_bl_get_version\(\)](#)- 获取版本
- [ehsm_bl_set_uart_baudrate\(\)](#)- 设置串口波特率

2.3.6 OTP 与寄存器操作

- [ehsm_bl_read_otp\(\)](#)- 读取 OTP
- [ehsm_bl_write_otp\(\)](#)- 写入 OTP
- [ehsm_bl_read_reg\(\)](#)- 读取寄存器
- [ehsm_bl_write_reg\(\)](#)- 写入寄存器

3 驱动移植指南

3.1 概述

本章内容旨在指导用户如何将 eHSM Host 驱动库移植到新的硬件平台。移植过程主要涉及以下三个方面：

1. **宏定义**：根据目标平台的硬件特性，提供一组特定的宏定义。
2. **接口实现**：实现一组平台相关的硬件操作接口。
3. **构建脚本适配**：修改构建脚本以包含新平台的源文件和编译选项。

3.2 文件结构

建议在 `port` 目录下为新的目标平台创建一个独立的子目录，例如 `port/your_platform_name`。然后，可以参考 `port/osr/m130` 的实现，在新目录中创建以下文件：

- `ehsm_host_custom.h`：用于存放平台相关的宏定义。
- `ehsm_host_port.c`：用于存放平台移植接口的具体实现。
- `port.cmake`：用于存放平台相关的构建脚本配置。

3.3 宏定义

所有平台相关的宏定义都应在 `ehsm_host_custom.h` 文件中完成。该文件会被主移植头文件 `None` 自动包含。

以下是需要定义的宏列表及其说明：

宏	说明
<code>EHSM_PORT_MAILBOX_CHANNEL_COUNT</code>	Mailbox 的硬件通道数量。
<code>EHSM_PORT_MAILBOX_REG_BASE_ADDR</code>	Mailbox 寄存器的基地址。此地址空间应为 non-cacheable。
<code>EHSM_PORT_MAILBOX_NOTE_WRITE_1_CLEAR</code>	Mailbox 的 note 寄存器清零方式。 <code>1</code> 表示写 1 清零， <code>0</code> 表示写 0 清零。目前 eHSM 硬件为写 1 清零。
<code>EHSM_PORT_SHARE_MEM_BASE_ADDR</code>	共享内存的基地址。
<code>EHSM_PORT_SHARE_MEM_SIZE</code>	共享内存的总大小。
<code>EHSM_PORT_OTP_SIZE</code>	eHSM OTP 空间的大小（字节）。
<code>ehsm_port_printf</code>	(可选) 定义一个 <code>printf</code> 宏，用于在驱动库中输出调试信息。

3.3.1 示例

以下是 `port/osr/m130/port/ehsm_host_custom.h` 中的宏定义示例：

```
1| #ifndef EHSM_HOST_CUSTOM_H
2| #define EHSM_HOST_CUSTOM_H
3|
```

```
4| #include <stdio.h>
5|
6| #define EHSM_PORT_MAILBOX_CHANNEL_COUNT    16u
7| #define EHSM_PORT_MAILBOX_REG_BASE_ADDR    0x80000000u
8| #define EHSM_PORT_SHARE_MEM_BASE_ADDR     0x60000000u
9| #define EHSM_PORT_SHARE_MEM_SIZE          (256u * 1024u)
10| #define EHSM_PORT_OTP_SIZE                 1024
11| #define EHSM_PORT_MAILBOX_NOTE_WRITE_1_CLEAR 1
12|
13| #define ehsm_port_printf(...) printf(__VA_ARGS__)
14|
15| #endif // EHSM_HOST_CUSTOM_H
16|
```

3.4 接口实现

移植工作的核心是实现定义在 `None` 中的所有接口函数。这些函数的实现应放在 `ehsm_host_port.c` 文件中。

3.4.1 ehsm_port_init

- **功能：**初始化移植平台所需的外设，如串口（用于打印）、中断控制器等。
- **示例** (`port/osr/m130/port/ehsm_host_port.c`):

```
1| uint32_t ehsm_port_init(ehsm_drv_mode_e mode)
2| {
3|     (void)mode;
4|     UartStdOutInit(115200);
5|     cpu_platform_init();
6|     return EHSM_OK;
7| }
8|
```

3.4.2 ehsm_port_addr_to_raddr

- **功能：**将当前 CPU 的逻辑地址转换为 eHSM 可以访问的物理地址。如果 CPU 和 eHSM 使用统一的地址空间，则直接返回原地址即可。
- **示例：**

```
1| raddr_t ehsm_port_addr_to_raddr(const void *addr)
2| {
3|     return (raddr_t)(uintptr_t)addr;
4| }
5|
```

3.4.3 ehsm_port_raddr_to_addr

- **功能：**将 eHSM 使用的物理地址转换为当前 CPU 的逻辑地址。如果使用统一地址空间，则直接返回原地址。
- **示例：**

```
1| void *ehsm_port_raddr_to_addr(raddr_t raddr)
2| {
3|     return (void *)(uintptr_t)raddr;
4| }
5|
```

3.4.4 ehsm_port_flush_and_invalidate_cache

- **功能：**将 CPU cache 中的数据刷入 RAM 并使 cache 无效。如果用于 eHSM 通信的共享内存被配置为 non-cacheable，则此函数可以为空。否则应当使用 CPU CACHE 相关的接口来实现此功能。
- **示例：**

```
1| void ehsm_port_flush_and_invalidate_cache(void)
2| {
3| }
4|
```

3.4.5 ehsm_port_enable_mailbox_int

- **功能：**使能指定 mailbox channel 的中断，并注册中断回调函数。当中断触发时，必须调用 `int_func` 并传入对应的 `channel` ID。
- **示例：**

```
1| // 声明一个函数指针数组用于存储不同 channel 的回调函数
2| static ehsm_mailbox_isr_f g_mb_isrs[EHSM_PORT_MAILBOX_CHANNEL_COUNT] = { NULL };
3|
4| // Mailbox 中断处理函数
5| ATTR_INTERRUPT void MAILBOX_0_Handler(void)
6| {
7|     if (g_mb_isrs[0]) {
8|         g_mb_isrs[0](0);
9|     }
10| }
11|
12| uint32_t ehsm_port_enable_mailbox_int(uint32_t channel, ehsm_mailbox_isr_f int_func)
13| {
14|     if (channel >= 2 || int_func == NULL) { // 示例平台只用到了两个 channel
15|         return EHSM_ERR_PARAM_ERROR;
16|     }
17| }
```

```

16|     }
17|     g_mb_isrs[channel] = int_func;
18|     switch (channel) {
19|     case 0:
20|         (void)cpu_register_int(SOC_MAILBOX0_IRQn, INT_LEVEL_TRIGGER,
(cpu_int_handler)MAILBOX_0_Handler);
21|         cpu_enable_int(SOC_MAILBOX0_IRQn);
22|         break;
23|         // ... 其它 channel
24|     }
25|     return EHSM_OK;
26| }
27|

```

3.4.6 ehsm_port_reset_ehsm

- 功能：通过操作复位引脚或系统控制寄存器来硬件复位 eHSM。
- 示例：

```

1| void ehsm_port_reset_ehsm(void)
2| {
3|     *((volatile uint32_t *) (SYS_REG_BASE_ADDR + 0x58)) = 0x0fa66e27u; //
OSR_FPGA_MAGIC
4|     *((volatile uint32_t *) (SYS_REG_BASE_ADDR + 0x08)) = 0xffffffffu; // reset
5|     *((volatile uint32_t *) (SYS_REG_BASE_ADDR + 0x08)) = 0xffffffffu; // release
6| }
7|

```

3.4.7 ehsm_port_read_reg

- 功能：根据寄存器编号 `ehsm_reg_e` 读取 eHSM 的状态寄存器。
- 示例：

```

1| uint32_t ehsm_port_read_reg(ehsm_reg_e reg)
2| {
3|     volatile uint32_t *reg_addr = get_reg_addr(reg); // get_reg_addr 是一个辅助函数
4|     if (reg_addr != NULL) {
5|         return *reg_addr;
6|     } else {
7|         return 0;
8|     }
9| }
10|

```


3.4.8 ehsm_port_write_reg

- 功能：根据寄存器编号 `ehsm_reg_e` 写入 eHSM 的控制寄存器。
- 示例：

```
1| void ehsm_port_write_reg(ehsm_reg_e reg, uint32_t val)
2| {
3|     volatile uint32_t *reg_addr = get_reg_addr(reg);
4|     if (reg_addr != NULL) {
5|         *reg_addr = val;
6|     }
7| }
8|
```

3.4.9 ehsm_port_read_otp

- 功能：从 eHSM 的 OTP 空间读取数据。
- 示例：

```
1| uint32_t ehsm_port_read_otp(uint8_t *buf, uint32_t offset, uint32_t len)
2| {
3|     if (offset + len > EHSM_PORT_OTP_SIZE) {
4|         return EHSM_ERR_PARAM_ERROR;
5|     }
6|     memcpy(buf, (void *) (OTP_BASE_ADDR + offset), len);
7|     return EHSM_OK;
8| }
9|
```

3.4.10 ehsm_port_write_otp

- 功能：向 eHSM 的 OTP 空间写入数据。
- 示例：

```
1| uint32_t ehsm_port_write_otp(const uint8_t *otp_data, uint32_t offset, uint32_t len)
2| {
3|     if (offset + len > EHSM_PORT_OTP_SIZE) {
4|         return EHSM_ERR_PARAM_ERROR;
5|     }
6|     memcpy((void *) (OTP_BASE_ADDR + offset), otp_data, len);
7|     return EHSM_OK;
8| }
9|
```

3.4.11 ehsm_port_create_timer 和 ehsm_port_is_timeout

- **功能**：实现一个超时计时器。`ehsm_port_create_timer`用于记录当前时间点，`ehsm_port_is_timeout`用于判断从记录的时间点开始是否已经超时。超时阈值由具体实现决定。
- **示例** (简单实现，永不超时):

```

1| ehsm_port_timer_t ehsm_port_create_timer(void)
2| {
3|     return 0;
4| }
5|
6| bool_t ehsm_port_is_timeout(ehsm_port_timer_t timer)
7| {
8|     (void)timer;
9|     return false;
10| }
11|

```

3.5 构建脚本适配

完成代码实现后，需要提供一个 `port.cmake` 文件来将平台代码集成到项目的 CMake 构建系统中。

主 `CMakeLists.txt` 文件会通过 `include(${CMAKE_CURRENT_SOURCE_DIR}/port/${PORT}/port.cmake)` 来包含此文件，其中 `${PORT}` 变量指向您平台的目录名。

`port.cmake` 文件需要设置以下几个变量：

- `EHSM_PORT_SRC_FILES`：包含平台相关的源文件列表（.c, .S 文件）。
- `EHSM_PORT_INC_DIRS`：包含平台相关的头文件目录。
- `EHSM_RETARGET_SRC_FILE`：包含平台重定向的 C 库函数源文件，例如 `_sbrk`, `_write`
- `EHSM_PORT_LINK_FLAGS`：(可选) 包含平台相关的链接器选项，例如指定链接脚本（`-Tlink.ld`）。
- (可选) 如果需要特定的交叉编译工具链，可以在此文件中引入。

以下是 `port/osr/m130/port.cmake` 的示例：

```

1| # 引入交叉编译工具链配置文件
2| include(${CMAKE_CURRENT_LIST_DIR}/rv32-wing-gcc.cmake)
3|
4| # 配置重定向的 C 库函数文件，主要是实现 UART 对 _write 的重写，实现串口打印，对 _sbrk
   的重写实现堆内存分配，以及对 C 库函数的重写，防止编译告警
5| set(EHSM_RETARGET_SRC_FILE
6|     ${CMAKE_CURRENT_LIST_DIR}/port/sys_call.c
7|     CACHE STRING "Set retarget functions for libc"
8| )
9|
10| # 设置平台相关的源文件

```

```
11| set(EHSM_PORT_SRC_FILES
12|     ${CMAKE_CURRENT_LIST_DIR}/driver/src/clic.c
13|     ${CMAKE_CURRENT_LIST_DIR}/port/port_m130.c
14|     ${CMAKE_CURRENT_LIST_DIR}/driver/src/uart_stdout.c
15|     ${CMAKE_CURRENT_LIST_DIR}/port/ehsm_host_port.c
16|     ${EHSM_RETARGET_SRC_FILE}
17|     ${CMAKE_CURRENT_LIST_DIR}/wmsis/core/src/crt0.S
18| )
19|
20| # 设置平台相关的头文件目录
21| set(EHSM_PORT_INC_DIRS
22|     ${CMAKE_CURRENT_LIST_DIR}/driver/inc
23|     ${CMAKE_CURRENT_LIST_DIR}/wmsis/core/inc ${CMAKE_CURRENT_LIST_DIR}/port
24|     ${CMAKE_CURRENT_LIST_DIR}/inc
25| )
26|
27| # 设置链接选项
28| set(EHSM_PORT_LINK_FLAGS "-T${CMAKE_CURRENT_LIST_DIR}/link.ld")
29|
```

完成以上步骤后，在构建时通过指定 `PORT` 变量即可编译适用于新平台的驱动库。可以选择是否编译 Host demo，通过 `cmake` 命令指定 `BUILD_HOST_DEMO` 变量指定，具体请参考 [Host demo 指南](#) 文档。例如：

```
1| # 编译 Host demo
2| cmake -S . -B build -G "your_cmake_generator" -DPORT=your_platform_name -
DBUILD_HOST_DEMO=ON
3|
4| cmake --build build
5|
6| # 不编译 Host demo
7| cmake -S . -B build -G "your_cmake_generator" -DPORT=your_platform_name -
DBUILD_HOST_DEMO=OFF
8|
9| cmake --build build
10|
```

4 Host demo 指南

4.1 概述

Host demo 旨在为客户提供 eHSM Host 功能及 APIs 使用的示例，包含 Bootloader demo 和 Firmware demo。

4.2 Demo 示例功能

4.2.1 Bootloader demo

- 主要包含以下功能的示例代码：
 - 鉴权
 - 升级启动校验
 - OTP 读写、密钥获取
 - 设置 UART 频率分频值
 - REG 读写
 - 算法自检
- Bootloader demo 的入口函数位于 demo/bl_demo/bl_demo.c 的 `void ehsm_bl_demo_entry(void);`。该函数中每个被调用的 `ehsm_bl_demo_xxxx_entry()` 代表一个功能示例，可以根据实际需求开启或关闭（注释调代码表示关闭演示）。例如

```
1| void ehsm_bl_demo_entry(void)
2| {
3|     ... // 省略代码
4|
5|     // 开启 OTP 密钥生成演示
6|     ehsm_bl_demo_gen_otp_key_entry();
7|
8|     // 关闭鉴权演示
9|     // ehsm_bl_demo_debug_auth_entry();
10|
11|     ... // 省略代码
12| }
13|
```

4.2.2 Firmware demo

- 主要包含以下功能的示例代码：
 - 对称/非对称加解密
 - 签名验签
 - HASH/HMAC
 - MAC

- 随机数生成
 - 密钥管理
 - OTP 读写、密钥获取
 - 升级启动校验
 - UTC TIMER
 - 单调计数器
 - 设置 UART 频率分频值
 - REG 读写
- Bootloader demo 的入口函数位于 demo/fw_demo/fw_demo.c 的 `void ehsm_fw_demo_entry(void);`。该函数中每个被调用的 `ehsm_fw_demo_xxxx_entry()` 代表一个功能示例,, 可以根据实际需求开启或关闭 (注释调代码表示关闭演示)。例如

```
1| void ehsm_fw_demo_entry(void)
2| {
3|     ... // 省略代码
4|
5|     // 开启 HASH 算法演示
6|     ehsm_demo_hash_entry();
7|
8|     // 关闭对称加解密算法演示
9|     // ehsm_demo_symm_cipher_entry();
10|
11|     ... // 省略代码
12| }
13|
```

4.2.3 编译说明

4.2.3.1 编译方法

使用 cmake 配合 CMakeLists.txt 进行编译, 若要编译 Host demo 源代码, 需要在 cmake 指令中使用指定参数 `BUILD_HOST_DEMO` 为 `ON/YES/1/OFF/NO/0` 实现。具体的编译脚本适配可以参考[驱动移植指南](#)。

在项目根目录下执行以下命令:

```
1| # 配置构建环境, 指定 PORT 和启用 BUILD_HOST_DEMO
2| cmake -B build -G "your_cmake_generator" -DPORT=your_platform_path -
  DBUILD_HOST_DEMO=ON
3|
4| # 编译代码
5| cmake --build build
6|
```

必需参数说明:

- `-DPORT=your_platform_path`: 指定目标平台路径 (必需)
- `-DBUILD_HOST_DEMO=ON/OFF`: 启用或禁用 Host demo 程序编译 (必需)

可选参数说明:

- `-DOTP_DEFAULT1=<0|1>`: 指定 OTP 默认填充值 (可选)
 - `0`: OTP 填充全 0
 - `1`: OTP 填充全 1

示例:

```
1| # 编译 Host demo, 使用默认 OTP 配置
2| cmake -B build -G "your_cmake_generator" -DPORT=your_platform_path -
DBUILD_HOST_DEMO=ON
3| cmake --build build
4|
5| # 编译 Host demo, 指定 OTP 填充全 0
6| cmake -B build -G "your_cmake_generator" -DPORT=your_platform_path -
DBUILD_HOST_DEMO=ON -DOTP_DEFAULT1=0
7| cmake --build build
8|
9| # 编译 Host demo, 指定 OTP 填充全 1
10| cmake -B build -G "your_cmake_generator" -DPORT=your_platform_path -
DBUILD_HOST_DEMO=ON -DOTP_DEFAULT1=1
11| cmake --build build
12|
13| # 不编译 Host demo 源代码
14| cmake -B build -G "your_cmake_generator" -DPORT=your_platform_path -
DBUILD_HOST_DEMO=OFF
15| cmake --build build
16|
```

4.2.3.2 编译产物

编译成功后会在 `build/` 目录中生成 `ehsm_host_demo` 可执行文件。

4.2.4 运行说明

- 编译 demo 代码, 会编译 Bootloader 和 Firmware 的 demo 所有代码, 其中 main 函数位于 `demo/test.c` 文件, 启动运行即可。
- 运行后会先获取 eHSM 固件版本, 根据固件版本中的 `type` 参数识别运行的固件是 **Bootloader** 还是 **Firmware**, 并运行对应的 demo 演示, 无需手动指定。

5 密钥管理方案

5.1 密钥权限

权限定义见[密钥权限](#)和 eHSM Firmware 技术参考手册。

5.2 密钥导入/导出数据结构说明

密钥导入/导出的数据由密钥头和密钥值两部分构成。两部分连接成完成的数据块，在调用导入导出功能时向 eHSM 提供密钥头的地址。

密钥数据如下：

密钥头	密钥值
12 字节	密钥类型决定长度

- 非对称密钥中的所有大数值，都以大端字节数组形式存储
- 导入/导出时如果提供 MAC 校验，此校验值是对整个密钥数据明文的 MAC 校验，算法为指定验证密钥的 CMAC 算法。
- 对于加密导入/导出的场景，密钥值部分会被加密（公钥部分不会加密），使用的算法为指定传输密钥的 CBC 加密模式，填充算法为 PKCS7。

总的来说，先计算明文数据的 MAC 值，再对密钥值部分进行加密。

5.2.1 密钥头

导入导出密钥时，需要提供密钥头，包含密钥类型、权限、公/私钥长度等信息，详见[ehsm_key_format_st](#)。

5.2.2 密钥数据格式

密钥数据按不同的算法，最终会拼成一个字节数组，具体格式由不同的密钥类型和长度决定。另外，只有公钥、只有私钥和公私钥都存在的情况也不相同，不存在的部分不会体现在数据中。

密钥类型可参考[ehsm_key_type_e](#)。

辅助密钥导入/导出的结构体定义，请参考[密钥导入/导出数据格式结构](#)。根据相要导入/导出的密钥类型、公钥/私钥/密钥对，使用其中定义的相应结构体来组织数据。

5.2.2.1 对称密钥

对称密钥（包括 HMAC 密钥、RANDOM 密钥）为一个实际密钥长度的字节数组。

例如，AES-128 密钥明文为 16 字节，加密后的长度为 32 字节（进行了 PKCS7 填充），因此，

- 明文导入导出时，其密钥数据的总长度为 $12 + 16 = 28$ 字节。
- 密文导入导出时，其密钥数据的总长度为 $12 + 32 = 44$ 字节。

5.2.2.2 RSA 密钥

RSA 的密钥数据格式为:

场景/密钥元素	E	N	D
仅公钥	有	有	无
仅私钥	无	有	有
密钥对	有	有	有

其中:

- E 的长度在密钥头中由公钥长度指定 (不存在时对应长度为 0)
- D 的长度在密钥头中由私钥长度指定 (不存在时对应长度为 0)
- N 的长度由密钥头中的密钥类型推导
- 密文导入导出时, 只有 D 会被加密 (注意长度会有差别)

例如, 对于 RSA-2048 密钥对, E 的长度假设为 8, 则明文数据总长度为: $12 + 8(E) + 256(N) + 256(D) = 532$, 密文数据总长度为 548, 只有 D 会被加密。

5.2.2.3 RSA-CRT 密钥

RSA-CRT 的密钥数据格式为:

场景/密钥元素	E	N	P、Q、DP、DQ、U
仅公钥	有	有	无
仅私钥	无	无	有
密钥对	有	有	有

- E 的长度在密钥头中由公钥长度指定 (不存在时对应长度为 0)
- 私钥部分 5 分量的长度在密钥头中由私钥长度指定 (不存在时对应长度为 0)
- 5 分量的存储顺序为 $P \parallel Q \parallel DP \parallel DQ \parallel U$
- N 的长度由密钥头中的密钥类型推导
- 密文导入导出时, 只有私钥部分会被加密 (注意长度会有差别)

例如, 对于 RSA-2048 密钥对, E 的长度假设为 8, 则明文数据总长度为: $12 + 8(E) + 256(N) + 128 \times 5(5 \text{ 分量}) = 916$, 密文数据总长度为 932, 只有 5 分量会被加密。

5.2.2.4 ECC 密钥

ECC 的密钥数据格式为:

场景/密钥元素	公钥 (x, y)	私钥
仅公钥	有	无
仅私钥	无	有
密钥对	有	有

- 公私钥的长度与密钥曲线类型相关

- 公私钥长度在密钥头中由对应的字段指定（不存在时对应长度为 0）
- 密文导入导出时，只有私钥部分会被加密（注意长度会有差别）
- 公钥不需要前导字节 `0x04`

例如：对于 ECC-P256R1 密钥对，密钥明文数据总长度为： $12 + 32 \times 2(X,Y) + 32 = 108$ ，密文数据总长度为 124。

5.2.2.5 SM2 密钥

SM2 的密钥数据格式与 ECC 密钥类似，唯一的区别是公钥必须附带前导字节 `0x04`，密钥明文数据总长度为： $12 + 1 + 32 \times 2(X,Y) + 32 = 109$ ，密文数据总长度为 125。

5.2.2.6 DH 密钥

DH 的密钥数据格式为：

场景/密钥元素	公钥	私钥
仅公钥	有	无
仅私钥	无	有
密钥对	有	有

- 公私钥的长度与选择的模数有关
- 公私钥长度在密钥头中由对应的字段指定（不存在时对应长度为 0）
- 密文导入导出时，只有私钥部分会被加密（注意长度会有差别）

例如：假设 DH 密钥长度为 2048 bit，则密钥明文数据总长度为 $12 + 256 + 256 = 524$ ，密文密钥数据总长度为 540。

5.2.2.7 SM9 密钥

SM9 密钥导入导出的密钥分为四种：用户加密密钥、用户签名密钥、用户密钥协商密钥、密钥协商临时密钥。用户加密密钥、用户签名密钥和用户密钥协商密钥数据只包含用户私钥，密钥长度分别为 128 字节、64 字节、128 字节。密钥协商临时密钥包含公私钥对，公私钥长度为 64 字节和 32 字节。

例如：密钥协商临时密钥，则密钥明文数据总长度为 $12 + 64 + 32 = 108$ ，密文密钥数据总长度为 124。

5.3 明文密钥数据格式

函数名称中包含 `_with_plain_key` 的 API 支持直接传入明文密钥值而非 Key Handle，其输入参数 `key` 的格式定义如下：

密钥类型	格式	签名/解密	验签/加密
对称密钥	固定长度数组	-	-
SM2 密钥	ehsm_sm2_key_st	仅提供私钥，公钥字段被忽略	仅提供公钥，私钥字段被忽略
RSA 密钥	ehsm_rsa_key_st	需提供公钥和私钥	仅提供公钥，私钥字段被忽略
ECC 密钥	ehsm_ecc_key_st	仅提供私钥，公钥字段被忽略	仅提供公钥，私钥字段被忽略
SM9 密钥	特定长度数组	签名时传用户签名私钥，长度为 64；解密时传用户加密私钥，长度为 128	验签和加密时传 NULL

注意：具体使用方式可参考 demo 代码。

6 共享内存机制

eHSM 与 SOC 通过共享内存来交互数据，API 中的许多参数附带有 `EHSM_SHM` 标识，表示对应指针指向的空间必须位于共享内存中。否则，eHSM 无法访问相应的数据。

实际共享的空间范围通常由 SOC 芯片制造商指定，因此，此驱动库的使用者需要根据实际情况从共享内存中分配内存空间用于 API 的调用。建议提供一个内存分配模块来为共享内存空间提供完整的内存分配功能。

可参考 demo 目录中的示例代码。

7 eHSM Speed Test 程序使用说明

7.1 概述

eHSM Speed Test 是一个用于测试硬件安全模块(eHSM)各种密码学算法性能的基准测试工具。该程序可以测试对称加密、哈希、公钥加密、消息认证码、认证加密和 ChaCha 算法的运行速度。

7.2 程序结构

```
1| speed_test/
2| |—— main.c           # 主程序入口
3| |—— speed_test.h      # 头文件定义
4| |—— speed_test_core.c # 核心功能实现
5| |—— test_ske_speed.c  # 对称密钥加密速度测试
6| |—— test_hash_speed.c # 哈希算法速度测试
7| |—— test_pke_speed.c  # 公钥加密速度测试
8| |—— test_pke_cipher_speed.c # 公钥密码速度测试
9| |—— test_mac_speed.c  # 消息认证码速度测试
10| |—— test_aead_speed.c # 认证加密速度测试
11| |—— test_chacha_speed.c # ChaCha算法速度测试
12| |—— CMakeLists.txt   # CMake构建配置
13| |—— speed_test.cmake # 速度测试CMake配置
14| |—— build/           # 构建输出目录
15|
```

7.3 编译方法

7.3.1 使用 CMake 编译

在项目根目录下执行以下命令：

```
1| # 配置构建环境, 指定 PORT 和启用 BUILD_SPEED_TEST
2| cmake -B build -DPORT=<your-platform-dir> -DBUILD_SPEED_TEST=ON
3|
4| # 编译代码
5| cmake --build build
6|
```

请将 `<your-platform-dir>` 替换为您在 `port/` 目录下的目标平台路径 (例如 `osr/m130`)。

必需参数说明：

- `-DPORT=<your-platform-dir>`: 指定目标平台路径 (必需)
- `-DBUILD_SPEED_TEST=ON`: 启用速度测试程序编译 (必需)

可选参数说明：

- `-DOTP_DEFAULT1=<0|1>`: 指定 OTP 默认填充值（可选，默认为 1）
 - `0`: OTP 填充全 0
 - `1`: OTP 填充全 1（默认）

示例:

```
1| # 指定 OTP 填充全 0
2| cmake -B build -DPORT=osr/m130 -DBUILD_SPEED_TEST=ON -DOTP_DEFAULT1=0
3|
4| # 指定 OTP 填充全 1
5| cmake -B build -DPORT=osr/m130 -DBUILD_SPEED_TEST=ON -DOTP_DEFAULT1=1
6|
```

7.3.2 编译产物

编译成功后会在 `build/` 目录中生成 `ehsm_speed_test` 可执行文件。

7.4 运行方式

7.4.1 运行所有测试

```
1| ./ehsm_speed_test
2|
```

程序会自动运行所有支持的密码学算法测试，包括：

- SKE (对称密钥加密)
- HASH (哈希算法)
- PKE (公钥加密)
- PKE-CIPHER (公钥密码)
- MAC (消息认证码)
- AEAD (认证加密)
- CHACHA (ChaCha 算法)

7.5 测试类型详解

7.5.1 1. SKE (对称密钥加密) 测试

- 支持算法: AES-128/192/256, SM4, DES, 3DES-128/192
- 支持模式: ECB, CBC, CFB, OFB, CTR, XTS
- 测试内容: 加密/解密性能

7.5.2 2. HASH (哈希算法) 测试

- 支持算法 : MD5, SHA1, SHA224, SHA256, SHA384, SHA512, SM3, SHA3-224/256/384/512
- 测试内容 : 哈希计算性能

7.5.3 3. PKE (公钥加密) 测试

- 支持密钥类型 :
 - RSA: 1024/2048/3072/4096 位 (标准和 CRT 格式)
 - ECC: 各种椭圆曲线 (SECP, Brainpool 等)
 - SM2, ED25519, X25519
- 测试内容 : 密钥生成、签名、验证性能

7.5.4 4. MAC (消息认证码) 测试

- 支持模式 : CMAC, CBC-MAC, GMAC
- 测试内容 : MAC 生成和验证性能

7.5.5 5. AEAD (认证加密) 测试

- 支持模式 : CCM, GCM
- 测试内容 : 认证加密/解密性能

7.5.6 6. CHACHA 测试

- 算法 : ChaCha20/Poly1305
- 测试内容 : 流密码性能

7.6 测试参数配置

7.6.1 默认配置

- 测试循环次数 : 5 次 (`DEFAULT_TEST_LOOPS`)
- 最大测试数据大小 : 250KB (`MAX_TEST_DATA_SIZE`)

7.6.2 性能指标

程序会输出以下性能指标:

- 时间 : 算法执行时间 (毫秒)
- 吞吐量 : MB/s (兆字节每秒)
- TPS : 每秒事务数 (适用于公钥算法)

7.7 硬件要求

7.7.1 eHSM 硬件

- 程序需要连接到 eHSM 硬件模块
- 支持的状态寄存器地址: `0x40010000`
- 需要等待硬件启动完成 (`SYSSTA0_BOOT_DONE`) 和 HSM 就绪 (`SYSSTA0_HSM_READY`)

7.7.2 OTP 配置

程序会自动配置 OTP (一次性可编程) 数据:

- 默认值通过 CMake 参数 `-DEHSM_PORT_OTP_DEFAULT_VALUE` 配置
- 支持全 0 或全 1 填充模式
- 默认值为 1 (填充全 1)

7.8 输出示例

```
1| eHSM Speed Test Starting...
2|
3| === Running ske tests ===
4| [测试SKE算法性能结果]
5|
6| === Running hash tests ===
7| [测试HASH算法性能结果]
8|
9| === Running pke tests ===
10| [测试PKE算法性能结果]
11|
12| ...
13|
14| All speed tests completed successfully!
15|
```

7.9 注意事项

1. **硬件依赖**: 程序依赖 eHSM 硬件模块, 无硬件时无法正常运行
2. **权限要求**: 可能需要管理员权限访问硬件寄存器
3. **测试时间**: 完整测试可能需要较长时间, 特别是大密钥长度的公钥算法
4. **结果解读**: 性能结果受硬件配置、系统负载等因素影响

7.10 故障排除

7.10.1 常见错误

- "Failed to setup eHSM": 硬件初始化失败, 检查硬件连接
- "Unknown test type": 测试类型参数错误
- 超时问题: 硬件未就绪, 检查硬件状态

7.10.2 调试建议

- 检查硬件连接状态
- 确认 OTP 配置正确
- 查看硬件状态寄存器值

8 API Reference

8.1 Groups

8.1.1 Group FW (Firmware) 相关的 API 接口

- enum [ehsm_hash_algo_e](#)
- enumvalue [EHSM_HASH_ALGO_SM3](#)
- enumvalue [EHSM_HASH_ALGO_MD5](#)
- enumvalue [EHSM_HASH_ALGO_SHA256](#)
- enumvalue [EHSM_HASH_ALGO_SHA384](#)
- enumvalue [EHSM_HASH_ALGO_SHA512](#)
- enumvalue [EHSM_HASH_ALGO_SHA1](#)
- enumvalue [EHSM_HASH_ALGO_SHA224](#)
- enumvalue [EHSM_HASH_ALGO_SHA512_224](#)
- enumvalue [EHSM_HASH_ALGO_SHA512_256](#)
- enumvalue [EHSM_HASH_ALGO_SHA3_224](#)
- enumvalue [EHSM_HASH_ALGO_SHA3_256](#)
- enumvalue [EHSM_HASH_ALGO_SHA3_384](#)
- enumvalue [EHSM_HASH_ALGO_SHA3_512](#)
- enum [ehsm_rng_algo_e](#)
- enumvalue [EHSM_RNG_ALGO_SM4_CTR_DRBG](#)
- enumvalue [EHSM_RNG_ALGO_AES_CTR_DRBG](#)
- enum [ehsm_symm_algo_e](#)
- enumvalue [EHSM_SYMM_ALGO_DES](#)
- enumvalue [EHSM_SYMM_ALGO_TDES_128](#)
- enumvalue [EHSM_SYMM_ALGO_TDES_192](#)
- enumvalue [EHSM_SYMM_ALGO_AES_128](#)
- enumvalue [EHSM_SYMM_ALGO_AES_192](#)
- enumvalue [EHSM_SYMM_ALGO_AES_256](#)
- enumvalue [EHSM_SYMM_ALGO_SM4](#)
- enum [ehsm_mac_mode_e](#)
- enumvalue [EHSM_MAC_MODE_CMAC](#)

- enumvalue [EHSM_MAC_MODE_CBC_MAC](#)
- enumvalue [EHSM_MAC_MODE_GMAC](#)
- enum [ehsm_padding_mode_e](#)
- enumvalue [EHSM_PADDING_NONE](#)
- enumvalue [EHSM_PADDING_PKCS7](#)
- enumvalue [EHSM_PADDING_ONE_WITH_ZEROS](#)
- enum [ehsm_rsa_padding_mode_e](#)
- enumvalue [EHSM_RSA_PADDING_NONE](#)
- enumvalue [EHSM_RSA_PADDING_PSS](#)
- enum [ehsm_cipher_mode_e](#)
- enumvalue [EHSM_CIPHER_MODE_ECB](#)
- enumvalue [EHSM_CIPHER_MODE_XTS](#)
- enumvalue [EHSM_CIPHER_MODE_CBC](#)
- enumvalue [EHSM_CIPHER_MODE_CFB](#)
- enumvalue [EHSM_CIPHER_MODE_OFB](#)
- enumvalue [EHSM_CIPHER_MODE_CTR](#)
- enum [ehsm_aead_mode_e](#)
- enumvalue [EHSM_AEAD_MODE_GCM](#)
- enumvalue [EHSM_AEAD_MODE_CCM](#)
- enum [ehsm_key_type_e](#)
- enumvalue [EHSM_KEY_TYPE_DES](#)
- enumvalue [EHSM_KEY_TYPE_TDES_128](#)
- enumvalue [EHSM_KEY_TYPE_TDES_192](#)
- enumvalue [EHSM_KEY_TYPE_AES_128](#)
- enumvalue [EHSM_KEY_TYPE_AES_192](#)
- enumvalue [EHSM_KEY_TYPE_AES_256](#)
- enumvalue [EHSM_KEY_TYPE_SM4](#)
- enumvalue [EHSM_KEY_TYPE_SM2](#)
- enumvalue [EHSM_KEY_TYPE_RSA_1024](#)
- enumvalue [EHSM_KEY_TYPE_RSA_2048](#)
- enumvalue [EHSM_KEY_TYPE_RSA_3072](#)
- enumvalue [EHSM_KEY_TYPE_RSA_4096](#)

- enumvalue [EHSM_KEY_TYPE_RSA_1024_CRT](#)
- enumvalue [EHSM_KEY_TYPE_RSA_2048_CRT](#)
- enumvalue [EHSM_KEY_TYPE_RSA_3072_CRT](#)
- enumvalue [EHSM_KEY_TYPE_RSA_4096_CRT](#)
- enumvalue [EHSM_KEY_TYPE_DH](#)
- enumvalue [EHSM_KEY_TYPE_ECC_BRAINPOOLP_160R1](#)
- enumvalue [EHSM_KEY_TYPE_ECC_BRAINPOOLP_192R1](#)
- enumvalue [EHSM_KEY_TYPE_ECC_BRAINPOOLP_224R1](#)
- enumvalue [EHSM_KEY_TYPE_ECC_BRAINPOOLP_256R1](#)
- enumvalue [EHSM_KEY_TYPE_ECC_BRAINPOOLP_320R1](#)
- enumvalue [EHSM_KEY_TYPE_ECC_BRAINPOOLP_384R1](#)
- enumvalue [EHSM_KEY_TYPE_ECC_BRAINPOOLP_512R1](#)
- enumvalue [EHSM_KEY_TYPE_ECC_SECP_192R1](#)
- enumvalue [EHSM_KEY_TYPE_ECC_SECP_224R1](#)
- enumvalue [EHSM_KEY_TYPE_ECC_SECP_256R1](#)
- enumvalue [EHSM_KEY_TYPE_ECC_SECP_384R1](#)
- enumvalue [EHSM_KEY_TYPE_ECC_SECP_521R1](#)
- enumvalue [EHSM_KEY_TYPE_ED25519](#)
- enumvalue [EHSM_KEY_TYPE_X25519](#)
- enumvalue [EHSM_KEY_TYPE_SM4_XTS](#)
- enumvalue [EHSM_KEY_TYPE_AES_128_XTS](#)
- enumvalue [EHSM_KEY_TYPE_AES_192_XTS](#)
- enumvalue [EHSM_KEY_TYPE_AES_256_XTS](#)
- enumvalue [EHSM_KEY_TYPE_CHACHA](#)
- enumvalue [EHSM_KEY_TYPE_HMAC](#)
- enumvalue [EHSM_KEY_TYPE_ECC_SECP_160K1](#)
- enumvalue [EHSM_KEY_TYPE_ECC_SECP_192K1](#)
- enumvalue [EHSM_KEY_TYPE_ECC_SECP_224K1](#)
- enumvalue [EHSM_KEY_TYPE_ECC_SECP_256K1](#)
- enumvalue [EHSM_KEY_TYPE_SM9_ENC_USERPRIV](#)
- enumvalue [EHSM_KEY_TYPE_SM9_SIGN_USERPRIV](#)
- enumvalue [EHSM_KEY_TYPE_SM9_EXCHG_USERPRIV](#)

- enumvalue [EHSM_KEY_TYPE_SM9_EXCHG_TEMP](#)
- enum [ehsm_key_part_e](#)
- enumvalue [EHSM_KEY_PART_PUBLIC_KEY](#)
- enumvalue [EHSM_KEY_PART_PRIVATE_KEY](#)
- enumvalue [EHSM_KEY_PART_SYMM_KEY](#)
- enumvalue [EHSM_KEY_PART_KEY_PAIR](#)
- enum [ehsm_derive_algo_e](#)
- enumvalue [EHSM_DERIVE_ALGO_KDFX963](#)
- enumvalue [EHSM_DERIVE_ALGO_PBKDF2](#)
- enum [ehsm_derive_type_e](#)
- enumvalue [EHSM_DERIVE_TYPE_PASSWORD](#)
- enumvalue [EHSM_DERIVE_TYPE_FROM_PARENT_KEY](#)
- enum [ehsm_sm2_role_e](#)
- enumvalue [EHSM_SM2_ROLE_SPONSOR](#)
- enumvalue [EHSM_SM2_ROLE_RESPONSOR](#)
- enum [ehsm_sm9_enc_type_e](#)
- enumvalue [EHSM_SM9_ENC_TYPE_STREAM](#)
- enumvalue [EHSM_SM9_ENC_TYPE_BLOCK](#)
- enum [ehsm_sm9_padding_mode_e](#)
- enumvalue [EHSM_SM9_PADDING_NONE](#)
- enumvalue [EHSM_SM9_PADDING_PKCS7](#)
- enum [ehsm_sm9_role_e](#)
- enumvalue [EHSM_SM9_ROLE_SPONSOR](#)
- enumvalue [EHSM_SM9_ROLE_RESPONSOR](#)
- enum [ehsm_proc_mode_e](#)
- enumvalue [EHSM_PROC_INIT](#)
- enumvalue [EHSM_PROC_UPDATE](#)
- enumvalue [EHSM_PROC_FINISH](#)
- enumvalue [EHSM_PROC_ONEPASS](#)
- enum [ehsm_drv_mode_e](#)
- enumvalue [EHSM_DRV_MODE_INTERRUPT](#)
- enumvalue [EHSM_DRV_MODE_WAIT_AND_POLL](#)

- enumvalue EHSM_DRV_MODE_SEND_AND_PEEK
- enum ehsm_key_level_e
- enumvalue EHSM_KEY_LEVEL_1
- enumvalue EHSM_KEY_LEVEL_2
- enum ehsm_install_key_type_e
- enumvalue EHSM_INSTALL_KEY_TYPE_SYMM
- enumvalue EHSM_INSTALL_KEY_TYPE_ASYM_PRIV_KEY
- enumvalue EHSM_INSTALL_KEY_TYPE_ASYM_PUB_KEY_HASH
- enum ehsm_challenge_type_e
- enumvalue EHSM_CHALLENGE_TYPE_EHSM_DEBUG
- enumvalue EHSM_CHALLENGE_TYPE_SHE_DEBUG
- enumvalue EHSM_CHALLENGE_TYPE_SOC_DEBUG
- enumvalue EHSM_CHALLENGE_TYPE_USER_AUTH
- enumvalue EHSM_CHALLENGE_TYPE_FW_AUTH
- enum ehsm_auth_algo_e
- enumvalue EHSM_AUTH_ALGO_SM3_SM2
- enumvalue EHSM_AUTH_ALGO_SHA256_ECDSA_P256R1
- enumvalue EHSM_AUTH_ALGO_SM4_CMAC
- enumvalue EHSM_AUTH_ALGO_AES128_CMAC
- enumvalue EHSM_AUTH_ALGO_SHA256_RSA
- enum ehsm_lifecycle_e
- enumvalue EHSM_LC_TEST
- enumvalue EHSM_LC_DEVELOP
- enumvalue EHSM_LC_MANUFACTURE
- enumvalue EHSM_LC_USER
- enumvalue EHSM_LC_DEBUG
- enumvalue EHSM_LC_DESTROY
- enum ehsm_ctrl_field_e
- enumvalue EHSM_CTRL_FIELD_HW
- enumvalue EHSM_CTRL_FIELD_EHSM
- enumvalue EHSM_CTRL_FIELD_SOC
- typedef ehsm_rsp_cb_func_t

- **function** [ehsm_driver_init_library](#)
- **function** [ehsm_driver_get_version](#)
- **function** [ehsm_ctx_init](#)
- **function** [ehsm_ctx_poll](#)
- **function** [ehsm_gen_random](#)
- **function** [ehsm_hash_init](#)
- **function** [ehsm_hash_update](#)
- **function** [ehsm_hash_finish](#)
- **function** [ehsm_hash_onepass](#)
- **function** [ehsm_hmac_init](#)
- **function** [ehsm_hmac_update](#)
- **function** [ehsm_hmac_finish_gen](#)
- **function** [ehsm_hmac_finish_verify](#)
- **function** [ehsm_hmac_onepass_gen](#)
- **function** [ehsm_hmac_onepass_verify](#)
- **function** [ehsm_hmac_init_with_plain_key](#)
- **function** [ehsm_hmac_onepass_gen_with_plain_key](#)
- **function** [ehsm_hmac_onepass_verify_with_plain_key](#)
- **function** [ehsm_mac_init](#)
- **function** [ehsm_mac_update](#)
- **function** [ehsm_mac_finish_gen](#)
- **function** [ehsm_mac_finish_verify](#)
- **function** [ehsm_mac_onepass_gen](#)
- **function** [ehsm_mac_onepass_verify](#)
- **function** [ehsm_mac_init_with_plain_key](#)
- **function** [ehsm_mac_onepass_gen_with_plain_key](#)
- **function** [ehsm_mac_onepass_verify_with_plain_key](#)
- **function** [ehsm_symm_cipher_init](#)
- **function** [ehsm_symm_cipher_update](#)
- **function** [ehsm_symm_cipher_finish](#)
- **function** [ehsm_symm_cipher_onepass](#)
- **function** [ehsm_symm_cipher_init_with_plain_key](#)

- **function** [ehsm_symm_cipher_onepass_with_plain_key](#)
- **function** [ehsm_sm2_cipher](#)
- **function** [ehsm_sm2_cipher_with_plain_key](#)
- **function** [ehsm_sm2_sign_init](#)
- **function** [ehsm_sm2_sign_init_with_plain_key](#)
- **function** [ehsm_sm2_sign_update](#)
- **function** [ehsm_sm2_sign_finish_gen](#)
- **function** [ehsm_sm2_sign_finish_verify](#)
- **function** [ehsm_sm2_sign_onepass_ex](#)
- **function** [ehsm_sm2_sign_onepass_gen](#)
- **function** [ehsm_sm2_sign_onepass_verify](#)
- **function** [ehsm_sm2_sign_onepass_gen_with_digest](#)
- **function** [ehsm_sm2_sign_onepass_verify_with_digest](#)
- **function** [ehsm_sm2_sign_onepass_gen_with_plain_key](#)
- **function** [ehsm_sm2_sign_onepass_verify_with_plain_key](#)
- **function** [ehsm_sm2_sign_onepass_gen_with_plain_key_and_digest](#)
- **function** [ehsm_sm2_sign_onepass_verify_with_plain_key_and_digest](#)
- **function** [ehsm_rsa_cipher](#)
- **function** [ehsm_rsa_cipher_with_plain_key](#)
- **function** [ehsm_rsa_sign_init](#)
- **function** [ehsm_rsa_sign_init_with_plain_key](#)
- **function** [ehsm_rsa_sign_update](#)
- **function** [ehsm_rsa_sign_finish_gen](#)
- **function** [ehsm_rsa_sign_finish_verify](#)
- **function** [ehsm_rsa_sign_onepass_ex](#)
- **function** [ehsm_rsa_sign_onepass_gen](#)
- **function** [ehsm_rsa_sign_onepass_verify](#)
- **function** [ehsm_rsa_sign_onepass_gen_with_digest](#)
- **function** [ehsm_rsa_sign_onepass_verify_with_digest](#)
- **function** [ehsm_rsa_sign_onepass_gen_with_plain_key](#)
- **function** [ehsm_rsa_sign_onepass_verify_with_plain_key](#)
- **function** [ehsm_rsa_sign_onepass_gen_with_plain_key_and_digest](#)

- **function** [ehsm_rsa_sign_onepass_verify_with_plain_key_and_digest](#)
- **function** [ehsm_ecdsa_init](#)
- **function** [ehsm_ecdsa_init_with_plain_key](#)
- **function** [ehsm_ecdsa_update](#)
- **function** [ehsm_ecdsa_finish_gen](#)
- **function** [ehsm_ecdsa_finish_verify](#)
- **function** [ehsm_ecdsa_onepass_ex](#)
- **function** [ehsm_ecdsa_onepass_gen](#)
- **function** [ehsm_ecdsa_onepass_verify](#)
- **function** [ehsm_ecdsa_onepass_gen_with_digest](#)
- **function** [ehsm_ecdsa_onepass_verify_with_digest](#)
- **function** [ehsm_ecdsa_onepass_gen_with_plain_key](#)
- **function** [ehsm_ecdsa_onepass_verify_with_plain_key](#)
- **function** [ehsm_ecdsa_onepass_gen_with_plain_key_and_digest](#)
- **function** [ehsm_ecdsa_onepass_verify_with_plain_key_and_digest](#)
- **function** [ehsm_sm9_cipher](#)
- **function** [ehsm_sm9_cipher_with_plain_key](#)
- **function** [ehsm_sm9_sign_onepass_gen](#)
- **function** [ehsm_sm9_sign_onepass_verify](#)
- **function** [ehsm_sm9_sign_onepass_gen_with_plain_key](#)
- **function** [ehsm_chacha_init](#)
- **function** [ehsm_chacha_update](#)
- **function** [ehsm_chacha_finish_enc](#)
- **function** [ehsm_chacha_finish_dec](#)
- **function** [ehsm_chacha_onepass_enc](#)
- **function** [ehsm_chacha_onepass_dec](#)
- **function** [ehsm_chacha_init_with_plain_key](#)
- **function** [ehsm_chacha_onepass_enc_with_plain_key](#)
- **function** [ehsm_chacha_onepass_dec_with_plain_key](#)
- **function** [ehsm_aead_init](#)
- **function** [ehsm_aead_init_with_plain_key](#)
- **function** [ehsm_aead_update](#)

- **function** [ehsm_aead_finish_enc](#)
- **function** [ehsm_aead_finish_dec](#)
- **function** [ehsm_aead_onepass_enc](#)
- **function** [ehsm_aead_onepass_dec](#)
- **function** [ehsm_aead_onepass_enc_with_plain_key](#)
- **function** [ehsm_aead_onepass_dec_with_plain_key](#)
- **function** [ehsm_km_gen_key](#)
- **function** [ehsm_km_import_key](#)
- **function** [ehsm_km_export_key](#)
- **function** [ehsm_km_derive_key](#)
- **function** [ehsm_km_derive_key_to_soc](#)
- **function** [ehsm_km_exchange_key](#)
- **function** [ehsm_km_sm9_exchange_key](#)
- **function** [ehsm_km_get_pub_from_priv](#)
- **function** [ehsm_km_remove_key](#)
- **function** [ehsm_get_version](#)
- **function** [ehsm_verify_image](#)
- **function** [ehsm_upgrade_fw_image_ex](#)
- **function** [ehsm_upgrade_fw_image](#)
- **function** [ehsm_upgrade_fw_image_init](#)
- **function** [ehsm_upgrade_fw_image_update](#)
- **function** [ehsm_upgrade_fw_image_finish](#)
- **function** [ehsm_set_utc_time](#)
- **function** [ehsm_get_utc_time](#)
- **function** [ehsm_create_counter](#)
- **function** [ehsm_read_counter](#)
- **function** [ehsm_increase_counter](#)
- **function** [ehsm_delete_counter](#)
- **function** [ehsm_enter_wfi](#)
- **function** [ehsm_set_uart_baudrate](#)
- **function** [ehsm_install_random_key](#)
- **function** [ehsm_install_encrypted_key](#)

- function [ehsm_read_otp](#)
- function [ehsm_write_otp](#)
- function [ehsm_read_reg](#)
- function [ehsm_write_reg](#)
- function [ehsm_get_challenge](#)
- function [ehsm_debug_auth](#)
- function [ehsm_close_debug](#)
- function [ehsm_get_emu_status](#)
- function [ehsm_change_lifecycle](#)
- function [ehsm_change_control_field](#)
- define [EHSM_KEY_HANDLE_INVALID](#)

8.1.2 Group 密钥导入/导入数据格式结构

- typedef [ehsm_key_format_des_plain_st](#)
- typedef [ehsm_key_format_des_cipher_st](#)
- typedef [ehsm_key_format_aes128_plain_st](#)
- typedef [ehsm_key_format_aes128_cipher_st](#)
- typedef [ehsm_key_format_tdes128_plain_st](#)
- typedef [ehsm_key_format_tdes128_cipher_st](#)
- typedef [ehsm_key_format_sm4_plain_st](#)
- typedef [ehsm_key_format_sm4_cipher_st](#)
- typedef [ehsm_key_format_tdes192_plain_st](#)
- typedef [ehsm_key_format_tdes192_cipher_st](#)
- typedef [ehsm_key_format_aes192_plain_st](#)
- typedef [ehsm_key_format_aes192_cipher_st](#)
- typedef [ehsm_key_format_aes256_plain_st](#)
- typedef [ehsm_key_format_aes256_cipher_st](#)
- typedef [ehsm_key_format_aes128_xts_plain_st](#)
- typedef [ehsm_key_format_aes128_xts_cipher_st](#)
- typedef [ehsm_key_format_sm4_xts_plain_st](#)
- typedef [ehsm_key_format_sm4_xts_cipher_st](#)
- typedef [ehsm_key_format_aes192_xts_plain_st](#)

- typedef ehsm_key_format_aes192_xts_cipher_st
- typedef ehsm_key_format_aes256_xts_plain_st
- typedef ehsm_key_format_aes256_xts_cipher_st
- typedef ehsm_key_format_hmac64_plain_st
- typedef ehsm_key_format_hmac64_cipher_st
- typedef ehsm_key_format_hmac128_plain_st
- typedef ehsm_key_format_hmac128_cipher_st
- typedef ehsm_key_format_hmac192_plain_st
- typedef ehsm_key_format_hmac192_cipher_st
- typedef ehsm_key_format_hmac256_plain_st
- typedef ehsm_key_format_hmac256_cipher_st
- typedef ehsm_key_format_hmac384_plain_st
- typedef ehsm_key_format_hmac384_cipher_st
- typedef ehsm_key_format_hmac512_plain_st
- typedef ehsm_key_format_hmac512_cipher_st
- typedef ehsm_key_format_ecc_bp_p160r1_plain_pubkey_st
- typedef ehsm_key_format_ecc_bp_p160r1_plain_privkey_st
- typedef ehsm_key_format_ecc_bp_p160r1_cipher_privkey_st
- typedef ehsm_key_format_ecc_bp_p160r1_plain_keypair_st
- typedef ehsm_key_format_ecc_bp_p160r1_cipher_keypair_st
- typedef ehsm_key_format_ecc_bp_p192r1_plain_pubkey_st
- typedef ehsm_key_format_ecc_bp_p192r1_plain_privkey_st
- typedef ehsm_key_format_ecc_bp_p192r1_cipher_privkey_st
- typedef ehsm_key_format_ecc_bp_p192r1_plain_keypair_st
- typedef ehsm_key_format_ecc_bp_p192r1_cipher_keypair_st
- typedef ehsm_key_format_ecc_bp_p224r1_plain_pubkey_st
- typedef ehsm_key_format_ecc_bp_p224r1_plain_privkey_st
- typedef ehsm_key_format_ecc_bp_p224r1_cipher_privkey_st
- typedef ehsm_key_format_ecc_bp_p224r1_plain_keypair_st
- typedef ehsm_key_format_ecc_bp_p224r1_cipher_keypair_st
- typedef ehsm_key_format_ecc_bp_p256r1_plain_pubkey_st
- typedef ehsm_key_format_ecc_bp_p256r1_plain_privkey_st

- `typedef ehsm_key_format_ecc_bp_p256r1_cipher_privkey_st`
- `typedef ehsm_key_format_ecc_bp_p256r1_plain_keypair_st`
- `typedef ehsm_key_format_ecc_bp_p256r1_cipher_keypair_st`
- `typedef ehsm_key_format_ecc_bp_p320r1_plain_pubkey_st`
- `typedef ehsm_key_format_ecc_bp_p320r1_plain_privkey_st`
- `typedef ehsm_key_format_ecc_bp_p320r1_cipher_privkey_st`
- `typedef ehsm_key_format_ecc_bp_p320r1_plain_keypair_st`
- `typedef ehsm_key_format_ecc_bp_p320r1_cipher_keypair_st`
- `typedef ehsm_key_format_ecc_bp_p384r1_plain_pubkey_st`
- `typedef ehsm_key_format_ecc_bp_p384r1_plain_privkey_st`
- `typedef ehsm_key_format_ecc_bp_p384r1_cipher_privkey_st`
- `typedef ehsm_key_format_ecc_bp_p384r1_plain_keypair_st`
- `typedef ehsm_key_format_ecc_bp_p384r1_cipher_keypair_st`
- `typedef ehsm_key_format_ecc_bp_p512r1_plain_pubkey_st`
- `typedef ehsm_key_format_ecc_bp_p512r1_plain_privkey_st`
- `typedef ehsm_key_format_ecc_bp_p512r1_cipher_privkey_st`
- `typedef ehsm_key_format_ecc_bp_p512r1_plain_keypair_st`
- `typedef ehsm_key_format_ecc_bp_p512r1_cipher_keypair_st`
- `typedef ehsm_key_format_ecc_sec_p192r1_plain_pubkey_st`
- `typedef ehsm_key_format_ecc_sec_p192r1_plain_privkey_st`
- `typedef ehsm_key_format_ecc_sec_p192r1_cipher_privkey_st`
- `typedef ehsm_key_format_ecc_sec_p192r1_plain_keypair_st`
- `typedef ehsm_key_format_ecc_sec_p192r1_cipher_keypair_st`
- `typedef ehsm_key_format_ecc_sec_p224r1_plain_pubkey_st`
- `typedef ehsm_key_format_ecc_sec_p224r1_plain_privkey_st`
- `typedef ehsm_key_format_ecc_sec_p224r1_cipher_privkey_st`
- `typedef ehsm_key_format_ecc_sec_p224r1_plain_keypair_st`
- `typedef ehsm_key_format_ecc_sec_p224r1_cipher_keypair_st`
- `typedef ehsm_key_format_ecc_sec_p256r1_plain_pubkey_st`
- `typedef ehsm_key_format_ecc_sec_p256r1_plain_privkey_st`
- `typedef ehsm_key_format_ecc_sec_p256r1_cipher_privkey_st`
- `typedef ehsm_key_format_ecc_sec_p256r1_plain_keypair_st`

- typedef ehsm_key_format_ecc_sec_p256r1_cipher_keypair_st
- typedef ehsm_key_format_ecc_sec_p384r1_plain_pubkey_st
- typedef ehsm_key_format_ecc_sec_p384r1_plain_privkey_st
- typedef ehsm_key_format_ecc_sec_p384r1_cipher_privkey_st
- typedef ehsm_key_format_ecc_sec_p384r1_plain_keypair_st
- typedef ehsm_key_format_ecc_sec_p384r1_cipher_keypair_st
- typedef ehsm_key_format_ecc_sec_p521r1_plain_pubkey_st
- typedef ehsm_key_format_ecc_sec_p521r1_plain_privkey_st
- typedef ehsm_key_format_ecc_sec_p521r1_cipher_privkey_st
- typedef ehsm_key_format_ecc_sec_p521r1_plain_keypair_st
- typedef ehsm_key_format_ecc_sec_p521r1_cipher_keypair_st
- typedef ehsm_key_format_ecc_sec_p160k1_plain_pubkey_st
- typedef ehsm_key_format_ecc_sec_p160k1_plain_privkey_st
- typedef ehsm_key_format_ecc_sec_p160k1_cipher_privkey_st
- typedef ehsm_key_format_ecc_sec_p160k1_plain_keypair_st
- typedef ehsm_key_format_ecc_sec_p160k1_cipher_keypair_st
- typedef ehsm_key_format_ecc_sec_p192k1_plain_pubkey_st
- typedef ehsm_key_format_ecc_sec_p192k1_plain_privkey_st
- typedef ehsm_key_format_ecc_sec_p192k1_cipher_privkey_st
- typedef ehsm_key_format_ecc_sec_p192k1_plain_keypair_st
- typedef ehsm_key_format_ecc_sec_p192k1_cipher_keypair_st
- typedef ehsm_key_format_ecc_sec_p224k1_plain_pubkey_st
- typedef ehsm_key_format_ecc_sec_p224k1_plain_privkey_st
- typedef ehsm_key_format_ecc_sec_p224k1_cipher_privkey_st
- typedef ehsm_key_format_ecc_sec_p224k1_plain_keypair_st
- typedef ehsm_key_format_ecc_sec_p224k1_cipher_keypair_st
- typedef ehsm_key_format_ecc_sec_p256k1_plain_pubkey_st
- typedef ehsm_key_format_ecc_sec_p256k1_plain_privkey_st
- typedef ehsm_key_format_ecc_sec_p256k1_cipher_privkey_st
- typedef ehsm_key_format_ecc_sec_p256k1_plain_keypair_st
- typedef ehsm_key_format_ecc_sec_p256k1_cipher_keypair_st

8.1.3 Group 密钥权限

- **define** [EHSM_KEY_PRIV_SIGN](#)
- **define** [EHSM_KEY_PRIV_VERIFY](#)
- **define** [EHSM_KEY_PRIV_ENCRYPT](#)
- **define** [EHSM_KEY_PRIV_DECRYPT](#)
- **define** [EHSM_KEY_PRIV_TIMESTAMP](#)
- **define** [EHSM_KEY_PRIV_SECURE_BOOT](#)
- **define** [EHSM_KEY_PRIV_SECURE_STORAGE](#)
- **define** [EHSM_KEY_PRIV_KEY_CREATION](#)
- **define** [EHSM_KEY_PRIV_TRANSPORT_KEY_CREATION](#)
- **define** [EHSM_KEY_PRIV_UTCSYNC](#)
- **define** [EHSM_KEY_PRIV_TRANSPORT](#)
- **define** [EHSM_KEY_PRIV_REMOVE](#)
- **define** [EHSM_KEY_PRIV_IMPORT_PLAIN](#)
- **define** [EHSM_KEY_PRIV_IMPORT_CIPHER](#)
- **define** [EHSM_KEY_PRIV_EXPORT_PLAIN](#)
- **define** [EHSM_KEY_PRIV_EXPORT_CIPHER](#)
- **define** [EHSM_KEY_PRIV_BOOT_FAIL_USAGE](#)
- **define** [EHSM_KEY_PRIV_DEBUG_USAGE](#)
- **define** [EHSM_KEY_PRIV_WILDCARD_PROTECT](#)
- **define** [EHSM_KEY_PRIV_WRITE_PROTECT](#)

8.1.4 Group BL (Bootloader) 相关的 API 接口

- **enum** [ehsm_bl_gen_key_type_e](#)
- **enumvalue** [EHSM_BL_GEN_KEY_TYPE_SYMM](#)
- **enumvalue** [EHSM_BL_GEN_KEY_TYPE_SM2](#)
- **enumvalue** [EHSM_BL_GEN_KEY_TYPE_ECC_P256R1](#)
- **function** [ehsm_bl_self_test](#)
- **function** [ehsm_bl_get_self_test_result](#)
- **function** [ehsm_bl_fw_auth](#)
- **function** [ehsm_bl_get_random_key](#)
- **function** [ehsm_bl_encrypt_key](#)

- **define** [ehsm_bl_get_version](#)
- **define** [ehsm_bl_verify_image](#)
- **define** [ehsm_bl_upgrade_fw_image](#)
- **define** [ehsm_bl_upgrade_fw_image_init](#)
- **define** [ehsm_bl_upgrade_fw_image_update](#)
- **define** [ehsm_bl_upgrade_fw_image_finish](#)
- **define** [ehsm_bl_set_uart_baudrate](#)
- **define** [ehsm_bl_get_challenge](#)
- **define** [ehsm_bl_debug_auth](#)
- **define** [ehsm_bl_close_debug](#)
- **define** [ehsm_bl_read_reg](#)
- **define** [ehsm_bl_write_reg](#)
- **define** [ehsm_bl_read_otp](#)
- **define** [ehsm_bl_write_otp](#)

8.1.5 Group 自检算法

- **define** [EHSM_SELF_TEST_SM4_ECB](#)
- **define** [EHSM_SELF_TEST_SM4_CBC](#)
- **define** [EHSM_SELF_TEST_SM4_CFB](#)
- **define** [EHSM_SELF_TEST_SM4_OFB](#)
- **define** [EHSM_SELF_TEST_SM4_CTR](#)
- **define** [EHSM_SELF_TEST_SM2_ENC](#)
- **define** [EHSM_SELF_TEST_SM2_VERIFY](#)
- **define** [EHSM_SELF_TEST_SM2_EXCH](#)
- **define** [EHSM_SELF_TEST_SM3](#)
- **define** [EHSM_SELF_TEST_DES](#)
- **define** [EHSM_SELF_TEST_TDES](#)
- **define** [EHSM_SELF_TEST_AES](#)
- **define** [EHSM_SELF_TEST_RSA](#)
- **define** [EHSM_SELF_TEST_ECC](#)
- **define** [EHSM_SELF_TEST_MD5](#)
- **define** [EHSM_SELF_TEST_SHA1](#)

- **define** [EHSM_SELF_TEST_SHA2](#)
- **define** [EHSM_SELF_TEST_SHA3](#)
- **define** [EHSM_SELF_TEST_SHA256](#)
- **define** [EHSM_SELF_TEST_TRNG](#)

8.2 Files

8.2.1 File api.h

- **define** [EHSM_KEY_PRIV_SIGN](#)
- **define** [EHSM_KEY_PRIV_VERIFY](#)
- **define** [EHSM_KEY_PRIV_ENCRYPT](#)
- **define** [EHSM_KEY_PRIV_DECRYPT](#)
- **define** [EHSM_KEY_PRIV_TIMESTAMP](#)
- **define** [EHSM_KEY_PRIV_SECURE_BOOT](#)
- **define** [EHSM_KEY_PRIV_SECURE_STORAGE](#)
- **define** [EHSM_KEY_PRIV_KEY_CREATION](#)
- **define** [EHSM_KEY_PRIV_TRANSPORT_KEY_CREATION](#)
- **define** [EHSM_KEY_PRIV_UTCSYNC](#)
- **define** [EHSM_KEY_PRIV_TRANSPORT](#)
- **define** [EHSM_KEY_PRIV_REMOVE](#)
- **define** [EHSM_KEY_PRIV_IMPORT_PLAIN](#)
- **define** [EHSM_KEY_PRIV_IMPORT_CIPHER](#)
- **define** [EHSM_KEY_PRIV_EXPORT_PLAIN](#)
- **define** [EHSM_KEY_PRIV_EXPORT_CIPHER](#)
- **define** [EHSM_KEY_PRIV_BOOT_FAIL_USAGE](#)
- **define** [EHSM_KEY_PRIV_DEBUG_USAGE](#)
- **define** [EHSM_KEY_PRIV_WILDCARD_PROTECT](#)
- **define** [EHSM_KEY_PRIV_WRITE_PROTECT](#)
- **define** [EHSM_KEY_HANDLE_INVALID](#)
- **enum** [ehsm_hash_algo_e](#)
- **enum** [ehsm_rng_algo_e](#)
- **enum** [ehsm_symm_algo_e](#)
- **enum** [ehsm_mac_mode_e](#)

- **enum** [ehsm_padding_mode_e](#)
- **enum** [ehsm_rsa_padding_mode_e](#)
- **enum** [ehsm_cipher_mode_e](#)
- **enum** [ehsm_aead_mode_e](#)
- **enum** [ehsm_key_type_e](#)
- **enum** [ehsm_key_part_e](#)
- **enum** [ehsm_derive_algo_e](#)
- **enum** [ehsm_derive_type_e](#)
- **enum** [ehsm_sm2_role_e](#)
- **enum** [ehsm_sm9_enc_type_e](#)
- **enum** [ehsm_sm9_padding_mode_e](#)
- **enum** [ehsm_sm9_role_e](#)
- **enum** [ehsm_proc_mode_e](#)
- **enum** [ehsm_drv_mode_e](#)
- **enum** [ehsm_key_level_e](#)
- **enum** [ehsm_install_key_type_e](#)
- **enum** [ehsm_challenge_type_e](#)
- **enum** [ehsm_auth_algo_e](#)
- **enum** [ehsm_lifecycle_e](#)
- **enum** [ehsm_ctrl_field_e](#)
- **typedef** [ehsm_rsp_cb_func_t](#)
- **function** [ehsm_driver_init_library](#)
- **function** [ehsm_driver_get_version](#)
- **function** [ehsm_ctx_init](#)
- **function** [ehsm_ctx_poll](#)
- **function** [ehsm_gen_random](#)
- **function** [ehsm_hash_init](#)
- **function** [ehsm_hash_update](#)
- **function** [ehsm_hash_finish](#)
- **function** [ehsm_hash_onepass](#)
- **function** [ehsm_hmac_init](#)
- **function** [ehsm_hmac_update](#)

- **function** [ehsm_hmac_finish_gen](#)
- **function** [ehsm_hmac_finish_verify](#)
- **function** [ehsm_hmac_onepass_gen](#)
- **function** [ehsm_hmac_onepass_verify](#)
- **function** [ehsm_hmac_init_with_plain_key](#)
- **function** [ehsm_hmac_onepass_gen_with_plain_key](#)
- **function** [ehsm_hmac_onepass_verify_with_plain_key](#)
- **function** [ehsm_mac_init](#)
- **function** [ehsm_mac_update](#)
- **function** [ehsm_mac_finish_gen](#)
- **function** [ehsm_mac_finish_verify](#)
- **function** [ehsm_mac_onepass_gen](#)
- **function** [ehsm_mac_onepass_verify](#)
- **function** [ehsm_mac_init_with_plain_key](#)
- **function** [ehsm_mac_onepass_gen_with_plain_key](#)
- **function** [ehsm_mac_onepass_verify_with_plain_key](#)
- **function** [ehsm_symm_cipher_init](#)
- **function** [ehsm_symm_cipher_update](#)
- **function** [ehsm_symm_cipher_finish](#)
- **function** [ehsm_symm_cipher_onepass](#)
- **function** [ehsm_symm_cipher_init_with_plain_key](#)
- **function** [ehsm_symm_cipher_onepass_with_plain_key](#)
- **function** [ehsm_sm2_cipher](#)
- **function** [ehsm_sm2_cipher_with_plain_key](#)
- **function** [ehsm_sm2_sign_init](#)
- **function** [ehsm_sm2_sign_init_with_plain_key](#)
- **function** [ehsm_sm2_sign_update](#)
- **function** [ehsm_sm2_sign_finish_gen](#)
- **function** [ehsm_sm2_sign_finish_verify](#)
- **function** [ehsm_sm2_sign_onepass_ex](#)
- **function** [ehsm_sm2_sign_onepass_gen](#)
- **function** [ehsm_sm2_sign_onepass_verify](#)

- **function** [ehsm_sm2_sign_onepass_gen_with_digest](#)
- **function** [ehsm_sm2_sign_onepass_verify_with_digest](#)
- **function** [ehsm_sm2_sign_onepass_gen_with_plain_key](#)
- **function** [ehsm_sm2_sign_onepass_verify_with_plain_key](#)
- **function** [ehsm_sm2_sign_onepass_gen_with_plain_key_and_digest](#)
- **function** [ehsm_sm2_sign_onepass_verify_with_plain_key_and_digest](#)
- **function** [ehsm_rsa_cipher](#)
- **function** [ehsm_rsa_cipher_with_plain_key](#)
- **function** [ehsm_rsa_sign_init](#)
- **function** [ehsm_rsa_sign_init_with_plain_key](#)
- **function** [ehsm_rsa_sign_update](#)
- **function** [ehsm_rsa_sign_finish_gen](#)
- **function** [ehsm_rsa_sign_finish_verify](#)
- **function** [ehsm_rsa_sign_onepass_ex](#)
- **function** [ehsm_rsa_sign_onepass_gen](#)
- **function** [ehsm_rsa_sign_onepass_verify](#)
- **function** [ehsm_rsa_sign_onepass_gen_with_digest](#)
- **function** [ehsm_rsa_sign_onepass_verify_with_digest](#)
- **function** [ehsm_rsa_sign_onepass_gen_with_plain_key](#)
- **function** [ehsm_rsa_sign_onepass_verify_with_plain_key](#)
- **function** [ehsm_rsa_sign_onepass_gen_with_plain_key_and_digest](#)
- **function** [ehsm_rsa_sign_onepass_verify_with_plain_key_and_digest](#)
- **function** [ehsm_ecdsa_init](#)
- **function** [ehsm_ecdsa_init_with_plain_key](#)
- **function** [ehsm_ecdsa_update](#)
- **function** [ehsm_ecdsa_finish_gen](#)
- **function** [ehsm_ecdsa_finish_verify](#)
- **function** [ehsm_ecdsa_onepass_ex](#)
- **function** [ehsm_ecdsa_onepass_gen](#)
- **function** [ehsm_ecdsa_onepass_verify](#)
- **function** [ehsm_ecdsa_onepass_gen_with_digest](#)
- **function** [ehsm_ecdsa_onepass_verify_with_digest](#)

- **function** [ehsm_ecdsa_onepass_gen_with_plain_key](#)
- **function** [ehsm_ecdsa_onepass_verify_with_plain_key](#)
- **function** [ehsm_ecdsa_onepass_gen_with_plain_key_and_digest](#)
- **function** [ehsm_ecdsa_onepass_verify_with_plain_key_and_digest](#)
- **function** [ehsm_sm9_cipher](#)
- **function** [ehsm_sm9_cipher_with_plain_key](#)
- **function** [ehsm_sm9_sign_onepass_gen](#)
- **function** [ehsm_sm9_sign_onepass_verify](#)
- **function** [ehsm_sm9_sign_onepass_gen_with_plain_key](#)
- **function** [ehsm_chacha_init](#)
- **function** [ehsm_chacha_update](#)
- **function** [ehsm_chacha_finish_enc](#)
- **function** [ehsm_chacha_finish_dec](#)
- **function** [ehsm_chacha_onepass_enc](#)
- **function** [ehsm_chacha_onepass_dec](#)
- **function** [ehsm_chacha_init_with_plain_key](#)
- **function** [ehsm_chacha_onepass_enc_with_plain_key](#)
- **function** [ehsm_chacha_onepass_dec_with_plain_key](#)
- **function** [ehsm_aead_init](#)
- **function** [ehsm_aead_init_with_plain_key](#)
- **function** [ehsm_aead_update](#)
- **function** [ehsm_aead_finish_enc](#)
- **function** [ehsm_aead_finish_dec](#)
- **function** [ehsm_aead_onepass_enc](#)
- **function** [ehsm_aead_onepass_dec](#)
- **function** [ehsm_aead_onepass_enc_with_plain_key](#)
- **function** [ehsm_aead_onepass_dec_with_plain_key](#)
- **function** [ehsm_km_gen_key](#)
- **function** [ehsm_km_import_key](#)
- **function** [ehsm_km_export_key](#)
- **function** [ehsm_km_derive_key](#)
- **function** [ehsm_km_derive_key_to_soc](#)

- **function** [ehsm_km_exchange_key](#)
- **function** [ehsm_km_sm9_exchange_key](#)
- **function** [ehsm_km_get_pub_from_priv](#)
- **function** [ehsm_km_remove_key](#)
- **function** [ehsm_get_version](#)
- **function** [ehsm_verify_image](#)
- **function** [ehsm_upgrade_fw_image_ex](#)
- **function** [ehsm_upgrade_fw_image](#)
- **function** [ehsm_upgrade_fw_image_init](#)
- **function** [ehsm_upgrade_fw_image_update](#)
- **function** [ehsm_upgrade_fw_image_finish](#)
- **function** [ehsm_set_utc_time](#)
- **function** [ehsm_get_utc_time](#)
- **function** [ehsm_create_counter](#)
- **function** [ehsm_read_counter](#)
- **function** [ehsm_increase_counter](#)
- **function** [ehsm_delete_counter](#)
- **function** [ehsm_enter_wfi](#)
- **function** [ehsm_set_uart_baudrate](#)
- **function** [ehsm_install_random_key](#)
- **function** [ehsm_install_encrypted_key](#)
- **function** [ehsm_read_otp](#)
- **function** [ehsm_write_otp](#)
- **function** [ehsm_read_reg](#)
- **function** [ehsm_write_reg](#)
- **function** [ehsm_get_challenge](#)
- **function** [ehsm_debug_auth](#)
- **function** [ehsm_close_debug](#)
- **function** [ehsm_get_emu_status](#)
- **function** [ehsm_change_lifecycle](#)
- **function** [ehsm_change_control_field](#)

8.2.2 File bl_api.h

- **define** [EHSM_SELF_TEST_SM4_ECB](#)
- **define** [EHSM_SELF_TEST_SM4_CBC](#)
- **define** [EHSM_SELF_TEST_SM4_CFB](#)
- **define** [EHSM_SELF_TEST_SM4_OFB](#)
- **define** [EHSM_SELF_TEST_SM4_CTR](#)
- **define** [EHSM_SELF_TEST_SM2_ENC](#)
- **define** [EHSM_SELF_TEST_SM2_VERIFY](#)
- **define** [EHSM_SELF_TEST_SM2_EXCH](#)
- **define** [EHSM_SELF_TEST_SM3](#)
- **define** [EHSM_SELF_TEST_DES](#)
- **define** [EHSM_SELF_TEST_TDES](#)
- **define** [EHSM_SELF_TEST_AES](#)
- **define** [EHSM_SELF_TEST_RSA](#)
- **define** [EHSM_SELF_TEST_ECC](#)
- **define** [EHSM_SELF_TEST_MD5](#)
- **define** [EHSM_SELF_TEST_SHA1](#)
- **define** [EHSM_SELF_TEST_SHA2](#)
- **define** [EHSM_SELF_TEST_SHA3](#)
- **define** [EHSM_SELF_TEST_SHA256](#)
- **define** [EHSM_SELF_TEST_TRNG](#)
- **define** [ehsm_bl_get_version](#)
- **define** [ehsm_bl_verify_image](#)
- **define** [ehsm_bl_upgrade_fw_image](#)
- **define** [ehsm_bl_upgrade_fw_image_init](#)
- **define** [ehsm_bl_upgrade_fw_image_update](#)
- **define** [ehsm_bl_upgrade_fw_image_finish](#)
- **define** [ehsm_bl_set_uart_baudrate](#)
- **define** [ehsm_bl_get_challenge](#)
- **define** [ehsm_bl_debug_auth](#)
- **define** [ehsm_bl_close_debug](#)
- **define** [ehsm_bl_read_reg](#)

- **define** [ehsm_bl_write_reg](#)
- **define** [ehsm_bl_read_otp](#)
- **define** [ehsm_bl_write_otp](#)
- **enum** [ehsm_bl_gen_key_type_e](#)
- **function** [ehsm_bl_self_test](#)
- **function** [ehsm_bl_get_self_test_result](#)
- **function** [ehsm_bl_fw_auth](#)
- **function** [ehsm_bl_get_random_key](#)
- **function** [ehsm_bl_encrypt_key](#)

8.2.3 File `key_assist.h`

- **typedef** [ehsm_key_format_des_plain_st](#)
- **typedef** [ehsm_key_format_des_cipher_st](#)
- **typedef** [ehsm_key_format_aes128_plain_st](#)
- **typedef** [ehsm_key_format_aes128_cipher_st](#)
- **typedef** [ehsm_key_format_tdes128_plain_st](#)
- **typedef** [ehsm_key_format_tdes128_cipher_st](#)
- **typedef** [ehsm_key_format_sm4_plain_st](#)
- **typedef** [ehsm_key_format_sm4_cipher_st](#)
- **typedef** [ehsm_key_format_tdes192_plain_st](#)
- **typedef** [ehsm_key_format_tdes192_cipher_st](#)
- **typedef** [ehsm_key_format_aes192_plain_st](#)
- **typedef** [ehsm_key_format_aes192_cipher_st](#)
- **typedef** [ehsm_key_format_aes256_plain_st](#)
- **typedef** [ehsm_key_format_aes256_cipher_st](#)
- **typedef** [ehsm_key_format_aes128_xts_plain_st](#)
- **typedef** [ehsm_key_format_aes128_xts_cipher_st](#)
- **typedef** [ehsm_key_format_sm4_xts_plain_st](#)
- **typedef** [ehsm_key_format_sm4_xts_cipher_st](#)
- **typedef** [ehsm_key_format_aes192_xts_plain_st](#)
- **typedef** [ehsm_key_format_aes192_xts_cipher_st](#)
- **typedef** [ehsm_key_format_aes256_xts_plain_st](#)

- typedef ehsm_key_format_aes256_xts_cipher_st
- typedef ehsm_key_format_hmac64_plain_st
- typedef ehsm_key_format_hmac64_cipher_st
- typedef ehsm_key_format_hmac128_plain_st
- typedef ehsm_key_format_hmac128_cipher_st
- typedef ehsm_key_format_hmac192_plain_st
- typedef ehsm_key_format_hmac192_cipher_st
- typedef ehsm_key_format_hmac256_plain_st
- typedef ehsm_key_format_hmac256_cipher_st
- typedef ehsm_key_format_hmac384_plain_st
- typedef ehsm_key_format_hmac384_cipher_st
- typedef ehsm_key_format_hmac512_plain_st
- typedef ehsm_key_format_hmac512_cipher_st
- typedef ehsm_key_format_ecc_bp_p160r1_plain_pubkey_st
- typedef ehsm_key_format_ecc_bp_p160r1_plain_privkey_st
- typedef ehsm_key_format_ecc_bp_p160r1_cipher_privkey_st
- typedef ehsm_key_format_ecc_bp_p160r1_plain_keypair_st
- typedef ehsm_key_format_ecc_bp_p160r1_cipher_keypair_st
- typedef ehsm_key_format_ecc_bp_p192r1_plain_pubkey_st
- typedef ehsm_key_format_ecc_bp_p192r1_plain_privkey_st
- typedef ehsm_key_format_ecc_bp_p192r1_cipher_privkey_st
- typedef ehsm_key_format_ecc_bp_p192r1_plain_keypair_st
- typedef ehsm_key_format_ecc_bp_p192r1_cipher_keypair_st
- typedef ehsm_key_format_ecc_bp_p224r1_plain_pubkey_st
- typedef ehsm_key_format_ecc_bp_p224r1_plain_privkey_st
- typedef ehsm_key_format_ecc_bp_p224r1_cipher_privkey_st
- typedef ehsm_key_format_ecc_bp_p224r1_plain_keypair_st
- typedef ehsm_key_format_ecc_bp_p224r1_cipher_keypair_st
- typedef ehsm_key_format_ecc_bp_p256r1_plain_pubkey_st
- typedef ehsm_key_format_ecc_bp_p256r1_plain_privkey_st
- typedef ehsm_key_format_ecc_bp_p256r1_cipher_privkey_st
- typedef ehsm_key_format_ecc_bp_p256r1_plain_keypair_st

- typedef ehsm_key_format_ecc_bp_p256r1_cipher_keypair_st
- typedef ehsm_key_format_ecc_bp_p320r1_plain_pubkey_st
- typedef ehsm_key_format_ecc_bp_p320r1_plain_privkey_st
- typedef ehsm_key_format_ecc_bp_p320r1_cipher_privkey_st
- typedef ehsm_key_format_ecc_bp_p320r1_plain_keypair_st
- typedef ehsm_key_format_ecc_bp_p320r1_cipher_keypair_st
- typedef ehsm_key_format_ecc_bp_p384r1_plain_pubkey_st
- typedef ehsm_key_format_ecc_bp_p384r1_plain_privkey_st
- typedef ehsm_key_format_ecc_bp_p384r1_cipher_privkey_st
- typedef ehsm_key_format_ecc_bp_p384r1_plain_keypair_st
- typedef ehsm_key_format_ecc_bp_p384r1_cipher_keypair_st
- typedef ehsm_key_format_ecc_bp_p512r1_plain_pubkey_st
- typedef ehsm_key_format_ecc_bp_p512r1_plain_privkey_st
- typedef ehsm_key_format_ecc_bp_p512r1_cipher_privkey_st
- typedef ehsm_key_format_ecc_bp_p512r1_plain_keypair_st
- typedef ehsm_key_format_ecc_bp_p512r1_cipher_keypair_st
- typedef ehsm_key_format_ecc_sec_p192r1_plain_pubkey_st
- typedef ehsm_key_format_ecc_sec_p192r1_plain_privkey_st
- typedef ehsm_key_format_ecc_sec_p192r1_cipher_privkey_st
- typedef ehsm_key_format_ecc_sec_p192r1_plain_keypair_st
- typedef ehsm_key_format_ecc_sec_p192r1_cipher_keypair_st
- typedef ehsm_key_format_ecc_sec_p224r1_plain_pubkey_st
- typedef ehsm_key_format_ecc_sec_p224r1_plain_privkey_st
- typedef ehsm_key_format_ecc_sec_p224r1_cipher_privkey_st
- typedef ehsm_key_format_ecc_sec_p224r1_plain_keypair_st
- typedef ehsm_key_format_ecc_sec_p224r1_cipher_keypair_st
- typedef ehsm_key_format_ecc_sec_p256r1_plain_pubkey_st
- typedef ehsm_key_format_ecc_sec_p256r1_plain_privkey_st
- typedef ehsm_key_format_ecc_sec_p256r1_cipher_privkey_st
- typedef ehsm_key_format_ecc_sec_p256r1_plain_keypair_st
- typedef ehsm_key_format_ecc_sec_p256r1_cipher_keypair_st
- typedef ehsm_key_format_ecc_sec_p384r1_plain_pubkey_st

- typedef ehsm_key_format_ecc_sec_p384r1_plain_privkey_st
- typedef ehsm_key_format_ecc_sec_p384r1_cipher_privkey_st
- typedef ehsm_key_format_ecc_sec_p384r1_plain_keypair_st
- typedef ehsm_key_format_ecc_sec_p384r1_cipher_keypair_st
- typedef ehsm_key_format_ecc_sec_p521r1_plain_pubkey_st
- typedef ehsm_key_format_ecc_sec_p521r1_plain_privkey_st
- typedef ehsm_key_format_ecc_sec_p521r1_cipher_privkey_st
- typedef ehsm_key_format_ecc_sec_p521r1_plain_keypair_st
- typedef ehsm_key_format_ecc_sec_p521r1_cipher_keypair_st
- typedef ehsm_key_format_ecc_sec_p160k1_plain_pubkey_st
- typedef ehsm_key_format_ecc_sec_p160k1_plain_privkey_st
- typedef ehsm_key_format_ecc_sec_p160k1_cipher_privkey_st
- typedef ehsm_key_format_ecc_sec_p160k1_plain_keypair_st
- typedef ehsm_key_format_ecc_sec_p160k1_cipher_keypair_st
- typedef ehsm_key_format_ecc_sec_p192k1_plain_pubkey_st
- typedef ehsm_key_format_ecc_sec_p192k1_plain_privkey_st
- typedef ehsm_key_format_ecc_sec_p192k1_cipher_privkey_st
- typedef ehsm_key_format_ecc_sec_p192k1_plain_keypair_st
- typedef ehsm_key_format_ecc_sec_p192k1_cipher_keypair_st
- typedef ehsm_key_format_ecc_sec_p224k1_plain_pubkey_st
- typedef ehsm_key_format_ecc_sec_p224k1_plain_privkey_st
- typedef ehsm_key_format_ecc_sec_p224k1_cipher_privkey_st
- typedef ehsm_key_format_ecc_sec_p224k1_plain_keypair_st
- typedef ehsm_key_format_ecc_sec_p224k1_cipher_keypair_st
- typedef ehsm_key_format_ecc_sec_p256k1_plain_pubkey_st
- typedef ehsm_key_format_ecc_sec_p256k1_plain_privkey_st
- typedef ehsm_key_format_ecc_sec_p256k1_cipher_privkey_st
- typedef ehsm_key_format_ecc_sec_p256k1_plain_keypair_st
- typedef ehsm_key_format_ecc_sec_p256k1_cipher_keypair_st

8.2.4 File mailbox.h

- typedef ehsm_mb_interrupt_func_t

- function [ehsm_mb_init](#)
- function [ehsm_mb_send](#)
- function [ehsm_mb_poll](#)

8.2.5 File `ehsm_host_port.h`

- define [ehsm_port_printf](#)
- enum [ehsm_reg_e](#)
- enumvalue [REG_HSM_STATUS_0](#)
- enumvalue [REG_HSM_STATUS_1](#)
- enumvalue [REG_HSM_ERR_SENSOR](#)
- enumvalue [REG_HSM_ERR_HW_0](#)
- enumvalue [REG_HSM_ERR_HW_1](#)
- enumvalue [REG_HSM_ERR_FW_0](#)
- enumvalue [REG_HSM_ERR_FW_1](#)
- enumvalue [REG_HSM_FUSA_ALARM_0](#)
- enumvalue [REG_HSM_FUSA_ALARM_1](#)
- enumvalue [REG_SOC_DBG_EN_0](#)
- enumvalue [REG_SOC_DBG_EN_1](#)
- enumvalue [REG_SOC_DBG_EN_2](#)
- enumvalue [REG_SOC_DBG_EN_3](#)
- enumvalue [REG_SOC_SENSOR](#)
- typedef [ehsm_mailbox_isr_f](#)
- typedef [ehsm_port_timer_t](#)
- function [ehsm_port_init](#)
- function [ehsm_port_addr_to_raddr](#)
- function [ehsm_port_raddr_to_addr](#)
- function [ehsm_port_flush_and_invalidate_cache](#)
- function [ehsm_port_enable_mailbox_int](#)
- function [ehsm_port_reset_ehsm](#)
- function [ehsm_port_read_reg](#)
- function [ehsm_port_write_reg](#)
- function [ehsm_port_read_otp](#)

- function [ehsm_port_write_otp](#)
- function [ehsm_port_create_timer](#)
- function [ehsm_port_is_timeout](#)
- function [ehsm_port_get_time_ms](#)
- function [ehsm_port_memory_barrier_write](#)
- function [ehsm_port_memory_barrier_read](#)
- function [ehsm_port_enter_critical](#)
- function [ehsm_port_exit_critical](#)

8.2.6 File types.h

- define [NULL](#)
- define [EHSM_OK](#)
- define [EHSM_ERR_PARAM_ERROR](#)
- define [EHSM_ERR_BUSY](#)
- define [EHSM_ERR_CTX_NOT_INIT](#)
- define [EHSM_ERR_CTX_STATUS](#)
- define [EHSM_ERR_TIMEOUT](#)
- define [EHSM_ERR_CTX_INVALID](#)
- define [EHSM_ERR_ILLEGAL_USE](#)
- define [EHSM_ERR_DEV_ERROR](#)
- define [EHSM_ERR_FW_BASE](#)
- define [EHSM_ERR_NEED_POLL](#)
- define [EHSM_SHM](#)
- typedef [bool_t](#)
- typedef [raddr_t](#)

8.2.7 File version.h

- define [EHSM_DRV_VERSION_MAJOR](#)
- define [EHSM_DRV_VERSION_MINOR](#)
- define [EHSM_DRV_VERSION_PATCH](#)
- define [EHSM_DRV_VERSION_PRE_RELEASE](#)

8.3 Structs

8.3.1 ehsm_ctx_st

为 eHSM 的功能调用提供上下文环境。

成员:

- `uint32_t _data[56]`: 内部维护数据

8.3.2 ehsm_dh_params_1024_st

1024 位 DH 参数, 参数值以大端形式存储在 value 数组中

成员:

- `uint32_t p_len`: 参数 P 的长度, 应当固定为 128
- `uint8_t p_value[128]`: 参数 P 的值
- `uint32_t q_len`: 参数 Q 的长度, 应当固定为 128
- `uint8_t q_value[128]`: 参数 Q 的值
- `uint32_t g_len`: 参数 G 的长度, 应当固定为 128
- `uint8_t g_value[128]`: 参数 G 的值
- `uint32_t h_len`: 参数 H 的长度, 此参数用于加速计算, 可以设为 0, 不提供此参数, 或提供时应当固定为 128
- `uint8_t h_value[128]`: 参数 H 的值

8.3.3 ehsm_dh_params_2048_st

2048 位 DH 参数, 参数值以大端形式存储在 value 数组中

成员:

- `uint32_t p_len`: 参数 P 的长度, 应当固定为 256
- `uint8_t p_value[256]`: 参数 P 的值
- `uint32_t q_len`: 参数 Q 的长度, 应当固定为 256
- `uint8_t q_value[256]`: 参数 Q 的值
- `uint32_t g_len`: 参数 G 的长度, 应当固定为 256
- `uint8_t g_value[256]`: 参数 G 的值
- `uint32_t h_len`: 参数 H 的长度, 此参数用于加速计算, 可以设为 0, 不提供此参数, 或提供时应当固定为 256
- `uint8_t h_value[256]`: 参数 H 的值

8.3.4 ehsm_dh_params_3072_st

3072 位 DH 参数, 参数值以大端形式存储在 value 数组中

成员:

- `uint32_t p_len`: 参数 P 的长度, 应当固定为 384
- `uint8_t p_value[384]`: 参数 P 的值
- `uint32_t q_len`: 参数 Q 的长度, 应当固定为 384
- `uint8_t q_value[384]`: 参数 Q 的值
- `uint32_t g_len`: 参数 G 的长度, 应当固定为 384
- `uint8_t g_value[384]`: 参数 G 的值
- `uint32_t h_len`: 参数 H 的长度, 此参数用于加速计算, 可以设为 0, 不提供此参数, 或提供时应当固定为 384
- `uint8_t h_value[384]`: 参数 H 的值

8.3.5 ehsm_dh_params_4096_st

4096 位 DH 参数, 参数值以大端形式存储在 value 数组中

成员:

- `uint32_t p_len`: 参数 P 的长度, 应当固定为 512
- `uint8_t p_value[512]`: 参数 P 的值
- `uint32_t q_len`: 参数 Q 的长度, 应当固定为 512
- `uint8_t q_value[512]`: 参数 Q 的值
- `uint32_t g_len`: 参数 G 的长度, 应当固定为 512
- `uint8_t g_value[512]`: 参数 G 的值
- `uint32_t h_len`: 参数 H 的长度, 此参数用于加速计算, 可以设为 0, 不提供此参数, 或提供时应当固定为 512
- `uint8_t h_value[512]`: 参数 H 的值

8.3.6 ehsm_ecc_key_st

ECC 明文密钥数据结构

成员:

- `uint32_t curve_id`: ECC 曲线对应的密钥编号, 见 `ehsm_key_type_e`
- `EHSM_SHM raddr_t privkey`: ECC 私钥的存储地址, 长度由 `curve_id` 对应的曲线类型决定
- `EHSM_SHM raddr_t pubkey`: ECC 公钥的存储地址, 长度由 `curve_id` 对应的曲线类型决定, 不包含前缀 04

8.3.7 ehsm_emu_status_st

eHSM EMU 状态信息结构体。

成员:

- `uint32_t o_hsm_status[2]`: eHSM 状态
- `uint32_t o_hsm_err_sensor`: 传感器检测到的错误
- `uint32_t o_hsm_err_hw[2]`: 硬件检测到的错误
- `uint32_t o_hsm_err_fw[2]`: 固件检测到的错误

8.3.8 ehsm_key_format_dh_1024_cipher_keypair_st

用于辅助 DH 1024 位密钥对密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 EHSM_KEY_TYPE_DH
- `uint8_t part_info`: 应当设置为 EHSM_KEY_PART_KEY_PAIR
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 128
- `uint16_t priv_key_size`: 私钥长度, 应为 128
- `uint8_t pub_value[128]`: 公钥值明文, 128 字节
- `uint8_t priv_value[144]`: 私钥值密文, 有 PKCS7 填充, 144 字节

8.3.9 ehsm_key_format_dh_1024_cipher_privkey_st

用于辅助 DH 1024 位私钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 EHSM_KEY_TYPE_DH
- `uint8_t part_info`: 应当设置为 EHSM_KEY_PART_PRIVATE_KEY
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 128
- `uint8_t priv_value[144]`: 私钥值密文, 有 PKCS7 填充, 144 字节

8.3.10 ehsm_key_format_dh_1024_plain_keypair_st

用于辅助 DH 1024 位密钥对明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 EHSM_KEY_TYPE_DH

- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 128
- `uint16_t priv_key_size`: 私钥长度, 应为 128
- `uint8_t pub_value[128]`: 公钥值明文, 128 字节
- `uint8_t priv_value[128]`: 私钥值明文, 128 字节

8.3.11 ehsm_key_format_dh_1024_plain_privkey_st

用于辅助 DH 1024 位私钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_DH`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 128
- `uint8_t priv_value[128]`: 私钥值明文, 128 字节

8.3.12 ehsm_key_format_dh_1024_plain_pubkey_st

用于辅助 DH 1024 位公钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_DH`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PUBLIC_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 128
- `uint16_t priv_key_size`: 为 0
- `uint8_t pub_value[128]`: 公钥值明文, 128 字节

8.3.13 ehsm_key_format_dh_2048_cipher_keypair_st

用于辅助 DH 2048 位密钥对密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限

- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_DH`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 256
- `uint16_t priv_key_size`: 私钥长度, 应为 256
- `uint8_t pub_value[256]`: 公钥值明文, 256 字节
- `uint8_t priv_value[272]`: 私钥值密文, 有 PKCS7 填充, 272 字节

8.3.14 ehsm_key_format_dh_2048_cipher_privkey_st

用于辅助 DH 2048 位私钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_DH`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 256
- `uint8_t priv_value[272]`: 私钥值密文, 有 PKCS7 填充, 272 字节

8.3.15 ehsm_key_format_dh_2048_plain_keypair_st

用于辅助 DH 2048 位密钥对明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_DH`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 256
- `uint16_t priv_key_size`: 私钥长度, 应为 256
- `uint8_t pub_value[256]`: 公钥值明文, 256 字节
- `uint8_t priv_value[256]`: 私钥值明文, 256 字节

8.3.16 ehsm_key_format_dh_2048_plain_privkey_st

用于辅助 DH 2048 位私钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_DH`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 256
- `uint8_t priv_value[256]`: 私钥值明文, 256 字节

8.3.17 ehsm_key_format_dh_2048_plain_pubkey_st

用于辅助 DH 2048 位公钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_DH`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PUBLIC_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 256
- `uint16_t priv_key_size`: 为 0
- `uint8_t pub_value[256]`: 公钥值明文, 256 字节

8.3.18 ehsm_key_format_dh_3072_cipher_keypair_st

用于辅助 DH 3072 位密钥对密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_DH`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 384
- `uint16_t priv_key_size`: 私钥长度, 应为 384
- `uint8_t pub_value[384]`: 公钥值明文, 384 字节
- `uint8_t priv_value[400]`: 私钥值密文, 有 PKCS7 填充, 400 字节

8.3.19 ehsm_key_format_dh_3072_cipher_privkey_st

用于辅助 DH 3072 位私钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_DH`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 384
- `uint8_t priv_value[400]`: 私钥值密文, 有 PKCS7 填充, 400 字节

8.3.20 ehsm_key_format_dh_3072_plain_keypair_st

用于辅助 DH 3072 位密钥对明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_DH`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 384
- `uint16_t priv_key_size`: 私钥长度, 应为 384
- `uint8_t pub_value[384]`: 公钥值明文, 384 字节
- `uint8_t priv_value[384]`: 私钥值明文, 384 字节

8.3.21 ehsm_key_format_dh_3072_plain_privkey_st

用于辅助 DH 3072 位私钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_DH`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 384
- `uint8_t priv_value[384]`: 私钥值明文, 384 字节

8.3.22 ehsm_key_format_dh_3072_plain_pubkey_st

用于辅助 DH 3072 位公钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_DH`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PUBLIC_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 384
- `uint16_t priv_key_size`: 为 0
- `uint8_t pub_value[384]`: 公钥值明文, 384 字节

8.3.23 ehsm_key_format_dh_4096_cipher_keypair_st

用于辅助 DH 4096 位密钥对密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_DH`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 512
- `uint16_t priv_key_size`: 私钥长度, 应为 512
- `uint8_t pub_value[512]`: 公钥值明文, 512 字节
- `uint8_t priv_value[528]`: 私钥值密文, 有 PKCS7 填充, 528 字节

8.3.24 ehsm_key_format_dh_4096_cipher_privkey_st

用于辅助 DH 4096 位私钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_DH`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 512
- `uint8_t priv_value[528]`: 私钥值密文, 有 PKCS7 填充, 528 字节

8.3.25 ehsm_key_format_dh_4096_plain_keypair_st

用于辅助 DH 4096 位密钥对明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_DH`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 512
- `uint16_t priv_key_size`: 私钥长度, 应为 512
- `uint8_t pub_value[512]`: 公钥值明文, 512 字节
- `uint8_t priv_value[512]`: 私钥值明文, 512 字节

8.3.26 ehsm_key_format_dh_4096_plain_privkey_st

用于辅助 DH 4096 位私钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_DH`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 512
- `uint8_t priv_value[512]`: 私钥值明文, 512 字节

8.3.27 ehsm_key_format_dh_4096_plain_pubkey_st

用于辅助 DH 4096 位公钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_DH`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PUBLIC_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 512
- `uint16_t priv_key_size`: 为 0
- `uint8_t pub_value[512]`: 公钥值明文, 512 字节

8.3.28 ehsm_key_format_ecc_160_cipher_keypair_st

用于辅助 ECC160 密钥对密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 160 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 40
- `uint16_t priv_key_size`: 私钥长度, 应为 20
- `uint8_t x_value[20]`: 公钥 X 值明文, 20 字节
- `uint8_t y_value[20]`: 公钥 Y 值明文, 20 字节
- `uint8_t priv_value[32]`: 私钥密文值, 有 PKCS7 填充, 32 字节

8.3.29 ehsm_key_format_ecc_160_cipher_privkey_st

用于辅助 ECC160 私钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 160 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 20
- `uint8_t priv_value[32]`: 私钥密文值, 有 PKCS7 填充, 32 字节

8.3.30 ehsm_key_format_ecc_160_plain_keypair_st

用于辅助 ECC160 密钥对明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 160 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 40
- `uint16_t priv_key_size`: 私钥长度, 应为 20
- `uint8_t x_value[20]`: 公钥 X 值明文, 20 字节
- `uint8_t y_value[20]`: 公钥 Y 值明文, 20 字节
- `uint8_t priv_value[20]`: 私钥明文值, 20 字节

8.3.31 ehsm_key_format_ecc_160_plain_privkey_st

用于辅助 ECC160 私钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 160 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 20
- `uint8_t priv_value[20]`: 私钥明文值, 20 字节

8.3.32 ehsm_key_format_ecc_160_plain_pubkey_st

用于辅助 ECC160 公钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 160 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PUBLIC_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应当设置为 40
- `uint16_t priv_key_size`: 为 0
- `uint8_t x_value[20]`: 公钥 X 值明文, 20 字节
- `uint8_t y_value[20]`: 公钥 Y 值明文, 20 字节

8.3.33 ehsm_key_format_ecc_192_cipher_keypair_st

用于辅助 ECC192 密钥对密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 192 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 48
- `uint16_t priv_key_size`: 私钥长度, 应为 24
- `uint8_t x_value[24]`: 公钥 X 值明文, 24 字节

- `uint8_t y_value[24]`: 公钥 Y 值明文, 24 字节
- `uint8_t priv_value[32]`: 私钥密文值, 有 PKCS7 填充, 32 字节

8.3.34 ehsm_key_format_ecc_192_cipher_privkey_st

用于辅助 ECC192 私钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 192 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 24
- `uint8_t priv_value[32]`: 私钥密文值, 有 PKCS7 填充, 32 字节

8.3.35 ehsm_key_format_ecc_192_plain_keypair_st

用于辅助 ECC192 密钥对明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 192 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 48
- `uint16_t priv_key_size`: 私钥长度, 应为 24
- `uint8_t x_value[24]`: 公钥 X 值明文, 24 字节
- `uint8_t y_value[24]`: 公钥 Y 值明文, 24 字节
- `uint8_t priv_value[24]`: 私钥明文值, 24 字节

8.3.36 ehsm_key_format_ecc_192_plain_privkey_st

用于辅助 ECC192 私钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 192 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`

- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 24
- `uint8_t priv_value[24]`: 私钥明文值, 24 字节

8.3.37 ehsm_key_format_ecc_192_plain_pubkey_st

用于辅助 ECC192 公钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 192 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PUBLIC_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应当设置为 48
- `uint16_t priv_key_size`: 为 0
- `uint8_t x_value[24]`: 公钥 X 值明文, 24 字节
- `uint8_t y_value[24]`: 公钥 Y 值明文, 24 字节

8.3.38 ehsm_key_format_ecc_224_cipher_keypair_st

用于辅助 ECC224 密钥对密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 224 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 56
- `uint16_t priv_key_size`: 私钥长度, 应为 28
- `uint8_t x_value[28]`: 公钥 X 值明文, 28 字节
- `uint8_t y_value[28]`: 公钥 Y 值明文, 28 字节
- `uint8_t priv_value[32]`: 私钥密文值, 有 PKCS7 填充, 32 字节

8.3.39 ehsm_key_format_ecc_224_cipher_privkey_st

用于辅助 ECC224 私钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 224 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 28
- `uint8_t priv_value[32]`: 私钥密文值, 有 PKCS7 填充, 32 字节

8.3.40 `ehsm_key_format_ecc_224_plain_keypair_st`

用于辅助 ECC224 密钥对明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 224 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 56
- `uint16_t priv_key_size`: 私钥长度, 应为 28
- `uint8_t x_value[28]`: 公钥 X 值明文, 28 字节
- `uint8_t y_value[28]`: 公钥 Y 值明文, 28 字节
- `uint8_t priv_value[28]`: 私钥明文值, 28 字节

8.3.41 `ehsm_key_format_ecc_224_plain_privkey_st`

用于辅助 ECC224 私钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 224 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 28
- `uint8_t priv_value[28]`: 私钥明文值, 28 字节

8.3.42 `ehsm_key_format_ecc_224_plain_pubkey_st`

用于辅助 ECC224 公钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 224 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PUBLIC_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应当设置为 56
- `uint16_t priv_key_size`: 为 0
- `uint8_t x_value[28]`: 公钥 X 值明文, 28 字节
- `uint8_t y_value[28]`: 公钥 Y 值明文, 28 字节

8.3.43 ehsm_key_format_ecc_25519_cipher_keypair_st

用于辅助 ED25519/X25519 密钥对密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 ED25519/X25519 类型, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 32
- `uint16_t priv_key_size`: 私钥长度, 应为 32
- `uint8_t pub_value[32]`: 公钥值明文, 32 字节
- `uint8_t priv_value[48]`: 私钥密文值, 有 PKCS7 填充, 48 字节

8.3.44 ehsm_key_format_ecc_25519_cipher_privkey_st

用于辅助 ED25519/X25519 私钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 ED25519/X25519 类型, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 32
- `uint8_t priv_value[48]`: 私钥密文值, 有 PKCS7 填充, 48 字节

8.3.45 ehsm_key_format_ecc_25519_plain_keypair_st

用于辅助 ED25519/X25519 密钥对明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 ED25519/X25519 类型, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 32
- `uint16_t priv_key_size`: 私钥长度, 应为 32
- `uint8_t pub_value[32]`: 公钥值明文, 32 字节
- `uint8_t priv_value[32]`: 私钥明文值, 32 字节

8.3.46 ehsm_key_format_ecc_25519_plain_privkey_st

用于辅助 ED25519/X25519 私钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 ED25519/X25519 类型, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 32
- `uint8_t priv_value[32]`: 私钥明文值, 32 字节

8.3.47 ehsm_key_format_ecc_25519_plain_pubkey_st

用于辅助 ED25519/X25519 公钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 ED25519/X25519 类型, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PUBLIC_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应当设置为 32
- `uint16_t priv_key_size`: 为 0
- `uint8_t pub_value[32]`: 公钥值明文, 32 字节

8.3.48 ehsm_key_format_ecc_256_cipher_keypair_st

用于辅助 ECC256 密钥对密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 256 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 64
- `uint16_t priv_key_size`: 私钥长度, 应为 32
- `uint8_t x_value[32]`: 公钥 X 值明文, 32 字节
- `uint8_t y_value[32]`: 公钥 Y 值明文, 32 字节
- `uint8_t priv_value[48]`: 私钥密文值, 有 PKCS7 填充, 48 字节

8.3.49 ehsm_key_format_ecc_256_cipher_privkey_st

用于辅助 ECC256 私钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 256 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 32
- `uint8_t priv_value[48]`: 私钥密文值, 有 PKCS7 填充, 48 字节

8.3.50 ehsm_key_format_ecc_256_plain_keypair_st

用于辅助 ECC256 密钥对明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 256 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 64
- `uint16_t priv_key_size`: 私钥长度, 应为 32

- `uint8_t x_value[32]`: 公钥 X 值明文, 32 字节
- `uint8_t y_value[32]`: 公钥 Y 值明文, 32 字节
- `uint8_t priv_value[32]`: 私钥明文值, 32 字节

8.3.51 ehsm_key_format_ecc_256_plain_privkey_st

用于辅助 ECC256 私钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 256 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 32
- `uint8_t priv_value[32]`: 私钥明文值, 32 字节

8.3.52 ehsm_key_format_ecc_256_plain_pubkey_st

用于辅助 ECC256 公钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 256 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PUBLIC_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应当设置为 64
- `uint16_t priv_key_size`: 为 0
- `uint8_t x_value[32]`: 公钥 X 值明文, 32 字节
- `uint8_t y_value[32]`: 公钥 Y 值明文, 32 字节

8.3.53 ehsm_key_format_ecc_320_cipher_keypair_st

用于辅助 ECC320 密钥对密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 320 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`

- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 80
- `uint16_t priv_key_size`: 私钥长度, 应为 40
- `uint8_t x_value[40]`: 公钥 X 值明文, 40 字节
- `uint8_t y_value[40]`: 公钥 Y 值明文, 40 字节
- `uint8_t priv_value[48]`: 私钥密文值, 有 PKCS7 填充, 48 字节

8.3.54 ehsm_key_format_ecc_320_cipher_privkey_st

用于辅助 ECC320 私钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 320 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 40
- `uint8_t priv_value[48]`: 私钥密文值, 有 PKCS7 填充, 48 字节

8.3.55 ehsm_key_format_ecc_320_plain_keypair_st

用于辅助 ECC320 密钥对明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 320 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 80
- `uint16_t priv_key_size`: 私钥长度, 应为 40
- `uint8_t x_value[40]`: 公钥 X 值明文, 40 字节
- `uint8_t y_value[40]`: 公钥 Y 值明文, 40 字节
- `uint8_t priv_value[40]`: 私钥明文值, 40 字节

8.3.56 ehsm_key_format_ecc_320_plain_privkey_st

用于辅助 ECC320 私钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 320 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 40
- `uint8_t priv_value[40]`: 私钥明文值, 40 字节

8.3.57 `ehsm_key_format_ecc_320_plain_pubkey_st`

用于辅助 ECC320 公钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 320 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PUBLIC_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应当设置为 80
- `uint16_t priv_key_size`: 为 0
- `uint8_t x_value[40]`: 公钥 X 值明文, 40 字节
- `uint8_t y_value[40]`: 公钥 Y 值明文, 40 字节

8.3.58 `ehsm_key_format_ecc_384_cipher_keypair_st`

用于辅助 ECC384 密钥对密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 384 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 96
- `uint16_t priv_key_size`: 私钥长度, 应为 48
- `uint8_t x_value[48]`: 公钥 X 值明文, 48 字节
- `uint8_t y_value[48]`: 公钥 Y 值明文, 48 字节
- `uint8_t priv_value[64]`: 私钥密文值, 有 PKCS7 填充, 64 字节

8.3.59 ehsm_key_format_ecc_384_cipher_privkey_st

用于辅助 ECC384 私钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 384 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 48
- `uint8_t priv_value[64]`: 私钥密文值, 有 PKCS7 填充, 64 字节

8.3.60 ehsm_key_format_ecc_384_plain_keypair_st

用于辅助 ECC384 密钥对明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 384 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 96
- `uint16_t priv_key_size`: 私钥长度, 应为 48
- `uint8_t x_value[48]`: 公钥 X 值明文, 48 字节
- `uint8_t y_value[48]`: 公钥 Y 值明文, 48 字节
- `uint8_t priv_value[48]`: 私钥明文值, 48 字节

8.3.61 ehsm_key_format_ecc_384_plain_privkey_st

用于辅助 ECC384 私钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 384 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 48

- `uint8_t priv_value[48]`: 私钥明文值, 48 字节

8.3.62 ehsm_key_format_ecc_384_plain_pubkey_st

用于辅助 ECC384 公钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 384 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PUBLIC_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应当设置为 96
- `uint16_t priv_key_size`: 为 0
- `uint8_t x_value[48]`: 公钥 X 值明文, 48 字节
- `uint8_t y_value[48]`: 公钥 Y 值明文, 48 字节

8.3.63 ehsm_key_format_ecc_512_cipher_keypair_st

用于辅助 ECC512 密钥对密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 512 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 128
- `uint16_t priv_key_size`: 私钥长度, 应为 64
- `uint8_t x_value[64]`: 公钥 X 值明文, 64 字节
- `uint8_t y_value[64]`: 公钥 Y 值明文, 64 字节
- `uint8_t priv_value[80]`: 私钥密文值, 有 PKCS7 填充, 80 字节

8.3.64 ehsm_key_format_ecc_512_cipher_privkey_st

用于辅助 ECC512 私钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 512 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`

- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 64
- `uint8_t priv_value[80]`: 私钥密文值, 有 PKCS7 填充, 80 字节

8.3.65 ehsm_key_format_ecc_512_plain_keypair_st

用于辅助 ECC512 密钥对明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 512 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 128
- `uint16_t priv_key_size`: 私钥长度, 应为 64
- `uint8_t x_value[64]`: 公钥 X 值明文, 64 字节
- `uint8_t y_value[64]`: 公钥 Y 值明文, 64 字节
- `uint8_t priv_value[64]`: 私钥明文值, 64 字节

8.3.66 ehsm_key_format_ecc_512_plain_privkey_st

用于辅助 ECC512 私钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 512 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 64
- `uint8_t priv_value[64]`: 私钥明文值, 64 字节

8.3.67 ehsm_key_format_ecc_512_plain_pubkey_st

用于辅助 ECC512 公钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限

- `uint8_t key_type`: 密钥类型, 应当设置为 512 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PUBLIC_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应当设置为 128
- `uint16_t priv_key_size`: 为 0
- `uint8_t x_value[64]`: 公钥 X 值明文, 64 字节
- `uint8_t y_value[64]`: 公钥 Y 值明文, 64 字节

8.3.68 `ehsm_key_format_ecc_521_cipher_keypair_st`

用于辅助 ECC521 密钥对密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 521 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 132
- `uint16_t priv_key_size`: 私钥长度, 应为 66
- `uint8_t x_value[66]`: 公钥 X 值明文, 66 字节
- `uint8_t y_value[66]`: 公钥 Y 值明文, 66 字节
- `uint8_t priv_value[80]`: 私钥密文值, 有 PKCS7 填充, 80 字节

8.3.69 `ehsm_key_format_ecc_521_cipher_privkey_st`

用于辅助 ECC521 私钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 521 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 66
- `uint8_t priv_value[80]`: 私钥密文值, 有 PKCS7 填充, 80 字节

8.3.70 `ehsm_key_format_ecc_521_plain_keypair_st`

用于辅助 ECC521 密钥对明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 521 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 132
- `uint16_t priv_key_size`: 私钥长度, 应为 66
- `uint8_t x_value[66]`: 公钥 X 值明文, 66 字节
- `uint8_t y_value[66]`: 公钥 Y 值明文, 66 字节
- `uint8_t priv_value[66]`: 私钥明文值, 66 字节

8.3.71 ehsm_key_format_ecc_521_plain_privkey_st

用于辅助 ECC521 私钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 521 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 66
- `uint8_t priv_value[66]`: 私钥明文值, 66 字节

8.3.72 ehsm_key_format_ecc_521_plain_pubkey_st

用于辅助 ECC521 公钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 521 位的 ECC 曲线参数, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PUBLIC_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应当设置为 132
- `uint16_t priv_key_size`: 为 0
- `uint8_t x_value[66]`: 公钥 X 值明文, 66 字节
- `uint8_t y_value[66]`: 公钥 Y 值明文, 66 字节

8.3.73 ehsm_key_format_rsa1024_cipher_keypair_st

用于辅助 RSA1024 密钥对密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_1024`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: E 值长度, 应当设置为 8
- `uint16_t priv_key_size`: D 值明文长度, 这里应当设置为 128
- `uint8_t e_value[8]`: E 值明文, 8 字节
- `uint8_t n_value[128]`: N 值明文, 128 字节
- `uint8_t d_value[144]`: D 值密文, 144 字节

8.3.74 ehsm_key_format_rsa1024_cipher_privkey_st

用于辅助 RSA1024 私钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_1024`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: D 值明文长度, 这里应当设置为 128
- `uint8_t n_value[128]`: N 值明文, 128 字节
- `uint8_t d_value[144]`: D 值密文, 进行了 PKCS7 填充, 因此是 144 字节

8.3.75 ehsm_key_format_rsa1024 crt_cipher_keypair_st

用于辅助 RSA1024-CRT 密钥对密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_1024`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: E 值长度, 应当设置为 8

- `uint16_t priv_key_size`: 5 分量明文长度, 这里应当设置为 320
- `uint8_t e_value[8]`: E 值明文, 8 字节
- `uint8_t n_value[128]`: N 值明文, 128 字节
- `uint8_t p_value[64]`: P 值密文, 64 字节
- `uint8_t q_value[64]`: Q 值密文, 64 字节
- `uint8_t dp_value[64]`: DP 值密文, 64 字节
- `uint8_t dq_value[64]`: DQ 值密文, 64 字节
- `uint8_t u_value[64]`: U 值密文, 64 字节
- `uint8_t padding[16]`: PKCS7 填充后的密文值

8.3.76 ehsm_key_format_rsa1024_crt_cipher_privkey_st

用于辅助 RSA1024-CRT 私钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_1024`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 5 分量明文长度, 这里应当设置为 320
- `uint8_t p_value[64]`: P 值密文, 64 字节
- `uint8_t q_value[64]`: Q 值密文, 64 字节
- `uint8_t dp_value[64]`: DP 值密文, 64 字节
- `uint8_t dq_value[64]`: DQ 值密文, 64 字节
- `uint8_t u_value[64]`: U 值密文, 64 字节
- `uint8_t padding[16]`: PKCS7 填充后的密文值

8.3.77 ehsm_key_format_rsa1024_crt_plain_keypair_st

用于辅助 RSA1024-CRT 密钥对明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_1024`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留

- `uint16_t pub_key_size`: E 值长度, 应当设置为 8
- `uint16_t priv_key_size`: 5 分量明文长度, 这里应当设置为 320
- `uint8_t e_value[8]`: E 值明文, 8 字节
- `uint8_t n_value[128]`: N 值明文, 128 字节
- `uint8_t p_value[64]`: P 值明文, 64 字节
- `uint8_t q_value[64]`: Q 值明文, 64 字节
- `uint8_t dp_value[64]`: DP 值明文, 64 字节
- `uint8_t dq_value[64]`: DQ 值明文, 64 字节
- `uint8_t u_value[64]`: U 值明文, 64 字节

8.3.78 ehsm_key_format_rsa1024_crt_plain_privkey_st

用于辅助 RSA1024-CRT 私钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_1024`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 5 分量明文长度, 这里应当设置为 320
- `uint8_t p_value[64]`: P 值明文, 64 字节
- `uint8_t q_value[64]`: Q 值明文, 64 字节
- `uint8_t dp_value[64]`: DP 值明文, 64 字节
- `uint8_t dq_value[64]`: DQ 值明文, 64 字节
- `uint8_t u_value[64]`: U 值明文, 64 字节

8.3.79 ehsm_key_format_rsa1024_plain_keypair_st

用于辅助 RSA1024 密钥对明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_1024`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: E 值长度, 应当设置为 8

- `uint16_t priv_key_size`: D 值明文长度, 这里应当设置为 128
- `uint8_t e_value[8]`: E 值明文, 8 字节
- `uint8_t n_value[128]`: N 值明文, 128 字节
- `uint8_t d_value[128]`: D 值明文, 128 字节

8.3.80 ehsm_key_format_rsa1024_plain_privkey_st

用于辅助 RSA1024 私钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 EHSM_KEY_TYPE_RSA_1024
- `uint8_t part_info`: 应当设置为 EHSM_KEY_PART_PRIVATE_KEY
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: D 值明文长度, 这里应当设置为 128
- `uint8_t n_value[128]`: N 值明文
- `uint8_t d_value[128]`: D 值明文

8.3.81 ehsm_key_format_rsa1024_plain_pubkey_st

用于辅助 RSA1024 公钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 EHSM_KEY_TYPE_RSA_1024
- `uint8_t part_info`: 应当设置为 EHSM_KEY_PART_PUBLIC_KEY
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 应当设置为 8
- `uint16_t priv_key_size`: 为 0
- `uint8_t e_value[8]`: E 值
- `uint8_t n_value[128]`: N 值

8.3.82 ehsm_key_format_rsa2048_cipher_keypair_st

用于辅助 RSA2048 密钥对密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限

- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_2048`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: E 值长度, 应当设置为 8
- `uint16_t priv_key_size`: D 值明文长度, 这里应当设置为 256
- `uint8_t e_value[8]`: E 值明文, 8 字节
- `uint8_t n_value[256]`: N 值明文, 256 字节
- `uint8_t d_value[272]`: D 值密文, 272 字节

8.3.83 ehsm_key_format_rsa2048_cipher_privkey_st

用于辅助 RSA2048 私钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_2048`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: D 值明文长度, 这里应当设置为 256
- `uint8_t n_value[256]`: N 值明文, 256 字节
- `uint8_t d_value[272]`: D 值密文, 进行了 PKCS7 填充, 因此是 272 字节

8.3.84 ehsm_key_format_rsa2048 crt_cipher_keypair_st

用于辅助 RSA2048-CRT 密钥对密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_2048`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: E 值长度, 应当设置为 8
- `uint16_t priv_key_size`: 5 分量明文长度, 这里应当设置为 640
- `uint8_t e_value[8]`: E 值明文, 8 字节
- `uint8_t n_value[256]`: N 值明文, 256 字节
- `uint8_t p_value[128]`: P 值密文, 128 字节

- `uint8_t q_value[128]`: Q 值密文, 128 字节
- `uint8_t dp_value[128]`: DP 值密文, 128 字节
- `uint8_t dq_value[128]`: DQ 值密文, 128 字节
- `uint8_t u_value[128]`: U 值密文, 128 字节
- `uint8_t padding[16]`: PKCS7 填充后的密文值

8.3.85 ehsm_key_format_rsa2048_crt_cipher_privkey_st

用于辅助 RSA2048-CRT 私钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_2048`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 5 分量明文长度, 这里应当设置为 640
- `uint8_t p_value[128]`: P 值密文, 128 字节
- `uint8_t q_value[128]`: Q 值密文, 128 字节
- `uint8_t dp_value[128]`: DP 值密文, 128 字节
- `uint8_t dq_value[128]`: DQ 值密文, 128 字节
- `uint8_t u_value[128]`: U 值密文, 128 字节
- `uint8_t padding[16]`: PKCS7 填充后的密文值

8.3.86 ehsm_key_format_rsa2048_crt_plain_keypair_st

用于辅助 RSA2048-CRT 密钥对明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_2048`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: E 值长度, 应当设置为 8
- `uint16_t priv_key_size`: 5 分量明文长度, 这里应当设置为 640
- `uint8_t e_value[8]`: E 值明文, 8 字节
- `uint8_t n_value[256]`: N 值明文, 256 字节

- `uint8_t p_value[128]`: P 值明文, 128 字节
- `uint8_t q_value[128]`: Q 值明文, 128 字节
- `uint8_t dp_value[128]`: DP 值明文, 128 字节
- `uint8_t dq_value[128]`: DQ 值明文, 128 字节
- `uint8_t u_value[128]`: U 值明文, 128 字节

8.3.87 ehsm_key_format_rsa2048_crt_plain_privkey_st

用于辅助 RSA2048-CRT 私钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_2048`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 5 分量明文长度, 这里应当设置为 640
- `uint8_t p_value[128]`: P 值明文, 128 字节
- `uint8_t q_value[128]`: Q 值明文, 128 字节
- `uint8_t dp_value[128]`: DP 值明文, 128 字节
- `uint8_t dq_value[128]`: DQ 值明文, 128 字节
- `uint8_t u_value[128]`: U 值明文, 128 字节

8.3.88 ehsm_key_format_rsa2048_plain_keypair_st

用于辅助 RSA2048 密钥对明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_2048`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: E 值长度, 应当设置为 8
- `uint16_t priv_key_size`: D 值明文长度, 这里应当设置为 256
- `uint8_t e_value[8]`: E 值明文, 8 字节
- `uint8_t n_value[256]`: N 值明文, 256 字节
- `uint8_t d_value[256]`: D 值明文, 256 字节

8.3.89 ehsm_key_format_rsa2048_plain_privkey_st

用于辅助 RSA2048 私钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_2048`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: D 值明文长度, 这里应当设置为 256
- `uint8_t n_value[256]`: N 值明文
- `uint8_t d_value[256]`: D 值明文

8.3.90 ehsm_key_format_rsa2048_plain_pubkey_st

用于辅助 RSA2048 公钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_2048`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PUBLIC_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 应当设置为 8
- `uint16_t priv_key_size`: 为 0
- `uint8_t e_value[8]`: E 值
- `uint8_t n_value[256]`: N 值

8.3.91 ehsm_key_format_rsa3072_cipher_keypair_st

用于辅助 RSA3072 密钥对密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_3072`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: E 值长度, 应当设置为 8
- `uint16_t priv_key_size`: D 值明文长度, 这里应当设置为 384

- `uint8_t e_value[8]`: E 值明文, 8 字节
- `uint8_t n_value[384]`: N 值明文, 384 字节
- `uint8_t d_value[400]`: D 值密文, 400 字节

8.3.92 ehsm_key_format_rsa3072_cipher_privkey_st

用于辅助 RSA3072 私钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 EHSM_KEY_TYPE_RSA_3072
- `uint8_t part_info`: 应当设置为 EHSM_KEY_PART_PRIVATE_KEY
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: D 值明文长度, 这里应当设置为 384
- `uint8_t n_value[384]`: N 值明文, 384 字节
- `uint8_t d_value[400]`: D 值密文, 进行了 PKCS7 填充, 因此是 400 字节

8.3.93 ehsm_key_format_rsa3072_crt_cipher_keypair_st

用于辅助 RSA3072-CRT 密钥对密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 EHSM_KEY_TYPE_RSA_3072
- `uint8_t part_info`: 应当设置为 EHSM_KEY_PART_KEY_PAIR
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: E 值长度, 应当设置为 8
- `uint16_t priv_key_size`: 5 分量明文长度, 这里应当设置为 960
- `uint8_t e_value[8]`: E 值明文, 8 字节
- `uint8_t n_value[384]`: N 值明文, 384 字节
- `uint8_t p_value[192]`: P 值密文, 192 字节
- `uint8_t q_value[192]`: Q 值密文, 192 字节
- `uint8_t dp_value[192]`: DP 值密文, 192 字节
- `uint8_t dq_value[192]`: DQ 值密文, 192 字节
- `uint8_t u_value[192]`: U 值密文, 192 字节
- `uint8_t padding[16]`: PKCS7 填充后的密文值

8.3.94 ehsm_key_format_rsa3072_crt_cipher_privkey_st

用于辅助 RSA3072-CRT 私钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_3072`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 5 分量明文长度, 这里应当设置为 960
- `uint8_t p_value[192]`: P 值密文, 192 字节
- `uint8_t q_value[192]`: Q 值密文, 192 字节
- `uint8_t dp_value[192]`: DP 值密文, 192 字节
- `uint8_t dq_value[192]`: DQ 值密文, 192 字节
- `uint8_t u_value[192]`: U 值密文, 192 字节
- `uint8_t padding[16]`: PKCS7 填充后的密文值

8.3.95 ehsm_key_format_rsa3072_crt_plain_keypair_st

用于辅助 RSA3072-CRT 密钥对明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_3072`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: E 值长度, 应当设置为 8
- `uint16_t priv_key_size`: 5 分量明文长度, 这里应当设置为 960
- `uint8_t e_value[8]`: E 值明文, 8 字节
- `uint8_t n_value[384]`: N 值明文, 384 字节
- `uint8_t p_value[192]`: P 值明文, 192 字节
- `uint8_t q_value[192]`: Q 值明文, 192 字节
- `uint8_t dp_value[192]`: DP 值明文, 192 字节
- `uint8_t dq_value[192]`: DQ 值明文, 192 字节
- `uint8_t u_value[192]`: U 值明文, 192 字节

8.3.96 ehsm_key_format_rsa3072_crt_plain_privkey_st

用于辅助 RSA3072-CRT 私钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_3072`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 5 分量明文长度, 这里应当设置为 960
- `uint8_t p_value[192]`: P 值明文, 192 字节
- `uint8_t q_value[192]`: Q 值明文, 192 字节
- `uint8_t dp_value[192]`: DP 值明文, 192 字节
- `uint8_t dq_value[192]`: DQ 值明文, 192 字节
- `uint8_t u_value[192]`: U 值明文, 192 字节

8.3.97 ehsm_key_format_rsa3072_plain_keypair_st

用于辅助 RSA3072 密钥对明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_3072`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: E 值长度, 应当设置为 8
- `uint16_t priv_key_size`: D 值明文长度, 这里应当设置为 384
- `uint8_t e_value[8]`: E 值明文, 8 字节
- `uint8_t n_value[384]`: N 值明文, 384 字节
- `uint8_t d_value[384]`: D 值明文, 384 字节

8.3.98 ehsm_key_format_rsa3072_plain_privkey_st

用于辅助 RSA3072 私钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_3072`

- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: D 值明文长度, 这里应当设置为 384
- `uint8_t n_value[384]`: N 值明文
- `uint8_t d_value[384]`: D 值明文

8.3.99 ehsm_key_format_rsa3072_plain_pubkey_st

用于辅助 RSA3072 公钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_3072`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PUBLIC_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 应当设置为 8
- `uint16_t priv_key_size`: 为 0
- `uint8_t e_value[8]`: E 值
- `uint8_t n_value[384]`: N 值

8.3.100 ehsm_key_format_rsa4096_cipher_keypair_st

用于辅助 RSA4096 密钥对密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_4096`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: E 值长度, 应当设置为 8
- `uint16_t priv_key_size`: D 值明文长度, 这里应当设置为 512
- `uint8_t e_value[8]`: E 值明文, 8 字节
- `uint8_t n_value[512]`: N 值明文, 512 字节
- `uint8_t d_value[528]`: D 值密文, 528 字节

8.3.101 ehsm_key_format_rsa4096_cipher_privkey_st

用于辅助 RSA4096 私钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_4096`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: D 值明文长度, 这里应当设置为 512
- `uint8_t n_value[512]`: N 值明文, 512 字节
- `uint8_t d_value[528]`: D 值密文, 进行了 PKCS7 填充, 因此是 528 字节

8.3.102 ehsm_key_format_rsa4096_crt_cipher_keypair_st

用于辅助 RSA4096-CRT 密钥对密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_4096`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: E 值长度, 应当设置为 8
- `uint16_t priv_key_size`: 5 分量明文长度, 这里应当设置为 1280
- `uint8_t e_value[8]`: E 值明文, 8 字节
- `uint8_t n_value[512]`: N 值明文, 512 字节
- `uint8_t p_value[256]`: P 值密文, 256 字节
- `uint8_t q_value[256]`: Q 值密文, 256 字节
- `uint8_t dp_value[256]`: DP 值密文, 256 字节
- `uint8_t dq_value[256]`: DQ 值密文, 256 字节
- `uint8_t u_value[256]`: U 值密文, 256 字节
- `uint8_t padding[16]`: PKCS7 填充后的密文值

8.3.103 ehsm_key_format_rsa4096_crt_cipher_privkey_st

用于辅助 RSA4096-CRT 私钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_4096`

- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 5 分量明文长度, 这里应当设置为 1280
- `uint8_t p_value[256]`: P 值密文, 256 字节
- `uint8_t q_value[256]`: Q 值密文, 256 字节
- `uint8_t dp_value[256]`: DP 值密文, 256 字节
- `uint8_t dq_value[256]`: DQ 值密文, 256 字节
- `uint8_t u_value[256]`: U 值密文, 256 字节
- `uint8_t padding[16]`: PKCS7 填充后的密文值

8.3.104 `ehsm_key_format_rsa4096_crt_plain_keypair_st`

用于辅助 RSA4096-CRT 密钥对明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_4096`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: E 值长度, 应当设置为 8
- `uint16_t priv_key_size`: 5 分量明文长度, 这里应当设置为 1280
- `uint8_t e_value[8]`: E 值明文, 8 字节
- `uint8_t n_value[512]`: N 值明文, 512 字节
- `uint8_t p_value[256]`: P 值明文, 256 字节
- `uint8_t q_value[256]`: Q 值明文, 256 字节
- `uint8_t dp_value[256]`: DP 值明文, 256 字节
- `uint8_t dq_value[256]`: DQ 值明文, 256 字节
- `uint8_t u_value[256]`: U 值明文, 256 字节

8.3.105 `ehsm_key_format_rsa4096_crt_plain_privkey_st`

用于辅助 RSA4096-CRT 私钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_4096`

- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 5 分量明文长度, 这里应当设置为 1280
- `uint8_t p_value[256]`: P 值明文, 256 字节
- `uint8_t q_value[256]`: Q 值明文, 256 字节
- `uint8_t dp_value[256]`: DP 值明文, 256 字节
- `uint8_t dq_value[256]`: DQ 值明文, 256 字节
- `uint8_t u_value[256]`: U 值明文, 256 字节

8.3.106 ehsm_key_format_rsa4096_plain_keypair_st

用于辅助 RSA4096 密钥对明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_4096`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: E 值长度, 应当设置为 8
- `uint16_t priv_key_size`: D 值明文长度, 这里应当设置为 512
- `uint8_t e_value[8]`: E 值明文, 8 字节
- `uint8_t n_value[512]`: N 值明文, 512 字节
- `uint8_t d_value[512]`: D 值明文, 512 字节

8.3.107 ehsm_key_format_rsa4096_plain_privkey_st

用于辅助 RSA4096 私钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_4096`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: D 值明文长度, 这里应当设置为 512
- `uint8_t n_value[512]`: N 值明文

- `uint8_t d_value[512]`: D 值明文

8.3.108 ehsm_key_format_rsa4096_plain_pubkey_st

用于辅助 RSA4096 公钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_RSA_4096`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PUBLIC_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 应当设置为 8
- `uint16_t priv_key_size`: 为 0
- `uint8_t e_value[8]`: E 值
- `uint8_t n_value[512]`: N 值

8.3.109 ehsm_key_format_sm2_cipher_keypair_st

用于辅助 SM2 密钥对密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_SM2`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 65
- `uint16_t priv_key_size`: 私钥长度, 应为 32
- `uint8_t prefix`: 前导字节, 必须为 0x04
- `uint8_t x_value[32]`: 公钥 X 值明文, 32 字节
- `uint8_t y_value[32]`: 公钥 Y 值明文, 32 字节
- `uint8_t priv_value[48]`: 私钥密文值, 有 PKCS7 填充, 48 字节

8.3.110 ehsm_key_format_sm2_cipher_privkey_st

用于辅助 SM2 私钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_SM2`

- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 32
- `uint8_t priv_value[48]`: 私钥密文值, 有 PKCS7 填充, 48 字节

8.3.111 `ehsm_key_format_sm2_plain_keypair_st`

用于辅助 SM2 密钥对明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_SM2`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_KEY_PAIR`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应为 65
- `uint16_t priv_key_size`: 私钥长度, 应为 32
- `uint8_t prefix`: 前导字节, 必须为 0x04
- `uint8_t x_value[32]`: 公钥 X 值明文, 32 字节
- `uint8_t y_value[32]`: 公钥 Y 值明文, 32 字节
- `uint8_t priv_value[32]`: 私钥明文值, 32 字节

8.3.112 `ehsm_key_format_sm2_plain_privkey_st`

用于辅助 SM2 私钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_SM2`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PRIVATE_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 为 0
- `uint16_t priv_key_size`: 私钥长度, 应为 32
- `uint8_t priv_value[32]`: 私钥明文值, 32 字节

8.3.113 `ehsm_key_format_sm2_plain_pubkey_st`

用于辅助 SM2 公钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 应当设置为 `EHSM_KEY_TYPE_SM2`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_PUBLIC_KEY`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥长度, 应当设置为 65
- `uint16_t priv_key_size`: 为 0
- `uint8_t prefix`: 前导字节, 必须为 0x04
- `uint8_t x_value[32]`: 公钥 X 值明文, 32 字节
- `uint8_t y_value[32]`: 公钥 Y 值明文, 32 字节

8.3.114 ehsm_key_format_st

导入/导出密钥数据结构的头部。

结构体中 `size` 字段无法表示实际密钥长度, `key_value` 长度为 0 表示后续的密钥值根据算法不同, 其具体格式和长度是可变的, 实际长度由 `pub_key_size` 和 `priv_key_size` 之和来表示。如果是对称密钥, 则 `pub_key_size` 为 0, 密钥长度由 `priv_key_size` 指示。关于密钥值的具体格式, 见[密钥数据格式](#)。实现使用时, 可使用其它辅助结构体, 见[密钥导入/导出数据格式结构](#)。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 密钥包含的数据信息, 见 `ehsm_key_part_e`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 公钥数据长度, 如果是 RSA, 这里是 E 的长度 (必须 4 字节对齐), N 的长度由密钥类型推导
- `uint16_t priv_key_size`: 私钥数据或对称密钥数据长度, 如果是 RSA, 这里是 D 的长度, N 的长度由密钥类型推导; 如果是 RSA-CRT, 这里是私钥 5 分量的总长度
- `uint8_t key_value[0]`: 密钥值, 对称密钥和非对称密钥的结构见描述 密钥数据格式

8.3.115 ehsm_key_format_symm_cipher_128_st

用于辅助 128 位对称密钥密文导入/导出的结构体。

注意, 此结构体仅针对传输密钥的块大小是 16 字节的情况, 对于传输密钥为 DES/TDES 时, 密钥密文长度为 24 字节

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_SYMM_KEY`, 见 `ehsm_key_part_e`

- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 应当设置为 0
- `uint16_t priv_key_size`: 对称密钥明文数据长度, 这里应当设置为 16
- `uint8_t key_value[32]`: 密文密钥值, 这里进行了 PKCS7 填充, 因此为 32 字节

8.3.116 ehsm_key_format_symm_cipher_192_st

用于辅助 192 位对称密钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_SYMM_KEY`, 见 `ehsm_key_part_e`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 应当设置为 0
- `uint16_t priv_key_size`: 对称密钥明文数据长度, 这里应当设置为 24
- `uint8_t key_value[32]`: 密文密钥值, 这里进行了 PKCS7 填充, 因此为 32 字节

8.3.117 ehsm_key_format_symm_cipher_256_st

用于辅助 256 位对称密钥密文导入/导出的结构体。

注意, 此结构体仅针对传输密钥的块大小是 16 字节的情况, 对于传输密钥为 DES/TDES 时, 密钥密文长度为 40 字节

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_SYMM_KEY`, 见 `ehsm_key_part_e`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 应当设置为 0
- `uint16_t priv_key_size`: 对称密钥明文数据长度, 这里应当设置为 32
- `uint8_t key_value[48]`: 密文密钥值, 这里进行了 PKCS7 填充, 因此为 48 字节

8.3.118 ehsm_key_format_symm_cipher_384_st

用于辅助 384 位对称密钥密文导入/导出的结构体。

注意, 此结构体仅针对传输密钥的块大小是 16 字节的情况, 对于传输密钥为 DES/TDES 时, 密钥密文长度为 56 字节

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_SYMM_KEY`, 见 `ehsm_key_part_e`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 应当设置为 0
- `uint16_t priv_key_size`: 对称密钥明文数据长度, 这里应当设置为 48
- `uint8_t key_value[64]`: 密文密钥值, 这里进行了 PKCS7 填充, 因此为 64 字节

8.3.119 ehsm_key_format_symm_cipher_512_st

用于辅助 512 位对称密钥密文导入/导出的结构体。

注意, 此结构体仅针对传输密钥的块大小是 16 字节的情况, 对于传输密钥为 DES/TDES 时, 密钥密文长度为 72 字节

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_SYMM_KEY`, 见 `ehsm_key_part_e`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 应当设置为 0
- `uint16_t priv_key_size`: 对称密钥明文数据长度, 这里应当设置为 64
- `uint8_t key_value[80]`: 密文密钥值, 这里进行了 PKCS7 填充, 因此为 80 字节

8.3.120 ehsm_key_format_symm_cipher_64_st

用于辅助 64 位对称密钥密文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_SYMM_KEY`, 见 `ehsm_key_part_e`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 应当设置为 0
- `uint16_t priv_key_size`: 对称密钥明文数据长度, 这里应当设置为 8
- `uint8_t key_value[16]`: 密文密钥值, 这里进行了 PKCS7 填充, 因此为 16 字节

8.3.121 ehsm_key_format_symm_plain_128_st

用于辅助 128 位对称密钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_SYMM_KEY`, 见 `ehsm_key_part_e`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 应当设置为 0
- `uint16_t priv_key_size`: 对称密钥明文数据长度, 这里应当设置为 16
- `uint8_t key_value[16]`: 明文密钥值

8.3.122 ehsm_key_format_symm_plain_192_st

用于辅助 192 位对称密钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_SYMM_KEY`, 见 `ehsm_key_part_e`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 应当设置为 0
- `uint16_t priv_key_size`: 对称密钥明文数据长度, 这里应当设置为 24
- `uint8_t key_value[24]`: 明文密钥值

8.3.123 ehsm_key_format_symm_plain_256_st

用于辅助 256 位对称密钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_SYMM_KEY`, 见 `ehsm_key_part_e`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 应当设置为 0
- `uint16_t priv_key_size`: 对称密钥明文数据长度, 这里应当设置为 32
- `uint8_t key_value[32]`: 明文密钥值

8.3.124 ehsm_key_format_symm_plain_384_st

用于辅助 384 位对称密钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_SYMM_KEY`, 见 `ehsm_key_part_e`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 应当设置为 0
- `uint16_t priv_key_size`: 对称密钥明文数据长度, 这里应当设置为 48
- `uint8_t key_value[48]`: 明文密钥值

8.3.125 ehsm_key_format_symm_plain_512_st

用于辅助 512 位对称密钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_SYMM_KEY`, 见 `ehsm_key_part_e`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 应当设置为 0
- `uint16_t priv_key_size`: 对称密钥明文数据长度, 这里应当设置为 64
- `uint8_t key_value[64]`: 明文密钥值

8.3.126 ehsm_key_format_symm_plain_64_st

用于辅助 64 位对称密钥明文导入/导出的结构体。

成员:

- `uint32_t privilege`: 权限位, 见 密钥权限
- `uint8_t key_type`: 密钥类型, 见 `ehsm_key_type_e`
- `uint8_t part_info`: 应当设置为 `EHSM_KEY_PART_SYMM_KEY`, 见 `ehsm_key_part_e`
- `uint8_t reserved[2]`: 保留
- `uint16_t pub_key_size`: 应当设置为 0
- `uint16_t priv_key_size`: 对称密钥明文数据长度, 这里应当设置为 8
- `uint8_t key_value[8]`: 明文密钥值

8.3.127 ehsm_mb_cmd_st**成员:**

- `uint16_t cmd_id`
- `uint16_t cmd_id_inv`
- `uint32_t data[39]`

8.3.128 ehsm_mb_packet_st

成员:

- `raddr_t cmd_addr`
- `raddr_t rsp_addr`

8.3.129 ehsm_mb_rsp_st

成员:

- `uint32_t ret_code`
- `uint32_t data[4]`

8.3.130 ehsm_rsa_key_st

RSA 明文密钥数据结构

成员:

- `uint8_t crt_mode`: CRT 模式, 1 代表私钥使用 CRT 模式
- `uint8_t reserved0[3]`: 保留字段
- `uint32_t n_byte_sz`: n 值的字节数
- `uint32_t e_byte_sz`: e 值的字节数
- `EHSM_SHM raddr_t n`: n 值的存储地址
- `EHSM_SHM raddr_t e`: e 值的存储地址
- `EHSM_SHM raddr_t d`: d 值的存储地址
- `EHSM_SHM raddr_t p`: p 值的存储地址
- `EHSM_SHM raddr_t q`: q 值的存储地址
- `EHSM_SHM raddr_t dp`: dp 值的存储地址
- `EHSM_SHM raddr_t dq`: dq 值的存储地址
- `EHSM_SHM raddr_t u`: u 值的存储地址

8.3.131 ehsm_self_test_result_st

算法自检结果。

成员:

- `uint32_t result`: 指示哪些算法测试通过, bit 为 1 的对应算法测试通过 (mask 中未测试的算法没有结果)
- `uint32_t mask`: 指示哪些算法被测试, bit 为 1 的对应算法被测试, 见 自检算法

8.3.132 ehsm_session_st

为 eHSM 的三段式(init/update/finish)算法计算提供存储空间, 此内存空间应当在 init/update/finish 调用期间保持有效状态。

成员:

- `uint32_t _data[512/4]`: 内部维护数据

8.3.133 ehsm_sm2_key_st

SM2 明文密钥数据结构

成员:

- `uint8_t privkey[32]`: SM2 私钥
- `uint8_t pubkey[65]`: SM2 公钥, 04 + X + Y

8.3.134 ehsm_sm2_params_st

SM2 密钥交换算法相关的参数。

成员:

- `uint8_t sm2_role`: SM2 密钥交换角色, 见 `ehsm_sm2_role_e`
- `uint8_t reserved0[3]`: 保留字段
- `uint32_t local_tmp_key_handle`: SM2 密钥交换本地临时密钥句柄
- `EHSM_SHM raddr_t s1_s2_value_addr`: SM2 密钥交换 s1_s2 值的主机地址
- `EHSM_SHM raddr_t sa_sb_value_addr`: SM2 密钥交换 sa_sb 值的主机地址
- `EHSM_SHM raddr_t peer_temp_pubkey_addr`: SM2 密钥交换对方临时公钥的主机地址
- `uint32_t peer_temp_pubkey_size`: 对方临时公钥的长度

8.3.135 ehsm_sm9_params_st

SM9 密钥交换算法相关的参数。

成员:

- `EHSM_SHM raddr_t peer_tmp_pub_addr`: 对方临时公钥的主机地址
- `EHSM_SHM raddr_t kgc_pub_key_addr`: KGC 系统加密主公钥的主机地址, 公钥大小为 64 字节
- `EHSM_SHM raddr_t fp12g_addr`: $e(P1, pub_key)$ 值的主机地址, 如果设置为 NULL, 将在 eHSM 内部计算
- `EHSM_SHM raddr_t self_id_addr`: 本方用户 ID 的主机地址

- `EHSM_SHM raddr_t peer_id_addr`: 对方用户 ID 的主机地址
- `uint32_t peer_id_size`: 对方用户 ID 的字节数, 最大可为 1024
- `uint32_t self_id_size`: 本方用户 ID 的字节数, 最大可为 1024
- `EHSM_SHM raddr_t s1_s2_addr`: S1 (或 S2) 验证值的主机地址, 大小为 32 字节
- `EHSM_SHM raddr_t sa_sb_addr`: SA (或 SB) 验证值的主机地址, 大小为 32 字节

8.3.136 ehsm_soc_dbg_bitmap_st

SOC 调试开启/关闭对应的使能位图。

成员:

- `uint32_t bitmaps[5]`: 前 4 个 word, 每个 bit 对应 128 个 SOC 调试端口中的一个; 最后一个 word 的最低位对应第 129 个 SOC 调试端口

8.3.137 ehsm_version_st

Bootloader/Firmware 的版本号结构体。

成员:

- `uint8_t type`: 固件类型, 0: Bootloader, 1: 固件
- `uint8_t ver_major`: 主版本号
- `uint8_t ver_minor`: 次版本号
- `uint8_t ver_patch`: 修订版本号
- `uint8_t ver_pre_release[8]`: 预发布版本字符串, 如 alpha.1、rc3; 正式版本则为空字符串
- `uint8_t reserved0[8]`: 保留
- `uint32_t pke_engine_ver`: PKE 引擎版本号
- `uint32_t pke_lib_ver`: PKE 库版本号。格式: 0x23080301 表示 2023-08-03 版本 1
- `uint32_t ske_engine_ver`: SKE 引擎版本号
- `uint32_t ske_lib_ver`: SKE 库版本号。格式同上
- `uint32_t hash_engine_ver`: HASH 引擎版本号
- `uint32_t hash_lib_ver`: HASH 库版本号。格式同上
- `uint32_t trng_engine_ver`: TRNG 引擎版本号
- `uint32_t trng_lib_ver`: TRNG 库版本号。格式同上
- `uint32_t hw_ver`: 硬件版本号, 详见 OSR eHSM TRM 手册中的 SYS_VER1
- `uint8_t uid[16]`: 从 OTP 读取的 16 字节 UID
- `uint8_t reserved1[56]`: 保留

8.4 Functions

8.4.1 ehsm_mb_init

原型:

```
1| uint32_t ehsm_mb_init(ehsm_mb_interrupt_func_t func, ehsm_drv_mode_e drv_mode)
```

参数:

- `func`
- `drv_mode`

8.4.2 ehsm_mb_send

原型:

```
1| uint32_t ehsm_mb_send(uint32_t channel, raddr_t packet)
```

参数:

- `channel`
- `packet`

8.4.3 ehsm_mb_poll

原型:

```
1| uint32_t ehsm_mb_poll(uint32_t channel)
```

参数:

- `channel`

8.4.4 ehsm_port_init

原型:

```
1| uint32_t ehsm_port_init(ehsm_drv_mode_e drv_mode)
```

说明:

初始化移植平台。

参数:

- `drv_mode`

返回说明:

EHSM_OK 表示成功, 其它值表示失败

8.4.5 ehsm_port_addr_to_raddr

原型:

```
1| raddr_t ehsm_port_addr_to_raddr(const void *addr)
```

说明:

将逻辑地址转换为硬件地址。用于将当前进程的逻辑地址转换为硬件地址并放到 mailbox 命令中, 此硬件地址是 eHSM 可以识别的地址。

参数:

- `addr`: 当前进程的逻辑地址

返回说明:

硬件地址

8.4.6 ehsm_port_raddr_to_addr

原型:

```
1| void * ehsm_port_raddr_to_addr(raddr_t raddr)
```

说明:

将硬件地址转换为当前进程的逻辑地址。

参数:

- `raddr`: 硬件地址

返回说明:

逻辑地址

8.4.7 ehsm_port_flush_and_invalidate_cache

原型:

```
1| void ehsm_port_flush_and_invalidate_cache(void)
```

说明:

将 CPU cache 数据刷入 RAM 并失效 cache。将 CPU cache 中的数据刷入 RAM 然后将 cache 失效, 确保 cache 数据写入 RAM 并且从 RAM 中读取最新数据, 同时要考虑增加合适内存屏障操

作避免 CPU 乱序执行引起的问题。实现者可以考虑将一段有限的地址空间作为共享内存, 这样仅针对有限空间刷 cache 可提高性能; 或者将共享内存空间配置为 non-cacheable, 这样此函数可以什么都不做。此函数会在每次发送 Mailbox 命令之前调用

8.4.8 ehsm_port_enable_mailbox_int

原型:

```
1| uint32_t ehsm_port_enable_mailbox_int(uint32_t channel, ehsm_mailbox_isr_f int_func)
```

说明:

使能指定 mailbox channel 的中断, 并注册回调函数。当中断触发时, 传入的 int_func 必须被调用, 且参数为对应 channel 值。

参数:

- `channel`: Mailbox channel ID
- `int_func`: 中断回调函数

返回说明:

EHSM_OK 表示成功, 其它值为错误码

8.4.9 ehsm_port_reset_ehsm

原型:

```
1| void ehsm_port_reset_ehsm(void)
```

说明:

复位 eHSM。

8.4.10 ehsm_port_read_reg

原型:

```
1| uint32_t ehsm_port_read_reg(ehsm_reg_e reg)
```

说明:

读取 eHSM 寄存器值。

参数:

- `reg`: 寄存器编号。

返回说明:

寄存器的值。

8.4.11 ehsm_port_write_reg

原型:

```
1| void ehsm_port_write_reg(ehsm_reg_e reg, uint32_t val)
```

说明:

写入 eHSM 寄存器值。

注意:

- 目前仅支持 REG_SOC_SENSOR 寄存器。

参数:

- `reg`: 寄存器编号。
- `val`: 寄存器的值。

8.4.12 ehsm_port_read_otp

原型:

```
1| uint32_t ehsm_port_read_otp(uint8_t *buf, uint32_t offset, uint32_t len)
```

说明:

读取 eHSM OTP 空间数据。

参数:

- `buf`: 存储读取数据的 buffer
- `offset`: 起始偏移
- `len`: 读取长度

返回说明:

EHSM_OK 表示成功, 其它值为错误码

8.4.13 ehsm_port_write_otp

原型:

```
1| uint32_t ehsm_port_write_otp(const uint8_t *otp_data, uint32_t offset, uint32_t len)
```

说明:

写入 eHSM OTP 空间数据。

参数:

- `otp_data`: 存储待写入数据的 buffer
- `offset`: 起始偏移
- `len`: 写入长度

返回说明:

EHSM_OK 表示成功, 其它值为错误码

8.4.14 ehsm_port_create_timer

原型:

```
1| ehsm_port_timer_t ehsm_port_create_timer(void)
```

说明:

创建一个计时器。

返回说明:

计时器 ID (可能仅是当前系统时钟值)

8.4.15 ehsm_port_is_timeout

原型:

```
1| bool_t ehsm_port_is_timeout(ehsm_port_timer_t timer)
```

说明:

查看计时器是否超时。由实现者判断多少时间是超时, ehsm_host 库仅使用这个判断结果来决定操作是否继续 进行。

参数:

- `timer`: 计时器 ID (可能仅是创建时的系统时钟值)

返回说明:

是否超时

8.4.16 ehsm_port_get_time_ms

原型:

```
1| double ehsm_port_get_time_ms(void)
```

说明:

获取系统运行时间, 微秒数。

返回说明:

EHSM_OK 表示成功, 其它值表示失败

8.4.17 ehsm_port_memory_barrier_write

原型:

```
1| void ehsm_port_memory_barrier_write(void)
```

说明:

写内存屏障, 确保屏障前的所有写操作在屏障后的写操作之前完成。

8.4.18 ehsm_port_memory_barrier_read

原型:

```
1| void ehsm_port_memory_barrier_read(void)
```

说明:

读内存屏障, 确保屏障前的所有读操作在屏障后的读操作之前完成。

8.4.19 ehsm_port_enter_critical

原型:

```
1| void ehsm_port_enter_critical(void)
```

说明:

进入临界区 (禁用中断)。

8.4.20 ehsm_port_exit_critical

原型:

```
1| void ehsm_port_exit_critical(void)
```

说明:

退出临界区 (恢复中断)。

8.4.21 ehsm_driver_init_library

原型:

```
1| uint32_t ehsm_driver_init_library(ehsm_drv_mode_e drv_mode)
```

说明:

初始化驱动库。

注意:

- 此函数不涉及向 eHSM 发送命令。

参数:

- `drv_mode`: 底层使用的驱动模式, 想要支持异步必须使用中断模式, 见 [ehsm_drv_mode_e](#)

返回值:

- `EHSM_OK`: 初始化驱动库成功。
- 其它值: 初始化驱动库失败。

8.4.22 ehsm_driver_get_version

原型:

```
1| uint32_t ehsm_driver_get_version(void)
```

说明:

获取驱动库的版本号。版本号的格式为 major.minor.patch, 使用 32 位整数的低 24 位, 每字节表示一个字段。例如: 0x00010203 表示 v1.2.3。

注意:

- 此函数不涉及向 eHSM 发送命令。

返回说明:

驱动库的版本号

8.4.23 ehsm_ctx_init

原型:

```
1| void ehsm_ctx_init(EHSM_SHM ehsm_ctx_st *ctx, uint8_t mb_ch, bool_t async,
ehsm_rsp_cb_func_t rsp_callback)
```

说明:

初始化 eHSM API 调用的 context。说明:

- 当配置了 `async=true` 且为 `EHSM_DRV_MODE_INTERRUPT` 模式时, API 函数不会等待 eHSM 处理完才返回, 而是先返回 `EHSM_ERR_NEED_POLL`
- 异步时, 如果 `rsp_callback` 不为 NULL, 则任务完成时 `rsp_callback` 会被调用; 否则, 需要主动调用 [ehsm_ctx_poll](#) 获取任务是否完成

注意:

- 此函数仅初始化 `ctx` 内存, 不涉及向 eHSM 发送命令。

参数:

- `ctx`: 新分配的 ehsm context 地址
- `mb_ch`: 使用的 mailbox channel
- `async`: 是否异步模式, 仅在 `EHSM_DRV_MODE_INTERRUPT` 模式时有效
- `rsp_callback`: 异步模式下, 任务完成时的回调函数, 可以为 NULL。同步模式下此参数忽略。

8.4.24 ehsm_ctx_poll

原型:

```
1| uint32_t ehsm_ctx_poll(EHSM_SHM ehsm_ctx_st *ctx)
```

说明:

异步模式下, 查询之前调用的任务是否已完成。

注意:

- 此函数仅查询 `ctx` 中保存的状态, 不涉及向 eHSM 发送命令。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.25 ehsm_gen_random

原型:

```
1| uint32_t ehsm_gen_random(EHSM_SHM ehsm_ctx_st *ctx, ehsm_rng_algo_e algo, EHSM_SHM  
uint8_t *rand_buf, uint32_t rand_size)
```

说明:

生成随机数。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化

- `algo`: 后处理算法类型, 见 [ehsm_rng_algo_e](#)
- `rand_buf`: 输出 buffer
- `rand_size`: 生成的随机数长度

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.26 ehsm_hash_init

原型:

```
1| uint32_t ehsm_hash_init(EHSM_SHM ehsm_ctx_st *ctx, ehsm_hash_algo_e algo, EHSM_SHM  
ehsm_session_st *session)
```

说明:

HASH 计算, 初始化。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: HASH 算法类型, 见 [ehsm_hash_algo_e](#)
- `session`: 用于 ehsm 内部缓存 HASH 计算中间值的 buffer, 此参数指向的内存存在 [ehsm_hash_update](#) 和 [ehsm_hash_finish](#) 中仍会使用, 因此需保证其有效性, 不能提前释放。

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.27 ehsm_hash_update

原型:

```
1| uint32_t ehsm_hash_update(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t *msg,  
uint32_t msg_size)
```

说明:

HASH 计算, 更新消息数据。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_hash_init](#) 或 [ehsm_hash_update](#)
- `msg`: 输入更新的消息数据。
- `msg_size`: 消息数据长度。

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.28 ehsm_hash_finish

原型:

```
1| uint32_t ehsm_hash_finish(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM uint8_t *digest,
uint32_t *digest_size)
```

说明:

HASH 计算, 生成 HASH 结果。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_hash_init](#) 或 [ehsm_hash_update](#)
- `digest`: 存放 HASH 值的 buffer。
- `digest_size`: 输入时存储的是 digest buffer 长度, 输出时是实际生成的 HASH 值长度。

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.29 ehsm_hash_onepass

原型:

```
1| uint32_t ehsm_hash_onepass(EHSM_SHM ehsm_ctx_st *ctx, ehsm_hash_algo_e algo, EHSM_SHM
const uint8_t *msg, uint32_t msg_size, EHSM_SHM uint8_t *digest, uint32_t *digest_size)
```

说明:

HASH 计算, 一次完成计算并输出结果。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化

- `algo`: HASH 算法类型, 见 [ehsm_hash_algo_e](#)
- `msg`: 输入的消息数据。
- `msg_size`: 消息数据长度。
- `digest`: 存放 HASH 值的 buffer。
- `digest_size`: 输入时存储的是 digest buffer 长度, 输出时是实际生成的 HASH 值长度。

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.30 ehsm_hmac_init

原型:

```
1| uint32_t ehsm_hmac_init(EHSM_SHM ehsm_ctx_st *ctx, ehsm_hash_algo_e algo, uint32_t  
key_handle, bool_t gen_hmac, EHSM_SHM ehsm_session_st *session)
```

说明:

HMAC 计算, 初始化。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: HASH 算法类型, 见 [ehsm_hash_algo_e](#)
- `key_handle`: 用于 HMAC 计算的密钥句柄
- `gen_hmac` : 操作方式, 必须跟最后调用的 [ehsm_hmac_finish_gen](#) 或 [ehsm_hmac_finish_verify](#) 相匹配
 - `true` 生成 HMAC 值
 - `false` 校验 HMAC 值
- `session` : 用于 ehsm 内部缓存 HMAC 计算中间值的 buffer, 此参数指向的内存存在 [ehsm_hmac_update](#), [ehsm_hmac_finish_gen](#) 和 [ehsm_hmac_finish_verify](#) 中仍会使用, 因此需保证其有效性, 不能提前释放。

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.31 ehsm_hmac_update

原型:

```
1| uint32_t ehsm_hmac_update(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t *msg,  
uint32_t msg_size)
```

说明:

HMAC 计算, 更新消息数据。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_hmac_init](#) 或 [ehsm_hmac_update](#)
- `msg`: 输入更新的消息数据。
- `msg_size`: 消息数据长度。

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.32 ehsm_hmac_finish_gen

原型:

```
1| uint32_t ehsm_hmac_finish_gen(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM uint8_t *hmac,  
uint32_t hmac_size)
```

说明:

HMAC 计算, 生成 HMAC 结果。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_hmac_init](#) 或 [ehsm_hmac_update](#)
- `hmac`: 存放 HMAC 值的 buffer
- `hmac_size`: 期望生成的 HMAC 长度, 不能小于 8, 也不能大于实际算法对应的 HMAC 长度

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.33 ehsm_hmac_finish_verify

原型:

```
1| uint32_t ehsm_hmac_finish_verify(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t *hmac, uint32_t hmac_size, bool_t *verify_result)
```

说明:

HMAC 计算, 验证 HMAC 值。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_hmac_init](#) 或 [ehsm_hmac_update](#)
- `hmac`: 待验证的 HMAC 值存放的位置
- `hmac_size`: HMAC 值大小, 不能小于 8
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.34 ehsm_hmac_onepass_gen

原型:

```
1| uint32_t ehsm_hmac_onepass_gen(EHSM_SHM ehsm_ctx_st *ctx, ehsm_hash_algo_e algo, uint32_t key_handle, EHSM_SHM const uint8_t *msg, uint32_t msg_size, EHSM_SHM uint8_t *hmac, uint32_t hmac_size)
```

说明:

HMAC 计算, 一次完成计算并生成 HMAC 结果。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: HASH 算法类型, 见 [ehsm_hash_algo_e](#)
- `key_handle`: 用于 HMAC 计算的密钥句柄
- `msg`: 输入更新的消息数据。
- `msg_size`: 消息数据长度。

- `hmac`: 存放 HMAC 值的 buffer
- `hmac_size`: 期望生成的 HMAC 长度, 不能小于 8, 也不能大于实际算法对应的 HMAC 长度

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.35 ehsm_hmac_onepass_verify

原型:

```
1| uint32_t ehsm_hmac_onepass_verify(EHSM_SHM ehsm_ctx_st *ctx, ehsm_hash_algo_e algo,  
uint32_t key_handle, EHSM_SHM const uint8_t *msg, uint32_t msg_size, EHSM_SHM const uint8_t  
*hmac, uint32_t hmac_size, bool_t *verify_result)
```

说明:

HMAC 计算, 一次完成计算并验证 HMAC 值。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: HASH 算法类型, 见 [ehsm_hash_algo_e](#)
- `key_handle`: 用于 HMAC 计算的密钥句柄
- `msg`: 输入更新的消息数据。
- `msg_size`: 消息数据长度。
- `hmac`: 待验证的 HMAC 值存放的位置
- `hmac_size`: HMAC 值大小, 不能小于 8
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.36 ehsm_hmac_init_with_plain_key

原型:

```
1| uint32_t ehsm_hmac_init_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx, ehsm_hash_algo_e  
algo, EHSM_SHM const uint8_t *key, uint32_t key_size, bool_t gen_hmac, EHSM_SHM  
ehsm_session_st *session)
```

说明:

HMAC 计算, 初始化, 使用明文密钥计算。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: HASH 算法类型, 见 [ehsm_hash_algo_e](#)
- `key`: 用于 HMAC 计算的明文密钥, 对应的内存在 finish 接口之前不能释放, 参考[明文密钥数据格式](#)
- `key_size`: 明文密钥字节数
- `gen_hmac`: 操作方式, 必须跟最后调用的 [ehsm_hmac_finish_gen](#) 或 [ehsm_hmac_finish_verify](#) 相匹配
 - `true` 生成 HMAC 值
 - `false` 校验 HMAC 值
- `session`: 用于 ehsm 内部缓存 HMAC 计算中间值的 buffer, 此参数指向的内存在 [ehsm_hmac_update](#), [ehsm_hmac_finish_gen](#) 和 [ehsm_hmac_finish_verify](#) 中仍会使用, 因此需保证其有效性, 不能提前释放。

返回值:

- EHSM_OK: 任务成功完成
- EHSM_ERR_NEED_POLL: 任务未完成
- 其它值: 任务发生错误

8.4.37 ehsm_hmac_onepass_gen_with_plain_key

原型:

```
1| uint32_t ehsm_hmac_onepass_gen_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx,  
ehsm_hash_algo_e algo, EHSM_SHM const uint8_t *key, uint32_t key_size, EHSM_SHM const  
uint8_t *msg, uint32_t msg_size, EHSM_SHM uint8_t *hmac, uint32_t hmac_size)
```

说明:

HMAC 计算, 一次完成计算并生成 HMAC 结果, 使用明文密钥计算。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: HASH 算法类型, 见 [ehsm_hash_algo_e](#)

- `key`: 用于 HMAC 计算的明文密钥, 参考[明文密钥数据格式](#)
- `key_size`: 明文密钥字节数
- `msg`: 输入更新的消息数据。
- `msg_size`: 消息数据长度。
- `hmac`: 存放 HMAC 值的 buffer
- `hmac_size`: 期望生成的 HMAC 长度, 不能小于 8, 也不能大于实际算法对应的 HMAC 长度

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.38 ehsm_hmac_onepass_verify_with_plain_key

原型:

```
1| uint32_t ehsm_hmac_onepass_verify_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx,  
ehsm_hash_algo_e algo, EHSM_SHM const uint8_t *key, uint32_t key_size, EHSM_SHM const  
uint8_t *msg, uint32_t msg_size, EHSM_SHM const uint8_t *hmac, uint32_t hmac_size, bool_t  
*verify_result)
```

说明:

HMAC 计算, 一次完成计算并验证 HMAC 值, 使用明文密钥计算。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: HASH 算法类型, 见 [ehsm_hash_algo_e](#)
- `key`: 用于 HMAC 计算的明文密钥, 参考[明文密钥数据格式](#)
- `key_size`: 明文密钥字节数
- `msg`: 输入更新的消息数据。
- `msg_size`: 消息数据长度。
- `hmac`: 待验证的 HMAC 值存放的位置
- `hmac_size`: HMAC 值大小, 不能小于 8
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.39 ehsm_mac_init

原型:

```
1| uint32_t ehsm_mac_init(EHSM_SHM ehsm_ctx_st *ctx, ehsm_symm_algo_e algo,  
ehsm_mac_mode_e mode, uint32_t key_handle, bool_t gen_mac, EHSM_SHM const uint8_t *iv,  
uint32_t iv_size, uint32_t mac_size, EHSM_SHM ehsm_session_st *session)
```

说明:

MAC 计算, 初始化。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: 对称算法的 ID, 见 [ehsm_symm_algo_e](#)
- `mode`: MAC 算法模式, 见 [ehsm_mac_mode_e](#)
- `key_handle`: 用于计算 MAC 的对称密钥的句柄
- `gen_mac`: 操作方式, 必须跟最后调用的 [ehsm_mac_finish_gen](#) 或 [ehsm_mac_finish_verify](#) 相匹配
 - `true` 生成 MAC 值
 - `false` 校验 MAC 值
- `iv`: GMAC 算法使用的 IV 值, 其它模式请传 NULL。
- `iv_size`: IV 值的字节长度, GMAC 必须是 12 字节, 其它模式请填 0。
- `mac_size`: 期望生成或需要校验的 MAC 长度, 不能小于 8, 也不能大于实际算法对应的 MAC 长度
- `session`: 用于 ehsm 内部缓存 MAC 计算中间值的 buffer, 此参数指向的内存存在 [ehsm_mac_update](#), [ehsm_mac_finish_gen](#) 和 [ehsm_mac_finish_verify](#) 中仍会使用, 因此需保证其有效性, 不能提前释放。

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.40 ehsm_mac_update

原型:

```
1| uint32_t ehsm_mac_update(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t *msg,  
uint32_t msg_size)
```

说明:

MAC 计算, 更新消息数据。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_mac_init](#) 或 [ehsm_mac_update](#)
- `msg`: 输入更新的消息数据。
- `msg_size`: 消息数据长度,
 - 对于 CBC-MAC, 要求输入的总长度不能为 0

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.41 ehsm_mac_finish_gen

原型:

```
1| uint32_t ehsm_mac_finish_gen(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t *msg,  
uint32_t msg_size, EHSM_SHM uint8_t *mac)
```

说明:

MAC 计算, 生成 MAC 结果。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_mac_init](#) 或 [ehsm_mac_update](#)
- `msg`: 输入更新的消息数据。
- `msg_size`: 消息数据长度,
 - 对于 CBC-MAC, 要求输入的总长度不能为 0
 - 对于 CMAC、CBC-MAC、GMAC, 要求当前接口 `msg_size` 不能为 0
- `mac`: 存放 MAC 值的 buffer, 其长度需大于等于 [ehsm_mac_init](#) 传入的参数 `mac_size`

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.42 ehsm_mac_finish_verify

原型:

```
1| uint32_t ehsm_mac_finish_verify(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t *msg, uint32_t msg_size, EHSM_SHM const uint8_t *mac, bool_t *verify_result)
```

说明:

MAC 计算, 验证 MAC 值。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_mac_init](#) 或 [ehsm_mac_update](#)
- `msg`: 输入更新的消息数据。
- `msg_size`: 消息数据长度,
 - 对于 CBC-MAC, 要求输入的总长度不能为 0
 - 对于 CMAC、CBC-MAC、GMAC, 要求当前接口 `msg_size` 不能为 0
- `mac`: 待验证的 MAC 值存放的位置, 其长度需等于 [ehsm_mac_init](#) 传入的参数 `mac_size`
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- EHSM_OK: 任务成功完成
- EHSM_ERR_NEED_POLL: 任务未完成
- 其它值: 任务发生错误

8.4.43 ehsm_mac_onepass_gen

原型:

```
1| uint32_t ehsm_mac_onepass_gen(EHSM_SHM ehsm_ctx_st *ctx, ehsm_symm_algo_e algo, ehsm_mac_mode_e mode, uint32_t key_handle, EHSM_SHM const uint8_t *iv, uint32_t iv_size, EHSM_SHM const uint8_t *msg, uint32_t msg_size, EHSM_SHM uint8_t *mac, uint32_t mac_size)
```

说明:

MAC 计算, 一次完成计算并生成 MAC 结果。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: 对称算法的 ID, 见 [ehsm_symm_algo_e](#)
- `mode`: MAC 算法模式, 见 [ehsm_mac_mode_e](#)

- `key_handle`: 用于计算 MAC 的对称密钥的句柄
- `iv`: GMAC 算法使用的 IV 值, 其它模式请传 NULL。
- `iv_size`: IV 值的字节长度, GMAC 必须是 12 字节, 其它模式请填 0。
- `msg`: 输入更新的消息数据。
- `msg_size`: 消息数据长度。
- `mac`: 存放 MAC 值的 buffer
- `mac_size`: 期望生成的 MAC 长度, 不能小于 8, 也不能大于实际算法对应的 MAC 长度

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.44 ehsm_mac_onepass_verify

原型:

```
1| uint32_t ehsm_mac_onepass_verify(EHSM_SHM ehsm_ctx_st *ctx, ehsm_symm_algo_e algo,  
ehsm_mac_mode_e mode, uint32_t key_handle, EHSM_SHM const uint8_t *iv, uint32_t iv_size,  
EHSM_SHM const uint8_t *msg, uint32_t msg_size, EHSM_SHM const uint8_t *mac, uint32_t  
mac_size, bool_t *verify_result)
```

说明:

MAC 计算, 一次完成计算并验证 MAC 值。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `algo`: 对称算法的 ID, 见 `ehsm_symm_algo_e`
- `mode`: MAC 算法模式, 见 `ehsm_mac_mode_e`
- `key_handle`: 用于计算 MAC 的对称密钥的句柄
- `iv`: GMAC 算法使用的 IV 值, 其它模式请传 NULL。
- `iv_size`: IV 值的字节长度, GMAC 必须是 12 字节, 其它模式请填 0。
- `msg`: 输入更新的消息数据。
- `msg_size`: 消息数据长度。
- `mac`: 待验证的 MAC 值存放的位置
- `mac_size`: MAC 值大小, 不能小于 8
- `verify_result`: 存放验证结果的内存位置

- `true` 验证通过
- `false` 验证不通过

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.45 ehsm_mac_init_with_plain_key

原型:

```
1| uint32_t ehsm_mac_init_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx, ehsm_symm_algo_e  
algo, ehsm_mac_mode_e mode, EHSM_SHM const uint8_t *key, uint32_t key_size, bool_t gen_mac,  
EHSM_SHM const uint8_t *iv, uint32_t iv_size, uint32_t mac_size, EHSM_SHM ehsm_session_st  
*session)
```

说明:

MAC 计算, 初始化, 使用明文密钥计算。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: 对称算法的 ID, 见 [ehsm_symm_algo_e](#)
- `mode`: MAC 算法模式, 见 [ehsm_mac_mode_e](#)
- `key`: 用于 MAC 计算的明文密钥, 对应的内存在 finish 接口之前不能释放, 参考[明文密钥数据格式](#)
- `key_size`: 明文密钥字节数
- `gen_mac`: 操作方式, 必须跟最后调用的 [ehsm_mac_finish_gen](#) 或 [ehsm_mac_finish_verify](#) 相匹配
 - `true` 生成 MAC 值
 - `false` 校验 MAC 值
- `iv`: GMAC 算法使用的 IV 值, 其它模式请传 NULL。
- `iv_size`: IV 值的字节长度, GMAC 必须是 12 字节, 其它模式请填 0。
- `mac_size`: 期望生成或需要校验的 MAC 长度, 不能小于 8, 也不能大于实际算法对应的 MAC 长度
- `session`: 用于 ehsm 内部缓存 MAC 计算中间值的 buffer, 此参数指向的内存在 [ehsm_mac_update](#), [ehsm_mac_finish_gen](#) 和 [ehsm_mac_finish_verify](#) 中仍会使用, 因此需保证其有效性, 不能提前释放。

返回值:

- `EHSM_OK`: 任务成功完成

- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.46 ehsm_mac_onepass_gen_with_plain_key

原型:

```
1| uint32_t ehsm_mac_onepass_gen_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx,  
ehsm_symm_algo_e algo, ehsm_mac_mode_e mode, EHSM_SHM const uint8_t *key, uint32_t  
key_size, EHSM_SHM const uint8_t *iv, uint32_t iv_size, EHSM_SHM const uint8_t *msg,  
uint32_t msg_size, EHSM_SHM uint8_t *mac, uint32_t mac_size)
```

说明:

MAC 计算, 一次完成计算并生成 MAC 结果, 使用明文密钥计算。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: 对称算法的 ID, 见 [ehsm_symm_algo_e](#)
- `mode`: MAC 算法模式, 见 [ehsm_mac_mode_e](#)
- `key`: 用于 MAC 计算的明文密钥, 参考[明文密钥数据格式](#)
- `key_size`: 明文密钥字节数
- `iv`: GMAC 算法使用的 IV 值, 其它模式请传 NULL。
- `iv_size`: IV 值的字节长度, GMAC 必须是 12 字节, 其它模式请填 0。
- `msg`: 输入更新的消息数据。
- `msg_size`: 消息数据长度。
- `mac`: 存放 MAC 值的 buffer
- `mac_size`: 期望生成的 MAC 长度, 不能小于 8, 也不能大于实际算法对应的 MAC 长度

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.47 ehsm_mac_onepass_verify_with_plain_key

原型:

```
1| uint32_t ehsm_mac_onepass_verify_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx,  
ehsm_symm_algo_e algo, ehsm_mac_mode_e mode, EHSM_SHM const uint8_t *key, uint32_t
```

```
key_size, EHSM_SHM const uint8_t *iv, uint32_t iv_size, EHSM_SHM const uint8_t *msg,
uint32_t msg_size, EHSM_SHM const uint8_t *mac, uint32_t mac_size, bool_t *verify_result)
```

说明:

MAC 计算，一次完成计算并验证 MAC 值，使用明文密钥计算。

参数:

- `ctx`: ehsm context 地址，需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: 对称算法的 ID，见 [ehsm_symm_algo_e](#)
- `mode`: MAC 算法模式，见 [ehsm_mac_mode_e](#)
- `key`: 用于 MAC 计算的明文密钥，参考[明文密钥数据格式](#)
- `key_size`: 明文密钥字节数
- `iv`: GMAC 算法使用的 IV 值，其它模式请传 NULL。
- `iv_size`: IV 值的字节长度，GMAC 必须是 12 字节，其它模式请填 0。
- `msg`: 输入更新的消息数据。
- `msg_size`: 消息数据长度。
- `mac`: 待验证的 MAC 值存放的位置
- `mac_size`: MAC 值大小，不能小于 8
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.48 ehsm_symm_cipher_init

原型:

```
1| uint32_t ehsm_symm_cipher_init(EHSM_SHM ehsm_ctx_st *ctx, ehsm_symm_algo_e algo,
ehsm_cipher_mode_e mode, ehsm_padding_mode_e padding, uint32_t key_handle, bool_t enc,
EHSM_SHM const uint8_t *iv, uint32_t iv_size, EHSM_SHM ehsm_session_st *session)
```

说明:

对称加密/解密计算，初始化。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: 对称加密算法, 见 [ehsm_symm_algo_e](#)
- `mode`: 对称加密模式, 见 [ehsm_cipher_mode_e](#)
- `padding`: 对称加密填充算法, 见 [ehsm_padding_mode_e](#), 对于 XTS 模式, 此参数被忽略
- `key_handle`: 对称密钥句柄
- `enc`: 是否加密
 - `true` 加密
 - `false` 解密
- `iv`: IV/Nonce 值
 - ECB 模式下忽略此值, 应传入 `NULL`
 - CTR 模式下为 Initial Counter
 - XTS 模式下为 initial i value
- `iv_size`: `iv` 数据的长度, 应为算法的 block size
- `session`: 用于 ehsm 内部缓存 cipher 计算中间值的 buffer, 此参数指向的内存存在 [ehsm_symm_cipher_update](#) 和 [ehsm_symm_cipher_finish](#) 中仍会使用, 因此需保证其有效性, 不能提前释放。

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.49 ehsm_symm_cipher_update**原型:**

```
1| uint32_t ehsm_symm_cipher_update(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t
*input, uint32_t input_size, EHSM_SHM uint8_t *output)
```

说明:

对称加密/解密计算, 更新数据。 此函数处理连续的块数据, 输入数据长度必须 block size 对齐, 实际写入 `output` 中的数据与输入数据长度相同。

此函数不会进行填充或去填充:

- 在解密模式下, 必须保留至少一个 block 密文数据给 [ehsm_symm_cipher_finish](#) 调用, 以便去掉填充
- 在加密模式下, 保证输入数据 block size 对齐即可

在 XTS 模式下，无论是加密还是解密，必须保留至少 2 个 block 的数据给 `ehsm_symm_cipher_finish` 处理

参数：

- `ctx`: ehsm context 地址，需事先调用过 `ehsm_symm_cipher_init` 初始化
- `input`: 输入数据
- `input_size`: 输入数据长度，必须 block size 对齐
- `output`: 输出 buffer，其空间大小不能小于 `input_size`

返回值：

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.50 ehsm_symm_cipher_finish

原型：

```
1| uint32_t ehsm_symm_cipher_finish(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t  
*input, uint32_t input_size, EHSM_SHM uint8_t *output, uint32_t *output_size)
```

说明：

对称加密/解密计算，结束计算。此函数处理最后一部分数据：

- 在加密模式下，如果填充是 `EHSM_PADDING_NONE`，则 `input_size` 应当为 0 或 block size 对齐
- 在加密模式下，如果填充不是 `EHSM_PADDING_NONE`，则 `input_size` 可以为任意长度
- 在解密模式下，如果填充是 `EHSM_PADDING_NONE`，则 `input_size` 应当为 0 或 block size 对齐
- 在解密模式下，如果填充不是 `EHSM_PADDING_NONE`，则 `input_size` 必须是 block size 对齐且至少有 1 个 block。

在 XTS 模式下，无论是加密还是解密，必须保留至少 2 个 block 的数据给此函数处理

参数：

- `ctx`: ehsm context 地址，需事先调用过 `ehsm_symm_cipher_init` 或 `ehsm_symm_cipher_update`
- `input`: 输入数据
- `input_size`: 输入数据长度，见说明
- `output`: 输出 buffer

- `output_size`: 传入时为输出 buffer 长度, 加密且有填充时, 其长度应比 `input_size` 大一个 block size; 完成时为实际的输出数据长度

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.51 ehsm_symm_cipher_onepass

原型:

```
1| uint32_t ehsm_symm_cipher_onepass(EHSM_SHM ehsm_ctx_st *ctx, ehsm_symm_algo_e algo,
ehsm_cipher_mode_e mode, ehsm_padding_mode_e padding, uint32_t key_handle, bool_t enc,
EHSM_SHM const uint8_t *iv, uint32_t iv_size, EHSM_SHM const uint8_t *input, uint32_t
input_size, EHSM_SHM uint8_t *output, uint32_t *output_size)
```

说明:

对称加密/解密计算, 一次完成计算。此函数一次处理所有数据:

- 在加密模式下, 如果填充是 `EHSM_PADDING_NONE`, 则 `input_size` 应当为 0 或 block size 对齐
- 在加密模式下, 如果填充不是 `EHSM_PADDING_NONE`, 则 `input_size` 可以为任意长度
- 在解密模式下, 如果填充是 `EHSM_PADDING_NONE`, 则 `input_size` 应当为 0 或 block size 对齐
- 在解密模式下, 如果填充不是 `EHSM_PADDING_NONE`, 则 `input_size` 必须是 block size 对齐且至少有 1 个 block。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `algo`: 对称加密算法, 见 `ehsm_symm_algo_e`
- `mode`: 对称加密模式, 见 `ehsm_cipher_mode_e`
- `padding`: 对称加密填充算法, 见 `ehsm_padding_mode_e`, 对于 XTS 模式, 此参数被忽略
- `key_handle`: 对称密钥句柄
- `enc`: 是否加密
 - `true` 加密
 - `false` 解密
- `iv`: IV/Nonce 值
 - ECB 模式下忽略此值, 应传入 `NULL`
 - CTR 模式下为 Initial Counter
 - XTS 模式下为 initial i value

- `iv_size`: `iv` 数据的长度, 应为算法的 block size
- `input`: 输入数据
- `input_size`: 输入数据长度, 见说明
- `output`: 输出 buffer
- `output_size`: 传入时为输出 buffer 长度, 加密且有填充时, 其长度应比 `input_size` 大一个 block size; 完成时为实际的输出数据长度

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.52 ehsm_symm_cipher_init_with_plain_key

原型:

```
1| uint32_t ehsm_symm_cipher_init_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx,
ehsm_symm_algo_e algo, ehsm_cipher_mode_e mode, ehsm_padding_mode_e padding, EHSM_SHM const
uint8_t *key, uint32_t key_size, bool_t enc, EHSM_SHM const uint8_t *iv, uint32_t iv_size,
EHSM_SHM ehsm_session_st *session)
```

说明:

对称加密/解密计算, 初始化, 使用明文密钥计算。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `algo`: 对称加密算法, 见 `ehsm_symm_algo_e`
- `mode`: 对称加密模式, 见 `ehsm_cipher_mode_e`
- `padding`: 对称加密填充算法, 见 `ehsm_padding_mode_e`, 对于 XTS 模式, 此参数被忽略
- `key`: 用于加密/解密的明文密钥, 对应的内存在 finish 接口之前不能释放, 参考明文密钥数据格式
- `key_size`: 明文密钥字节数
- `enc`: 是否加密
 - `true` 加密
 - `false` 解密
- `iv`: IV/Nonce 值
 - ECB 模式下忽略此值, 应传入 `NULL`
 - CTR 模式下为 Initial Counter
 - XTS 模式下为 initial i value

- `iv_size`: `iv` 数据的长度, 应为算法的 block size
- `session`: 用于 ehsm 内部缓存 cipher 计算中间值的 buffer, 此参数指向的内存存在 [ehsm_symm_cipher_update](#) 和 [ehsm_symm_cipher_finish](#) 中仍会使用, 因此需保证其有效性, 不能提前释放。

返回值:

- EHSM_OK: 任务成功完成
- EHSM_ERR_NEED_POLL: 任务未完成
- 其它值: 任务发生错误

8.4.53 ehsm_symm_cipher_onepass_with_plain_key

原型:

```
1| uint32_t ehsm_symm_cipher_onepass_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx,
ehsm_symm_algo_e algo, ehsm_cipher_mode_e mode, ehsm_padding_mode_e padding, EHSM_SHM const
uint8_t *key, uint32_t key_size, bool_t enc, EHSM_SHM const uint8_t *iv, uint32_t iv_size,
EHSM_SHM const uint8_t *input, uint32_t input_size, EHSM_SHM uint8_t *output, uint32_t
*output_size)
```

说明:

对称加密/解密计算, 一次完成计算, 使用明文密钥计算。此函数一次处理所有数据:

- 在加密模式下, 如果填充是 [EHSM_PADDING_NONE](#), 则 `input_size` 应当为 0 或 block size 对齐
- 在加密模式下, 如果填充不是 [EHSM_PADDING_NONE](#), 则 `input_size` 可以为任意长度
- 在解密模式下, 如果填充是 [EHSM_PADDING_NONE](#), 则 `input_size` 应当为 0 或 block size 对齐
- 在解密模式下, 如果填充不是 [EHSM_PADDING_NONE](#), 则 `input_size` 必须是 block size 对齐且至少有 1 个 block。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: 对称加密算法, 见 [ehsm_symm_algo_e](#)
- `mode`: 对称加密模式, 见 [ehsm_cipher_mode_e](#)
- `padding`: 对称加密填充算法, 见 [ehsm_padding_mode_e](#), 对于 XTS 模式, 此参数被忽略
- `key`: 用于加密/解密的明文密钥, 参考[明文密钥数据格式](#)
- `key_size`: 明文密钥字节数
- `enc`: 是否加密
 - `true` 加密
 - `false` 解密

- `iv`: IV/Nonce 值
 - ECB 模式下忽略此值, 应传入 `NULL`
 - CTR 模式下为 Initial Counter
 - XTS 模式下为 initial i value
- `iv_size`: `iv` 数据的长度, 应为算法的 block size
- `input`: 输入数据
- `input_size`: 输入数据长度, 见说明
- `output`: 输出 buffer
- `output_size`: 传入时为输出 buffer 长度, 加密且有填充时, 其长度应比 `input_size` 大一个 block size; 完成时为实际的输出数据长度

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.54 ehsm_sm2_cipher

原型:

```
1| uint32_t ehsm_sm2_cipher(EHSM_SHM ehsm_ctx_st *ctx, uint32_t key_handle, bool_t enc,
EHSM_SHM const uint8_t *input, uint32_t input_size, EHSM_SHM uint8_t *output, uint32_t
*output_size)
```

说明:

SM2 加密/解密计算, 一次完成计算。

注意:

- SM2 加解密不支持三段式调用。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `key_handle`: SM2 密钥句柄
- `enc`: 是否加密
 - `true` 加密, 密钥句柄对应的密钥中必须包含公钥
 - `false` 解密, 密钥句柄对应的密钥中必须包含私钥
- `input`: 输入的明文或密文, 密文的格式为 C1C3C2, 其中
 - C1 为曲线上的点, 65 字节, 以 `0x04` 开头, 以及完整 x 和 y 坐标
 - C2 为实际的密文数据, 与对应的明文长度相同
 - C3 为 SM3 的结果, 32 字节, 用于保证完整性

- `input_size`: 输入数据的长度, 最大不能超过 1024 字节
- `output`: 保存输出数据的 buffer, 如果是加密, 则密文格式 C1C3C2, 见 `input` 的描述
- `output_size`: 传入时为输出 buffer 的长度, 如果是加密, 应大于等于 `input_size + 97`; 如果是解密, 应大于等于 `input_size - 97`

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.55 ehsm_sm2_cipher_with_plain_key

原型:

```
1| uint32_t ehsm_sm2_cipher_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const
uint8_t *key, bool_t enc, EHSM_SHM const uint8_t *input, uint32_t input_size, EHSM_SHM
uint8_t *output, uint32_t *output_size)
```

说明:

SM2 加密/解密计算, 一次完成计算, 使用明文密钥计算。

注意:

- SM2 加解密不支持三段式调用。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `key`: 用于加密/解密的明文密钥, 参考明文密钥数据格式和 `ehsm_sm2_key_st`
- `enc`: 是否加密
 - `true` 加密, 密钥数据必须包含公钥
 - `false` 解密, 密钥数据必须包含私钥
- `input`: 输入的明文或密文, 密文的格式为 C1C3C2, 其中
 - C1 为曲线上的点, 65 字节, 以 `0x04` 开头, 以及完整 x 和 y 坐标
 - C2 为实际的密文数据, 与对应的明文长度相同
 - C3 为 SM3 的结果, 32 字节, 用于保证完整性
- `input_size`: 输入数据的长度, 最大不能超过 1024 字节
- `output`: 保存输出数据的 buffer, 如果是加密, 则密文格式 C1C3C2, 见 `input` 的描述
- `output_size`: 传入时为输出 buffer 的长度, 如果是加密, 应大于等于 `input_size + 97`; 如果是解密, 应大于等于 `input_size - 97`

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.56 ehsm_sm2_sign_init

原型:

```
1| uint32_t ehsm_sm2_sign_init(EHSM_SHM ehsm_ctx_st *ctx, uint32_t key_handle, bool_t  
gen_sig, EHSM_SHM ehsm_session_st *session)
```

说明:

SM2 签名/验签计算, 初始化。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `key_handle`: SM2 密钥句柄
- `gen_sig` : 是否生成签名, 必须跟最后调用的 [ehsm_sm2_sign_finish_gen](#) 或 [ehsm_sm2_sign_finish_verify](#) 相匹配
 - `true` 生成签名值
 - `false` 验证签名值
- `session` : 用于 ehsm 内部缓存 HASH 计算中间值的 buffer, 此参数指向的内存在 [ehsm_sm2_sign_update](#)、[ehsm_sm2_sign_finish_gen](#) 和 [ehsm_sm2_sign_finish_verify](#) 中仍会使用, 因此需保证其有效性, 不能提前释放。

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.57 ehsm_sm2_sign_init_with_plain_key

原型:

```
1| uint32_t ehsm_sm2_sign_init_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const  
uint8_t *key, bool_t gen_sig, EHSM_SHM ehsm_session_st *session)
```

说明:

SM2 签名/验签计算, 初始化, 使用明文密钥计算。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `key`: 用于签名/验签的密钥值, 对应的内存在 finish 接口之前不能释放, 参考[明文密钥数据格式](#)和 [ehsm_sm2_key_st](#)
- `gen_sig`: 是否生成签名, 必须跟最后调用的 [ehsm_sm2_sign_finish_gen](#) 或 [ehsm_sm2_sign_finish_verify](#) 相匹配
 - `true` 生成签名值
 - `false` 验证签名值
- `session`: 用于 ehsm 内部缓存 HASH 计算中间值的 buffer, 此参数指向的内存在 [ehsm_sm2_sign_update](#)、[ehsm_sm2_sign_finish_gen](#) 和 [ehsm_sm2_sign_finish_verify](#) 中仍会使用, 因此需保证其有效性, 不能提前释放。

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.58 ehsm_sm2_sign_update

原型:

```
1| uint32_t ehsm_sm2_sign_update(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t *msg,  
uint32_t msg_size)
```

说明:

SM2 签名/验签计算, 更新数据。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_sm2_sign_init](#) 或 [ehsm_sm2_sign_init_with_plain_key](#) 或 [ehsm_sm2_sign_update](#)
- `msg`: 输入更新的消息数据
- `msg_size`: 输入的消息数据长度

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.59 ehsm_sm2_sign_finish_gen

原型:

```
1| uint32_t ehsm_sm2_sign_finish_gen(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM uint8_t *sig,
uint32_t sig_size)
```

说明:

SM2 签名/验签计算, 生成签名值。生成的 SM2 签名值的长度固定为 64 字节。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_sm2_sign_init](#) 或 [ehsm_sm2_sign_update](#)
- `sig`: 保存签名值的 buffer
- `sig_size`: 签名 buffer 的长度, 不能小于 64

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.60 ehsm_sm2_sign_finish_verify

原型:

```
1| uint32_t ehsm_sm2_sign_finish_verify(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t
*sig, uint32_t sig_size, bool_t *verify_result)
```

说明:

SM2 签名/验签计算, 验证签名值。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_sm2_sign_init](#) 或 [ehsm_sm2_sign_update](#)
- `sig`: 待验证的签名值
- `sig_size`: 签名值长度, 仅接受 64 字节签名值
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.61 ehsm_sm2_sign_onepass_ex

原型:

```
1| uint32_t ehsm_sm2_sign_onepass_ex(EHSM_SHM ehsm_ctx_st *ctx, bool_t use_plain_key,  
uint32_t key_handle, EHSM_SHM const uint8_t *key, bool_t gen_sig, bool_t is_digest,  
EHSM_SHM const uint8_t *input, uint32_t input_size, EHSM_SHM uintptr_t sig_addr, uint32_t  
sig_size, bool_t *verify_result)
```

说明:

SM2 签名生成/验证扩展接口, 支持明文密钥、密钥句柄、摘要输入等多种模式的一次性签名操作。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `use_plain_key`: 是否使用明文密钥
 - `true` 使用明文密钥, `key` 参数有效, `key_handle` 参数被忽略
 - `false` 使用密钥句柄, `key_handle` 参数有效, `key` 参数被忽略
- `key_handle`: SM2 密钥句柄, 当 `use_plain_key` 为 `false` 时使用
- `key`: 用于签名/验证的明文密钥, 当 `use_plain_key` 为 `true` 时使用, 格式参考[明文密钥数据格式](#)和 [ehsm_sm2_key_st](#)
- `gen_sig`: 操作方式
 - `true` 生成签名值
 - `false` 验证签名值
- `is_digest`: 输入数据类型
 - `true` `input` 为已计算好的消息摘要(E 值), 长度必须为 32 字节
 - `false` `input` 为原始消息数据, 由 eHSM 内部计算摘要
- `input`: 输入数据, 根据 `is_digest` 参数决定是消息原文还是消息摘要
- `input_size`: 输入数据长度, 当 `is_digest` 为 `true` 时必须为 32 字节
- `sig_addr`: 签名值的地址
 - 生成签名时: 作为输出 buffer 的地址, 存放生成的签名值
 - 验证签名时: 作为输入 buffer 的地址, 存放待验证的签名值
- `sig_size`: 签名 buffer 长度, SM2 签名固定为 64 字节
- `verify_result`: 验证结果存放地址, 仅在 `gen_sig` 为 `false` 时有效
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成

- 其它值：任务发生错误

8.4.62 ehsm_sm2_sign_onepass_gen

原型：

```
1| uint32_t ehsm_sm2_sign_onepass_gen(EHSM_SHM ehsm_ctx_st *ctx, uint32_t key_handle,  
EHSM_SHM const uint8_t *msg, uint32_t msg_size, EHSM_SHM uint8_t *sig, uint32_t sig_size)
```

说明：

SM2 签名值生成，一次计算完成。

参数：

- `ctx`：ehsm context 地址，需事先调用过 `ehsm_ctx_init` 初始化
- `key_handle`：SM2 密钥句柄
- `msg`：输入更新的消息数据
- `msg_size`：输入的消息数据长度
- `sig`：保存签名值的 buffer
- `sig_size`：签名 buffer 的长度，不能小于 64

返回值：

- EHSM_OK：任务成功完成
- EHSM_ERR_NEED_POLL：任务未完成
- 其它值：任务发生错误

8.4.63 ehsm_sm2_sign_onepass_verify

原型：

```
1| uint32_t ehsm_sm2_sign_onepass_verify(EHSM_SHM ehsm_ctx_st *ctx, uint32_t key_handle,  
EHSM_SHM const uint8_t *msg, uint32_t msg_size, EHSM_SHM const uint8_t *sig, uint32_t  
sig_size, bool_t *verify_result)
```

说明：

SM2 验证签名值，一次计算完成。

参数：

- `ctx`：ehsm context 地址，需事先调用过 `ehsm_ctx_init` 初始化
- `key_handle`：SM2 密钥句柄
- `msg`：输入更新的消息数据

- `msg_size`: 输入的消息数据长度
- `sig`: 待验证的签名值
- `sig_size`: 签名值长度, 仅接受 64 字节签名值
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.64 ehsm_sm2_sign_onepass_gen_with_digest

原型:

```
1 | uint32_t ehsm_sm2_sign_onepass_gen_with_digest(EHSM_SHM ehsm_ctx_st *ctx, uint32_t  
key_handle, EHSM_SHM const uint8_t *digest, uint32_t digest_size, EHSM_SHM uint8_t *sig,  
uint32_t sig_size)
```

说明:

SM2 签名值生成, 一次计算完成, 外部传入计算好的消息摘要(E 值)方式。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `key_handle`: SM2 密钥句柄
- `digest`: 输入的消息的摘要(E 值)数据
- `digest_size`: 输入的消息的摘要数据长度, 仅支持 32 字节长度。
- `sig`: 保存签名值的 buffer
- `sig_size`: 签名 buffer 的长度, 不能小于 64

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.65 ehsm_sm2_sign_onepass_verify_with_digest

原型:

```
1| uint32_t ehsm_sm2_sign_onepass_verify_with_digest(EHSM_SHM ehsm_ctx_st *ctx, uint32_t  
key_handle, EHSM_SHM const uint8_t *digest, uint32_t digest_size, EHSM_SHM const uint8_t  
*sig, uint32_t sig_size, bool_t *verify_result)
```

说明:

SM2 验证签名值, 一次计算完成, 外部传入计算好的消息摘要(E 值)方式。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `key_handle`: SM2 密钥句柄
- `digest`: 输入消息的摘要数据
- `digest_size`: 输入摘要数据长度, 仅支持 32 字节长度。
- `sig`: 待验证的签名值
- `sig_size`: 签名值长度, 仅接受 64 字节签名值
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.66 ehsm_sm2_sign_onepass_gen_with_plain_key

原型:

```
1| uint32_t ehsm_sm2_sign_onepass_gen_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM  
const uint8_t *key, EHSM_SHM const uint8_t *msg, uint32_t msg_size, EHSM_SHM uint8_t *sig,  
uint32_t sig_size)
```

说明:

SM2 签名值生成, 一次计算完成, 使用明文密钥计算。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `key`: 用于签名/验签的明文密钥, 参考明文密钥数据格式和 `ehsm_sm2_key_st`
- `msg`: 输入更新的消息数据

- `msg_size`: 输入的消息数据长度
- `sig`: 保存签名值的 buffer
- `sig_size`: 签名 buffer 的长度, 不能小于 64

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.67 ehsm_sm2_sign_onepass_verify_with_plain_key

原型:

```
1| uint32_t ehsm_sm2_sign_onepass_verify_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx,  
EHSM_SHM const uint8_t *key, EHSM_SHM const uint8_t *msg, uint32_t msg_size, EHSM_SHM const  
uint8_t *sig, uint32_t sig_size, bool_t *verify_result)
```

说明:

SM2 验证签名值, 一次计算完成, 使用明文密钥计算。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `key`: 用于签名/验签的明文密钥, 参考[明文密钥数据格式](#)和 [ehsm_sm2_key_st](#)
- `msg`: 输入更新的消息数据
- `msg_size`: 输入的消息数据长度
- `sig`: 待验证的签名值
- `sig_size`: 签名值长度, 仅接受 64 字节签名值
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.68 ehsm_sm2_sign_onepass_gen_with_plain_key_and_digest

原型:

```
1| uint32_t ehsm_sm2_sign_onepass_gen_with_plain_key_and_digest(EHSM_SHM ehsm_ctx_st
*ctx, EHSM_SHM const uint8_t *key, EHSM_SHM const uint8_t *digest, uint32_t digest_size,
EHSM_SHM uint8_t *sig, uint32_t sig_size)
```

说明:

SM2 签名值生成, 一次计算完成, 使用明文密钥计算。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `key`: 用于签名/验签的明文密钥, 参考[明文密钥数据格式](#)和 [ehsm_sm2_key_st](#)
- `digest`: 输入的消息的摘要(E 值)数据
- `digest_size`: 输入的消息的摘要数据长度, 仅支持 32 字节长度
- `sig`: 保存签名值的 buffer
- `sig_size`: 签名 buffer 的长度, 不能小于 64

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.69 ehsm_sm2_sign_onepass_verify_with_plain_key_and_digest

原型:

```
1| uint32_t ehsm_sm2_sign_onepass_verify_with_plain_key_and_digest(EHSM_SHM ehsm_ctx_st
*ctx, EHSM_SHM const uint8_t *key, EHSM_SHM const uint8_t *digest, uint32_t digest_size,
EHSM_SHM const uint8_t *sig, uint32_t sig_size, bool_t *verify_result)
```

说明:

SM2 验证签名值, 一次计算完成, 使用明文密钥计算。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `key`: 用于签名/验签的明文密钥, 参考[明文密钥数据格式](#)和 [ehsm_sm2_key_st](#)
- `digest`: 输入消息的摘要数据
- `digest_size`: 输入摘要数据长度, 仅支持 32 字节长度
- `sig`: 待验证的签名值
- `sig_size`: 签名值长度, 仅接受 64 字节签名值

- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.70 ehsm_rsa_cipher

原型:

```
1| uint32_t ehsm_rsa_cipher(EHSM_SHM ehsm_ctx_st *ctx, uint32_t key_handle, bool_t enc,  
EHSM_SHM const uint8_t *input, uint32_t input_size, EHSM_SHM uint8_t *output, uint32_t  
*output_size)
```

说明:

RSA 加密/解密计算，一次完成计算。

注意:

- RSA 加解密不支持三段式调用。

参数:

- `ctx`: ehsm context 地址，需事先调用过 `ehsm_ctx_init` 初始化
- `key_handle`: RSA 密钥句柄
- `enc`: 是否加密
 - `true` 加密，密钥句柄对应的密钥中必须包含公钥
 - `false` 解密，密钥句柄对应的密钥中必须包含私钥
- `input`: 输入的明文或密文，输入后会作为一个大数参与运算，其值应小于 N，长度可以小于对应密钥长度，字节序为大端
- `input_size`: 输入数据的长度
- `output`: 保存输出数据的 buffer
- `output_size`: 传入时为输出 buffer 的长度，完成时为实际的输出数据长度

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.71 ehsm_rsa_cipher_with_plain_key

原型:

```
1| uint32_t ehsm_rsa_cipher_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const  
uint8_t *key, bool_t enc, EHSM_SHM const uint8_t *input, uint32_t input_size, EHSM_SHM  
uint8_t *output, uint32_t *output_size)
```

说明:

RSA 加密/解密计算，一次完成计算，使用明文密钥计算。

注意:

- RSA 加解密不支持三段式调用。

参数:

- `ctx`: ehsm context 地址，需事先调用过 `ehsm_ctx_init` 初始化
- `key`: 用于加密/解密的明文密钥，参考[明文密钥数据格式](#)和 `ehsm_rsa_key_st`
- `enc`: 是否加密
 - `true` 加密，密钥句柄对应的密钥中必须包含公钥
 - `false` 解密，密钥句柄对应的密钥中必须包含私钥
- `input`: 输入的明文或密文，输入后会作为一个大数参与运算，其值应小于 N，长度可以小于对应密钥长度，字节序为大端
- `input_size`: 输入数据的长度
- `output`: 保存输出数据的 buffer
- `output_size`: 传入时为输出 buffer 的长度，完成时为实际的输出数据长度

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.72 ehsm_rsa_sign_init

原型:

```
1| uint32_t ehsm_rsa_sign_init(EHSM_SHM ehsm_ctx_st *ctx, ehsm_hash_algo_e algo, uint32_t  
key_handle, bool_t gen_sig, ehsm_rsa_padding_mode_e padding, EHSM_SHM ehsm_session_st  
*session)
```

说明:

RSA 签名/验签计算，初始化。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: 签名对应的 HASH 算法, 见 [ehsm_hash_algo_e](#), 仅支持 SHA1 和 SHA2 系列算法, MD5、SM3 和 SHA3 系列不支持
- `key_handle`: RSA 密钥句柄
- `gen_sig`: 是否生成签名, 必须跟最后调用的 [ehsm_rsa_sign_finish_gen](#) 或 [ehsm_rsa_sign_finish_verify](#) 相匹配
 - `true` 生成签名值
 - `false` 验证签名值
- `padding`: 填充方式, 见 [ehsm_rsa_padding_mode_e](#)
- `session`: 用于 ehsm 内部缓存 HASH 计算中间值的 buffer, 此参数指向的内存在 [ehsm_rsa_sign_update](#)、[ehsm_rsa_sign_finish_gen](#) 和 [ehsm_rsa_sign_finish_verify](#) 中仍会使用, 因此需保证其有效性, 不能提前释放。

返回值:

- EHSM_OK: 任务成功完成
- EHSM_ERR_NEED_POLL: 任务未完成
- 其它值: 任务发生错误

8.4.73 ehsm_rsa_sign_init_with_plain_key

原型:

```
1| uint32_t ehsm_rsa_sign_init_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx, ehsm_hash_algo_e  
algo, EHSM_SHM const uint8_t *key, bool_t gen_sig, ehsm_rsa_padding_mode_e padding,  
EHSM_SHM ehsm_session_st *session)
```

说明:

RSA 签名/验签计算, 初始化。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: 签名对应的 HASH 算法, 见 [ehsm_hash_algo_e](#), 仅支持 SHA1 和 SHA2 系列算法, MD5、SM3 和 SHA3 系列不支持
- `key`: 用于 RSA 签名/验签计算的明文密钥, 对应的内存在 finish 接口之前不能释放, 参考[明文密钥数据格式](#)和 [ehsm_rsa_key_st](#)
- `gen_sig`: 是否生成签名, 必须跟最后调用的 [ehsm_rsa_sign_finish_gen](#) 或 [ehsm_rsa_sign_finish_verify](#) 相匹配
 - `true` 生成签名值
 - `false` 验证签名值

- `padding`: 填充方式, 见 [ehsm_rsa_padding_mode_e](#)
- `session`: 用于 ehsm 内部缓存 HASH 计算中间值的 buffer, 此参数指向的内存在 [ehsm_rsa_sign_update](#)、[ehsm_rsa_sign_finish_gen](#) 和 [ehsm_rsa_sign_finish_verify](#) 中仍会使用, 因此需保证其有效性, 不能提前释放。

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.74 ehsm_rsa_sign_update

原型:

```
1| uint32_t ehsm_rsa_sign_update(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t *msg,  
uint32_t msg_size)
```

说明:

RSA 签名/验签计算, 更新数据。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_rsa_sign_init](#) 或 [ehsm_rsa_sign_update](#)
- `msg`: 输入更新的消息数据
- `msg_size`: 输入的消息数据长度

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.75 ehsm_rsa_sign_finish_gen

原型:

```
1| uint32_t ehsm_rsa_sign_finish_gen(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM uint8_t *sig,  
uint32_t *sig_size, uint32_t salt_size)
```

说明:

RSA 签名/验签计算, 生成签名值。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_rsa_sign_init` 或 `ehsm_rsa_sign_update`
- `sig`: 保存签名值的 buffer
- `sig_size`: 输入时为 buffer 长度, 完成时为写入 buffer 中的签名数据长度
- `salt_size`: 当填充方式为 `EHSM_RSA_PADDING_PSS` 时, 生成的盐值的长度; 当填充方式为 `EHSM_RSA_PADDING_NONE` 时, 'salt_size' 应为 0

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.76 ehsm_rsa_sign_finish_verify

原型:

```
1 | uint32_t ehsm_rsa_sign_finish_verify(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t  
*sig, uint32_t sig_size, uint32_t salt_size, bool_t *verify_result)
```

说明:

RSA 签名/验签计算, 验证签名值。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_rsa_sign_init` 或 `ehsm_rsa_sign_update`
- `sig`: 待验证的签名值
- `sig_size`: 签名值长度
- `salt_size`: 当填充方式为 `EHSM_RSA_PADDING_PSS` 时, 待验签数据的盐值的长度; 当填充方式为 `EHSM_RSA_PADDING_NONE` 时, 'salt_size' 应为 0
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.77 ehsm_rsa_sign_onepass_ex

原型:

```
1| uint32_t ehsm_rsa_sign_onepass_ex(EHSM_SHM ehsm_ctx_st *ctx, ehsm_hash_algo_e algo,
bool_t use_plain_key, uint32_t key_handle, EHSM_SHM const uint8_t *key,
ehsm_rsa_padding_mode_e padding, bool_t gen_sig, bool_t is_digest, EHSM_SHM const uint8_t
*input, uint32_t input_size, EHSM_SHM uintptr_t sig_addr, uint32_t *sig_size, uint32_t
salt_size, bool_t *verify_result)
```

说明:

RSA 签名生成/验证扩展接口，支持明文密钥、密钥句柄、摘要输入等多种模式的一次性签名操作。

参数:

- `ctx`: ehsm context 地址，需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: HASH 算法，见 [ehsm_hash_algo_e](#)
- `use_plain_key`: 是否使用明文密钥
 - `true` 使用明文密钥，key 参数有效，key_handle 参数被忽略
 - `false` 使用密钥句柄，key_handle 参数有效，key 参数被忽略
- `key_handle`: RSA 密钥句柄，当 `use_plain_key` 为 `false` 时使用
- `key`: 用于签名/验证的明文密钥，当 `use_plain_key` 为 `true` 时使用，格式见 [ehsm_rsa_key_st](#)
- `padding`: RSA 填充模式，见 [ehsm_rsa_padding_mode_e](#)
- `gen_sig`: 操作方式
 - `true` 生成签名值
 - `false` 验证签名值
- `is_digest`: 输入数据类型
 - `true` input 为已计算好的消息摘要，长度由 algo 参数决定
 - `false` input 为原始消息数据，由 eHSM 内部计算摘要
- `input`: 输入数据，根据 `is_digest` 参数决定是消息原文还是消息摘要
- `input_size`: 输入数据长度
- `sig_addr`: 签名值的地址
 - 生成签名时：作为输出 buffer 的地址，存放生成的签名值
 - 验证签名时：作为输入 buffer 的地址，存放待验证的签名值
- `sig_size`: 签名 buffer 长度
 - 生成签名时：输入时为 buffer 长度，输出时为实际签名数据长度
 - 验证签名时：输入时为待验证的签名数据长度
- `salt_size`: PSS 填充模式使用的 salt 长度，其他填充模式应设置为 0
- `verify_result`: 验证结果存放地址，仅在 `gen_sig` 为 `false` 时有效
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.78 ehsm_rsa_sign_onepass_gen

原型:

```
1| uint32_t ehsm_rsa_sign_onepass_gen(EHSM_SHM ehsm_ctx_st *ctx, ehsm_hash_algo_e algo,  
uint32_t key_handle, ehsm_rsa_padding_mode_e padding, EHSM_SHM const uint8_t *msg, uint32_t  
msg_size, EHSM_SHM uint8_t *sig, uint32_t *sig_size, uint32_t salt_size)
```

说明:

RSA 签名值生成, 一次计算完成。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: 签名对应的 HASH 算法, 见 [ehsm_hash_algo_e](#), 仅支持 SHA1 和 SHA2 系列算法, MD5、SM3 和 SHA3 系列不支持
- `key_handle`: RSA 密钥句柄
- `padding`: 填充方式, 见 [ehsm_rsa_padding_mode_e](#)
- `msg`: 输入更新的消息数据
- `msg_size`: 输入的消息数据长度
- `sig`: 保存签名值的 buffer
- `sig_size`: 输入时为 buffer 长度, 完成时为写入 buffer 中的签名数据长度
- `salt_size`: 当填充方式为 `EHSM_RSA_PADDING_PSS` 时, 生成的盐值的长度; 当填充方式为 `EHSM_RSA_PADDING_NONE` 时, 'salt_size' 应为 0

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.79 ehsm_rsa_sign_onepass_verify

原型:

```
1| uint32_t ehsm_rsa_sign_onepass_verify(EHSM_SHM ehsm_ctx_st *ctx, ehsm_hash_algo_e  
algo, uint32_t key_handle, ehsm_rsa_padding_mode_e padding, EHSM_SHM const uint8_t *msg,
```

```
uint32_t msg_size, EHSM_SHM const uint8_t *sig, uint32_t sig_size, uint32_t salt_size,
bool_t *verify_result)
```

说明:

RSA 验证签名值，一次计算完成。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `algo`: 签名对应的 HASH 算法, 见 `ehsm_hash_algo_e`, 仅支持 SHA1 和 SHA2 系列算法, SM3 和 SHA3 系列不支持
- `key_handle`: RSA 密钥句柄
- `padding`: 填充方式, 见 `ehsm_rsa_padding_mode_e`
- `msg`: 输入更新的消息数据
- `msg_size`: 输入的消息数据长度
- `sig`: 待验证的签名值
- `sig_size`: 签名值长度
- `salt_size`: 当填充方式为 `EHSM_RSA_PADDING_PSS` 时, 待验签数据的盐值的长度; 当填充方式为 `EHSM_RSA_PADDING_NONE` 时, 'salt_size' 应为 0
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.80 ehsm_rsa_sign_onepass_gen_with_digest

原型:

```
1| uint32_t ehsm_rsa_sign_onepass_gen_with_digest(EHSM_SHM ehsm_ctx_st *ctx,
ehsm_hash_algo_e algo, uint32_t key_handle, ehsm_rsa_padding_mode_e padding, EHSM_SHM const
uint8_t *digest, uint32_t digest_size, EHSM_SHM uint8_t *sig, uint32_t *sig_size, uint32_t
salt_size)
```

说明:

RSA 签名值生成，一次计算完成, 外部传入计算好的消息摘要方式。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: 签名对应的 HASH 算法, 见 [ehsm_hash_algo_e](#), 仅支持 SHA1 和 SHA2 系列算法, MD5、SM3 和 SHA3 系列不支持 该参数仅 PSS 填充模式需要输入, 需要输入的 hash 算法需要和计算消息的摘要算法一致。
- `key_handle`: RSA 密钥句柄
- `padding`: 填充方式, 见 [ehsm_rsa_padding_mode_e](#)
- `digest`: 输入的消息摘要数据
- `digest_size`: 输入的消息摘要数据长度, 最大支持 64 字节长度。
- `sig`: 保存签名值的 buffer
- `sig_size`: 输入时为 buffer 长度, 完成时为写入 buffer 中的签名数据长度
- `salt_size`: 当填充方式为 `EHSM_RSA_PADDING_PSS` 时, 生成的盐值的长度; 当填充方式为 `EHSM_RSA_PADDING_NONE` 时, 'salt_size' 应为 0

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.81 ehsm_rsa_sign_onepass_verify_with_digest

原型:

```
1| uint32_t ehsm_rsa_sign_onepass_verify_with_digest(EHSM_SHM ehsm_ctx_st *ctx,  
ehsm_hash_algo_e algo, uint32_t key_handle, ehsm_rsa_padding_mode_e padding, EHSM_SHM const  
uint8_t *digest, uint32_t digest_size, EHSM_SHM const uint8_t *sig, uint32_t sig_size,  
uint32_t salt_size, bool_t *verify_result)
```

说明:

RSA 验证签名值, 一次计算完成, 外部传入计算好的消息摘要方式。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: 签名对应的 HASH 算法, 见 [ehsm_hash_algo_e](#), 仅支持 SHA1 和 SHA2 系列算法, SM3 和 SHA3 系列不支持, 该参数仅 PSS 填充模式需要输入, 需要输入的 hash 算法需要和计算消息的摘要算法一致。
- `key_handle`: RSA 密钥句柄
- `padding`: 填充方式, 见 [ehsm_rsa_padding_mode_e](#)
- `digest`: 输入的消息摘要数据

- `digest_size`: 输入的消息摘要数据长度, 最大支持 64 字节长度。
- `sig`: 待验证的签名值
- `sig_size`: 签名值长度
- `salt_size`: 当填充方式为 `EHSM_RSA_PADDING_PSS` 时, 待验签数据的盐值的长度; 当填充方式为 `EHSM_RSA_PADDING_NONE` 时, 'salt_size' 应为 0
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.82 ehsm_rsa_sign_onepass_gen_with_plain_key

原型:

```
1| uint32_t ehsm_rsa_sign_onepass_gen_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx,  
ehsm_hash_algo_e algo, EHSM_SHM const uint8_t *key, ehsm_rsa_padding_mode_e padding,  
EHSM_SHM const uint8_t *msg, uint32_t msg_size, EHSM_SHM uint8_t *sig, uint32_t *sig_size,  
uint32_t salt_size)
```

说明:

RSA 签名值生成, 一次计算完成。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `algo`: 签名对应的 HASH 算法, 见 `ehsm_hash_algo_e`, 仅支持 SHA1 和 SHA2 系列算法, MD5、SM3 和 SHA3 系列不支持
- `key`: 用于 RSA 签名值生成的明文密钥, 参考明文密钥数据格式和 `ehsm_rsa_key_st`
- `padding`: 填充方式, 见 `ehsm_rsa_padding_mode_e`
- `msg`: 输入更新的消息数据
- `msg_size`: 输入的消息数据长度
- `sig`: 保存签名值的 buffer
- `sig_size`: 输入时为 buffer 长度, 完成时为写入 buffer 中的签名数据长度
- `salt_size`: 当填充方式为 `EHSM_RSA_PADDING_PSS` 时, 生成的盐值的长度; 当填充方式为 `EHSM_RSA_PADDING_NONE` 时, 'salt_size' 应为 0

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.83 ehsm_rsa_sign_onepass_verify_with_plain_key

原型:

```
1| uint32_t ehsm_rsa_sign_onepass_verify_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx,  
ehsm_hash_algo_e algo, EHSM_SHM const uint8_t *key, ehsm_rsa_padding_mode_e padding,  
EHSM_SHM const uint8_t *msg, uint32_t msg_size, EHSM_SHM const uint8_t *sig, uint32_t  
sig_size, uint32_t salt_size, bool_t *verify_result)
```

说明:

RSA 验证签名值, 一次计算完成。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: 签名对应的 HASH 算法, 见 [ehsm_hash_algo_e](#), 仅支持 SHA1 和 SHA2 系列算法, SM3 和 SHA3 系列不支持
- `key`: 用于 RSA 验证签名值的明文密钥, 参考[明文密钥数据格式](#)和 [ehsm_rsa_key_st](#)
- `padding`: 填充方式, 见 [ehsm_rsa_padding_mode_e](#)
- `msg`: 输入更新的消息数据
- `msg_size`: 输入的消息数据长度
- `sig`: 待验证的签名值
- `sig_size`: 签名值长度
- `salt_size`: 当填充方式为 `EHSM_RSA_PADDING_PSS` 时, 待验签数据的盐值的长度; 当填充方式为 `EHSM_RSA_PADDING_NONE` 时, ‘salt_size’ 应为 0
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.84 ehsm_rsa_sign_onepass_gen_with_plain_key_and_digest

原型:


```
1| uint32_t ehsm_rsa_sign_onepass_gen_with_plain_key_and_digest(EHSM_SHM ehsm_ctx_st
*ctx, ehsm_hash_algo_e algo, EHSM_SHM const uint8_t *key, ehsm_rsa_padding_mode_e padding,
EHSM_SHM const uint8_t *digest, uint32_t digest_size, EHSM_SHM uint8_t *sig, uint32_t
*sig_size, uint32_t salt_size)
```

说明:

RSA 签名值生成, 一次计算完成。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: 签名对应的 HASH 算法, 见 [ehsm_hash_algo_e](#), 仅支持 SHA1 和 SHA2 系列算法, MD5、SM3 和 SHA3 系列不支持
- `key`: 用于 RSA 签名值生成的明文密钥, 参考[明文密钥数据格式](#)和 [ehsm_rsa_key_st](#)
- `padding`: 填充方式, 见 [ehsm_rsa_padding_mode_e](#)
- `digest`: 输入的消息摘要数据
- `digest_size`: 输入的消息摘要长度
- `sig`: 保存签名值的 buffer
- `sig_size`: 输入时为 buffer 长度, 完成时为写入 buffer 中的签名数据长度
- `salt_size`: 当填充方式为 `EHSM_RSA_PADDING_PSS` 时, 生成的盐值的长度; 当填充方式为 `EHSM_RSA_PADDING_NONE` 时, ‘salt_size’ 应为 0

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.85 ehsm_rsa_sign_onepass_verify_with_plain_key_and_digest

原型:

```
1| uint32_t ehsm_rsa_sign_onepass_verify_with_plain_key_and_digest(EHSM_SHM ehsm_ctx_st
*ctx, ehsm_hash_algo_e algo, EHSM_SHM const uint8_t *key, ehsm_rsa_padding_mode_e padding,
EHSM_SHM const uint8_t *digest, uint32_t digest_size, EHSM_SHM const uint8_t *sig, uint32_t
sig_size, uint32_t salt_size, bool_t *verify_result)
```

说明:

RSA 验证签名值, 一次计算完成。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: 签名对应的 HASH 算法, 见 [ehsm_hash_algo_e](#), 仅支持 SHA1 和 SHA2 系列算法, SM3 和 SHA3 系列不支持
- `key`: 用于 RSA 验证签名值的明文密钥, 参考[明文密钥数据格式](#)和 [ehsm_rsa_key_st](#)
- `padding`: 填充方式, 见 [ehsm_rsa_padding_mode_e](#)
- `digest`: 输入的消息摘要数据
- `digest_size`: 输入的消息摘要长度
- `sig`: 待验证的签名值
- `sig_size`: 签名值长度
- `salt_size`: 当填充方式为 `EHSM_RSA_PADDING_PSS` 时, 待验签数据的盐值的长度; 当填充方式为 `EHSM_RSA_PADDING_NONE` 时, 'salt_size' 应为 0
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.86 ehsm_ecdsa_init

原型:

```
1| uint32_t ehsm_ecdsa_init(EHSM_SHM ehsm_ctx_st *ctx, ehsm_hash_algo_e algo, uint32_t
key_handle, bool_t gen_sig, EHSM_SHM ehsm_session_st *session)
```

说明:

ECDSA 签名/验签计算, 初始化。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: 签名对应的 HASH 算法, 见 [ehsm_hash_algo_e](#), 支持所有 HASH 算法
- `key_handle`: ECDSA 密钥句柄
- `gen_sig`: 是否生成签名, 必须跟最后调用的 [ehsm_ecdsa_finish_gen](#) 或 [ehsm_ecdsa_finish_verify](#) 相匹配
 - `true` 生成签名值
 - `false` 验证签名值

- `session` : 用于 ehsm 内部缓存 HASH 计算中间值的 buffer, 此参数指向的内存存在 [ehsm_ecdsa_update](#)、[ehsm_ecdsa_finish_gen](#) 和 [ehsm_ecdsa_finish_verify](#) 中仍会使用, 因此需保证其有效性, 不能提前释放。

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.87 ehsm_ecdsa_init_with_plain_key

原型:

```
1| uint32_t ehsm_ecdsa_init_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx, ehsm_hash_algo_e  
algo, EHSM_SHM const uint8_t *key, bool_t gen_sig, EHSM_SHM ehsm_session_st *session)
```

说明:

ECDSA 签名/验签计算, 初始化。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: 签名对应的 HASH 算法, 见 [ehsm_hash_algo_e](#), 支持所有 HASH 算法
- `key`: 用于 ECDSA 签名/验签计算的明文密钥, 对应的内存存在 finish 接口之前不能释放, 参考[明文密钥数据格式](#)和 [ehsm_ecc_key_st](#)
- `gen_sig` : 是否生成签名, 必须跟最后调用的 [ehsm_ecdsa_finish_gen](#) 或 [ehsm_ecdsa_finish_verify](#) 相匹配
 - `true` 生成签名值
 - `false` 验证签名值
- `session` : 用于 ehsm 内部缓存 HASH 计算中间值的 buffer, 此参数指向的内存存在 [ehsm_ecdsa_update](#)、[ehsm_ecdsa_finish_gen](#) 和 [ehsm_ecdsa_finish_verify](#) 中仍会使用, 因此需保证其有效性, 不能提前释放。

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.88 ehsm_ecdsa_update

原型:

```
1| uint32_t ehsm_ecdsa_update(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t *msg,  
uint32_t msg_size)
```

说明:

ECDSA 签名/验签计算, 更新数据。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ecdsa_init` 或 `ehsm_ecdsa_update`
- `msg`: 输入更新的消息数据
- `msg_size`: 输入的消息数据长度

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.89 ehsm_ecdsa_finish_gen

原型:

```
1| uint32_t ehsm_ecdsa_finish_gen(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM uint8_t *sig,  
uint32_t *sig_size)
```

说明:

ECDSA 签名/验签计算, 生成签名值。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ecdsa_init` 或 `ehsm_ecdsa_update`
- `sig`: 保存签名值的 buffer
- `sig_size`: 输入时为 buffer 长度, 完成时为写入 buffer 中的签名数据长度

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.90 ehsm_ecdsa_finish_verify

原型:

```
1| uint32_t ehsm_ecdsa_finish_verify(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t *sig, uint32_t sig_size, bool_t *verify_result)
```

说明:

ECDSA 签名/验签计算, 验证签名值。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ecdsa_init` 或 `ehsm_ecdsa_update`
- `sig`: 待验证的签名值
- `sig_size`: 签名值长度
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.91 ehsm_ecdsa_onepass_ex

原型:

```
1| uint32_t ehsm_ecdsa_onepass_ex(EHSM_SHM ehsm_ctx_st *ctx, ehsm_hash_algo_e algo, bool_t use_plain_key, uint32_t key_handle, EHSM_SHM const uint8_t *key, bool_t gen_sig, bool_t is_digest, EHSM_SHM const uint8_t *input, uint32_t input_size, EHSM_SHM uintptr_t sig_addr, uint32_t *sig_size, bool_t *verify_result)
```

说明:

ECDSA 签名值生成或验证, 一次计算完成。

注意:

- 此函数提供了最大的灵活性, 但参数较多, 建议使用对应的封装版函数。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `algo`: 签名对应的 HASH 算法, 见 `ehsm_hash_algo_e`, 支持所有 HASH 算法
- `use_plain_key`: 是否使用外部输入的明文密钥值进行签名生成或验证
- `key_handle`: ECDSA 密钥句柄, 当 `use_plain_key` 为 `false` 时有效, 否则忽略

- `key`: ECDSA 公钥或私钥值, 当 `use_plain_key` 为 `true` 时有效, 否则忽略
- `gen_sig`: 是进行签名的生成, 还是验证
- `is_digest`: 输入的数据是已计算好的消息摘要值, 还是消息明文
- `input`: 输入数据地址
- `input_size`: 输入数据长度
- `sig_addr`: 在生成签名时, 此地址表示保存输出的签名值的 buffer 地址; 在验证签名时, 此地址表示用于验证的签名值数据地址
- `sig_size`: 在生成签名时, 此地址保存了输出 buffer 的长度, 并在计算完成后更新为签名值的长度; 在验证签名时, 此地址应当保存签名值的长度
- `verify_result`: 在验证签名时, 存放验证结果的内存位置 (生成签名时忽略此参数)
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.92 ehsm_ecdsa_onepass_gen

原型:

```
1| uint32_t ehsm_ecdsa_onepass_gen(EHSM_SHM ehsm_ctx_st *ctx, ehsm_hash_algo_e algo,
uint32_t key_handle, EHSM_SHM const uint8_t *msg, uint32_t msg_size, EHSM_SHM uint8_t *sig,
uint32_t *sig_size)
```

说明:

ECDSA 签名值生成, 一次计算完成。此函数使用 key handle 和原始消息数据生成签名值。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `algo`: 签名对应的 HASH 算法, 见 `ehsm_hash_algo_e`, 支持所有 HASH 算法
- `key_handle`: ECDSA 密钥句柄
- `msg`: 输入更新的消息数据
- `msg_size`: 输入的消息数据长度
- `sig`: 保存签名值的 buffer
- `sig_size`: 输入时为 buffer 长度, 完成时为写入 buffer 中的签名数据长度

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.93 ehsm_ecdsa_onepass_verify

原型:

```
1| uint32_t ehsm_ecdsa_onepass_verify(EHSM_SHM ehsm_ctx_st *ctx, ehsm_hash_algo_e algo,  
uint32_t key_handle, EHSM_SHM const uint8_t *msg, uint32_t msg_size, EHSM_SHM const uint8_t  
*sig, uint32_t sig_size, bool_t *verify_result)
```

说明:

ECDSA 验证签名值, 一次计算完成。此函数使用 key handle 和原始消息数据验证签名值。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `algo`: 签名对应的 HASH 算法, 见 `ehsm_hash_algo_e`, 支持所有 HASH 算法
- `key_handle`: ECDSA 密钥句柄
- `msg`: 输入更新的消息数据
- `msg_size`: 输入的消息数据长度
- `sig`: 待验证的签名值
- `sig_size`: 签名值长度
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.94 ehsm_ecdsa_onepass_gen_with_digest

原型:

```
1| uint32_t ehsm_ecdsa_onepass_gen_with_digest(EHSM_SHM ehsm_ctx_st *ctx,  
ehsm_hash_algo_e algo, uint32_t key_handle, EHSM_SHM const uint8_t *digest, uint32_t  
digest_size, EHSM_SHM uint8_t *sig, uint32_t *sig_size)
```

说明:

ECDSA 签名值生成，一次计算完成。此函数使用 key handle 和消息数据摘要值生成签名值。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: HASH 算法, 见 [ehsm_hash_algo_e](#)
- `key_handle`: ECDSA 密钥句柄
- `digest`: 输入的消息摘要数据
- `digest_size`: 输入的消息摘要长度, 最大长度是 64 字节
- `sig`: 保存签名值的 buffer
- `sig_size`: 输入时为 buffer 长度, 完成时为写入 buffer 中的签名数据长度

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.95 ehsm_ecdsa_onepass_verify_with_digest

原型:

```
1| uint32_t ehsm_ecdsa_onepass_verify_with_digest(EHSM_SHM ehsm_ctx_st *ctx,  
ehsm_hash_algo_e algo, uint32_t key_handle, EHSM_SHM const uint8_t *digest, uint32_t  
digest_size, EHSM_SHM const uint8_t *sig, uint32_t sig_size, bool_t *verify_result)
```

说明:

ECDSA 验证签名值，一次计算完成。此函数使用 key handle 和消息数据摘要值验证签名值。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: HASH 算法, 见 [ehsm_hash_algo_e](#)
- `key_handle`: ECDSA 密钥句柄
- `digest`: 输入的消息摘要数据
- `digest_size`: 输入的消息摘要长度, 最大长度是 64 字节
- `sig`: 待验证的签名值
- `sig_size`: 签名值长度
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过

- `false` 验证不通过

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.96 ehsm_ecdsa_onepass_gen_with_plain_key

原型:

```
1| uint32_t ehsm_ecdsa_onepass_gen_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx,  
ehsm_hash_algo_e algo, EHSM_SHM const uint8_t *key, EHSM_SHM const uint8_t *msg, uint32_t  
msg_size, EHSM_SHM uint8_t *sig, uint32_t *sig_size)
```

说明:

ECDSA 签名值生成, 一次计算完成。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: 签名对应的 HASH 算法, 见 [ehsm_hash_algo_e](#), 支持所有 HASH 算法
- `key`: 用于 ECDSA 签名值生成的明文密钥, 参考[明文密钥数据格式](#)和 [ehsm_ecc_key_st](#)
- `msg`: 输入更新的消息数据
- `msg_size`: 输入的消息数据长度
- `sig`: 保存签名值的 buffer
- `sig_size`: 输入时为 buffer 长度, 完成时为写入 buffer 中的签名数据长度

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.97 ehsm_ecdsa_onepass_verify_with_plain_key

原型:

```
1| uint32_t ehsm_ecdsa_onepass_verify_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx,  
ehsm_hash_algo_e algo, EHSM_SHM const uint8_t *key, EHSM_SHM const uint8_t *msg, uint32_t  
msg_size, EHSM_SHM const uint8_t *sig, uint32_t sig_size, bool_t *verify_result)
```

说明:

ECDSA 验证签名值，一次计算完成。

参数：

- `ctx`: ehsm context 地址，需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: 签名对应的 HASH 算法，见 [ehsm_hash_algo_e](#)，支持所有 HASH 算法
- `key`: 用于 ECDSA 验证签名值的明文密钥，参考[明文密钥数据格式](#)和 [ehsm_ecc_key_st](#)
- `msg`: 输入更新的消息数据
- `msg_size`: 输入的消息数据长度
- `sig`: 待验证的签名值
- `sig_size`: 签名值长度
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值：

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.98 ehsm_ecdsa_onepass_gen_with_plain_key_and_digest

原型：

```
1| uint32_t ehsm_ecdsa_onepass_gen_with_plain_key_and_digest(EHSM_SHM ehsm_ctx_st *ctx,  
ehsm_hash_algo_e algo, EHSM_SHM const uint8_t *key, EHSM_SHM const uint8_t *digest,  
uint32_t digest_size, EHSM_SHM uint8_t *sig, uint32_t *sig_size)
```

说明：

ECDSA 签名值生成，一次计算完成。

参数：

- `ctx`: ehsm context 地址，需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: 签名对应的 HASH 算法，见 [ehsm_hash_algo_e](#)，支持所有 HASH 算法
- `key`: 用于 ECDSA 签名值生成的明文密钥，参考[明文密钥数据格式](#)和 [ehsm_ecc_key_st](#)
- `digest`: 输入的消息摘要数据
- `digest_size`: 输入的消息摘要长度,最大长度是 64 字节
- `sig`: 保存签名值的 buffer
- `sig_size`: 输入时为 buffer 长度，完成时为写入 buffer 中的签名数据长度

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.99 ehsm_ecdsa_onepass_verify_with_plain_key_and_digest**原型:**

```
1| uint32_t ehsm_ecdsa_onepass_verify_with_plain_key_and_digest(EHSM_SHM ehsm_ctx_st
*ctx, ehsm_hash_algo_e algo, EHSM_SHM const uint8_t *key, EHSM_SHM const uint8_t *digest,
uint32_t digest_size, EHSM_SHM uint8_t *sig, uint32_t sig_size, bool_t *verify_result)
```

说明:

ECDSA 验证签名值, 一次计算完成。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: 签名对应的 HASH 算法, 见 [ehsm_hash_algo_e](#), 支持所有 HASH 算法
- `key`: 用于 ECDSA 验证签名值的明文密钥, 参考[明文密钥数据格式](#)和 [ehsm_ecc_key_st](#)
- `digest`: 输入的消息摘要数据
- `digest_size`: 输入的消息摘要长度, 最大长度是 64 字节
- `sig`: 待验证的签名值
- `sig_size`: 签名值长度
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.100 ehsm_sm9_cipher**原型:**

```
1| uint32_t ehsm_sm9_cipher(EHSM_SHM ehsm_ctx_st *ctx, uint32_t key_handle, bool_t enc,
ehsm_sm9_enc_type_e enc_type, ehsm_sm9_padding_mode_e padding, uint8_t key2_size, uint8_t
hid, EHSM_SHM const uint8_t *kgc_pub_key, EHSM_SHM const uint8_t *input, uint32_t
```

```
input_size, EHSM_SHM uint8_t *output, uint32_t *output_size, EHSM_SHM const uint8_t *id,
uint32_t id_size, EHSM_SHM const uint8_t *fp12g)
```

说明:

SM9 加密/解密计算，一次完成计算。

注意:

- SM9 加解密不支持三段式调用。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `key_handle`: SM9 密钥句柄, 解密有效, 加密时传入 0
- `enc`: 是否加密
 - `true` 加密
 - `false` 解密
- `enc_type`: 加密类型 `ehsm_sm9_enc_type_e`
- `padding`: padding 类型 `ehsm_sm9_padding_mode_e`, 在 `enc_type` 为 `EHSM_SM9_ENC_TYPE_BLOCK` 的情况有效
- `key2_size`: SM9 内部计算 MAC 的密钥长度
- `hid`: SM9 加密密钥生成标识, 默认取值为 0x3
- `kgc_pub_key`: KGC 公钥数据, 长度为 64 字节
- `input`: 输入的明文或密文, 密文的格式为 C1C3C2, 其中
 - C1 为曲线上的点, 65 字节, 以 `0x04` 开头, 以及完整 x 和 y 坐标
 - C2 为实际的密文数据, 与对应的明文长度相同
 - C3 为 SM3 的结果, 32 字节, 用于保证完整性
- `input_size`: 输入数据长度, 不能大于 1024
- `output`: 输出数据的 buffer
- `output_size`: 传入时为输出 buffer 的长度, 如果是加密, 应大于等于 `input_size + 97`; 如果是解密, 应大于等于 `input_size - 97`, 完成时为实际的输出数据长度
- `id`: 用户的 ID
- `id_size`: 用户 ID 的长度, 长度必须小于 1024 字节
- `fp12g`: SM9 算法内部中间值, 长度为 384 字节, 可以传空值, 如果是空值, HSM 内部进行计算

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成

- 其它值：任务发生错误

8.4.101 ehsm_sm9_cipher_with_plain_key

原型：

```
1| uint32_t ehsm_sm9_cipher_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const
uint8_t *key, bool_t enc, ehsm_sm9_enc_type_e enc_type, ehsm_sm9_padding_mode_e padding,
uint8_t key2_size, uint8_t hid, EHSM_SHM const uint8_t *kgc_pub_key, EHSM_SHM const uint8_t
*input, uint32_t input_size, EHSM_SHM uint8_t *output, uint32_t *output_size, EHSM_SHM
const uint8_t *id, uint32_t id_size, EHSM_SHM const uint8_t *fp12g)
```

说明：

SM9 加密/解密计算，一次完成计算，解密时私钥使用明文密钥。

注意：

- SM9 加解密不支持三段式调用。

参数：

- `ctx`：ehsm context 地址，需事先调用过 [ehsm_ctx_init](#) 初始化
- `key`：SM9 用户私钥，解密有效，加密时传入 NULL，参考[明文密钥数据格式](#)
- `enc`：是否加密
 - `true` 加密
 - `false` 解密
- `enc_type`：加密类型 [ehsm_sm9_enc_type_e](#)
- `padding`：padding 类型 [ehsm_sm9_padding_mode_e](#)，在 `enc_type` 为 `EHSM_SM9_ENC_TYPE_BLOCK` 的情况有效
- `key2_size`：SM9 内部计算 MAC 的密钥长度
- `hid`：SM9 加密密钥生成标识，默认取值为 0x3
- `kgc_pub_key`：KGC 公钥数据，长度为 64 字节
- `input`：输入的明文或密文，密文的格式为 C1C3C2，其中
 - C1 为曲线上的点，65 字节，以 `0x04` 开头，以及完整 x 和 y 坐标
 - C2 为实际的密文数据，与对应的明文长度相同
 - C3 为 SM3 的结果，32 字节，用于保证完整性
- `input_size`：输入数据长度，不能大于 1024
- `output`：输出数据的 buffer
- `output_size`：传入时为输出 buffer 的长度，如果是加密，应大于等于 `input_size + 97`；如果是解密，应大于等于 `input_size - 97`，完成时为实际的输出数据长度
- `id`：用户的 ID

- `id_size`: 用户 ID 的长度, 长度必须小于 1024 字节
- `fp12g`: SM9 算法内部中间值, 长度为 384 字节, 可以传空值, 如果是空值, HSM 内部进行计算

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.102 ehsm_sm9_sign_onepass_gen

原型:

```
1| uint32_t ehsm_sm9_sign_onepass_gen(EHSM_SHM ehsm_ctx_st *ctx, uint32_t key_handle,  
EHSM_SHM const uint8_t *msg, uint32_t msg_size, EHSM_SHM uint8_t *sig, uint32_t sig_size,  
EHSM_SHM const uint8_t *kgc_pub_key, EHSM_SHM const uint8_t *fp12g)
```

说明:

SM9 签名值生成, 一次计算完成。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `key_handle`: SM9 密钥句柄
- `msg`: 输入的消息数据
- `msg_size`: 输入的消息数据长度, 不能大于 1024
- `sig`: 保存签名值的 buffer
- `sig_size`: 签名 buffer 的长度, 不能小于 97
- `kgc_pub_key`: KGC 公钥数据, 长度为 64 字节
- `fp12g`: SM9 算法内部中间值, 可以传空值, 如果是空值, HSM 内部进行计算

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.103 ehsm_sm9_sign_onepass_verify

原型:

```
1| uint32_t ehsm_sm9_sign_onepass_verify(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const
uint8_t *msg, uint32_t msg_size, EHSM_SHM const uint8_t *id, uint32_t id_size, uint8_t hid,
EHSM_SHM const uint8_t *kgc_pub_key, EHSM_SHM const uint8_t *fp12g, EHSM_SHM const uint8_t
*sig, uint32_t sig_size, bool_t *verify_result)
```

说明:

SM9 验证签名值, 一次计算完成。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `msg`: 输入的消息数据
- `msg_size`: 输入的消息数据长度, 不能大于 1024
- `id`: 用户的 ID
- `id_size`: 用户 ID 的长度, 不能大于 1024 字节
- `hid`: SM9 签名密钥生成标识, 默认取值为 0x1
- `kgc_pub_key`: KGC 公钥数据, 长度为 64 字节
- `fp12g`: SM9 算法内部中间值, 可以传空值, 如果是空值, HSM 内部进行计算
- `sig`: 待验证的签名值
- `sig_size`: 签名值长度, 仅接受 97 字节签名值
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.104 ehsm_sm9_sign_onepass_gen_with_plain_key

原型:

```
1| uint32_t ehsm_sm9_sign_onepass_gen_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM
const uint8_t *key, EHSM_SHM const uint8_t *msg, uint32_t msg_size, EHSM_SHM uint8_t *sig,
uint32_t sig_size, EHSM_SHM const uint8_t *kgc_pub_key, EHSM_SHM const uint8_t *fp12g)
```

说明:

SM9 签名值生成, 一次计算完成, 使用明文密钥。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `key`: SM9 用户私钥, 参考[明文密钥数据格式](#)
- `msg`: 输入的消息数据
- `msg_size`: 输入的消息数据长度, 不能大于 1024
- `sig`: 保存签名值的 buffer
- `sig_size`: 签名 buffer 的长度, 不能小于 97
- `kgc_pub_key`: KGC 公钥数据, 长度为 64 字节
- `fp12g`: SM9 算法内部中间值, 可以传空值, 如果是空值, HSM 内部进行计算

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.105 ehsm_chacha_init

原型:

```
1| uint32_t ehsm_chacha_init(EHSM_SHM ehsm_ctx_st *ctx, uint32_t key_handle, bool_t enc,  
EHSM_SHM const uint8_t *nonce, uint32_t nonce_size, EHSM_SHM const uint8_t *aad, uint32_t  
aad_size, uint32_t constant, EHSM_SHM ehsm_session_st *session)
```

说明:

CHACHA 加密/解密计算, 初始化。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `key_handle`: 对称密钥句柄
- `enc` : 是否加密, 此参数必须与最后调用的 [ehsm_chacha_finish_enc](#) 或 [ehsm_chacha_finish_dec](#) 相匹配
 - `true` 加密
 - `false` 解密
- `nonce`: nonce 值
- `nonce_size`: 必须为 8 字节
- `aad`: 附加数据, 可以为 NULL
- `aad_size`: 附加数据长度

- `constant`: 计数器初始值
- `session`: 用于 ehsm 内部缓存 CHACHA 计算中间值的 buffer, 此参数指向的内存存在 `ehsm_chacha_update`、`ehsm_chacha_finish_enc` 和 `ehsm_chacha_finish_dec` 中仍会使用, 因此需保证其有效性, 不能提前释放。

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.106 ehsm_chacha_update

原型:

```
1| uint32_t ehsm_chacha_update(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t *input,  
uint32_t input_size, EHSM_SHM uint8_t *output)
```

说明:

CHACHA 加密/解密计算, 更新数据。此函数处理连续的块数据, 实际写入 `output` 中的数据与输入数据长度相同。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_chacha_init` 初始化
- `input`: 输入数据
- `input_size`: 输入数据长度, 严格要求 block size 对齐
 - 若为加密, 则必须保留至少一个字节的输入数据给 `ehsm_chacha_finish_enc` 使用;
 - 若为解密, 则必须保留至少一个字节的输入数据给 `ehsm_chacha_finish_dec` 使用;
- `output`: 输出 buffer, 其空间大小不能小于 `input_size`

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.107 ehsm_chacha_finish_enc

原型:

```
1| uint32_t ehsm_chacha_finish_enc(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t  
*input, uint32_t input_size, EHSM_SHM uint8_t *output, EHSM_SHM uint8_t *tag, uint32_t  
tag_size)
```

说明:

CHACHA 加密/解密计算, 计算结果。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_chacha_init](#) 或 [ehsm_chacha_update](#)
- `input`: 输入数据, 不能为空
- `input_size`: 输入数据长度, 不能为 0, 不要求对齐 block size
- `output`: 输出 buffer, 其空间大小不能小于 `input_size`
- `tag`: 存入生成的 TAG 值的 buffer, 其空间大小不能小于 `tag_size`
- `tag_size`: 指定生成的 TAG 值长度, 必须为 16 字节

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.108 ehsm_chacha_finish_dec

原型:

```
1| uint32_t ehsm_chacha_finish_dec(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t  
*input, uint32_t input_size, EHSM_SHM uint8_t *output, EHSM_SHM const uint8_t *tag,  
uint32_t tag_size, bool_t *verify_result)
```

说明:

CHACHA 加密/解密计算, 验证结果。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_chacha_init](#) 或 [ehsm_chacha_update](#)
- `input`: 输入数据, 不能为空
- `input_size`: 输入数据长度, 不能为 0, 不要求对齐 block size
- `output`: 输出 buffer, 其空间大小不能小于 `input_size`
- `tag`: 输入的 TAG 值
- `tag_size`: TAG 值长度, 必须为 16 字节
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.109 ehsm_chacha_onepass_enc

原型:

```
1| uint32_t ehsm_chacha_onepass_enc(EHSM_SHM ehsm_ctx_st *ctx, uint32_t key_handle,  
EHSM_SHM const uint8_t *nonce, uint32_t nonce_size, EHSM_SHM const uint8_t *aad, uint32_t  
aad_size, uint32_t constant, EHSM_SHM const uint8_t *input, uint32_t input_size, EHSM_SHM  
uint8_t *output, EHSM_SHM uint8_t *tag, uint32_t tag_size)
```

说明:

CHACHA 加密计算，一次计算完成。此函数处理连续的块数据，输入数据长度为任意长度，实际写入 `output` 中的数据与输入数据长度相同。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `key_handle`: 对称密钥句柄
- `nonce`: nonce 值
- `nonce_size`: nonce_size 必须为 8 字节
- `aad`: 附加数据, 可以为 NULL
- `aad_size`: 附加数据长度
- `constant`: 计数器初始值
- `input`: 输入数据
- `input_size`: 输入数据长度, 可以是任意长度, 包括 0
- `output`: 输出 buffer, 其空间大小不能小于 `input_size`
- `tag`: 存入生成的 TAG 值的 buffer
- `tag_size`: 指定生成的 TAG 值长度, 必须为 16 字节

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.110 ehsm_chacha_onepass_dec

原型:

```
1| uint32_t ehsm_chacha_onepass_dec(EHSM_SHM ehsm_ctx_st *ctx, uint32_t key_handle,  
EHSM_SHM const uint8_t *nonce, uint32_t nonce_size, EHSM_SHM const uint8_t *aad, uint32_t  
aad_size, uint32_t constant, EHSM_SHM const uint8_t *input, uint32_t input_size, EHSM_SHM  
uint8_t *output, EHSM_SHM const uint8_t *tag, uint32_t tag_size, bool_t *verify_result)
```

说明:

CHACHA 解密计算，一次计算完成。此函数处理连续的块数据，输入数据长度为任意长度，实际写入 `output` 中的数据与输入数据长度相同。

参数:

- `ctx`: ehsm context 地址，需事先调用过 `ehsm_ctx_init` 初始化
- `key_handle`: 对称密钥句柄
- `nonce`: nonce 值
- `nonce_size`: nonce_size 必须为 8 字节
- `aad`: 附加数据，可以为 NULL
- `aad_size`: 附加数据长度
- `constant`: 计数器初始值
- `input`: 输入数据
- `input_size`: 输入数据长度，可以是任意长度，包括 0
- `output`: 输出 buffer，其空间大小不能小于 `input_size`
- `tag`: 输入的 TAG 值
- `tag_size`: 指定生成的 TAG 值长度，必须为 16 字节
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.111 ehsm_chacha_init_with_plain_key

原型:

```
1| uint32_t ehsm_chacha_init_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const  
uint8_t *key, uint32_t key_size, bool_t enc, EHSM_SHM const uint8_t *nonce, uint32_t
```

```
nonce_size, EHSM_SHM const uint8_t *aad, uint32_t aad_size, uint32_t constant, EHSM_SHM
ehsm_session_st *session)
```

说明:

CHACHA 加密/解密计算, 初始化, 使用明文密钥

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `key`: 用于 CHACHA 加密/解密计算的明文密钥, 参考[明文密钥数据格式](#)
- `key_size`: 明文密钥字节数
- `enc`: 是否加密, 此参数必须与最后调用的 [ehsm_chacha_finish_enc](#) 或 [ehsm_chacha_finish_dec](#) 相匹配
 - `true` 加密
 - `false` 解密
- `nonce`: nonce 值
- `nonce_size`: 必须为 8 字节
- `aad`: 附加数据, 可以为 NULL
- `aad_size`: 附加数据长度
- `constant`: 计数器初始值
- `session`: 用于 ehsm 内部缓存 CHACHA 计算中间值的 buffer, 此参数指向的内存存在 [ehsm_chacha_update](#)、[ehsm_chacha_finish_enc](#) 和 [ehsm_chacha_finish_dec](#) 中仍会使用, 因此需保证其有效性, 不能提前释放。

返回值:

- EHSM_OK: 任务成功完成
- EHSM_ERR_NEED_POLL: 任务未完成
- 其它值: 任务发生错误

8.4.112 ehsm_chacha_onepass_enc_with_plain_key

原型:

```
1| uint32_t ehsm_chacha_onepass_enc_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM
const uint8_t *key, uint32_t key_size, EHSM_SHM const uint8_t *nonce, uint32_t nonce_size,
EHSM_SHM const uint8_t *aad, uint32_t aad_size, uint32_t constant, EHSM_SHM const uint8_t
*input, uint32_t input_size, EHSM_SHM uint8_t *output, EHSM_SHM uint8_t *tag, uint32_t
tag_size)
```

说明:

CHACHA 加密计算，一次计算完成，使用明文密钥 此函数处理连续的块数据，输入数据长度为任意长度，实际写入 `output` 中的数据与输入数据长度相同。

参数：

- `ctx`: ehsm context 地址，需事先调用过 `ehsm_ctx_init` 初始化
- `key`: 用于 CHACHA 加密计算的明文密钥，参考[明文密钥数据格式](#)
- `key_size`: 明文密钥字节数
- `nonce`: nonce 值
- `nonce_size`: nonce_size 必须为 8 字节
- `aad`: 附加数据，可以为 NULL
- `aad_size`: 附加数据长度
- `constant`: 计数器初始值
- `input`: 输入数据
- `input_size`: 输入数据长度，可以是任意长度，包括 0
- `output`: 输出 buffer，其空间大小不能小于 `input_size`
- `tag`: 存入生成的 TAG 值的 buffer
- `tag_size`: 指定生成的 TAG 值长度，必须为 16 字节

返回值：

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.113 ehsm_chacha_onepass_dec_with_plain_key

原型：

```
1| uint32_t ehsm_chacha_onepass_dec_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM
const uint8_t *key, uint32_t key_size, EHSM_SHM const uint8_t *nonce, uint32_t nonce_size,
EHSM_SHM const uint8_t *aad, uint32_t aad_size, uint32_t constant, EHSM_SHM const uint8_t
*input, uint32_t input_size, EHSM_SHM uint8_t *output, EHSM_SHM const uint8_t *tag,
uint32_t tag_size, bool_t *verify_result)
```

说明：

CHACHA 解密计算，一次计算完成，使用明文密钥 此函数处理连续的块数据，输入数据长度为任意长度，实际写入 `output` 中的数据与输入数据长度相同。

参数：

- `ctx`: ehsm context 地址，需事先调用过 `ehsm_ctx_init` 初始化

- `key`: 用于 CHACHA 解密计算的明文密钥, 参考[明文密钥数据格式](#)
- `key_size`: 明文密钥字节数
- `nonce`: nonce 值
- `nonce_size`: nonce_size 必须为 8 字节
- `aad`: 附加数据, 可以为 NULL
- `aad_size`: 附加数据长度
- `constant`: 计数器初始值
- `input`: 输入数据
- `input_size`: 输入数据长度, 可以是任意长度, 包括 0
- `output`: 输出 buffer, 其空间大小不能小于 `input_size`
- `tag`: 输入的 TAG 值
- `tag_size`: 指定生成的 TAG 值长度, 必须为 16 字节
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.114 ehsm_aead_init

原型:

```
1| uint32_t ehsm_aead_init(EHSM_SHM ehsm_ctx_st *ctx, ehsm_symm_algo_e algo,  
ehsm_aead_mode_e mode, uint32_t key_handle, bool_t enc, EHSM_SHM const uint8_t *nonce,  
uint32_t nonce_size, EHSM_SHM const uint8_t *aad, uint32_t aad_size, uint32_t data_size,  
uint32_t tag_size, EHSM_SHM ehsm_session_st *session)
```

说明:

AEAD 加密/解密计算, 初始化。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: AEAD 使用的对称算法类型, 见 [ehsm_symm_algo_e](#), 不支持 DES 和 TDES
- `mode`: AEAD 模式, 见 [ehsm_aead_mode_e](#)

- `key_handle`: 对称密钥句柄
- `enc`: 是否加密, 此参数必须与最后调用的 `ehsm_aead_finish_enc` 或 `ehsm_aead_finish_dec` 相匹配
 - `true` 加密
 - `false` 解密
- `nonce`: nonce 值
- `nonce_size`: nonce 值的长度, 对于 GCM, 必须为 12 字节, 对于 CCM, 取值范围是 7~13
- `aad`: 附加数据, 可以为 NULL
- `aad_size`: 附加数据长度, 不能大于 128
- `data_size`: 后续调用 `ehsm_aead_update` 更新的所有数据的长度之和, 必须与后续实际更新的数据长度相等
- `tag_size`: 指定生成或校验的 TAG 值长度, 对于 GCM, 取值范围是 1~16, 对于 CCM, 取值范围是 {4,6,8,10,12,14,16}
- `session`: 用于 ehsm 内部缓存 HASH 计算中间值的 buffer, 此参数指向的内存存在 `ehsm_aead_update`、`ehsm_aead_finish_enc` 和 `ehsm_aead_finish_dec` 中仍会使用, 因此需保证其有效性, 不能提前释放。

返回值:

- EHSM_OK: 任务成功完成
- EHSM_ERR_NEED_POLL: 任务未完成
- 其它值: 任务发生错误

8.4.115 ehsm_aead_init_with_plain_key

原型:

```
1| uint32_t ehsm_aead_init_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx, ehsm_symm_algo_e
    algo, ehsm_aead_mode_e mode, EHSM_SHM const uint8_t *key, uint32_t key_size, bool_t enc,
    EHSM_SHM const uint8_t *nonce, uint32_t nonce_size, EHSM_SHM const uint8_t *aad, uint32_t
    aad_size, uint32_t data_size, uint32_t tag_size, EHSM_SHM ehsm_session_st *session)
```

说明:

AEAD 加密/解密计算, 初始化, 使用明文密钥

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `algo`: AEAD 使用的对称算法类型, 见 `ehsm_symm_algo_e`, 不支持 DES 和 TDES
- `mode`: AEAD 模式, 见 `ehsm_aead_mode_e`
- `key`: 用于 AEAD 加密/解密计算的明文密钥, 对应的内存存在 finish 接口之前不能释放, 参考[明文密钥数据格式](#)

- `key_size`: 明文密钥字节数
- `enc`: 是否加密, 此参数必须与最后调用的 `ehsm_aead_finish_enc` 或 `ehsm_aead_finish_dec` 相匹配
 - `true` 加密
 - `false` 解密
- `nonce`: nonce 值
- `nonce_size`: nonce 值的长度, 对于 GCM, 必须为 12 字节, 对于 CCM, 取值范围是 7~13
- `aad`: 附加数据, 可以为 NULL
- `aad_size`: 附加数据长度, 不能大于 128
- `data_size`: 后续调用 `ehsm_aead_update` 更新的所有数据的长度之和, 必须与后续实际更新的数据长度相等
- `tag_size`: 指定生成或校验的 TAG 值长度, 对于 GCM, 取值范围是 1~16, 对于 CCM, 取值范围是 {4,6,8,10,12,14,16}
- `session`: 用于 ehsm 内部缓存 HASH 计算中间值的 buffer, 此参数指向的内存存在 `ehsm_aead_update`、`ehsm_aead_finish_enc` 和 `ehsm_aead_finish_dec` 中仍会使用, 因此需保证其有效性, 不能提前释放。

返回值:

- EHSM_OK: 任务成功完成
- EHSM_ERR_NEED_POLL: 任务未完成
- 其它值: 任务发生错误

8.4.116 ehsm_aead_update

原型:

```
1| uint32_t ehsm_aead_update(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t *input,  
uint32_t input_size, EHSM_SHM uint8_t *output)
```

说明:

AEAD 加密/解密计算, 更新数据。 此函数处理连续的块数据, 输入数据长度必须 block size 对齐, 实际写入 `output` 中的数据与输入数据长度相同。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_aead_init` 初始化
- `input`: 输入数据
- `input_size`: 输入数据长度, 必须 block size 对齐
- `output`: 输出 buffer, 其空间大小不能小于 `input_size`

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.117 ehsm_aead_finish_enc

原型:

```
1| uint32_t ehsm_aead_finish_enc(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t  
*input, uint32_t input_size, EHSM_SHM uint8_t *output, EHSM_SHM uint8_t *tag)
```

说明:

AEAD 加密/解密计算, 计算结果。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_aead_init` 或 `ehsm_aead_update`
- `input`: 输入数据
- `input_size`: 输入数据长度, 不要求对齐 block size
- `output`: 输出 buffer, 其空间大小不能小于 `input_size`
- `tag`: 存入生成的 TAG 值的 buffer, 其空间大小不能小于 `ehsm_aead_init` 所传入的参数 `tag_size`

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.118 ehsm_aead_finish_dec

原型:

```
1| uint32_t ehsm_aead_finish_dec(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t  
*input, uint32_t input_size, EHSM_SHM uint8_t *output, EHSM_SHM const uint8_t *tag, bool_t  
*verify_result)
```

说明:

AEAD 加密/解密计算, 验证结果。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_aead_init` 或 `ehsm_aead_update`
- `input`: 输入数据

- `input_size`: 输入数据长度, 不要求对齐 block size
- `output`: 输出 buffer, 其空间大小不能小于 `input_size`
- `tag`: 输入的 TAG 值
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.119 ehsm_aead_onepass_enc

原型:

```
1| uint32_t ehsm_aead_onepass_enc(EHSM_SHM ehsm_ctx_st *ctx, ehsm_symm_algo_e algo,
ehsm_aead_mode_e mode, uint32_t key_handle, EHSM_SHM const uint8_t *nonce, uint32_t
nonce_size, EHSM_SHM const uint8_t *aad, uint32_t aad_size, EHSM_SHM const uint8_t *input,
uint32_t input_size, EHSM_SHM uint8_t *output, EHSM_SHM uint8_t *tag, uint32_t tag_size)
```

说明:

AEAD 加密计算, 一次计算完成。 此函数处理连续的块数据, 输入数据长度为任意长度, 实际写入 `output` 中的数据与输入数据长度相同。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `algo`: AEAD 使用的对称算法类型, 见 `ehsm_symm_algo_e`, 不支持 DES 和 TDES
- `mode`: AEAD 模式, 见 `ehsm_aead_mode_e`
- `key_handle`: 对称密钥句柄
- `nonce`: nonce 值
- `nonce_size`: nonce 值的长度, 对于 GCM, 必须为 12 字节, 对于 CCM, 取值范围是 7~13
- `aad`: 附加数据, 可以为 NULL
- `aad_size`: 附加数据长度, 不能大于 128
- `input`: 输入数据
- `input_size`: 输入数据长度, 可以是任意长度, 包括 0
- `output`: 输出 buffer, 其空间大小不能小于 `input_size`
- `tag`: 存入生成的 TAG 值的 buffer

- `tag_size`: 指定生成的 TAG 值长度, 对于 GCM, 取值范围是 1~16, 对于 CCM, 取值范围是 {4,6,8,10,12,14,16}

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.120 ehsm_aead_onepass_dec

原型:

```
1| uint32_t ehsm_aead_onepass_dec(EHSM_SHM ehsm_ctx_st *ctx, ehsm_symm_algo_e algo,  
ehsm_aead_mode_e mode, uint32_t key_handle, EHSM_SHM const uint8_t *nonce, uint32_t  
nonce_size, EHSM_SHM const uint8_t *aad, uint32_t aad_size, EHSM_SHM const uint8_t *input,  
uint32_t input_size, EHSM_SHM uint8_t *output, EHSM_SHM const uint8_t *tag, uint32_t  
tag_size, bool_t *verify_result)
```

说明:

AEAD 解密计算, 一次计算完成。 此函数处理连续的块数据, 输入数据长度为任意长度, 实际写入 `output` 中的数据与输入数据长度相同。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `algo`: AEAD 使用的对称算法类型, 见 `ehsm_symm_algo_e`, 不支持 DES 和 TDES
- `mode`: AEAD 模式, 见 `ehsm_aead_mode_e`
- `key_handle`: 对称密钥句柄
- `nonce`: nonce 值
- `nonce_size`: nonce 值的长度, 对于 GCM, 必须为 12 字节, 对于 CCM, 取值范围是 7~13
- `aad`: 附加数据, 可以为 NULL
- `aad_size`: 附加数据长度, 不能大于 128
- `input`: 输入数据
- `input_size`: 输入数据长度, 可以是任意长度, 包括 0
- `output`: 输出 buffer, 其空间大小不能小于 `input_size`
- `tag`: 输入的 TAG 值
- `tag_size`: 输入的 TAG 长度, 对于 GCM, 取值范围是 1~16, 对于 CCM, 取值范围是 {4,6,8,10,12,14,16}
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过

- `false` 验证不通过

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.121 `ehsm_aead_onepass_enc_with_plain_key`

原型:

```
1| uint32_t ehsm_aead_onepass_enc_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx,  
ehsm_symm_algo_e algo, ehsm_aead_mode_e mode, EHSM_SHM const uint8_t *key, uint32_t  
key_size, EHSM_SHM const uint8_t *nonce, uint32_t nonce_size, EHSM_SHM const uint8_t *aad,  
uint32_t aad_size, EHSM_SHM const uint8_t *input, uint32_t input_size, EHSM_SHM uint8_t  
*output, EHSM_SHM uint8_t *tag, uint32_t tag_size)
```

说明:

AEAD 加密计算，一次计算完成，使用明文密钥 此函数处理连续的块数据，输入数据长度为任意长度，实际写入 `output` 中的数据与输入数据长度相同。

参数:

- `ctx`: ehsm context 地址，需事先调用过 [ehsm_ctx_init](#) 初始化
- `algo`: AEAD 使用的对称算法类型，见 [ehsm_symm_algo_e](#)，不支持 DES 和 TDES
- `mode`: AEAD 模式，见 [ehsm_aead_mode_e](#)
- `key`: 用于 AEAD 加密计算的明文密钥，参考[明文密钥数据格式](#)
- `key_size`: 明文密钥字节数
- `nonce`: nonce 值
- `nonce_size`: nonce 值的长度，对于 GCM，必须为 12 字节，对于 CCM，取值范围是 7~13
- `aad`: 附加数据，可以为 NULL
- `aad_size`: 附加数据长度，不能大于 128
- `input`: 输入数据
- `input_size`: 输入数据长度，可以是任意长度，包括 0
- `output`: 输出 buffer，其空间大小不能小于 `input_size`
- `tag`: 存入生成的 TAG 值的 buffer
- `tag_size`: 指定生成的 TAG 值长度，对于 GCM，取值范围是 1~16，对于 CCM，取值范围是 {4,6,8,10,12,14,16}

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.122 ehsm_aead_onepass_dec_with_plain_key

原型:

```
1| uint32_t ehsm_aead_onepass_dec_with_plain_key(EHSM_SHM ehsm_ctx_st *ctx,
ehsm_symm_algo_e algo, ehsm_aead_mode_e mode, EHSM_SHM const uint8_t *key, uint32_t
key_size, EHSM_SHM const uint8_t *nonce, uint32_t nonce_size, EHSM_SHM const uint8_t *aad,
uint32_t aad_size, EHSM_SHM const uint8_t *input, uint32_t input_size, EHSM_SHM uint8_t
*output, EHSM_SHM const uint8_t *tag, uint32_t tag_size, bool_t *verify_result)
```

说明:

AEAD 解密计算，一次计算完成，使用明文密钥 此函数处理连续的块数据，输入数据长度为任意长度，实际写入 `output` 中的数据与输入数据长度相同。

参数:

- `ctx`: ehsm context 地址，需事先调用过 `ehsm_ctx_init` 初始化
- `algo`: AEAD 使用的对称算法类型，见 `ehsm_symm_algo_e`，不支持 DES 和 TDES
- `mode`: AEAD 模式，见 `ehsm_aead_mode_e`
- `key`: 用于 AEAD 解密计算的明文密钥，参考[明文密钥数据格式](#)
- `key_size`: 明文密钥字节数
- `nonce`: nonce 值
- `nonce_size`: nonce 值的长度，对于 GCM，必须为 12 字节，对于 CCM，取值范围是 7~13
- `aad`: 附加数据，可以为 NULL
- `aad_size`: 附加数据长度，不能大于 128
- `input`: 输入数据
- `input_size`: 输入数据长度，可以是任意长度，包括 0
- `output`: 输出 buffer，其空间大小不能小于 `input_size`
- `tag`: 输入的 TAG 值
- `tag_size`: 输入的 TAG 长度，对于 GCM，取值范围是 1~16，对于 CCM，取值范围是 {4,6,8,10,12,14,16}
- `verify_result`: 存放验证结果的内存位置
 - `true` 验证通过
 - `false` 验证不通过

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.123 ehsm_km_gen_key

原型:

```
1| uint32_t ehsm_km_gen_key(EHSM_SHM ehsm_ctx_st *ctx, ehsm_key_type_e key_type, uint32_t  
privilege, uint32_t rsa_e_bit_size, uint32_t hmac_key_size, EHSM_SHM const void *dh_params,  
uint32_t dh_params_size, uint32_t *key_handle)
```

说明:

让 eHSM 内部生成随机密钥 (对)。

注意:

- 请参考用户手册以了解完整的密钥管理方案。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `key_type`: 密钥类型, 见 [ehsm_key_type_e](#) (此函数不支持 DH 密钥对生成)
- `privilege`: 密钥权限, 由权限位组合而成, 见 [密钥权限](#)
- `rsa_e_bit_size`: RSA 的公钥 E 值的位长度, 例如 17 代表 E 值为 0x10001。生成非 RSA 密钥时应设置为 0。
- `hmac_key_size`: HMAC 的密钥长度, 生成非 HMAC 密钥时应设置为 0。
- `dh_params`: 指向 DH 密钥的参数, 生成非 DH 密钥对时应设置为 `NULL`。指向的参数只能是以下结构体的指针:
 - [ehsm_dh_params_1024_st](#)
 - [ehsm_dh_params_2048_st](#)
 - [ehsm_dh_params_3072_st](#)
 - [ehsm_dh_params_4096_st](#)
- `dh_params_size`: DH 密钥参数结构体的字节长度, 生成非 DH 密钥对时应设置为 0
- `key_handle`: 存放密钥句柄的指针
 - 如果提供的是 [EHSM_KEY_HANDLE_INVALID](#), 会由 eHSM 自动分配一个密钥句柄, 完成时写入此位置;
 - 否则使用指定的密钥句柄, 密钥句柄不合法时会失败;
 - 如果 eHSM 已有此句柄的密钥, 视其权限可能会返回失败或覆盖成功

返回值:

- EHSM_OK : 任务成功完成

- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.124 ehsm_km_import_key

原型:

```
1| uint32_t ehsm_km_import_key(EHSM_SHM ehsm_ctx_st *ctx, uint32_t transport_key_handle,  
uint32_t auth_key_handle, EHSM_SHM const ehsm_key_format_st *key_data, uint32_t  
key_data_size, EHSM_SHM const uint8_t *mac, uint32_t mac_size, uint32_t *key_handle)
```

说明:

导入密钥。

注意:

- 请参考[密钥管理方案](#)和用户手册以了解完整的密钥管理方案。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `transport_key_handle`: - 密文导入时, 此值是传输密钥句柄
 - 明文导入时此值应为 [EHSM_KEY_HANDLE_INVALID](#)。
- `auth_key_handle`: 用于明文/密文导出时指定验证的密钥句柄。
 - 如果是密文导入, 此值必须是合法密钥句柄
 - 如果是明文导入, 此值可以是 [EHSM_KEY_HANDLE_INVALID](#), 也可以是一个合法密钥句柄
 - 可以和 `transport_key_handle` 相同, 以使用同一个密钥
- `key_data`: 导入的密钥数据, 见 [ehsm_key_format_st](#)
- `key_data_size`: 导入密钥数据 buffer 的大小, 见[密钥数据格式](#)
- `mac`: 指向密钥数据结构的 MAC 值, 如果 `auth_key_handle` 为 [EHSM_KEY_HANDLE_INVALID](#), 这里应传入 `NULL`
- `mac_size`: MAC 值的长度, 如果 `mac` 为 `NULL`, 此值应为 0。
- `key_handle`: 存放密钥句柄的指针
 - 如果提供的是 [EHSM_KEY_HANDLE_INVALID](#), 会由 eHSM 自动分配一个密钥句柄, 完成时写入此位置;

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.125 ehsm_km_export_key

原型:

```
1| uint32_t ehsm_km_export_key(EHSM_SHM ehsm_ctx_st *ctx, uint32_t target_key_handle,
uint32_t transport_key_handle, uint32_t auth_key_handle, ehsm_key_part_e key_part, EHSM_SHM
ehsm_key_format_st *key_data, uint32_t *key_data_size, EHSM_SHM uint8_t *mac, uint32_t
*mac_size)
```

说明:

导出密钥。

注意:

- 请参考[密钥管理方案](#)和用户手册以了解完整的密钥管理方案。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `target_key_handle`: 要导出的目标密钥句柄
- `transport_key_handle`: - 密文导出时, 此值是传输密钥句柄
 - 明文导出时此值应为 [EHSM_KEY_HANDLE_INVALID](#)。
- `auth_key_handle`: 用于明文/密文导出时指定验证的密钥句柄。
 - 如果是密文导出, 此值必须是合法密钥句柄
 - 如果是明文导出, 此值可以是 [EHSM_KEY_HANDLE_INVALID](#), 也可以是一个合法密钥句柄
 - 可以和 `transport_key_handle` 相同, 以使用同一个密钥
- `key_part`: 导出密钥的哪部分, 见 [ehsm_key_part_e](#)
- `key_data`: 存放导出的密钥数据 buffer, 见 [ehsm_key_format_st](#)
- `key_data_size`: 传入时存放导出密钥数据 buffer 的大小, 不能小于对应密钥的导出数据格式所需空间, 见[密钥数据格式](#); 完成时存放实际输出到 `key_data` 中的数据长度
- `mac`: 存放输出的密钥数据 MAC 值的 buffer, 如果 `auth_key_handle` 为 [EHSM_KEY_HANDLE_INVALID](#), 则此参数应当传入 NULL
- `mac_size`: 传入时保存的是 `mac` 指向 buffer 的大小, 成功完成时保存的是输出的密钥 MAC 值长度。如果 `mac` 为 `NULL`, 此值应为 `NULL`。

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.126 ehsm_km_derive_key

原型:

```
1| uint32_t ehsm_km_derive_key(EHSM_SHM ehsm_ctx_st *ctx, ehsm_hash_algo_e hash_algo,  
ehsm_derive_algo_e derive_algo, ehsm_derive_type_e derive_type, uint32_t privilege,  
ehsm_key_type_e key_type, uint16_t key_size, uint32_t parent_key_handle, EHSM_SHM const  
uint8_t *salt, uint32_t salt_size, EHSM_SHM const uint8_t *password, uint32_t  
password_size, uint32_t iter_times, uint32_t *key_handle)
```

说明:

密钥派生。

注意:

- 请参考[密钥管理方案](#)和用户手册以了解完整的密钥管理方案。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `hash_algo`: 使用的 HASH 算法, 不支持摘要长度小于 32 字节的算法, 见 [ehsm_hash_algo_e](#)
- `derive_algo`: 派生算法类型, 见 [ehsm_derive_algo_e](#)
- `derive_type`: 派生方式, 见 [ehsm_derive_type_e](#)
- `privilege`: 生成的密钥权限, 由权限位组合而成, 见[密钥权限](#)
- `key_type`: 生成的密钥类型, 仅支持对称密钥类型, 见 [ehsm_key_type_e](#)
- `key_size`: 生成的密钥长度, 对于生成 HMAC 密钥, 最大不超过 512 字节; 其它密钥类型此长度必须与对应类型相匹配。
- `parent_key_handle` : 派生使用的父密钥句柄, 仅 `derive_type` 为 [EHSM_DERIVE_TYPE_FROM_PARENT_KEY](#) 时有效
- `salt`: 盐值, 可以为 `NULL`
- `salt_size`: 盐值长度, 不能大于 128
- `password`: 密码值, 仅 `derive_type` 为 [EHSM_DERIVE_TYPE_PASSWORD](#) 时有效, 否则应传入 `NULL`
- `password_size` : 密码值长度, 不能大于 512, 仅 `derive_type` 为 [EHSM_DERIVE_TYPE_PASSWORD](#) 时有效, 否则应传入 0
- `iter_times`: 派生算法的迭代次数
- `key_handle`: 存放密钥句柄的指针
 - 如果提供的是 [EHSM_KEY_HANDLE_INVALID](#), 会由 eHSM 自动分配一个密钥句柄, 完成时写入此位置;

返回值:

- `EHSM_OK`: 任务成功完成

- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.127 ehsm_km_derive_key_to_soc

原型:

```
1| uint32_t ehsm_km_derive_key_to_soc(EHSM_SHM ehsm_ctx_st *ctx, ehsm_hash_algo_e  
hash_algo, ehsm_derive_algo_e derive_algo, ehsm_derive_type_e derive_type, uint32_t  
parent_key_handle, EHSM_SHM const uint8_t *salt, uint32_t salt_size, uint32_t iter_times,  
uint8_t soc_channel_id)
```

说明:

派生密钥并将密钥值直接输出到 SOC 的物理安全端口。

注意:

- 此函数不会将密钥存储在 eHSM 内部, 不会有对应的 key_handle。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `hash_algo`: 使用的 HASH 算法, 见 `ehsm_hash_algo_e`
- `derive_algo`: 派生算法类型, 见 `ehsm_derive_algo_e`
- `derive_type`: 派生方式, 见 `ehsm_derive_type_e`
- `parent_key_handle`: 派生使用的父密钥句柄, 仅 `derive_type` 为 `EHSM_DERIVE_TYPE_FROM_PARENT_KEY` 时有效
- `salt`: 盐值, 可以为 `NULL`
- `salt_size`: 盐值长度, 不能大于 128
- `iter_times`: 派生算法的迭代次数
- `soc_channel_id`: 密钥值输出到 SOC 端的物理安全通道

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.128 ehsm_km_exchange_key

原型:

```
1| uint32_t ehsm_km_exchange_key(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t
*rmt_pub_key, uint32_t rmt_pub_key_size, uint32_t privilege, ehsm_key_type_e key_type,
uint32_t hmac_key_size, uint32_t local_key_handle, EHSM_SHM const void *dh_params, uint32_t
dh_params_size, EHSM_SHM ehsm_sm2_params_st *sm2_params, uint32_t *key_handle)
```

说明:

密钥交换。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `rmt_pub_key`: 对端公钥值, 仅公钥值内容, 格式见[密钥数据格式](#)
- `rmt_pub_key_size`: 对端公钥值长度
- `privilege`: 生成的目标密钥的权限, 见[密钥权限](#)
- `key_type`: 生成的目标密钥类型, 见 [ehsm_key_type_e](#), 仅能生成其中的对称密钥类型
- `hmac_key_size`: 生成的目标密钥长度, 仅生成 HMAC 密钥时有效, 不能超过 512 字节。其它密钥类型传入 0。
- `local_key_handle`: 本地私钥的密钥句柄
- `dh_params`: 如果是 DH 算法, 传入相应的交换参数, 否则应传入 `NULL`。指向的参数只能是以下结构体的指针:
 - [ehsm_dh_params_1024_st](#)
 - [ehsm_dh_params_2048_st](#)
 - [ehsm_dh_params_3072_st](#)
 - [ehsm_dh_params_4096_st](#)
- `dh_params_size`: DH 密钥参数结构体的字节长度, 生成非 DH 密钥对时应设置为 0
- `sm2_params`: 如果是 SM2 算法, 传入相应的交换参数, 否则应传入 `NULL`
- `key_handle`: 存放密钥句柄的指针
 - 如果提供的是 [EHSM_KEY_HANDLE_INVALID](#), 会由 eHSM 自动分配一个密钥句柄, 完成时写入此位置;

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.129 ehsm_km_sm9_exchange_key

原型:

```
1| uint32_t ehsm_km_sm9_exchange_key(EHSM_SHM ehsm_ctx_st *ctx, uint32_t privilege,  
ehsm_key_type_e key_type, uint8_t role, uint32_t user_priv_key_handle, uint32_t  
user_tmp_key_handle, uint32_t hmac_key_size, ehsm_sm9_params_st *extra_params, uint32_t  
*key_handle)
```

说明:

SM9 密钥协商。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `privilege`: 生成的目标密钥的权限, 见[密钥权限](#)
- `key_type`: 生成的目标密钥类型, 见 [ehsm_key_type_e](#), 仅能生成其中的对称密钥类型
- `role`: 密钥协商的角色, 见 [ehsm_sm9_role_e](#)
- `user_priv_key_handle`: 本地的密钥协商用户私钥句柄
- `user_tmp_key_handle`: 本地的密钥协商临时密钥句柄
- `hmac_key_size`: 生成的目标密钥长度, 仅生成 HMAC 密钥时有效, 不能超过 512 字节。其它密钥类型传入 0。
- `extra_params`: SM9 密钥协商的其他参数
- `key_handle`: 存放密钥句柄的指针, 由 eHSM 自动分配一个密钥句柄, 完成时写入此位置;

返回值:

- EHSM_OK: 任务成功完成
- EHSM_ERR_NEED_POLL: 任务未完成
- 其它值: 任务发生错误

8.4.130 ehsm_km_get_pub_from_priv

原型:

```
1| uint32_t ehsm_km_get_pub_from_priv(EHSM_SHM ehsm_ctx_st *ctx, uint32_t key_handle,  
EHSM_SHM const void *dh_params, uint32_t dh_params_size, EHSM_SHM uint8_t *pub_key,  
uint32_t *pub_key_size, ehsm_key_type_e *key_type)
```

说明:

从包含私钥的密钥句柄计算公钥值。

注意:

- RSA 算法不支持此功能。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `key_handle`: 密钥句柄
- `dh_params`: 指向 DH 密钥的参数, 生成非 DH 密钥对时应设置为 `NULL`。指向的参数只能是以下结构体的指针:
 - `ehsm_dh_params_1024_st`
 - `ehsm_dh_params_2048_st`
 - `ehsm_dh_params_3072_st`
 - `ehsm_dh_params_4096_st`
- `dh_params_size`: DH 密钥参数结构体的字节长度, 生成非 DH 密钥对时应设置为 0
- `pub_key`: 存放输出公钥值的 buffer, 仅包含公钥数据, 公钥格式见[密钥数据格式](#)
- `pub_key_size`: 输入时为 `pub_key` buffer 的长度, 完成时为写入 `pub_key` 的数据长度
- `key_type`: 输出密钥类型

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.131 ehsm_km_remove_key

原型:

```
1| uint32_t ehsm_km_remove_key(EHSM_SHM ehsm_ctx_st *ctx, uint32_t key_handle)
```

说明:

删除指定密钥。如果密钥没有可删除权限, 则删除会失败。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `key_handle`: 期望删除的密钥句柄

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.132 ehsm_get_version

原型:

```
1| uint32_t ehsm_get_version(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM ehsm_version_st  
*version)
```

说明:

获取 BootLoader/Firmware 的版本号。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `version`: 保存版本数据的 buffer, 见 `ehsm_version_st`

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.133 ehsm_verify_image

原型:

```
1| uint32_t ehsm_verify_image(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t *image,  
uint32_t image_size, bool_t check_version, bool_t boot, EHSM_SHM uint8_t *image_out)
```

说明:

校验安全启动镜像。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `image`: 镜像的存储位置
- `image_size`: 镜像的大小
- `check_version`: 是否检查 version counter
- `boot`: 校验成功后是否跳转到 FW 中执行, 仅对 eHSM 镜像有效
- `image_out`: 镜像解密输出的地址, 包含镜像头, 仅对 SOC 镜像有效, 如果是 eHSM 镜像此参数应为 `NULL`

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.134 ehsm_upgrade_fw_image_ex

原型:

```
1| uint32_t ehsm_upgrade_fw_image_ex(EHSM_SHM ehsm_ctx_st *ctx, ehsm_proc_mode_e mode,  
EHSM_SHM const uint8_t *input, uint32_t input_size, EHSM_SHM uint8_t *output)
```

说明:

升级固件镜像的扩展函数，支持不同的处理阶段。

注意:

- 这是底层接口，建议根据需求使用封装接口，参考
- [ehsm_upgrade_fw_image](#)
- [ehsm_upgrade_fw_image_init](#)
- [ehsm_upgrade_fw_image_update](#)
- [ehsm_upgrade_fw_image_finish](#)

参数:

- `ctx`: ehsm context 地址，需事先调用过 [ehsm_ctx_init](#) 初始化
- `mode`: 处理模式，见 [ehsm_proc_mode_e](#)
- `input`: 输入数据地址
- `input_size`: 输入数据长度
- `output`: 输出数据地址

返回值:

- EHSM_OK: 任务成功完成
- EHSM_ERR_NEED_POLL: 任务未完成
- 其它值: 任务发生错误

8.4.135 ehsm_upgrade_fw_image

原型:

```
1| uint32_t ehsm_upgrade_fw_image(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t  
*image, uint32_t image_size, EHSM_SHM uint8_t *image_out)
```

说明:

升级固件镜像，一次性完成。此操作输入一个安全升级镜像，得到一个安全启动镜像，根据安全启动镜像的 `Plain_Flag` 决定是否对安全启动镜像的 CODE 区域加密。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `image`: 完整安全升级镜像数据地址
- `image_size`: 完整安全升级镜像字节大小
- `image_out`: 输出安全启动镜像的地址, 可以等于 `image` + 1KB 的位置 (即在原地解密), 否则不要与输入的区域有重叠, 输出的数据长度为输入镜像长度 - 1KB, 应提供足够的空间

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.136 ehsm_upgrade_fw_image_init

原型:

```
1 | uint32_t ehsm_upgrade_fw_image_init(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t  
  | *image_header, uint32_t header_size, EHSM_SHM uint8_t *image_out)
```

说明:

升级固件镜像初始化。

注意:

- 必须将完整的镜像头输入, 且此函数只处理镜像头, 不会处理数据部分。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `image_header`: 安全升级镜像的镜像头地址
- `header_size`: 镜像头长度
- `image_out`: 用于存放输出的启动镜像的起始地址, 实际 init 时不会向此地址输出数据, 但不能传入 NULL

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.137 ehsm_upgrade_fw_image_update

原型:

```
1| uint32_t ehsm_upgrade_fw_image_update(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const
uint8_t *body_block, uint32_t block_size, EHSM_SHM uint8_t *image_out)
```

说明:

升级固件镜像更新。 此函数用于分块提交升级镜像的数据。

注意:

- 第一次调用此函数（即更新第一块升级镜像中的数据块）时，传入的数据长度不能小于镜像头的大小。因为第一块数据中需要 包含安全启动镜像的镜像头(注意不是升级镜像的镜像头)，以便 eHSM 一次性读取并处理。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `body_block`: 输入的安全升级镜像数据块地址 (镜像头必须在 `ehsm_upgrade_fw_image_init` 中输入)
- `block_size`: 输入数据长度, 必须是 16 的整数倍
- `image_out`: 用于存放输出的启动镜像对应数据的地址, 输出数据的长度与输入数据相同

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.138 ehsm_upgrade_fw_image_finish

原型:

```
1| uint32_t ehsm_upgrade_fw_image_finish(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const
uint8_t *body_block, uint32_t block_size, EHSM_SHM uint8_t *image_out)
```

说明:

升级固件镜像完成。 此函数用于分块提交升级镜像的最后一块数据并完成校验, 此函数应当只对最后一个数据块调用一次。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `body_block`: 输入的安全升级镜像数据最后一块地址 (镜像头必须在 `ehsm_upgrade_fw_image_init` 中输入)
- `block_size`: 输入数据长度, 必须是 16 的整数倍
- `image_out`: 用于存放输出的启动镜像对应数据的地址, 输出数据的长度与输入数据相同

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.139 ehsm_set_utc_time**原型:**

```
1| uint32_t ehsm_set_utc_time(EHSM_SHM ehsm_ctx_st *ctx, uint32_t utc_time)
```

说明:

设置 EHSM UTC-Timer 时间。此接口可以配置 UTC 时间到 EHSM 的 UTC-Timer 模块, 以此实现 UTC 时间配置或同步的功能。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `utc_time`: 需要配置的 UTC 时间

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.140 ehsm_get_utc_time**原型:**

```
1| uint32_t ehsm_get_utc_time(EHSM_SHM ehsm_ctx_st *ctx, uint32_t *utc_time)
```

说明:

获取 EHSM UTC-Timer 时间。此接口可以获取 EHSM 当前的 UTC-Timer 的值。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `utc_time`: 当前的 EHSM 的 UTC 时间

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成

- 其它值：任务发生错误

8.4.141 ehsm_create_counter

原型：

```
1| uint32_t ehsm_create_counter(EHSM_SHM ehsm_ctx_st *ctx, uint32_t *counter_id, uint64_t *counter_value)
```

说明：

创建（使能）一个计数器 此接口可以创建或使能一个 EHSM 的计数器。

参数：

- `ctx`：ehsm context 地址，需事先调用过 `ehsm_ctx_init` 初始化
- `counter_id`：被创建或使能的计数器的 ID
- `counter_value`：被创建或使能的计数器的数值

返回值：

- EHSM_OK：任务成功完成
- EHSM_ERR_NEED_POLL：任务未完成
- 其它值：任务发生错误

8.4.142 ehsm_read_counter

原型：

```
1| uint32_t ehsm_read_counter(EHSM_SHM ehsm_ctx_st *ctx, uint32_t counter_id, uint64_t *counter_value)
```

说明：

读取某个计数器的值 此接口可以读取某个计数器的值。

参数：

- `ctx`：ehsm context 地址，需事先调用过 `ehsm_ctx_init` 初始化
- `counter_id`：需要被读取的计数器的 ID
- `counter_value`：返回的计数器的值

返回值：

- EHSM_OK：任务成功完成
- EHSM_ERR_NEED_POLL：任务未完成
- 其它值：任务发生错误

8.4.143 ehsm_increase_counter

原型:

```
1| uint32_t ehsm_increase_counter(EHSM_SHM ehsm_ctx_st *ctx, uint32_t counter_id,  
uint64_t increase_value, uint64_t *current_value)
```

说明:

为某个计数器增加计数值 此接口可以为某个计数器增加计数值

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `counter_id`: 需要增加计数值的计数器 ID
- `increase_value`: 需要增加的计数值
- `current_value`: 增加数值后的计数值

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.144 ehsm_delete_counter

原型:

```
1| uint32_t ehsm_delete_counter(EHSM_SHM ehsm_ctx_st *ctx, uint32_t counter_id)
```

说明:

删除（使失能）某个计数器 此接口可以删除（使失能）某个计数值

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `counter_id`: 需要删除的计数器 ID

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.145 ehsm_enter_wfi

原型:

```
1| uint32_t ehsm_enter_wfi(EHSM_SHM ehsm_ctx_st *ctx)
```

说明:

eHSM 进入 WFI 状态。 eHSM 在收到命令后会, 当所有 Mailbox channel 上都没有命令时, 会进入 WFI 状态, 直到下一次被 Mailbox 或其它中断唤醒。

注意:

- 发送此命令后, 在 cpu_wfi 信号置位之前, 不应在任何 Mailbox channel 上发送命令。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.146 ehsm_set_uart_baudrate

原型:

```
1| uint32_t ehsm_set_uart_baudrate(EHSM_SHM ehsm_ctx_st *ctx, uint32_t baud_div)
```

说明:

设置串口波特率

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `baud_div`: UART 的频率分频值, 指 CPU 频率值除以 UART 波特率值的结果

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.147 ehsm_install_random_key

原型:

```
1| uint32_t ehsm_install_random_key(EHSM_SHM ehsm_ctx_st *ctx, ehsm_key_level_e  
key_level, ehsm_install_key_type_e key_type, uint16_t key_slot_id, bool_t last_key)
```

说明:

生成一个随机密钥并且写入指定的 OTP 密钥槽中

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `key_level`: 密钥级别, 见 `ehsm_key_level_e`
- `key_type`: 生成的密钥类型 见 `ehsm_install_key_type_e`
- `key_slot_id`: 安装的 OTP 密钥槽 ID 编号
- `last_key`: 是否是最后一个 OTP 密钥, true: 是最后一个, false: 不是最后一个

返回值:

- EHSM_OK : 任务成功完成
- EHSM_ERR_NEED_POLL : 任务未完成
- 其它值 : 任务发生错误

8.4.148 ehsm_install_encrypted_key

原型:

```
1| uint32_t ehsm_install_encrypted_key(EHSM_SHM ehsm_ctx_st *ctx, ehsm_key_level_e  
key_level, ehsm_install_key_type_e key_type, uint16_t key_slot_id, bool_t last_key,  
EHSM_SHM const uint8_t *input_data, uint32_t size)
```

说明:

将外部加密的密钥值写入到指定的 OTP 密钥槽中 此命令内部使用 `CHIP RTL KEK EHSM` 或 `CHIP RTL KEK SOC` 密钥对输入值进行 CBC (IV=全 0) 解密, 然后使用对应的 `CHIP ROOT KEY` 或 `DEVICE ROOT KEY` 对密钥明文进行加密, 并计算其 CRC32 值, 最后将加密后的密钥值和 CRC32 值写入指定的 OTP 密钥槽

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `key_level`: 密钥级别, 见 `ehsm_key_level_e`
- `key_type`: 生成的密钥类型 见 `ehsm_install_key_type_e`
- `key_slot_id`: 安装的 OTP 密钥槽 ID 编号
- `last_key`: 是否是最后一个 OTP 密钥, true: 是最后一个, false: 不是最后一个

- `input_data` : 密钥值或公钥 HASH 值, 根据 `key_level` 的取值, 被 `CHIP RTL KEK EHSM` 或 `CHIP RTL KEK SOC` 加密过 (CBC 模式, IV=全 0, 算法由 OTP HW-CTRL 配置中的 `KeyAlgSel` 决定是 AES128 还是 SM4)
- `size`: `input_data` 的字节长度

返回值:

- `EHSM_OK` : 任务成功完成
- `EHSM_ERR_NEED_POLL` : 任务未完成
- 其它值 : 任务发生错误

8.4.149 ehsm_read_otp

原型:

```
1| uint32_t ehsm_read_otp(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM uint8_t *buf, uint32_t ehsm_src_addr, uint32_t size)
```

说明:

读取 OTP 数据。此功能仅支持 `TEST_MODE` 和 `DEVELOPE_MODE`。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `buf`: 存储读取的 OTP 数据
- `ehsm_src_addr`: 从 ehsm 内部的 OTP 地址读取 (即 0x33000000 开始的 OTP 数据地址)
- `size`: 要读取的字节长度

返回值:

- `EHSM_OK` : 任务成功完成
- `EHSM_ERR_NEED_POLL` : 任务未完成
- 其它值 : 任务发生错误

8.4.150 ehsm_write_otp

原型:

```
1| uint32_t ehsm_write_otp(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t *src_data, uint32_t ehsm_dest_addr, uint32_t size)
```

说明:

写入 OTP 数据。此功能仅支持 `TEST_MODE` 和 `DEVELOPE_MODE`。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `src_data`: 写入的源数据
- `ehsm_dest_addr`: 写入的 ehsm 内部 OTP 地址 (即 0x33000000 开始的 OTP 数据地址)
- `size`: 写入数据的字节长度

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.151 ehsm_read_reg

原型:

```
1| uint32_t ehsm_read_reg(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM uint8_t *buf, uint32_t  
ehsm_src_addr, uint32_t size)
```

说明:

读取 CFG 地址范围的寄存器值。此功能仅支持 `TEST_MODE` 和 `DEVELOPE_MODE`。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 `ehsm_ctx_init` 初始化
- `buf`: 存储读取的 CFG 寄存器数据
- `ehsm_src_addr`: 从 ehsm 内部的 CFG 地址读取 (即 0x33400000~0x334FFFFFF 地址区间)
- `size`: 要读取的字节长度

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.152 ehsm_write_reg

原型:

```
1| uint32_t ehsm_write_reg(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM const uint8_t *src_data,  
uint32_t ehsm_dest_addr, uint32_t size)
```

说明:

写入 CFG 地址范围的寄存器值。此功能仅支持 `TEST_MODE` 和 `DEVELOPE_MODE`。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `src_data`: 写入的源数据
- `ehsm_dest_addr`: 写入的 ehsm 内部 CFG 地址 (即 0x33400000~0x334FFFFFF 地址区间)
- `size`: 写入数据的字节长度

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.153 ehsm_get_challenge

原型:

```
1| uint32_t ehsm_get_challenge(EHSM_SHM ehsm_ctx_st *ctx, ehsm_challenge_type_e  
challenge_type, EHSM_SHM uint8_t *output)
```

说明:

获取鉴权挑战值。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `challenge_type`: 挑战鉴权类型, 见 [ehsm_challenge_type_e](#)
- `output`: 存放挑战值的地址, 挑战值的长度见 [ehsm_challenge_type_e](#), `output` 的可用长度不能小于挑战值长度

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.154 ehsm_debug_auth

原型:

```
1| uint32_t ehsm_debug_auth(EHSM_SHM ehsm_ctx_st *ctx, ehsm_challenge_type_e  
challenge_type, ehsm_auth_algo_e algo, EHSM_SHM const uint8_t *sig, uint32_t sig_size,
```

```
EHSM_SHM const uint8_t *pub_key, uint32_t pub_key_size, EHSM_SHM const  
ehsm_soc_dbg_bitmap_st *soc_dbg_bitmap)
```

说明:

完成鉴权并进行相应的调试配置操作。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `challenge_type`: 挑战鉴权类型, 应当与调用 [ehsm_get_challenge](#) 时相同, 见 [ehsm_challenge_type_e](#)
- `algo`: 用于鉴权的算法类型, 见 [ehsm_auth_algo_e](#)
- `sig`: 签名/CMAC 值
- `sig_size`: 签名/CMAC 值的字节长度
 - 对于 SM2 和 ECDSA_P256R1, 为 64 字节
 - 对于 AES 和 SM4 的 CMAC, 为 16 字节
 - 对于 RSA2048 和 RSA4096, 分别为 256 字节和 512 字节
- `pub_key`: 公钥值, 仅挑战类型为 [EHSM_CHALLENGE_TYPE_SOC_DEBUG](#) 或 [EHSM_CHALLENGE_TYPE_EHSM_DEBUG](#) 时且使用非对称算法鉴权时有效, 其它时候传入 `NULL`
- `pub_key_size`: 公钥值的长度
 - 对于 SM2 算法, 长度为 64, 公钥值不带前面的 `0x04` 标记
 - 对于 ECDSA_P256R1 算法, 长度为 64
 - 对于 RSA2048, 长度为 320 (64 字节 E + 256 字节 N)
 - 对于 RSA4096, 长度为 576 (64 字节 E + 512 字节 N)
- `soc_dbg_bitmap`: 对挑战类型为 [EHSM_CHALLENGE_TYPE_SOC_DEBUG](#) 时有效, 其它类型传入 `NULL`, 对于 bitmap 中的值, 为 1 的 bit 时对应的调试端口被打开, 为 0 的 bit 对应的调试端口保持原状态。见 [ehsm_soc_dbg_bitmap_st](#)

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.155 ehsm_close_debug

原型:

```
1| uint32_t ehsm_close_debug(EHSM_SHM ehsm_ctx_st *ctx, ehsm_challenge_type_e type,  
EHSM_SHM const ehsm_soc_dbg_bitmap_st *soc_dbg_bitmap)
```

说明:

关闭调试。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `type`: 仅支持 [EHSM_CHALLENGE_TYPE_SOC_DEBUG](#) 和 [EHSM_CHALLENGE_TYPE_EHSM_DEBUG](#)
- `soc_dbg_bitmap`: 对挑战类型为 [EHSM_CHALLENGE_TYPE_SOC_DEBUG](#) 时有效, 其它类型传入 `NULL`, 对于 bitmap 中的值, 为 1 的 bit 时对应的调试端口被关闭, 为 0 的 bit 对应的调试端口保持原状态。见 [ehsm_soc_dbg_bitmap_st](#)

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.156 ehsm_get_emu_status

原型:

```
1| uint32_t ehsm_get_emu_status(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM ehsm_emu_status_st *status_buf)
```

说明:

获取 eHSM EMU 状态。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `status_buf`: 存储 EMU 状态信息的共享内存地址, 见 [ehsm_emu_status_st](#)

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.157 ehsm_change_lifecycle

原型:

```
1| uint32_t ehsm_change_lifecycle(EHSM_SHM ehsm_ctx_st *ctx, ehsm_lifecycle_e lifecycle)
```

说明:

更改 eHSM 生命周期。

注意:

- 如果当前 eHSM 生命周期为 EHSM_LC_USER，想要修改生命周期必须先鉴权，见 [EHSM_CHALLENGE_TYPE_USER_AUTH](#)

参数:

- `ctx`: ehsm context 地址，需事先调用过 [ehsm_ctx_init](#) 初始化
- `lifecycle`: 目标生命周期，不能比当前 eHSM 的生命周期小

返回值:

- EHSM_OK: 任务成功完成
- EHSM_ERR_NEED_POLL: 任务未完成
- 其它值: 任务发生错误

8.4.158 ehsm_change_control_field

原型:

```
1| uint32_t ehsm_change_control_field(EHSM_SHM ehsm_ctx_st *ctx, ehsm_ctrl_field_e field,  
EHSM_SHM const uint64_t *value)
```

说明:

修改 OTP 控制字段的值。

参数:

- `ctx`: ehsm context 地址，需事先调用过 [ehsm_ctx_init](#) 初始化
- `field`: 控制字段类型，见 [ehsm_ctrl_field_e](#)
- `value`: 存放 64 位值的共享内存地址。

返回值:

- EHSM_OK: 任务成功完成
- EHSM_ERR_NEED_POLL: 任务未完成
- 其它值: 任务发生错误

8.4.159 ehsm_bl_self_test

原型:

```
1| uint32_t ehsm_bl_self_test(EHSM_SHM ehsm_ctx_st *ctx)
```

说明:

启动算法自检。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.160 ehsm_bl_get_self_test_result

原型:

```
1| uint32_t ehsm_bl_get_self_test_result(EHSM_SHM ehsm_ctx_st *ctx, EHSM_SHM  
ehsm_self_test_result_st *result)
```

说明:

查询算法自检结果。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `result`: 算法自检结果, 见 [ehsm_self_test_result_st](#)

返回值:

- `EHSM_OK`: 任务成功完成
- `EHSM_ERR_NEED_POLL`: 任务未完成
- 其它值: 任务发生错误

8.4.161 ehsm_bl_fw_auth

原型:

```
1| uint32_t ehsm_bl_fw_auth(EHSM_SHM ehsm_ctx_st *ctx, uint8_t type, const uint8_t  
arg[16], EHSM_SHM const uint8_t *auth_data, EHSM_SHM uint8_t *out_addr)
```

说明:

国密固件认证。

注意:

- 需要先执行 [ehsm_bl_get_challenge](#)。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `type`: 认证类型: 1 表示删除给定的用户密钥, 2 表示版本计数器更新, 3 表示进入销毁模式, 删除所有密钥
- `arg`: 当类型为 1 时, arg 表示用户密钥 ID; 当类型为 2 时, 表示新的版本计数器; 否则忽略
- `auth_data`: 认证数据。认证数据, 表示随机数据 Ra||Rb 的 SM4-CBC 加密结果, Ra 由 `bl_get_challenge` 命令生成, Rb 由主机生成。Ra 和 Rb 均为 16 字节
- `out_addr`: OTP 输出 buffer, 可以为 NULL, 当不为 NULL 时, 应当至少有 720 字节空间

返回值:

- EHSM_OK: 任务成功完成
- EHSM_ERR_NEED_POLL: 任务未完成
- 其它值: 任务发生错误

8.4.162 ehsm_bl_get_random_key

原型:

```
1| uint32_t ehsm_bl_get_random_key(EHSM_SHM ehsm_ctx_st *ctx, ehsm_key_level_e key_level,  
ehsm_bl_gen_key_type_e key_type, EHSM_SHM uint8_t *key_out)
```

说明:

生成可写入 OTP 的被 ROOT KEY 加密的密钥值及相应的 CRC。

参数:

- `ctx`: ehsm context 地址, 需事先调用过 [ehsm_ctx_init](#) 初始化
- `key_level`: 密钥级别, 见 [ehsm_key_level_e](#)
- `key_type`: 生成的密钥类型 见 [ehsm_bl_gen_key_type_e](#)
- `key_out`: 输出加密密钥值和 CRC32 的地址, 空间不能小于 36 字节

返回值:

- EHSM_OK: 任务成功完成
- EHSM_ERR_NEED_POLL: 任务未完成
- 其它值: 任务发生错误

8.4.163 ehsm_bl_encrypt_key

原型:

```
1| uint32_t ehsm_bl_encrypt_key(EHSM_SHM ehsm_ctx_st *ctx, ehsm_key_level_e key_level,
EHSM_SHM const uint8_t *input_data, uint32_t size, EHSM_SHM uint8_t *key_out)
```

说明:

加密外部提供的密钥值，输出被对应 ROOT KEY 加密的密钥值及相应的 CRC。此命令内部使用 CHIP RTL KEK EHSM 或 CHIP RTL KEK SOC 密钥对输入值进行 CBC (IV=全 0) 解密，然后使用对应的 CHIP ROOT KEY 或 DEVICE ROOT KEY 对密钥明文进行加密，并计算其 CRC32 值。

参数:

- `ctx`: ehsm context 地址，需事先调用过 `ehsm_ctx_init` 初始化
- `key_level`: 密钥级别，见 `ehsm_key_level_e`
- `input_data`: 密钥值或公钥 HASH 值，根据 `key_level` 的取值，被 CHIP RTL KEK EHSM 或 CHIP RTL KEK SOC 加密过（CBC 模式，IV=全 0，算法由 OTP HW-CTRL 配置中的 KeyAlgSel 决定是 AES128 还是 SM4）
- `size`: `input_data` 的字节长度
- `key_out`: 输出加密密钥值和 CRC32 的地址，空间不能小于 36 字节

返回值:

- EHSM_OK: 任务成功完成
- EHSM_ERR_NEED_POLL: 任务未完成
- 其它值: 任务发生错误

8.5 Enums

8.5.1 ehsm_reg_e

eHSM 寄存器编号，用于读写。

枚举值:

- `REG_HSM_STATUS_0`: 对应 eHSM o_hsm_status[31:0]
- `REG_HSM_STATUS_1`: 对应 eHSM o_hsm_status[64:32]
- `REG_HSM_ERR_SENSOR`: 对应 eHSM o_hsm_err_seneor[31:0]
- `REG_HSM_ERR_HW_0`: 对应 eHSM o_hsm_err_hw[31:0]
- `REG_HSM_ERR_HW_1`: 对应 eHSM o_hsm_err_hw[64:32]
- `REG_HSM_ERR_FW_0`: 对应 eHSM o_hsm_err_fw[31:0]
- `REG_HSM_ERR_FW_1`: 对应 eHSM o_hsm_err_fw[64:32]

- `REG_HSM_FUSA_ALARM_0`: 对应 eHSM `o_hsm_fusa_alarm[31:0]`
- `REG_HSM_FUSA_ALARM_1`: 对应 eHSM `o_hsm_fusa_alarm[64:32]`
- `REG_SOC_DBG_EN_0`: 对应 eHSM `o_soc_dbg_en_128b[31:0]`
- `REG_SOC_DBG_EN_1`: 对应 eHSM `o_soc_dbg_en_128b[63:32]`
- `REG_SOC_DBG_EN_2`: 对应 eHSM `o_soc_dbg_en_128b[95:64]`
- `REG_SOC_DBG_EN_3`: 对应 eHSM `o_soc_dbg_en_128b[127:96]`
- `REG_SOC_SENSOR`: 输入 eHSM 的传感器告警寄存器, 可写入以触发 eHSM 的操作

8.5.2 ehsm_hash_algo_e

枚举值:

- `EHSM_HASH_ALGO_SM3`: SM3 哈希算法
- `EHSM_HASH_ALGO_MD5`: MD5 哈希算法
- `EHSM_HASH_ALGO_SHA256`: SHA-256 哈希算法
- `EHSM_HASH_ALGO_SHA384`: SHA-384 哈希算法
- `EHSM_HASH_ALGO_SHA512`: SHA-512 哈希算法
- `EHSM_HASH_ALGO_SHA1`: SHA-1 哈希算法
- `EHSM_HASH_ALGO_SHA224`: SHA-224 哈希算法
- `EHSM_HASH_ALGO_SHA512_224`: SHA-512/224 哈希算法
- `EHSM_HASH_ALGO_SHA512_256`: SHA-512/256 哈希算法
- `EHSM_HASH_ALGO_SHA3_224`: SHA3-224 哈希算法
- `EHSM_HASH_ALGO_SHA3_256`: SHA3-256 哈希算法
- `EHSM_HASH_ALGO_SHA3_384`: SHA3-384 哈希算法
- `EHSM_HASH_ALGO_SHA3_512`: SHA3-512 哈希算法

8.5.3 ehsm_rng_algo_e

枚举值:

- `EHSM_RNG_ALGO_SM4_CTR_DRBG`: 基于 SM4-CTR 的确定性随机位生成器
- `EHSM_RNG_ALGO_AES_CTR_DRBG`: 基于 AES-CTR 的确定性随机位生成器

8.5.4 ehsm_symm_algo_e

枚举值:

- `EHSM_SYMM_ALGO_DES`: DES 对称加密算法
- `EHSM_SYMM_ALGO_TDES_128`: 3DES 128 位对称加密算法
- `EHSM_SYMM_ALGO_TDES_192`: 3DES 192 位对称加密算法
- `EHSM_SYMM_ALGO_AES_128`: AES-128 对称加密算法
- `EHSM_SYMM_ALGO_AES_192`: AES-192 对称加密算法
- `EHSM_SYMM_ALGO_AES_256`: AES-256 对称加密算法
- `EHSM_SYMM_ALGO_SM4`: SM4 对称加密算法

8.5.5 ehsm_mac_mode_e

枚举值:

- `EHSM_MAC_MODE_CMAC`: CMAC 消息认证码模式
- `EHSM_MAC_MODE_CBC_MAC`: CBC-MAC 消息认证码模式
- `EHSM_MAC_MODE_GMAC`: GMAC 消息认证码模式

8.5.6 ehsm_padding_mode_e

枚举值:

- `EHSM_PADDING_NONE`
- `EHSM_PADDING_PKCS7`: PKCS7, 兼容 PKCS5
- `EHSM_PADDING_ONE_WITH_ZEROS`: ISO7816-4

8.5.7 ehsm_rsa_padding_mode_e

枚举值:

- `EHSM_RSA_PADDING_NONE`
- `EHSM_RSA_PADDING_PSS`

8.5.8 ehsm_cipher_mode_e

枚举值:

- `EHSM_CIPHER_MODE_ECB`: ECB (电子密码本) 模式
- `EHSM_CIPHER_MODE_XTS`: XTS (可调整密文块链接) 模式
- `EHSM_CIPHER_MODE_CBC`: CBC (密码块链接) 模式

- `EHSM_CIPHER_MODE_CFB`: CFB（密码反馈）模式
- `EHSM_CIPHER_MODE_OFB`: OFB（输出反馈）模式
- `EHSM_CIPHER_MODE_CTR`: CTR（计数器）模式

8.5.9 ehsm_aead_mode_e

枚举值:

- `EHSM_AEAD_MODE_GCM`: GCM（伽罗瓦计数器）认证加密模式
- `EHSM_AEAD_MODE_CCM`: CCM（计数器与 CBC-MAC）认证加密模式

8.5.10 ehsm_key_type_e

枚举值:

- `EHSM_KEY_TYPE_DES`: DES 密钥
- `EHSM_KEY_TYPE_TDES_128`: 3DES 128 位密钥
- `EHSM_KEY_TYPE_TDES_192`: 3DES 192 位密钥
- `EHSM_KEY_TYPE_AES_128`: AES-128 密钥
- `EHSM_KEY_TYPE_AES_192`: AES-192 密钥
- `EHSM_KEY_TYPE_AES_256`: AES-256 密钥
- `EHSM_KEY_TYPE_SM4`: SM4 密钥
- `EHSM_KEY_TYPE_SM2`: SM2 密钥
- `EHSM_KEY_TYPE_RSA_1024`: RSA-1024 密钥
- `EHSM_KEY_TYPE_RSA_2048`: RSA-2048 密钥
- `EHSM_KEY_TYPE_RSA_3072`: RSA-3072 密钥
- `EHSM_KEY_TYPE_RSA_4096`: RSA-4096 密钥
- `EHSM_KEY_TYPE_RSA_1024_CRT`: RSA-1024 CRT 格式密钥
- `EHSM_KEY_TYPE_RSA_2048_CRT`: RSA-2048 CRT 格式密钥
- `EHSM_KEY_TYPE_RSA_3072_CRT`: RSA-3072 CRT 格式密钥
- `EHSM_KEY_TYPE_RSA_4096_CRT`: RSA-4096 CRT 格式密钥
- `EHSM_KEY_TYPE_DH`: DH（Diffie-Hellman）密钥
- `EHSM_KEY_TYPE_ECC_BRAINPOOLP_160R1`: ECC BrainpoolP160r1 曲线密钥

- `EHSM_KEY_TYPE_ECC_BRAINPOOLP_192R1`: ECC BrainpoolP192r1 曲线密钥
- `EHSM_KEY_TYPE_ECC_BRAINPOOLP_224R1`: ECC BrainpoolP224r1 曲线密钥
- `EHSM_KEY_TYPE_ECC_BRAINPOOLP_256R1`: ECC BrainpoolP256r1 曲线密钥
- `EHSM_KEY_TYPE_ECC_BRAINPOOLP_320R1`: ECC BrainpoolP320r1 曲线密钥
- `EHSM_KEY_TYPE_ECC_BRAINPOOLP_384R1`: ECC BrainpoolP384r1 曲线密钥
- `EHSM_KEY_TYPE_ECC_BRAINPOOLP_512R1`: ECC BrainpoolP512r1 曲线密钥
- `EHSM_KEY_TYPE_ECC_SECP_192R1`: ECC secp192r1 曲线密钥 (NIST P-192)
- `EHSM_KEY_TYPE_ECC_SECP_224R1`: ECC secp224r1 曲线密钥 (NIST P-224)
- `EHSM_KEY_TYPE_ECC_SECP_256R1`: ECC secp256r1 曲线密钥 (NIST P-256)
- `EHSM_KEY_TYPE_ECC_SECP_384R1`: ECC secp384r1 曲线密钥 (NIST P-384)
- `EHSM_KEY_TYPE_ECC_SECP_521R1`: ECC secp521r1 曲线密钥 (NIST P-521)
- `EHSM_KEY_TYPE_ED25519`: Ed25519 签名密钥
- `EHSM_KEY_TYPE_X25519`: X25519 密钥交换密钥
- `EHSM_KEY_TYPE_SM4_XTS`: SM4 XTS 模式密钥
- `EHSM_KEY_TYPE_AES_128_XTS`: AES-128 XTS 模式密钥
- `EHSM_KEY_TYPE_AES_192_XTS`: AES-192 XTS 模式密钥
- `EHSM_KEY_TYPE_AES_256_XTS`: AES-256 XTS 模式密钥
- `EHSM_KEY_TYPE_CHACHA`: ChaCha20 流密码密钥
- `EHSM_KEY_TYPE_HMAC`: HMAC 消息认证码密钥
- `EHSM_KEY_TYPE_ECC_SECP_160K1`: ECC secp160k1 曲线密钥
- `EHSM_KEY_TYPE_ECC_SECP_192K1`: ECC secp192k1 曲线密钥
- `EHSM_KEY_TYPE_ECC_SECP_224K1`: ECC secp224k1 曲线密钥
- `EHSM_KEY_TYPE_ECC_SECP_256K1`: ECC secp256k1 曲线密钥
- `EHSM_KEY_TYPE_SM9_ENC_USERPRIV`: SM9 加密用户私钥
- `EHSM_KEY_TYPE_SM9_SIGN_USERPRIV`: SM9 签名用户私钥
- `EHSM_KEY_TYPE_SM9_EXCHG_USERPRIV`: SM9 密钥交换用户私钥
- `EHSM_KEY_TYPE_SM9_EXCHG_TEMP`: SM9 密钥交换临时密钥

8.5.11 ehsm_key_part_e

枚举值:

- `EHSM_KEY_PART_PUBLIC_KEY`: 密钥数据结构体中仅包含公钥
- `EHSM_KEY_PART_PRIVATE_KEY`: 密钥数据结构体中仅包含私钥（与 `EHSM_KEY_PART_SYMM_KEY` 共享同一值，这是有意设计）
- `EHSM_KEY_PART_SYMM_KEY`: 密钥数据结构体包含的是对称密钥（与 `EHSM_KEY_PART_PRIVATE_KEY` 共享同一值，这是有意设计）
- `EHSM_KEY_PART_KEY_PAIR`: 密钥数据结构体包含的是密钥对

8.5.12 ehsm_derive_algo_e

枚举值:

- `EHSM_DERIVE_ALGO_KDFX963`
- `EHSM_DERIVE_ALGO_PBKDF2`

8.5.13 ehsm_derive_type_e

枚举值:

- `EHSM_DERIVE_TYPE_PASSWORD`: 使用 password 派生
- `EHSM_DERIVE_TYPE_FROM_PARENT_KEY`: 从父密钥派生

8.5.14 ehsm_sm2_role_e

枚举值:

- `EHSM_SM2_ROLE_SPONSOR`
- `EHSM_SM2_ROLE_RESPONSOR`

8.5.15 ehsm_sm9_enc_type_e

枚举值:

- `EHSM_SM9_ENC_TYPE_STREAM`
- `EHSM_SM9_ENC_TYPE_BLOCK`

8.5.16 ehsm_sm9_padding_mode_e

枚举值:

- `EHSM_SM9_PADDING_NONE`
- `EHSM_SM9_PADDING_PKCS7`

8.5.17 ehsm_sm9_role_e

枚举值:

- `EHSM_SM9_ROLE_SPONSOR`
- `EHSM_SM9_ROLE_RESPONSOR`

8.5.18 ehsm_proc_mode_e

枚举值:

- `EHSM_PROC_INIT`: 三段式处理的开始阶段
- `EHSM_PROC_UPDATE`: 三段式处理的更新数据阶段
- `EHSM_PROC_FINISH`: 三段式处理的结束阶段
- `EHSM_PROC_ONEPASS`

8.5.19 ehsm_drv_mode_e

枚举值:

- `EHSM_DRV_MODE_INTERRUPT`: 中断模式。

会初始化 Mailbox 寄存器, 底层 Mailbox 使用中断收取数据, 根据 ctx 中的 async 配置 API 决定会立即返回 `EHSM_ERR_NEED_POLL` 还是等待操作完成。

- `EHSM_DRV_MODE_WAIT_AND_POLL`: 同步等待模式。

会初始化 Mailbox 寄存器, 底层 Mailbox 使用轮询收取数据, API 总是等待响应并返回结果, 忽略 ctx 中的 async 配置。

- `EHSM_DRV_MODE_SEND_AND_PEEK`: 发送并由用户 POLL。

不清 Mailbox 寄存器, 不配置中断, 也不轮询接收, API 总是立刻返回, 需要反复调用 `ehsm_ctx_poll` 判断是否完成, 忽略 ctx 中的 async 配置。

8.5.20 ehsm_key_level_e

枚举值:

- `EHSM_KEY_LEVEL_1`: 此 OTP 密钥由 CHIP ROOT KEY 加密, 仅支持 `TEST_MODE` 和 `DEVELOP_MODE`
- `EHSM_KEY_LEVEL_2`: 此 OTP 密钥由 DEVICE ROOT KEY 加密, 仅支持 `TEST_MODE`、`DEVELOP_MODE` 和 `MANUFACTURE_MODE`

8.5.21 ehsm_install_key_type_e

枚举值:

- `EHSM_INSTALL_KEY_TYPE_SYMM`
- `EHSM_INSTALL_KEY_TYPE_ASYM_PRIV_KEY`

- `EHSM_INSTALL_KEY_TYPE_ASYM_PUB_KEY_HASH`

8.5.22 ehsm_challenge_type_e

挑战鉴权类型。

枚举值:

- `EHSM_CHALLENGE_TYPE_EHSM_DEBUG`: EHSM 调试鉴权挑战, 挑战值的长度为 32+16 字节
- `EHSM_CHALLENGE_TYPE_SHE_DEBUG`: SHE 管理调试鉴权挑战, BootLoader 中不支持
- `EHSM_CHALLENGE_TYPE_SOC_DEBUG`: SOC 调试鉴权挑战, 挑战值的长度为 32+16 字节
- `EHSM_CHALLENGE_TYPE_USER_AUTH`: FW 用户行为鉴权, BootLoader 中不支持, 挑战值长度为 32+16 字节
- `EHSM_CHALLENGE_TYPE_FW_AUTH`: 国密固件认证鉴权挑战, 仅 BootLoader 中支持, 挑战值长度为 16 字节

8.5.23 ehsm_auth_algo_e

权限算法类型。

枚举值:

- `EHSM_AUTH_ALGO_SM3_SM2`
- `EHSM_AUTH_ALGO_SHA256_ECDSA_P256R1`
- `EHSM_AUTH_ALGO_SM4_CMAC`
- `EHSM_AUTH_ALGO_AES128_CMAC`
- `EHSM_AUTH_ALGO_SHA256_RSA`

8.5.24 ehsm_lifecycle_e

eHSM 生命周期枚举值。

枚举值:

- `EHSM_LC_TEST`
- `EHSM_LC_DEVELOP`
- `EHSM_LC_MANUFACTURE`
- `EHSM_LC_USER`
- `EHSM_LC_DEBUG`
- `EHSM_LC_DESTROY`

8.5.25 ehsm_ctrl_field_e

eHSM OTP 中控制字段类型。

枚举值:

- `EHSM_CTRL_FIELD_HW`
- `EHSM_CTRL_FIELD_EHSM`
- `EHSM_CTRL_FIELD_SOC`

8.5.26 ehsm_bl_gen_key_type_e

枚举值:

- `EHSM_BL_GEN_KEY_TYPE_SYMM`
- `EHSM_BL_GEN_KEY_TYPE_SM2`
- `EHSM_BL_GEN_KEY_TYPE_ECC_P256R1`

8.6 Defines

8.6.1 ehsm_port_printf

定义: `ehsm_port_printf`

提供一个 printf 相同的 ehsm_port_printf 调用宏, 用来打印数据。

8.6.2 NULL

定义: `NULL ((void *)0)`

8.6.3 EHSM_OK

定义: `EHSM_OK (0U)`

8.6.4 EHSM_ERR_PARAM_ERROR

定义: `EHSM_ERR_PARAM_ERROR (1U)`

8.6.5 EHSM_ERR_BUSY

定义: `EHSM_ERR_BUSY (2U)`

8.6.6 EHSM_ERR_CTX_NOT_INIT

定义: `EHSM_ERR_CTX_NOT_INIT (3U)`

8.6.7 EHSM_ERR_CTX_STATUS

定义: `EHSM_ERR_CTX_STATUS (4U)`

8.6.8 EHSM_ERR_TIMEOUT

定义: `EHSM_ERR_TIMEOUT (5U)`

8.6.9 EHSM_ERR_CTX_INVALID

定义: `EHSM_ERR_CTX_INVALID (6U)`

8.6.10 EHSM_ERR_ILLEGAL_USE

定义: `EHSM_ERR_ILLEGAL_USE (7U)`

8.6.11 EHSM_ERR_DEV_ERROR

定义: `EHSM_ERR_DEV_ERROR (8U)`

8.6.12 EHSM_ERR_FW_BASE

定义: `EHSM_ERR_FW_BASE (0x010000U)`

固件返回的错误码会加上此偏移值

8.6.13 EHSM_ERR_NEED_POLL

定义: `EHSM_ERR_NEED_POLL (0x55AABBCCU)`

异步标志

8.6.14 EHSM_SHM

定义: `EHSM_SHM`

仅用于提示对应的指针参数需指向 SOC 与 eHSM 共享的内存区域, 以便 eHSM 读写数据, 未指定此标志的参数则无需指向共享内存。参考[共享内存机制](#)

8.6.15 EHSM_DRV_VERSION_MAJOR

定义: `EHSM_DRV_VERSION_MAJOR 2`

8.6.16 EHSM_DRV_VERSION_MINOR

定义: `EHSM_DRV_VERSION_MINOR 3`

8.6.17 EHSM_DRV_VERSION_PATCH

定义: `EHSM_DRV_VERSION_PATCH 1`

8.6.18 EHSM_DRV_VERSION_PRE_RELEASE

定义: `EHSM_DRV_VERSION_PRE_RELEASE "\\\""`

8.6.19 EHSM_KEY_HANDLE_INVALID

定义: `EHSM_KEY_HANDLE_INVALID (0xFFFFFFFFFU)`

无效的密钥句柄, 如果希望由 eHSM 自动分配生成/导入/派生/协商的密钥句柄时, 可使用此值

8.6.20 EHSM_KEY_PRIV_SIGN

定义: `EHSM_KEY_PRIV_SIGN (0x0001U)`

密钥可用于签名或生成 MAC/HMAC

8.6.21 EHSM_KEY_PRIV_VERIFY

定义: `EHSM_KEY_PRIV_VERIFY (0x0002U)`

密钥可用于验签或验证 MAC/HMAC

8.6.22 EHSM_KEY_PRIV_ENCRYPT

定义: `EHSM_KEY_PRIV_ENCRYPT (0x0004U)`

密钥可用于加密

8.6.23 EHSM_KEY_PRIV_DECRYPT

定义: `EHSM_KEY_PRIV_DECRYPT (0x0008U)`

密钥可用于解密

8.6.24 EHSM_KEY_PRIV_TIMESTAMP

定义: `EHSM_KEY_PRIV_TIMESTAMP (0x0010U)`

保留

8.6.25 EHSM_KEY_PRIV_SECURE_BOOT

定义: `EHSM_KEY_PRIV_SECURE_BOOT (0x0020U)`

保留

8.6.26 EHSM_KEY_PRIV_SECURE_STORAGE

定义: `EHSM_KEY_PRIV_SECURE_STORAGE (0x0040U)`

保留

8.6.27 EHSM_KEY_PRIV_KEY_CREATION

定义: `EHSM_KEY_PRIV_KEY_CREATION (0x0080U)`

密钥可用于派生或协商新密钥

8.6.28 EHSM_KEY_PRIV_TRANSPORT_KEY_CREATION

定义: `EHSM_KEY_PRIV_TRANSPORT_KEY_CREATION (0x0100U)`

密钥可用于派生传输密钥

8.6.29 EHSM_KEY_PRIV_UTCSYNC

定义: `EHSM_KEY_PRIV_UTCSYNC (0x0200U)`

保留

8.6.30 EHSM_KEY_PRIV_TRANSPORT

定义: `EHSM_KEY_PRIV_TRANSPORT (0x0400U)`

密钥可作为传输密钥使用

8.6.31 EHSM_KEY_PRIV_REMOVE

定义: `EHSM_KEY_PRIV_REMOVE (0x0800U)`

密钥可以被移除

8.6.32 EHSM_KEY_PRIV_IMPORT_PLAIN

定义: `EHSM_KEY_PRIV_IMPORT_PLAIN (0x1000U)`

密钥可以被明文导入

8.6.33 EHSM_KEY_PRIV_IMPORT_CIPHER

定义: `EHSM_KEY_PRIV_IMPORT_CIPHER (0x2000U)`

密钥可以被密文导入

8.6.34 EHSM_KEY_PRIV_EXPORT_PLAIN

定义: `EHSM_KEY_PRIV_EXPORT_PLAIN (0x4000U)`

密钥可以被明文导出

8.6.35 EHSM_KEY_PRIV_EXPORT_CIPHER

定义: `EHSM_KEY_PRIV_EXPORT_CIPHER (0x8000U)`

密钥可以被密文导出

8.6.36 EHSM_KEY_PRIV_BOOT_FAIL_USAGE

定义: `EHSM_KEY_PRIV_BOOT_FAIL_USAGE (0x10000U)`

保留

8.6.37 EHSM_KEY_PRIV_DEBUG_USAGE

定义: `EHSM_KEY_PRIV_DEBUG_USAGE (0x20000U)`

保留

8.6.38 EHSM_KEY_PRIV_WILDCARD_PROTECT

定义: `EHSM_KEY_PRIV_WILDCARD_PROTECT (0x40000U)`

保留

8.6.39 EHSM_KEY_PRIV_WRITE_PROTECT

定义: `EHSM_KEY_PRIV_WRITE_PROTECT (0x80000U)`

密钥写保护（不能覆盖）

8.6.40 ehsm_bl_get_version

定义: `ehsm_bl_get_version`

BL 下获取版本号，见 [ehsm_get_version](#)。

8.6.41 ehsm_bl_verify_image

定义: `ehsm_bl_verify_image`

BL 下的校验安全启动镜像，见 [ehsm_verify_image](#)。

8.6.42 ehsm_bl_upgrade_fw_image

定义: `ehsm_bl_upgrade_fw_image`

BL 下校验升级固件镜像，见 [ehsm_upgrade_fw_image](#)。

8.6.43 ehsm_bl_upgrade_fw_image_init

定义: `ehsm_bl_upgrade_fw_image_init`

BL 下校验升级固件镜像初始化，见 [ehsm_upgrade_fw_image_init](#)。

8.6.44 ehsm_bl_upgrade_fw_image_update

定义: `ehsm_bl_upgrade_fw_image_update`

BL 下校验升级固件镜像更新数据，见 [ehsm_upgrade_fw_image_update](#)。

8.6.45 ehsm_bl_upgrade_fw_image_finish

定义: `ehsm_bl_upgrade_fw_image_finish`

BL 下校验升级固件镜像完成，见 [ehsm_upgrade_fw_image_finish](#)。

8.6.46 ehsm_bl_set_uart_baudrate

定义: `ehsm_bl_set_uart_baudrate`

BL 下设置串口波特率，见 [ehsm_set_uart_baudrate](#)。

8.6.47 ehsm_bl_get_challenge

定义: `ehsm_bl_get_challenge`

BL 下获取鉴权挑战值，见 [ehsm_get_challenge](#)。

8.6.48 ehsm_bl_debug_auth

定义: `ehsm_bl_debug_auth`

BL 下调试鉴权, 见 [ehsm_debug_auth](#)。

8.6.49 ehsm_bl_close_debug

定义: `ehsm_bl_close_debug`

BL 下关闭调试功能, 见 [ehsm_close_debug](#)。

8.6.50 ehsm_bl_read_reg

定义: `ehsm_bl_read_reg`

BL 下读取 CFG 寄存器值, 见 [ehsm_read_reg](#)。

8.6.51 ehsm_bl_write_reg

定义: `ehsm_bl_write_reg`

BL 下写入 CFG 寄存器值, 见 [ehsm_write_reg](#)。

8.6.52 ehsm_bl_read_otp

定义: `ehsm_bl_read_otp`

BL 下读取 OTP 值, 见 [ehsm_read_otp](#)。

8.6.53 ehsm_bl_write_otp

定义: `ehsm_bl_write_otp`

BL 下写入 OTP 值, 见 [ehsm_write_otp](#)。

8.6.54 EHSM_SELF_TEST_SM4_ECB

定义: `EHSM_SELF_TEST_SM4_ECB 0x01`

8.6.55 EHSM_SELF_TEST_SM4_CBC

定义: `EHSM_SELF_TEST_SM4_CBC 0x02`

8.6.56 EHSM_SELF_TEST_SM4_CFB

定义: `EHSM_SELF_TEST_SM4_CFB 0x04`

8.6.57 EHSM_SELF_TEST_SM4_OFB

定义: `EHSM_SELF_TEST_SM4_OFB 0x08`

8.6.58 EHSM_SELF_TEST_SM4_CTR

定义: `EHSM_SELF_TEST_SM4_CTR 0x10`

8.6.59 EHSM_SELF_TEST_SM2_ENC

定义: `EHSM_SELF_TEST_SM2_ENC 0x20`

8.6.60 EHSM_SELF_TEST_SM2_VERIFY

定义: `EHSM_SELF_TEST_SM2_VERIFY 0x40`

8.6.61 EHSM_SELF_TEST_SM2_EXCH

定义: `EHSM_SELF_TEST_SM2_EXCH 0x80`

8.6.62 EHSM_SELF_TEST_SM3

定义: `EHSM_SELF_TEST_SM3 0x100`

8.6.63 EHSM_SELF_TEST_DES

定义: `EHSM_SELF_TEST_DES 0x200`

8.6.64 EHSM_SELF_TEST_TDES

定义: `EHSM_SELF_TEST_TDES 0x400`

8.6.65 EHSM_SELF_TEST_AES

定义: `EHSM_SELF_TEST_AES 0x800`

8.6.66 EHSM_SELF_TEST_RSA

定义: `EHSM_SELF_TEST_RSA 0x1000`

8.6.67 EHSM_SELF_TEST_ECC

定义: `EHSM_SELF_TEST_ECC 0x2000`

8.6.68 EHSM_SELF_TEST_MD5

定义: `EHSM_SELF_TEST_MD5 0x4000`

8.6.69 EHSM_SELF_TEST_SHA1

定义: `EHSM_SELF_TEST_SHA1 0x8000`

8.6.70 EHSM_SELF_TEST_SHA2

定义: `EHSM_SELF_TEST_SHA2 0x10000`

8.6.71 EHSM_SELF_TEST_SHA3

定义: `EHSM_SELF_TEST_SHA3 0x20000`

8.6.72 EHSM_SELF_TEST_SHA256

定义: `EHSM_SELF_TEST_SHA256 0x40000`

8.6.73 EHSM_SELF_TEST_TRNG

定义: `EHSM_SELF_TEST_TRNG 0x80000`

8.7 Typedefs

8.7.1 ehsm_mb_interrupt_func_t

定义: `typedef void(* ehsm_mb_interrupt_func_t) (uint32_t channel, raddr_t packet_addr)`

8.7.2 ehsm_mailbox_isr_f

定义: `typedef void(* ehsm_mailbox_isr_f) (uint32_t channel)`

Mailbox 中断回调函数原型。channel 触发中断的 mailbox channel ID

8.7.3 ehsm_port_timer_t

定义: `typedef uint64_t ehsm_port_timer_t`

计时器的ID。对于简单的实现,可以直接使用当前的系统时钟值,因此这里定义为64位无符号整数。

8.7.4 bool_t

定义: `typedef bool bool_t`

8.7.5 raddr_t

定义: `typedef uint64_t raddr_t`

用于指示 Host 端的地址, eHSM 可对此地址值进行访问

8.7.6 ehsm_rsp_cb_func_t

定义: `typedef void(* ehsm_rsp_cb_func_t) (ehsm_ctx_st *ctx)`

命令处理完成的回调函数原型。

8.7.7 ehsm_key_format_des_plain_st

定义: `typedef ehsm_key_format_symm_plain_64_st ehsm_key_format_des_plain_st`

用于辅助 DES 密钥明文导入/导出的结构体

8.7.8 ehsm_key_format_des_cipher_st

定义: `typedef ehsm_key_format_symm_cipher_64_st ehsm_key_format_des_cipher_st`

用于辅助 DES 密钥密文导入/导出的结构体

8.7.9 ehsm_key_format_aes128_plain_st

定义: `typedef ehsm_key_format_symm_plain_128_st ehsm_key_format_aes128_plain_st`

用于辅助 AES128 密钥明文导入/导出的结构体

8.7.10 ehsm_key_format_aes128_cipher_st

定义: `typedef ehsm_key_format_symm_cipher_128_st ehsm_key_format_aes128_cipher_st`

用于辅助 AES128 密钥密文导入/导出的结构体

8.7.11 ehsm_key_format_tdes128_plain_st

定义: `typedef ehsm_key_format_symm_plain_128_st ehsm_key_format_tdes128_plain_st`

用于辅助 TDES128 密钥明文导入/导出的结构体

8.7.12 ehsm_key_format_tdes128_cipher_st

定义: `typedef ehsm_key_format_symm_cipher_128_st ehsm_key_format_tdes128_cipher_st`

用于辅助 TDES128 密钥密文导入/导出的结构体

8.7.13 ehsm_key_format_sm4_plain_st

定义: `typedef ehsm_key_format_symm_plain_128_st ehsm_key_format_sm4_plain_st`

用于辅助 SM4 密钥明文导入/导出的结构体

8.7.14 ehsm_key_format_sm4_cipher_st

定义: `typedef ehsm_key_format_symm_cipher_128_st ehsm_key_format_sm4_cipher_st`

用于辅助 SM4 密钥密文导入/导出的结构体

8.7.15 ehsm_key_format_tdes192_plain_st

定义: `typedef ehsm_key_format_symm_plain_192_st ehsm_key_format_tdes192_plain_st`

用于辅助 TDES192 密钥明文导入/导出的结构体

8.7.16 ehsm_key_format_tdes192_cipher_st

定义: `typedef ehsm_key_format_symm_cipher_192_st ehsm_key_format_tdes192_cipher_st`

用于辅助 TDES192 密钥密文导入/导出的结构体

8.7.17 ehsm_key_format_aes192_plain_st

定义: `typedef ehsm_key_format_symm_plain_192_st ehsm_key_format_aes192_plain_st`

用于辅助 AES192 密钥明文导入/导出的结构体

8.7.18 ehsm_key_format_aes192_cipher_st

定义: `typedef ehsm_key_format_symm_cipher_192_st ehsm_key_format_aes192_cipher_st`

用于辅助 AES192 密钥密文导入/导出的结构体

8.7.19 ehsm_key_format_aes256_plain_st

定义: `typedef ehsm_key_format_symm_plain_256_st ehsm_key_format_aes256_plain_st`

用于辅助 AES256 密钥明文导入/导出的结构体

8.7.20 ehsm_key_format_aes256_cipher_st

定义: `typedef ehsm_key_format_symm_cipher_256_st ehsm_key_format_aes256_cipher_st`

用于辅助 AES256 密钥密文导入/导出的结构体

8.7.21 ehsm_key_format_aes128_xts_plain_st

定义: `typedef ehsm_key_format_symm_plain_256_st ehsm_key_format_aes128_xts_plain_st`

用于辅助 AES128_XTS 密钥明文导入/导出的结构体

8.7.22 ehsm_key_format_aes128_xts_cipher_st

定义: `typedef ehsm_key_format_symm_cipher_256_st ehsm_key_format_aes128_xts_cipher_st`

用于辅助 AES128_XTS 密钥密文导入/导出的结构体

8.7.23 ehsm_key_format_sm4_xts_plain_st

定义: `typedef ehsm_key_format_symm_plain_256_st ehsm_key_format_sm4_xts_plain_st`

用于辅助 SM4_XTS 密钥明文导入/导出的结构体

8.7.24 ehsm_key_format_sm4_xts_cipher_st

定义: `typedef ehsm_key_format_symm_cipher_256_st ehsm_key_format_sm4_xts_cipher_st`

用于辅助 SM4_XTS 密钥密文导入/导出的结构体

8.7.25 ehsm_key_format_aes192_xts_plain_st

定义: `typedef ehsm_key_format_symm_plain_384_st ehsm_key_format_aes192_xts_plain_st`

用于辅助 AES192_XTS 密钥明文导入/导出的结构体

8.7.26 ehsm_key_format_aes192_xts_cipher_st

定义: `typedef ehsm_key_format_symm_cipher_384_st ehsm_key_format_aes192_xts_cipher_st`

用于辅助 AES192_XTS 密钥密文导入/导出的结构体

8.7.27 ehsm_key_format_aes256_xts_plain_st

定义: `typedef ehsm_key_format_symm_plain_512_st ehsm_key_format_aes256_xts_plain_st`

用于辅助 AES256_XTS 密钥明文导入/导出的结构体

8.7.28 ehsm_key_format_aes256_xts_cipher_st

定义: `typedef ehsm_key_format_symm_cipher_512_st ehsm_key_format_aes256_xts_cipher_st`

用于辅助 AES256_XTS 密钥密文导入/导出的结构体

8.7.29 ehsm_key_format_hmac64_plain_st

定义: `typedef ehsm_key_format_symm_plain_64_st ehsm_key_format_hmac64_plain_st`

用于辅助 HMAC64 密钥明文导入/导出的结构体

8.7.30 ehsm_key_format_hmac64_cipher_st

定义: `typedef ehsm_key_format_symm_cipher_64_st ehsm_key_format_hmac64_cipher_st`

用于辅助 HMAC64 密钥密文导入/导出的结构体

8.7.31 ehsm_key_format_hmac128_plain_st

定义: `typedef ehsm_key_format_symm_plain_128_st ehsm_key_format_hmac128_plain_st`

用于辅助 HMAC128 密钥明文导入/导出的结构体

8.7.32 ehsm_key_format_hmac128_cipher_st

定义: `typedef ehsm_key_format_symm_cipher_128_st ehsm_key_format_hmac128_cipher_st`

用于辅助 HMAC128 密钥密文导入/导出的结构体

8.7.33 ehsm_key_format_hmac192_plain_st

定义: `typedef ehsm_key_format_symm_plain_192_st ehsm_key_format_hmac192_plain_st`

用于辅助 HMAC192 密钥明文导入/导出的结构体

8.7.34 ehsm_key_format_hmac192_cipher_st

定义: `typedef ehsm_key_format_symm_cipher_192_st ehsm_key_format_hmac192_cipher_st`

用于辅助 HMAC192 密钥密文导入/导出的结构体

8.7.35 ehsm_key_format_hmac256_plain_st

定义: `typedef ehsm_key_format_symm_plain_256_st ehsm_key_format_hmac256_plain_st`

用于辅助 HMAC256 密钥明文导入/导出的结构体

8.7.36 ehsm_key_format_hmac256_cipher_st

定义: `typedef ehsm_key_format_symm_cipher_256_st ehsm_key_format_hmac256_cipher_st`

用于辅助 HMAC256 密钥密文导入/导出的结构体

8.7.37 ehsm_key_format_hmac384_plain_st

定义: `typedef ehsm_key_format_symm_plain_384_st ehsm_key_format_hmac384_plain_st`

用于辅助 HMAC384 密钥明文导入/导出的结构体

8.7.38 ehsm_key_format_hmac384_cipher_st

定义: `typedef ehsm_key_format_symm_cipher_384_st ehsm_key_format_hmac384_cipher_st`

用于辅助 HMAC384 密钥密文导入/导出的结构体

8.7.39 ehsm_key_format_hmac512_plain_st

定义: `typedef ehsm_key_format_symm_plain_512_st ehsm_key_format_hmac512_plain_st`

用于辅助 HMAC512 密钥明文导入/导出的结构体

8.7.40 ehsm_key_format_hmac512_cipher_st

定义: `typedef ehsm_key_format_symm_cipher_512_st ehsm_key_format_hmac512_cipher_st`

用于辅助 HMAC512 密钥密文导入/导出的结构体

8.7.41 ehsm_key_format_ecc_bp_p160r1_plain_pubkey_st

定义: `typedef ehsm_key_format_ecc_160_plain_pubkey_st ehsm_key_format_ecc_bp_p160r1_plain_pubkey_st`

8.7.42 ehsm_key_format_ecc_bp_p160r1_plain_privkey_st

定义: `typedef ehsm_key_format_ecc_160_plain_privkey_st ehsm_key_format_ecc_bp_p160r1_plain_privkey_st`

8.7.43 ehsm_key_format_ecc_bp_p160r1_cipher_privkey_st

定义: `typedef ehsm_key_format_ecc_160_cipher_privkey_st ehsm_key_format_ecc_bp_p160r1_cipher_privkey_st`

8.7.44 ehsm_key_format_ecc_bp_p160r1_plain_keypair_st

定义: `typedef ehsm_key_format_ecc_160_plain_keypair_st ehsm_key_format_ecc_bp_p160r1_plain_keypair_st`

8.7.45 ehsm_key_format_ecc_bp_p160r1_cipher_keypair_st

定义: `typedef ehsm_key_format_ecc_160_cipher_keypair_st ehsm_key_format_ecc_bp_p160r1_cipher_keypair_st`

8.7.46 ehsm_key_format_ecc_bp_p192r1_plain_pubkey_st

定义: `typedef ehsm_key_format_ecc_192_plain_pubkey_st ehsm_key_format_ecc_bp_p192r1_plain_pubkey_st`

8.7.47 ehsm_key_format_ecc_bp_p192r1_plain_privkey_st

定义: `typedef ehsm_key_format_ecc_192_plain_privkey_st ehsm_key_format_ecc_bp_p192r1_plain_privkey_st`

8.7.48 ehsm_key_format_ecc_bp_p192r1_cipher_privkey_st

定义: `typedef ehsm_key_format_ecc_192_cipher_privkey_st ehsm_key_format_ecc_bp_p192r1_cipher_privkey_st`

8.7.49 ehsm_key_format_ecc_bp_p192r1_plain_keypair_st

定义: `typedef ehsm_key_format_ecc_192_plain_keypair_st ehsm_key_format_ecc_bp_p192r1_plain_keypair_st`

8.7.50 ehsm_key_format_ecc_bp_p192r1_cipher_keypair_st

typedef
定义: ehsm_key_format_ecc_192_cipher_keypair_st ehsm_key_format_ecc_bp_p192r1_cipher_keypair_st

8.7.51 ehsm_key_format_ecc_bp_p224r1_plain_pubkey_st

定义 :
typedef ehsm_key_format_ecc_224_plain_pubkey_st ehsm_key_format_ecc_bp_p224r1_plain_pubkey_st

8.7.52 ehsm_key_format_ecc_bp_p224r1_plain_privkey_st

定义 :
typedef ehsm_key_format_ecc_224_plain_privkey_st ehsm_key_format_ecc_bp_p224r1_plain_privkey_st

8.7.53 ehsm_key_format_ecc_bp_p224r1_cipher_privkey_st

typedef
定义: ehsm_key_format_ecc_224_cipher_privkey_st ehsm_key_format_ecc_bp_p224r1_cipher_privkey_st

8.7.54 ehsm_key_format_ecc_bp_p224r1_plain_keypair_st

定义 :
typedef ehsm_key_format_ecc_224_plain_keypair_st ehsm_key_format_ecc_bp_p224r1_plain_keypair_st

8.7.55 ehsm_key_format_ecc_bp_p224r1_cipher_keypair_st

typedef
定义: ehsm_key_format_ecc_224_cipher_keypair_st ehsm_key_format_ecc_bp_p224r1_cipher_keypair_st

8.7.56 ehsm_key_format_ecc_bp_p256r1_plain_pubkey_st

定义 :
typedef ehsm_key_format_ecc_256_plain_pubkey_st ehsm_key_format_ecc_bp_p256r1_plain_pubkey_st

8.7.57 ehsm_key_format_ecc_bp_p256r1_plain_privkey_st

定义 :
typedef ehsm_key_format_ecc_256_plain_privkey_st ehsm_key_format_ecc_bp_p256r1_plain_privkey_st

8.7.58 ehsm_key_format_ecc_bp_p256r1_cipher_privkey_st

typedef
定义: ehsm_key_format_ecc_256_cipher_privkey_st ehsm_key_format_ecc_bp_p256r1_cipher_privkey_st

8.7.59 ehsm_key_format_ecc_bp_p256r1_plain_keypair_st

定义 :
typedef ehsm_key_format_ecc_256_plain_keypair_st ehsm_key_format_ecc_bp_p256r1_plain_keypair_st

8.7.60 ehsm_key_format_ecc_bp_p256r1_cipher_keypair_st

typedef
定义: ehsm_key_format_ecc_256_cipher_keypair_st ehsm_key_format_ecc_bp_p256r1_cipher_keypair_st

8.7.61 ehsm_key_format_ecc_bp_p320r1_plain_pubkey_st

定义 义 :

```
typedef ehsm_key_format_ecc_320_plain_pubkey_st ehsm_key_format_ecc_bp_p320r1_plain_pubkey_st
```

8.7.62 ehsm_key_format_ecc_bp_p320r1_plain_privkey_st

定义 义 :

```
typedef ehsm_key_format_ecc_320_plain_privkey_st ehsm_key_format_ecc_bp_p320r1_plain_privkey_st
```

8.7.63 ehsm_key_format_ecc_bp_p320r1_cipher_privkey_st

定义:

```
typedef  
ehsm_key_format_ecc_320_cipher_privkey_st ehsm_key_format_ecc_bp_p320r1_cipher_privkey_st
```

8.7.64 ehsm_key_format_ecc_bp_p320r1_plain_keypair_st

定义 义 :

```
typedef ehsm_key_format_ecc_320_plain_keypair_st ehsm_key_format_ecc_bp_p320r1_plain_keypair_st
```

8.7.65 ehsm_key_format_ecc_bp_p320r1_cipher_keypair_st

定义:

```
typedef  
ehsm_key_format_ecc_320_cipher_keypair_st ehsm_key_format_ecc_bp_p320r1_cipher_keypair_st
```

8.7.66 ehsm_key_format_ecc_bp_p384r1_plain_pubkey_st

定义 义 :

```
typedef ehsm_key_format_ecc_384_plain_pubkey_st ehsm_key_format_ecc_bp_p384r1_plain_pubkey_st
```

8.7.67 ehsm_key_format_ecc_bp_p384r1_plain_privkey_st

定义 义 :

```
typedef ehsm_key_format_ecc_384_plain_privkey_st ehsm_key_format_ecc_bp_p384r1_plain_privkey_st
```

8.7.68 ehsm_key_format_ecc_bp_p384r1_cipher_privkey_st

定义:

```
typedef  
ehsm_key_format_ecc_384_cipher_privkey_st ehsm_key_format_ecc_bp_p384r1_cipher_privkey_st
```

8.7.69 ehsm_key_format_ecc_bp_p384r1_plain_keypair_st

定义 义 :

```
typedef ehsm_key_format_ecc_384_plain_keypair_st ehsm_key_format_ecc_bp_p384r1_plain_keypair_st
```

8.7.70 ehsm_key_format_ecc_bp_p384r1_cipher_keypair_st

定义:

```
typedef  
ehsm_key_format_ecc_384_cipher_keypair_st ehsm_key_format_ecc_bp_p384r1_cipher_keypair_st
```

8.7.71 ehsm_key_format_ecc_bp_p512r1_plain_pubkey_st

定义 义 :

```
typedef ehsm_key_format_ecc_512_plain_pubkey_st ehsm_key_format_ecc_bp_p512r1_plain_pubkey_st
```

8.7.72 ehsm_key_format_ecc_bp_p512r1_plain_privkey_st

定义 义 :

```
typedef ehsm_key_format_ecc_512_plain_privkey_st ehsm_key_format_ecc_bp_p512r1_plain_privkey_st
```

8.7.73 ehsm_key_format_ecc_bp_p512r1_cipher_privkey_st

```
typedef
```

定义: ehsm_key_format_ecc_512_cipher_privkey_st ehsm_key_format_ecc_bp_p512r1_cipher_privkey_st

8.7.74 ehsm_key_format_ecc_bp_p512r1_plain_keypair_st

定义 义 :

```
typedef ehsm_key_format_ecc_512_plain_keypair_st ehsm_key_format_ecc_bp_p512r1_plain_keypair_st
```

8.7.75 ehsm_key_format_ecc_bp_p512r1_cipher_keypair_st

```
typedef
```

定义: ehsm_key_format_ecc_512_cipher_keypair_st ehsm_key_format_ecc_bp_p512r1_cipher_keypair_st

8.7.76 ehsm_key_format_ecc_sec_p192r1_plain_pubkey_st

定义 义 :

```
typedef ehsm_key_format_ecc_192_plain_pubkey_st ehsm_key_format_ecc_sec_p192r1_plain_pubkey_st
```

8.7.77 ehsm_key_format_ecc_sec_p192r1_plain_privkey_st

定义 义 :

```
typedef ehsm_key_format_ecc_192_plain_privkey_st ehsm_key_format_ecc_sec_p192r1_plain_privkey_st
```

8.7.78 ehsm_key_format_ecc_sec_p192r1_cipher_privkey_st

```
typedef
```

定义: ehsm_key_format_ecc_192_cipher_privkey_st ehsm_key_format_ecc_sec_p192r1_cipher_privkey_st

8.7.79 ehsm_key_format_ecc_sec_p192r1_plain_keypair_st

定义 义 :

```
typedef ehsm_key_format_ecc_192_plain_keypair_st ehsm_key_format_ecc_sec_p192r1_plain_keypair_st
```

8.7.80 ehsm_key_format_ecc_sec_p192r1_cipher_keypair_st

```
typedef
```

定义: ehsm_key_format_ecc_192_cipher_keypair_st ehsm_key_format_ecc_sec_p192r1_cipher_keypair_st

8.7.81 ehsm_key_format_ecc_sec_p224r1_plain_pubkey_st

定义 义 :

```
typedef ehsm_key_format_ecc_224_plain_pubkey_st ehsm_key_format_ecc_sec_p224r1_plain_pubkey_st
```

8.7.82 ehsm_key_format_ecc_sec_p224r1_plain_privkey_st

定义 义 :

```
typedef ehsm_key_format_ecc_224_plain_privkey_st ehsm_key_format_ecc_sec_p224r1_plain_privkey_st
```

8.7.83 ehsm_key_format_ecc_sec_p224r1_cipher_privkey_st

typedef
定义: ehsm_key_format_ecc_224_cipher_privkey_st ehsm_key_format_ecc_sec_p224r1_cipher_privkey_st

8.7.84 ehsm_key_format_ecc_sec_p224r1_plain_keypair_st

定义 :
typedef ehsm_key_format_ecc_224_plain_keypair_st ehsm_key_format_ecc_sec_p224r1_plain_keypair_st

8.7.85 ehsm_key_format_ecc_sec_p224r1_cipher_keypair_st

typedef
定义: ehsm_key_format_ecc_224_cipher_keypair_st ehsm_key_format_ecc_sec_p224r1_cipher_keypair_st

8.7.86 ehsm_key_format_ecc_sec_p256r1_plain_pubkey_st

定义 :
typedef ehsm_key_format_ecc_256_plain_pubkey_st ehsm_key_format_ecc_sec_p256r1_plain_pubkey_st

8.7.87 ehsm_key_format_ecc_sec_p256r1_plain_privkey_st

定义 :
typedef ehsm_key_format_ecc_256_plain_privkey_st ehsm_key_format_ecc_sec_p256r1_plain_privkey_st

8.7.88 ehsm_key_format_ecc_sec_p256r1_cipher_privkey_st

typedef
定义: ehsm_key_format_ecc_256_cipher_privkey_st ehsm_key_format_ecc_sec_p256r1_cipher_privkey_st

8.7.89 ehsm_key_format_ecc_sec_p256r1_plain_keypair_st

定义 :
typedef ehsm_key_format_ecc_256_plain_keypair_st ehsm_key_format_ecc_sec_p256r1_plain_keypair_st

8.7.90 ehsm_key_format_ecc_sec_p256r1_cipher_keypair_st

typedef
定义: ehsm_key_format_ecc_256_cipher_keypair_st ehsm_key_format_ecc_sec_p256r1_cipher_keypair_st

8.7.91 ehsm_key_format_ecc_sec_p384r1_plain_pubkey_st

定义 :
typedef ehsm_key_format_ecc_384_plain_pubkey_st ehsm_key_format_ecc_sec_p384r1_plain_pubkey_st

8.7.92 ehsm_key_format_ecc_sec_p384r1_plain_privkey_st

定义 :
typedef ehsm_key_format_ecc_384_plain_privkey_st ehsm_key_format_ecc_sec_p384r1_plain_privkey_st

8.7.93 ehsm_key_format_ecc_sec_p384r1_cipher_privkey_st

typedef
定义: ehsm_key_format_ecc_384_cipher_privkey_st ehsm_key_format_ecc_sec_p384r1_cipher_privkey_st

8.7.94 ehsm_key_format_ecc_sec_p384r1_plain_keypair_st

定义 义 :

```
typedef ehsm_key_format_ecc_384_plain_keypair_st ehsm_key_format_ecc_sec_p384r1_plain_keypair_st
```

8.7.95 ehsm_key_format_ecc_sec_p384r1_cipher_keypair_st

```
typedef
```

定义: `ehsm_key_format_ecc_384_cipher_keypair_st ehsm_key_format_ecc_sec_p384r1_cipher_keypair_st`

8.7.96 ehsm_key_format_ecc_sec_p521r1_plain_pubkey_st

定义 义 :

```
typedef ehsm_key_format_ecc_521_plain_pubkey_st ehsm_key_format_ecc_sec_p521r1_plain_pubkey_st
```

8.7.97 ehsm_key_format_ecc_sec_p521r1_plain_privkey_st

定义 义 :

```
typedef ehsm_key_format_ecc_521_plain_privkey_st ehsm_key_format_ecc_sec_p521r1_plain_privkey_st
```

8.7.98 ehsm_key_format_ecc_sec_p521r1_cipher_privkey_st

```
typedef
```

定义: `ehsm_key_format_ecc_521_cipher_privkey_st ehsm_key_format_ecc_sec_p521r1_cipher_privkey_st`

8.7.99 ehsm_key_format_ecc_sec_p521r1_plain_keypair_st

定义 义 :

```
typedef ehsm_key_format_ecc_521_plain_keypair_st ehsm_key_format_ecc_sec_p521r1_plain_keypair_st
```

8.7.100 ehsm_key_format_ecc_sec_p521r1_cipher_keypair_st

```
typedef
```

定义: `ehsm_key_format_ecc_521_cipher_keypair_st ehsm_key_format_ecc_sec_p521r1_cipher_keypair_st`

8.7.101 ehsm_key_format_ecc_sec_p160k1_plain_pubkey_st

定义 义 :

```
typedef ehsm_key_format_ecc_160_plain_pubkey_st ehsm_key_format_ecc_sec_p160k1_plain_pubkey_st
```

8.7.102 ehsm_key_format_ecc_sec_p160k1_plain_privkey_st

定义 义 :

```
typedef ehsm_key_format_ecc_160_plain_privkey_st ehsm_key_format_ecc_sec_p160k1_plain_privkey_st
```

8.7.103 ehsm_key_format_ecc_sec_p160k1_cipher_privkey_st

```
typedef
```

定义: `ehsm_key_format_ecc_160_cipher_privkey_st ehsm_key_format_ecc_sec_p160k1_cipher_privkey_st`

8.7.104 ehsm_key_format_ecc_sec_p160k1_plain_keypair_st

定义 义 :

```
typedef ehsm_key_format_ecc_160_plain_keypair_st ehsm_key_format_ecc_sec_p160k1_plain_keypair_st
```


8.7.105 ehsm_key_format_ecc_sec_p160k1_cipher_keypair_st

typedef
定义: ehsm_key_format_ecc_160_cipher_keypair_st ehsm_key_format_ecc_sec_p160k1_cipher_keypair_st

8.7.106 ehsm_key_format_ecc_sec_p192k1_plain_pubkey_st

定义 :
typedef ehsm_key_format_ecc_192_plain_pubkey_st ehsm_key_format_ecc_sec_p192k1_plain_pubkey_st

8.7.107 ehsm_key_format_ecc_sec_p192k1_plain_privkey_st

定义 :
typedef ehsm_key_format_ecc_192_plain_privkey_st ehsm_key_format_ecc_sec_p192k1_plain_privkey_st

8.7.108 ehsm_key_format_ecc_sec_p192k1_cipher_privkey_st

typedef
定义: ehsm_key_format_ecc_192_cipher_privkey_st ehsm_key_format_ecc_sec_p192k1_cipher_privkey_st

8.7.109 ehsm_key_format_ecc_sec_p192k1_plain_keypair_st

定义 :
typedef ehsm_key_format_ecc_192_plain_keypair_st ehsm_key_format_ecc_sec_p192k1_plain_keypair_st

8.7.110 ehsm_key_format_ecc_sec_p192k1_cipher_keypair_st

typedef
定义: ehsm_key_format_ecc_192_cipher_keypair_st ehsm_key_format_ecc_sec_p192k1_cipher_keypair_st

8.7.111 ehsm_key_format_ecc_sec_p224k1_plain_pubkey_st

定义 :
typedef ehsm_key_format_ecc_224_plain_pubkey_st ehsm_key_format_ecc_sec_p224k1_plain_pubkey_st

8.7.112 ehsm_key_format_ecc_sec_p224k1_plain_privkey_st

定义 :
typedef ehsm_key_format_ecc_224_plain_privkey_st ehsm_key_format_ecc_sec_p224k1_plain_privkey_st

8.7.113 ehsm_key_format_ecc_sec_p224k1_cipher_privkey_st

typedef
定义: ehsm_key_format_ecc_224_cipher_privkey_st ehsm_key_format_ecc_sec_p224k1_cipher_privkey_st

8.7.114 ehsm_key_format_ecc_sec_p224k1_plain_keypair_st

定义 :
typedef ehsm_key_format_ecc_224_plain_keypair_st ehsm_key_format_ecc_sec_p224k1_plain_keypair_st

8.7.115 ehsm_key_format_ecc_sec_p224k1_cipher_keypair_st

typedef
定义: ehsm_key_format_ecc_224_cipher_keypair_st ehsm_key_format_ecc_sec_p224k1_cipher_keypair_st

8.7.116 ehsm_key_format_ecc_sec_p256k1_plain_pubkey_st

定义 义 :

```
typedef ehsm_key_format_ecc_256_plain_pubkey_st ehsm_key_format_ecc_sec_p256k1_plain_pubkey_st
```

8.7.117 ehsm_key_format_ecc_sec_p256k1_plain_privkey_st

定义 义 :

```
typedef ehsm_key_format_ecc_256_plain_privkey_st ehsm_key_format_ecc_sec_p256k1_plain_privkey_st
```

8.7.118 ehsm_key_format_ecc_sec_p256k1_cipher_privkey_st

定义:

```
typedef  
ehsm_key_format_ecc_256_cipher_privkey_st ehsm_key_format_ecc_sec_p256k1_cipher_privkey_st
```

8.7.119 ehsm_key_format_ecc_sec_p256k1_plain_keypair_st

定义 义 :

```
typedef ehsm_key_format_ecc_256_plain_keypair_st ehsm_key_format_ecc_sec_p256k1_plain_keypair_st
```

8.7.120 ehsm_key_format_ecc_sec_p256k1_cipher_keypair_st

定义:

```
typedef  
ehsm_key_format_ecc_256_cipher_keypair_st ehsm_key_format_ecc_sec_p256k1_cipher_keypair_st
```